

MATHEMATICAL INFERENCE

SCIENCE AND LOGIC

Jelmer de Boer

This book grew out of a master's thesis in Artificial Intelligence submitted to the Open University in 2026.

© 2026 Jelmer de Boer

*“I got bored one day and I put everything on a bagel...
everything: All my hopes and dreams, my old report
cards, every breed of dog, every last personal ad on
Craigslist, sesame, poppy seed, salt... and it collapsed
in on itself. ’Cause you see, when you really put
everything on a bagel, it becomes this: The truth.”*

*Jobu Tupaki in *Everything Everywhere All At Once**

Contents

1	Prelude: Andante sostenuto	1
1.1	Inspiration	2
1.2	Introduction	2
1.3	Outline	4
1.4	Idiosyncrasies	5
2	First movement: Allegro con brio	7
2.1	Solid foundations for mathematics	8
2.2	Solid foundations for statistics	9
2.3	A good and honest attitude for doing these sciences	10
2.4	Mathematical manifesto	11
3	Second movement: Andante con moto	13
3.1	The Jaynes–Cox programme	14
3.2	Objections	15
4	Third movement: Adagio	21
4.1	Different types of logic	22
4.2	Type theory	23
4.3	Martin-Löf-style type theory	25
5	Fourth movement: Allegro maestoso	27
5.1	Context manipulation in Martin-Löf-style type theory	28
5.2	Modes and modalities	28
5.3	Probability types	29
5.4	Structural causal models in Martin-Löf-style type theory	31
5.5	Lean 4 implementation	36
5.6	Proof status	36
5.7	Intermediate results	37
6	Fifth movement: Presto con fuoco	39
6.1	Questionable research practices on a large scale	40
6.2	Start of a research career	42
6.3	Reporting in journals	43
6.4	Peer review and editorial process	44
6.5	Career advancement	45
6.6	Reform proposals	46

Contents

6.7	Synthesis	49
7	Postlude: Andante	51
7.1	Results	52
7.2	Discussion	52
7.3	Future work	54
	Appendices	57
A	Definitions	59
A.1	Type-theoretic definitions	61
A.2	Mode theory	78
A.3	Probabilistic definitions	79
A.4	Finite causal models	100
A.5	Do-calculus	126
A.6	Counterfactual semantics	139
A.7	Modal causal representation	144
A.8	Finite causal transport interfaces	159
A.9	Worked finite causal example	172
B	Proofs and arguments	177
B.1	Causality argument	179
B.2	Physics argument	180
B.3	Excluded middle is not derivable	181
B.4	Constructive type-theoretic facts	182
B.5	Lean verification and foundational boundary	184
B.6	Clayton–Waddington probability proof	188
B.7	Finite causal-model correspondence	199
B.8	Finite structural-causal-model construction and latent projection	207
B.9	Modal and structural causal execution	209
B.10	Counterfactual and multiworld execution	216
B.11	Transport of published identification	220
	Bibliography/sources	227

CHAPTER 1

Prelude: Andante sostenuto

“Science, including philosophy, is intrinsically polemical, and the polemical cannot be separated from the agonistic. In epochs in which great new things arise, the agonistic factor usually comes to the fore”

Johan Huizinga in *Homo Ludens*

1.1 Inspiration

When I was an undergraduate studying physics, probability theory often felt oddly under-defined. In statistical physics, and even more sharply in quantum physics, probability seemed to appear sometimes as ignorance, sometimes as frequency, sometimes as a physical property, and sometimes merely as a useful rule for calculation. The mathematical formalism worked well enough; that was not the issue. The problem was that there didn't seem to be a coherent interpretation, so the underlying logical story remained inconsistent.

After my studies I discovered E. T. Jaynes' *Probability Theory: The Logic of Science* [Jay03]. That book was, for me, the book I would have liked to have read much earlier. Jaynes, also trained as a physicist, refused to treat probability as a bag of unrelated conventions. He presented it, instead, as a disciplined extension of logic: a way of saying what one should believe, given what one already knows. The attraction was technical, and it restored a kind of intellectual honesty to the subject: hidden assumptions and convenient interpretations could no longer be smuggled in without being noticed.

Yet, Jaynes' book stops before the story is finished. Part of this is literal: the book was posthumously released on the basis of an unfinished manuscript. Part of it is conceptual: Jaynes gave a powerful solution for probability, following and extending Cox's programme [Cox46; Cox61], but several of his broader complaints about mathematics, subjectivity, causality and scientific practice remain only partially resolved. This book is therefore both a tribute and an attempt at continuation. It asks what Jaynes' programme might look like if it were done in an epistemological, constructive, type-theoretic and causal setting.

I dedicate this work to the memory of E. T. Jaynes, who saw the possibility of truth and explored it.

1.2 Introduction

The central concern of this work is the process by which new knowledge is added to existing information. I will call this process "mathematical inference". I regard this process as central to scientific reasoning. Jaynes' concern extended beyond the correctness of a few specific statistical formulae. He believed that the foundations of science itself were at stake, because large parts of mathematical and statistical practice had become too permissive and too tolerant. His polemical tone makes sense in this context: a pervasive and foundational problem calls for more than a technical correction.

The urgency that is felt in Jaynes' text also perfectly fits the opening quotation from Huizinga on the polemical and agonistic nature of defending new scientific paradigms. This urgency becomes even more justified if one looks at two interrelated phenomena: independent analysts reaching divergent conclusions from the same data [Bot+20; Bre+22; Sch+21; Sil+18], and large replication projects that reveal failures to replicate important and accepted scientific findings [Cam+16; Kle+14; Ope15]. Based on these phenomena, it becomes abundantly clear that a well-defined theory of mathematical inference is an urgent matter.

Three concerns run through this monograph:

1. Mathematics should have foundations that make assumptions explicit and prevent uncontrolled use of ideal objects like infinity;
2. Mathematical inference should be treated as a principled extension of logic, while also being honest about the limits of that derivation;
3. Scientific practice should be organised in such a way that good inference is encouraged rather than punished.

Jaynes addresses all three, but solves only part of the second. The present work takes these concerns together, since separating them would miss part of the problem.

The constructive setting used later is Martin-Löf-style type theory [DP24; Mar84; NPS90]. It forces constructions, contexts and rules to be stated, and it connects mathematics to computation, so that finite probability can be rebuilt in a checkable form, following finitary refinements of Cox-style arguments [CW17]. Causal questions beyond prediction will use Pearl’s structural causal models [Pea09; PM18].

The relevance for artificial intelligence is direct. If artificial intelligence is, at least in part, the formalisation and implementation of inference, then its reliability depends on the quality of the reasoning methods on which it is built. A finite, constructive and computational account of probability and causality would provide a clearer route from abstract reasoning to executable systems.

The causal extension is especially important. Modern intelligent systems are increasingly expected not only to predict, but to answer questions about intervention, responsibility and alternative outcomes. Pearl’s causal framework shows why prediction has to be supplemented by intervention and counterfactual reasoning [Pea09; PM18]. In the finite setting this book computes concrete interventional and counterfactual values and checks them. The later hiring example makes this framework concrete by supplying explicit interventional and counterfactual calculations.

The broader scientific-practice component is also relevant to artificial intelligence. McElreath’s presentation of Bayesian modelling keeps statistical inference close to modelling choices, scientific questions and workflow [McE20]. AI systems trained on, used within, or used to accelerate science inherit the weaknesses of the scientific pipeline. If the surrounding environment rewards fragile results, opaque reasoning or publication gamesmanship, then faster inference may simply produce bad science more efficiently. The formal and institutional questions therefore have to be considered together.

Scope and assumptions

This book is concerned with artificial intelligence, where artificial intelligence is understood broadly as the formalisation and implementation of inference. In this sense probability theory, logic, causality and proof checking all fall naturally within its scope. The aim is to ask what kind of mathematical inference such systems should perform when they reason under uncertainty, act under causal assumptions, or revise an epistemic state.

The monograph looks at inference on both the mathematical and sociological level. Mathematically, it asks which formal rules should ideally govern uncertain, subjective and causal inference. Sociologically, it asks in what

1. Prelude: Andante sostenuto

scientific environment mathematical and statistical inferences are generally made. This distinction matters, because the proposed formal machinery may not fit comfortably inside every existing scientific habit; and if that turns out to be the case, the problem may lie – at least partly – with those habits.

The mathematical scope is deliberately restricted: finite probability spaces; finite causal models; explicitly representable contexts and epistemic states; and central objects that are constructible and implementable. Thus the technical development is discrete throughout, but more narrowly so: the results below are finite results, not a general theory of arbitrary discrete spaces. This restriction follows Jaynes’ suspicion of uncontrolled infinities; it follows the constructive spirit of Martin-Löf-style type theory; and it fits the computational aspect of artificial intelligence.

Selected formal parts are implemented in Lean 4 [MU21] where this is useful and feasible. Lean is used as a proof checker for representative definitions, proofs and examples, so that the central constructions are precise enough for independent mechanical checking. Recent discussions of digitised proofs and AI-assisted mathematics make clear that this is becoming an increasingly relevant part of mathematical practice [Cas26; Kak26; Orn26; Slo26].

Several limitations follow directly from this scope: this work does not reconstruct all of classical probability theory; it does not provide a general theory of continuous probability spaces; it does not attempt to solve measure theory; it does not claim that Martin-Löf-style type theory is the only acceptable foundation for mathematics; it does not claim that Pearl’s framework captures everything that can meaningfully be called causality; and it does not pretend to give a complete empirical sociology of science. The aim is narrower: to construct a coherent finite framework in which probability, subjectivity and causality can be represented with as little hand-waving as possible, and to fit this framework within its practical context.

1.3 Outline

The book is structured as follows:

The first movement (Chapter 2) states Jaynes’ three complaints – foundations of mathematics, foundations of statistics, and a good scientific attitude – and fixes the manifesto that the rest of the monograph tries to obey.

The second movement (Chapter 3) reconstructs the Jaynes–Cox derivation and then asks what it actually buys. Consistency, reasonableness, universality, subjectivity and causality are treated as five different kinds of overreach.

The third movement (Chapter 4) chooses the language: Martin-Löf-style type theory, intuitionistic in its logic, epistemological in its interpretation, and finite enough to avoid infinite paradoxing.

The fourth movement (Chapter 5) does the reconstruction: contexts and modes, probability types, and finite structural causal models, with representative calculations checked in Lean.

The fifth movement (Chapter 6) returns to Jaynes’ vices as responses to a selection environment, then proposes reform.

The postlude (Chapter 7) restates what was constructed, where the boundary sits, and what remains open.

The aim is to represent finite Jaynes-style probabilistic updating and finite Pearl-style causal reasoning in the chosen Martin-Löf-style framework, with representative constructions checked in Lean 4, and to connect that reconstruction to the claim that good science needs both good inference and a good environment in which this inference takes place.

The appendices contain definitions, proofs, arguments and code-oriented material that would interrupt the main line of the text. They are not less important, but they are placed where they can be checked without interrupting the narrative.

1.4 Idiosyncrasies

Before continuing, a few habits of this text deserve some attention:

The structure is intentionally unconventional. The book is arranged as a symphony, with movements rather than ordinary parts. This is inspired by Huizinga’s insistence on the playful character of culture and science, and by the suspicion that some of this playfulness has been replaced by a more anxious and institutionalised competition. The musical structure is a reminder that scientific thought is not just a sequence of deliverables, but also a playful activity. The tempo markings are mood cues rather than a strict musical programme: the prelude is deliberate, the first movement polemical, the second reflective, the third slow and foundational, the fourth broad and constructive, the fifth quick and heated, and the postlude slower again.

Acronyms will be avoided where they do not genuinely help. Science already contains enough abbreviations that sound useful mainly because they sound fundable. Likewise, ordinary terms will not be visually inflated with unnecessary capital letters. This means there will be no grand acronym for “Mathematical Inference as a Logical Extension” (MILE); there will simply be “mathematical inference”.

This book was written with active AI collaboration. ChatGPT assisted with source discovery and general framing; Lumo with stylistic suggestions; Claude and ChatGPT with coding and proof-checking; Claude also contributed to the figures and diagrams. This use of agents places the project within the wider movement towards AI-assisted formal mathematics [Cas26; Kak26; Orn26; Sku26; Slo26]. I determined the storyline, critically assessed the literature and made the final decisions on the prose and proofs. Every paragraph was rewritten, weighed and approved by me, and responsibility for the argument and any errors remains mine. The book is 100% AI-assisted and 100% human-reviewed. The work follows the Leiden declaration on artificial intelligence and mathematics [Alp+26].

The intended audience is broad, and this is risky. The book lies at the crossing of logic, mathematics, artificial intelligence, philosophy of probability, causality and the sociology of science. Readers from any one of these domains may sometimes want more detail exactly where another reader needs less. The compromise adopted here is narrative-first: the main text preserves the flow of ideas, while technical definitions, proofs and code are moved to the appendices whenever they would otherwise break the story.

These idiosyncrasies are part of the method. A book about explicit reasoning, constructive mathematics and scientific honesty should itself try to be explicit

1. Prelude: Andante sostenuto

about the way it reasons and the choices it makes. The reader is invited to follow this movement.

CHAPTER 2

First movement: Allegro con brio

“Indeed, an insistence that all of mathematics be stated in that language all of the time can place unnecessary burdens on a theory, particularly one intended for application in the real world. It can also degenerate into an affectation, used only linguistically rather than functionally. To give every old, familiar notion a new, impressive name and symbol unknown to Gauss and Cauchy has nothing to do with rigor. It is, more often than not, a form of gamesmanship whose real purpose is to conceal the Mickey Mouse triviality of what is being done. One would blush to state it in plain English”

E. T. Jaynes in *Probability Theory: The Logic of Science*

2. First movement: Allegro con brio

This movement states Jaynes’ three complaints – foundations of mathematics, foundations of statistics, and a good scientific attitude. From these complaints a small mathematical manifesto follows.

2.1 Solid foundations for mathematics

Jaynes’ first complaint is directed at a particular kind of mathematical culture: a culture in which formal derivation is too easily confused with scientific usefulness, and in which the ability to write down a consistent-looking construction is mistaken for the ability to explain anything worth explaining; see the opening quote from Jaynes.

The most visible symptom is what is called “infinite paradoxing” by Jaynes. Modern mathematics is riddled with results that actively discover the limits of infinity (pun intended): Banach–Tarski decompositions, non-measurable Vitali sets, the Borel–Kolmogorov paradox, supertasks such as Thomson’s lamp, and Ross–Littlewood-style urn puzzles [AK91; AK95; BT24; GHR17; Tho54; Vit05]. These examples do not all have the same status: some are genuine theorems; some are philosophical stress tests; some are warnings about limiting procedures.

The deeper objection is therefore methodological. Axiomatic set theory, especially Zermelo–Fraenkel set theory with choice, easily allows for deeply abstract constructions. Its ease of use is the main reason for its success. By the same token, it is also deceptively easy to create nonsensical or non-applicable objects. The trained mathematician may admire the sheer possibility; the physicist may ask how it can be measured. Jaynes was a classically trained physicist: start with what can be stated, counted, constructed or approximated, and move to limits only with a clear account of what the limiting process achieves. If the point of mathematics in science is mathematical inference, then a method that converts finite observations into uncontrolled ideal objects should inherently be treated with suspicion. It may still be useful, but usefulness is not conferred by grandeur.

It is worth noting that, although Jaynes actively fulminates against using Zermelo–Fraenkel set theory with choice as a basis for mathematics, he uses it himself, albeit in a rather conservative way [Jay03]. This is therefore an objection that he extensively discusses, but that he is unable to ‘solve’. He proposes a more conservative approach to using the theory as it stands, but that approach asks users deliberately not to explore the full capability of the theory – an argument that is rather unconvincing to the mathematical community.

In this monograph I side with the physicists: for the formal reconstruction, the default mathematical environment will be constructive and finitary, and the motive is efficiency. A framework that forces objects to be constructed, contexts to be explicitly declared and dependencies to be visible is better suited to the kind of scientific reasoning Jaynes wanted. Rather than running the risk of spending valuable resources and research facilities on obtaining results that are questionable for applied research, it can be more efficient to construct everything in a more limiting environment, where results are more likely to be directly applicable and computationally checkable.

This is why the third movement, Chapter 4, turns to alternative logics and Martin-Löf-style type theory to find a setting in which constructions, contexts

and dependencies have to be stated openly. The fourth movement, Chapter 5, then uses this theory as a mathematical foundation for the reconstruction itself.

2.2 Solid foundations for statistics

Jaynes’ second complaint is more direct and more devastating. In his view, twentieth-century statistics had become a patchwork of procedures, tests, conventions and reputations. Some procedures worked well in practice; some were defended by tradition; some were inherited because nobody wanted to reopen old discussions. What was clearly missing was a single account of why statistical inference should have the form it had.

The force of Jaynes’ criticism becomes clearer when placed next to its possible consequences. Independent teams can receive the same data and reach sharply different conclusions, as shown in Figure 2.1 [Bot+20; Bre+22; Sch+21; Sil+18]. This poses a serious problem, because it threatens the foundations of statistical claims in economics, psychology, sociology and other sciences. Large replication projects make the same fragility visible at scale in Chapter 6 [Cam+16; Kle+14; Ope15]. This objection therefore bears even more serious consequences than the previous objection: time and energy can be wasted, and there is an acute risk of consciously propagating incorrect conclusions, which is directly contrary to doing science. This clearly illustrates that inference is fragile when assumptions are hidden, workflows are underdescribed and technical choices are treated as harmless details [GL13; Ioa05].

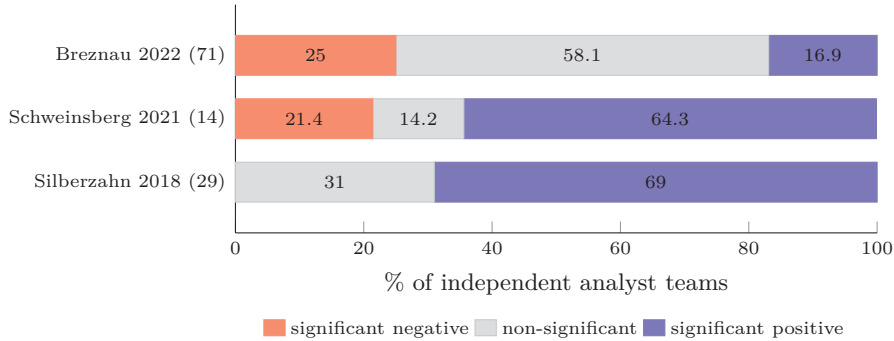


Figure 2.1: Three many-analyst projects in which independent teams analysed one dataset, classified by the *direction* of the reported result [Bre+22; Sch+21; Sil+18]. Botvinik-Nezer et al.’s neuroimaging study is discussed in the text but omitted here, since the experiment with 70 teams had a slightly different set-up [Bot+20].

Cox’s programme offered Jaynes exactly the kind of answer he wanted: if plausibility is represented numerically, if logically equivalent states of information are assigned equivalent plausibilities, and if reasoning is required to be consistent across different routes, then the familiar rules of probability can be recovered after a suitable rescaling [Cox46; Cox61; Jay03]. Probability is then not a bag of recipes; it is extended logic: the calculus of consistent reasoning under uncertainty.

2. First movement: Allegro con brio

This is the great attraction of the Jaynes–Cox view: statistical inference is what one must do if one wants uncertainty to obey the same coherence that ordinary deductive logic obeys in the limiting case of absolute certainty. Keynes, Ramsey and Jeffreys had already made probability philosophically serious as a calculus of partial belief; Cox and Jaynes gave the programme its sharpest logical form [Jef61; Key21; Ram31].

Of the three complaints filed in this movement, this one can be considered the most ‘solved’ and the most complete by Jaynes. I therefore see Jaynes’ solution as hugely important, but still partly incomplete. It gives a principled answer to a real problem by treating probability as a form of coherent inference. Yet its derivations themselves depend on assumptions that are not completely innocent, and the resulting framework lacks causality and a native representation of subjectivity.

The second movement, Chapter 3, will therefore critically assess the Jaynesian programme: what it derives, what it assumes and where its claims become too large. The fourth movement, Chapter 5, then rederives the finite Bayesian core and expands it such that subjectivity and causality are made explicit within the formal language.

2.3 A good and honest attitude for doing these sciences

The third complaint is less formal, but no less central. Jaynes objects to bad foundations and, more sharply, to bad habits. His prose is rather polemical and, at times, even develops into rather personal attacks¹. Since, for Jaynes, the very heart of scientific praxis is at stake – with an impractical basis for mathematics and a non-rigorous basis for statistics – this objection is absolutely the most serious. It directly takes aim at people who, in his view, can be considered personally responsible for the bad state of science as it stands.

The habits Jaynes attacks can be arranged under five headings:

1. **Gamesmanship:** the desire to display technical virtuosity, rather than to clarify a problem;
2. **Cutting corners:** the focus on polished results, without explaining the painstaking work that led to these outcomes;
3. **Hidden assumptions:** the silent transfer of what should have been stated into what is merely presumed;
4. **Hyperfocus:** the habit of investigating only an incredibly specialised subfield, whilst forgetting the wider context in which this research sits;
5. **Theoretical navel-gazing:** the production of mathematics that neither explains, predicts, nor helps anyone build or test anything practical.

For this complaint too, Jaynes discusses the problem at length but does not ‘solve’ it. He treats this complaint rather personally, while I believe it deserves a more institutional view. The vices mentioned by Jaynes are not limited to

¹Especially Ronald Fisher, but also William Feller, Nicolas Bourbaki/the Bourbaki group and others take personal blows.

‘pure’ mathematics and statistics, and they are, unfortunately, not limited to Jaynes’ opponents². They are temptations built into scholarly life.

The fifth movement, Chapter 6, returns to these vices in a more institutional setting. I will argue there that they appear whenever professional reward becomes too detached from intellectual responsibility. The formal reconstruction of the intervening movements is therefore not separate from the sociological diagnosis: both are attempts to make hidden assumptions harder to hide.

In software development, there has been some attempt to establish workflow standards as part of an effort to optimise operations, regardless of the particulars of each separate project. In mathematics, and science in general, there seems to be a general lack of such efforts. In the following, I will put forward a standard that I will adhere to, that is aimed at preventing most of these critiques.

2.4 Mathematical manifesto

From these complaints a small mathematical manifesto follows. It borrows its form from the Agile Manifesto and from later workflow frameworks such as Scrum: a short set of comparative preferences, meant to guide practice [Bec+01; SS20]. For the rest of the book, the discipline is this manifesto:

I am uncovering better ways of developing mathematical theories by doing it and helping others do it. Through this work I have come to value:

Algorithms over abstractions

Narrative over formulae

Explicit over implicit

Simple over complex

That is, while there is value in the items on the right, I value the items on the left more.

Algorithms over abstractions: abstraction should remain connected to construction, computation or procedure. If a concept cannot in principle be implemented, checked, traced or approximated, then its scientific role should be handled with care.

Narrative over formulae: formulae should serve the movement of thought. A derivation is not a heap of symbols; it is a story about why one statement follows from another. When the story is hidden, the formulae may still be true, but they become harder to inspect.

Explicit over implicit is the most Jaynesian rule: assumptions should be visible, contexts should be named and dependencies should be tracked. If a proof requires a specific domain, a decidable event, a specific logic or a specific causal graph, that requirement should not be secretly smuggled in. The constructive

²Cutting corners also appears inside Jaynes’ own work: some of his strongest claims about universality, subjectivity and completeness move faster than the derivations can justify. This is not a reason to discard Jaynes: it is a reason to take him seriously enough to apply his own standard to him.

2. First movement: Allegro con brio

framework used later is chosen because it makes hidden commitments harder to hide.

Simple over complex prefers the smallest mechanism that does the job. Complexity is sometimes necessary, but it should be paid for consciously. If a simpler derivation, formula, or even plain English sentence does the job equally well, then the more complex solution requires a justification.

CHAPTER 3

Second movement: Andante con moto

“A story is needed before one can set sail. Look at us. For the sake of the story, we are sitting here aboard a water taxi on our way to the Holy Grail, the unknown, and adventure. People think life becomes more enjoyable through stories. That is true enough, but it is not the point. Without stories, it becomes meaningless. Sense consists of sentences. The words that tell the story lay a reassuring relation of cause and effect over random facts and events. People crave plot because a plot reduces the unbearable and unmanageable chaos of the sublunary world to a human scale, and to a chain of initiatives and consequences that a person is able to comprehend. A plot gives a sense of control, provenance and destination, origin and direction”

Ilja Leonard Pfeijffer in *Grand Hotel Europa*

3. Second movement: Andante con moto

This movement reconstructs the Jaynes–Cox programme as a derivation of probability as extended logic and then asks what that derivation actually buys: what is derived, and what is claimed.

3.1 The Jaynes–Cox programme

Desiderata

The Jaynes–Cox programme begins with a modest question: what rules should govern reasonable degrees of belief [Cox46; Cox61; Jay03]? The exposition starts with three desiderata that, according to Jaynes, are the minimal, reasonable requirements that a calculus of plausible reasoning ought to satisfy. In Jaynes' own words, in the compact formulation [Jay03]:

- (I) Representation of degrees of plausibility by real numbers;
- (II) Qualitative correspondence with common sense;
- (III) Consistency.

In the ensuing discussion, Jaynes unpacks consistency into three demands: a conclusion should not depend on the route by which it is reached; relevant evidence should not be ignored; and equivalent states of knowledge should receive equivalent plausibility assignments. The first desideratum translates to divisibility and comparability: plausibility should be represented on a single ordered numerical scale, so that one state of information can be compared with another. The second is qualitative agreement with common sense: ordinary Aristotelian logic should appear as the limiting case in which propositions are known with certainty, and small changes in the premises should not produce arbitrary jumps in the conclusions. The third is consistency: if a plausibility can be obtained by two legitimate routes, the two answers should agree. These desiderata are then translated, via functional equations, into the familiar axioms of probability theory.

Derived calculus

Cox's formal move is to apply these desiderata to the logical operations of negation and conjunction. If A and B are propositions and I is the background information, then the plausibility of $\neg A$ should be determined by the plausibility of A , and the plausibility of $A \wedge B$ should be determined by the plausibility of A and the plausibility of B once A is assumed. Under the usual regularity assumptions, one can rescale the plausibility function to a probability function P satisfying

$$P(A \wedge B | I) = P(A | I)P(B | A, I)$$

and

$$P(A | I) + P(\neg A | I) = 1.$$

The first equation is the product rule; the second is the sum rule for complements.

Bayes' theorem then follows by applying the product rule in both orders:

$$P(A | B, I) = \frac{P(B | A, I)P(A | I)}{P(B | I)}$$

whenever $P(B \mid I)$ is non-zero. Here $P(A \mid I)$ is the prior probability of A relative to background information I , $P(B \mid A, I)$ is the likelihood of observing B under A , $P(B \mid I)$ is the evidence or normalising term, and $P(A \mid B, I)$ is the posterior probability after B has been taken into account. In general, many probabilities, such as the likelihood, are often clear from the context of the problem and therefore treated as relatively objective. The prior distribution is usually harder to fix, and is therefore often treated as more subjective. This degree of freedom makes this theory notoriously controversial, but allows the theory to introduce ‘subjectivity’ as the ability to freely choose one’s prior.

With these ingredients, one recovers the ordinary finite probability calculus: non-negativity, normalisation and finite additivity. In the classical measure-theoretic presentation, this calculus is extended to Kolmogorov’s axioms [Jay03]. In this classical reading, a probability function P satisfies Kolmogorov’s axioms if:

1. *Non-negativity*: $P(A) \geq 0$ for every $A \in \mathcal{F}$;
2. *Normalisation*: $P(\Omega) = 1$;
3. *Countable additivity*: for every countable pairwise disjoint family $(A_i)_{i=1}^{\infty}$ in $\mathcal{F}$,

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i).$$

Here $\mathcal{F}$ is a σ -algebra of events. The finite version of this calculus is reconstructed in the fourth movement, Chapter 5.

Jaynes did not invent Bayesian reasoning; Keynes, Jeffreys, Ramsey and Cox are the ancestors, and he explicitly inherits the Coxian line [Cox46; Cox61; Jef61; Key21; Ram31]. His contribution is the clarity of the synthesis: probability as extended logic, statistical inference as disciplined common sense, and scientific disagreement as often traceable to hidden assumptions or silently disregarded desiderata. The philosophical gain is large: what appears to be a specialised derivation sets the norm for coherent reasoning under uncertainty. Probability is then more than a matter of repeated experiments: it is about what follows from information.

Jaynes really maintained that he uncovered a universal and logical theory of doing science and that everything deviating from either Bayes’ rule or his logical desiderata should be considered fundamentally illogical and – even more strongly so – fundamentally unscientific. The attraction of this view is obvious. If correct, the programme gives mathematical inference a foundation. The remaining sections critically assess the programme and ask whether all its assumptions and conclusions can, in fact, be justified.

3.2 Objections

The objections to Jaynes do not all have the same focus: some are technical; some are philosophical; some concern scope. I will treat them in an order that moves from the derivation itself to the larger claims built on top of it: consistency, reasonableness, universality, subjectivity and causality.

3. Second movement: Andante con moto

The guiding distinction is the one already announced in the prelude: what is derived, and what is claimed. Jaynes derives a powerful representation result under assumptions. He sometimes writes as if he has also derived a universal account of rationality, subjectivity and scientific method. The route through these objections is summarised in Table 3.1.

Objection	Target	Problem for Jaynes	Response
Consistency	Cox-style derivation	Naive formulations of Cox's assumptions can fail on finite or insufficiently rich domains.	Treat the counterexamples as design constraints and rebuild the finite assumptions explicitly.
Reasonableness	Jaynes' desiderata	A consistent assumption can still be too technical, too strong or too poorly motivated.	Use the finite urn principles of Clayton and Waddington as the constructive route.
Universality	Probability as universal logic	A theorem inside one background logic is not automatically a logic of all uncertainty or all science.	Put probability inside a constructive and epistemological setting with explicit scope.
Subjectivity	Priors and background information	The subject is represented mainly by I and the prior, while the agent's position remains implicit.	Represent epistemic position through contexts, modes and modalities.
Causality	Prediction by probability	Statistical dependence does not by itself distinguish seeing, doing and imagining.	Add finite structural causal models and transport the relevant causal results.

Table 3.1: Route through the objections to the Jaynes–Cox programme.

Consistency

The first worry is whether Cox-style assumptions are even consistent in the form needed for Jaynes' conclusion. Halpern showed that naive formulations of Cox's theorem can fail – finite domains and insufficiently rich plausibility ranges can produce counterexamples to the intended representation [Hal99a; Hal99b].

Later work repairs the situation. Arnborg and Sjödin treat finite models and argue for additional principles under which Bayesian rules can be recovered or justified [AS00; AS01; AS99]. Van Horn reconstructs the assumptions needed for a continuous Cox-style theorem more carefully, and Snow defends finite-domain reasonableness from a related angle [Sno98; Van03].

These results illustrate that Cox-style derivations require additional assumptions: domain richness, regularity assumptions and a precise account of what counts as a permissible plausibility assignment. For this monograph, these critiques are taken as instructive sources of information: if probability is

to be rebuilt constructively, the needed assumptions must be made precise and checkable. This is the role of the finite reconstruction in the fourth movement, Chapter 5.

Reasonableness

A subtler worry is whether the assumptions are reasonable to begin with. An assumption can be consistent and still be philosophically unattractive. Shafer objects, among other things, to the idea that reasonable expectation must always be represented by a single real number [Sha04], and Van Horn’s reconstruction – although technically precise – does not by itself make every assumption compelling¹ [Van03].

Clayton and Waddington are especially important here because they aim to design explicit logical and intuitive rules that are more in line with Jaynes’ intentions. Instead of simply postulating technical conditions such as universality and strict monotonicity on large domains, they derive the needed structure from simple urn-based rules [CW17]. The finite assumptions needed later are listed in Definition A.3.34. The original paper contains some additional rules and derivations not needed here, as it operates in the classical axiomatic set-theoretic domain.

As Clayton and Waddington’s account of probability is perfectly in harmony with the mathematical manifesto stated earlier, Section 2.4, this monograph bases its derivation on their work. The reconstruction in this book only keeps the finite urn principles and changes the order of their derivations somewhat, but the idea remains the same. It is important not to take Jaynes’ desiderata as sacred, immutable tablets, but as a solid basis on which one can sensibly vary. Clayton and Waddington’s account might make the resulting probability theory less grand, but more explicit and, therefore, directly checkable and less evasive. The fourth movement, Chapter 5, implements these assumptions to derive the finitary Kolmogorov axioms and Bayes’ theorem.

Universality

Jaynes often writes that his derived probability theory is the universal logic of science. Shafer and Colyvan give useful warnings against this overreach: Cox-style arguments presuppose a background logic, and the choice of that logic matters [Col04; Sha04]. A theorem about plausible inference inside classical propositional logic is not automatically a theorem about all forms of uncertainty, all domains of discourse or all scientific reasoning.

The point becomes clearer once one distinguishes ontological and epistemological questions. An ontological logic asks what is the case; an epistemological logic asks what is known, knowable or justified from a given position [Hof23; SN25]. The question “Does God exist?” illustrates the contrast nicely: an ontological view forces one to give an immediate answer; an epistemological stance may answer that the question is simply not answerable.

Jaynes’ framework is epistemological in temperament – as he considers inference as the process of adding knowledge to a given, personal state of information – but formally ontological. He wants probabilities to describe

¹An objection explicitly mentioned by Van Horn himself.

3. Second movement: Andante con moto

states of knowledge, yet his derivation usually assumes a classical propositional background in which every proposition is immediate (true or false). However, vagueness, fictional discourse, undecided mathematics, limited interest and forgotten information are not well represented by simply assigning a number to such a classical proposition, as also maintained by Van Horn [Van03].

The response is to keep probability and drop the claim that a derivation in one background logic is the final logic of everything. The present monograph uses a constructive and epistemological setting, but remains modest about its applications to other domains. In that setting a judgement is always relative to a context. Probability remains central, placed inside a richer account of contexts, modes and epistemic states. The selection of the proper logic that sits more in line with Jaynes' programme is done in the third movement, Chapter 4.

Subjectivity

In the terminology of Bayes' theorem above, Jaynes' subjectivity enters mainly through the background information I and the prior $P(A | I)$: different agents may start from different information and therefore assign different probabilities. But the subject is still strangely absent from the formalism. The prior may vary from one agent to another, while the structure of the agent's position is not itself represented.

For a truly subjective representation, it is essential to know what information, per agent, is available. This means knowing what problems are in scope; what information has been forgotten; what information is of no interest; what connections are believed to hold true; and what can be imagined, based on the available information. These affect what can be asked, what can be inferred and what counts as an identical problem. These questions should be part of the mathematical formalism itself, rather than being added as an afterthought.

Jaynes rightly objects to treating probability as a property of objects in isolation, a stance often defended by frequentists. In this view, repeating a measurement until one 'uncovers the True Probability' simply means, as Jaynes correctly points out, an ill-defined procedure, as one can make a repeated coin flip have any experimental outcome one likes². A coin does not carry a metaphysical label "one half". A physical procedure, a personal state of information and a model of the experiment are often covertly doing the heavy work. However, stating this in prose does not yet build subjectivity into the mathematics.

Interest is a clear example, somewhat alluded to by Colyvan [Col04]. Classical logic lets every proposition sit in the same universe of discourse. Real agents do not work like that. To a real agent, a proposition may be true, false, unknown, irrelevant, forgotten or outside the present domain of concern. One may not care whether Achilles was considered the fastest among mortals by the ancients, nor what ending Gogol's Dead Souls would have had if he had lived a bit longer, while caring deeply about colugos. A logic of subjectivity should be able to represent this asymmetry without pretending that every statement is waiting for some kind of 'objective' probability to happen.

²Jaynes often refers to this phenomenon as the 'projection fallacy', where personal missing information is projected on a situation as a whole – like with the coin flip – or even is seen as being *fundamentally* missing – as with quantum mechanics.

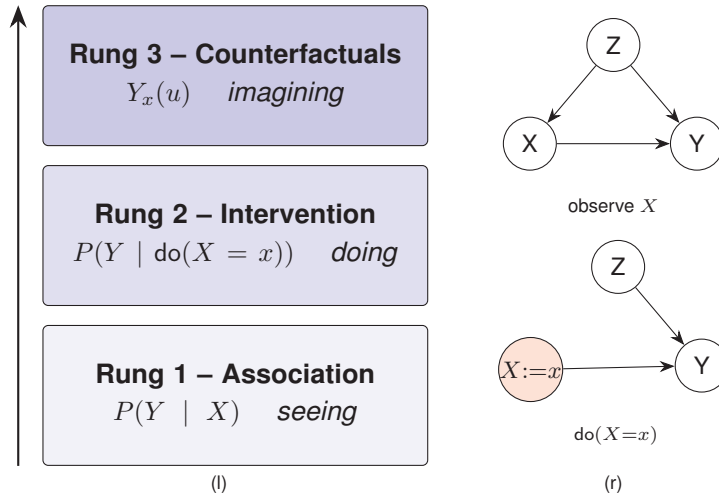


Figure 3.1: Pearl’s account of causal inquiry. (l) The ladder of causation: each rung answers a strictly stronger class of questions [PM18]. (r) Observing X leaves the confounding path through Z intact; intervening on X severs its incoming edges.

The constructive proposal in later sections is to make this epistemic position explicit. Subjectivity then becomes part of the mathematical machinery that is developed in the fourth movement, Chapter 5.

Causality

Jaynes’ probability theory can express statistical dependence, but dependence is not causation, as the famous adage goes. That is, a barometer and a storm can be tightly associated without the barometer causing the storm to happen. I give a pragmatic argument in Section B.1 for why a working theory of scientific reasoning – under which mathematical inference falls – should include causal models. The main components of this argument are already present in the opening quote by Pfeijffer, albeit there much more eloquently than I am capable of.

Pearl’s framework supplies the missing formal technology. This framework distinguishes three levels of causal inquiry. At the bottom sits association: questions such as $P(Y \mid X)$, where one asks how observations influence each other. Above that comes intervention, with questions such as $P(Y \mid \text{do}(X = x))$, asking what would happen if variables are forced to a specific value. At the top lives counterfactual reasoning – questions such as $Y_x(u)$, where one asks what the outcome for a particular unit hypothetically would have been under a different intervention; see Figure 3.1 (l) [Pea09; PM18]. These three levels require different assumptions, and they answer fundamentally different questions.

The first rung is still within Jaynes’ reach: association is a question about $P(Y \mid X)$. The second rung no longer is: if a variable is *set*, the mechanism that normally determines the variable is replaced. Observing leaves the ordinary causal paths in place; intervening severs the incoming edges of the variable that is set; see Figure 3.1 (r). The third rung is stronger still: a counterfactual

3. Second movement: Andante con moto

asks what the outcome for a particular unit would have been under a different intervention. That question needs a model in which the background conditions can be held fixed while a mechanism is replaced. Pearl's structural causal models supply that language [Pea09; PM18].

I consider Pearl's account an important extension of Jaynes' work. Structural causal models allow one to hypothesise while also recording a personal state of information: what is in scope and which relations are taken to hold. These models will be used in the fourth movement, Chapter 5, as a vehicle for subjectivity and causality.

CHAPTER 4

Third movement: Adagio

“A common mistake that people make when trying to design something completely foolproof is to underestimate the ingenuity of complete fools”

Douglas Adams in *Mostly Harmless*

4. Third movement: Adagio

This movement chooses a language for mathematical inference: Martin-Löf-style type theory, intuitionistic in its logic, epistemological in its interpretation, and finite enough to avoid infinite paradoxing.

4.1 Different types of logic

In light of the overly broad universality claims posited by Jaynes (Chapter 3), the next step is to enquire which logic is appropriate for which task. Classical logic is great when its assumptions fit: propositions are true or false; excluded middle is accepted (as propositions are always either true or false); contradiction is fatal; and the inferential world is crisp as all propositions are always available to everyone. However, reasoning is not always that crisp, and artificial intelligence does not have the luxury of pretending so.

Fuzzy and many-valued logics deal with predicates that do not behave as if they have sharp, binary borders. “Young”, “tall”, “similar” and “healthy” are not always best captured as if they only had two options, but rather a whole range of values or a degree of truth [CFN22]. Probability is sometimes misused here: a degree of truth is not the same thing as a degree of belief. Van Horn makes this distinction sharper, emphasising that plausible or probabilistic inference concerns uncertainty about propositions, but does not concern assigning gradients to propositions [Van03].

Modal logic adds structure by allowing truth to vary across worlds, states or situations. In Kripke-style semantics, one does not merely ask whether a proposition is true; one asks whether it is true at a specific world, and how different worlds are related [Gar24]. This is already close to the language needed later. An epistemic state can be treated like a world; moving between states can represent learning, forgetting or intervention; modal structure can record which movements are legitimate.

The contrast between **ontological** and **epistemological** logic is important for this monograph. The above logics are often framed in an ontological setting – although this is not strictly necessary, as will be seen later with modal logic. Ontological logic speaks as if it were describing what *is* or what *exists in reality*; epistemological logic speaks about *knowledge*: what is known and what can be inferred from it [Hof23; SN25]. Jaynes’ language is deeply epistemological, as it frequently refers to the personal knowledge of an agent, but his formal derivation remains close to a classical background.

Within epistemology there are a few candidates to consider. **Temporal** and **dynamic** logics are useful for reasoning about change and actions [BR16; GR25]. Both of these contain valuable components, and the later implementation can be considered a special flavour of a dynamic logic, where modalities govern transitions between epistemic states. **Intuitionism** is an epistemological logic in the constructive tradition: everything needs to be explicitly derived. It therefore does not accept a proposition because its negation cannot be proven, but it needs a positive proof of its truth. It immediately follows that the law of excluded middle is not generally accepted in this logic¹. Furthermore, real

¹Which is based on the premise that all propositions are either true or false. In the classical case, it then follows that if a proposition is explicitly not one, it must be automatically the other, e.g. if something is explicitly *not* false, then it *must* be true. This does not automatically follow in the intuitionistic case, where either value needs an explicit confirmation.

Logic	Type theory
proposition	type
proof	program / term
$A \wedge B$	$A \times B$ (pair)
$A \vee B$	$A + B$ (sum)
$A \Rightarrow B$	$A \rightarrow B$ (function)
$\top, \perp$	1 , 0 (unit, empty)
$\forall x. P(x)$	Π -type
$\exists x. P(x)$	Σ -type

Figure 4.1: The Curry–Howard correspondence: proofs are programs; propositions are types.

numbers, in the classical sense, are also not accepted, as these are, in axiomatic set theory, completed infinite objects, and not explicitly constructed; only numbers that are completely constructible are accepted [Iem24]. Especially this last point is attractive, as it aligns perfectly with Jaynes’ philosophy of explicit limits and constructions.

The philosophical stance I take here is agnostic about ontology and constructive about epistemology, therefore largely aligning with intuitionism. The explicit argument for this is laid down in Section B.2. The ensuing text is explicitly concerned with the influence of finite measurements and finite updates on the knowledge of finite agents. The manifesto in Section 2.4 also asks for an explicit and computational part; that part is type theory, introduced in the next section.

4.2 Type theory

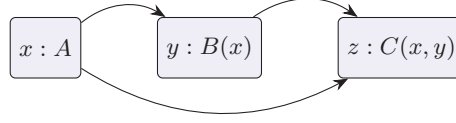
Type theory is the natural technical home for this stance. It connects mathematics to computation through the Curry–Howard correspondence: propositions can be read as types, and proofs as terms inhabiting those types, known as propositions-as-types; see Figure 4.1 and Definition A.1.28. To prove a proposition is, in this sense, to construct a proof, or, even more explicitly: compute a proof [Coq25].

A **type** may be understood, broadly, as a hypothesis or, specifically, as a domain of construction. A term $a : A$ says that a is an **inhabitant** of the type A , which means that a is a “witness” within the domain A . Within type theory, a **context**, Γ , records the assumptions and variables currently available. Importantly, these contexts are ordered: later entries in the context may depend on previous entries, but not vice versa; see Figure 4.2. What can be derived, given a specific context, is called a **judgement**. For example, a judgement of the form

$$\Gamma \vdash a : A$$

says that, in context Γ , it can be derived that the term a has type A . Furthermore, types themselves can be considered a term of even more general types, $A : B : C \dots$, creating a hierarchy, or **universe**, of types. The more formal vocabulary for types, contexts, judgements and universes is given in

4. Third movement: Adagio



order matters: later entries may depend on earlier ones

Figure 4.2: A context $\Gamma = (x : A, y : B(x), z : C(x, y))$ as an ordered telescope of dependencies. The notation $B(x)$ and $C(x, y)$, explained in the next section, anticipates dependent types.

Definitions A.1.1, A.1.2 and A.1.6. The above already brings subjectivity nearer: what can be derived depends heavily on the specific context considered.

Substitution explains how a term depending on a variable changes when that variable is replaced by a particular term. **Function types** transform inhabitants of one type into inhabitants of another. **Lambda abstractions** construct such functions, and **applications** use them. **Pair types** and **sum types** function like containers for two types at once². In the ensuing substitution and application steps, **equality** is not a purely metaphysical relation; it has computational content, for instance through **beta-reduction**. The more formal definitions and machinery are laid down in Definitions A.1.4, A.1.9, A.1.10 and A.1.12.

As this is all rather dense, a small example may help.

Example 4.2.1 (Mira’s tenure-track search, as a context). Suppose a hiring committee is considering one candidate, Mira, for a tenure-track position. Classically, $\text{Offer}(\text{Mira})$ is already true or false, even if nobody has decided. Constructively it is a type, and the committee’s conclusion is a judgement in a context. Let the committee’s file, in simplified form, be the ordered telescope

$$\Gamma_{\text{Mira}} = (p : \text{Pr}, q : \text{Qu}, c : \text{Fit}, s : \text{Short}, f : \text{Fund}),$$

where Pr denotes institutional prestige, Qu research dossier strength, Fit committee fit, Short being shortlisted and Fund funding-line availability. A shortlist witness may use the dossier together with prestige or fit; an offer witness may use the shortlist and the funding; the offer cannot be declared before the shortlist; see Figure 4.2. In this context the judgement

$$\Gamma_{\text{Mira}} \vdash \ell : \text{Offer}(\text{Mira})$$

says that, from the documents assembled so far, an offer letter can be constructed. A committee whose file does not yet contain a shortlist witness cannot form that judgement, even if every member is already privately certain that Mira deserves the post. I return to this situation as a finite causal model in Example 5.4.1: $\text{Offer}(\text{Mira})$ becomes the Boolean variable Off , and the variables are ordered so that later ones may depend only on earlier ones.

Type theory becomes epistemological in Martin-Löf-style type theory [DP24]. This type theory is a combination of intuitionism – explained in the previous

²Function types roughly correspond to implication, $\implies$, pair types roughly correspond to conjunction, $\wedge$, and sum types roughly to disjunction, $\vee$; see Definition A.1.30.

section – within the framework of type theory. The following section explains more details about this specific type theory.

4.3 Martin-Löf-style type theory

Martin-Löf-style type theory, or intuitionistic type theory, develops type theory as a constructive foundation for mathematics [DP24; Mar84; NPS90]. As with regular type theory, one begins with judgements about contexts, types, terms and equality.

Dependent types are a powerful extension: within this theory, a type may depend on a term. If A is a type and $B(x)$ is a type depending on $x : A$, then one can form dependent function and pair types. Dependent function types, Π , and dependent pair types, Σ , generalise the definitions of their ‘simpler’ versions above³. Furthermore, these functions and pairs can then themselves also be considered terms of yet more general types or functions and pairs. It is worth being explicit that this extension makes these functions and pairs extremely flexible⁴. The more formal definitions are presented in Definitions A.1.9 and A.1.10.

The law of excluded middle is not derivable, which is desirable within the intuitionistic discipline. If excluded middle is needed for a particular argument, it must be introduced as an assumption and paid for openly. A small demonstration of non-derivability is made explicit in Proposition B.3.1.

The above are generally accepted for Martin-Löf-style type theory, but there are some specific choices that can be made in the application of this theory, especially with respect to **identity types** and **inductive types**; see Definitions A.1.13 and A.1.15. The theory distinguishes equality by computation from equality as a proposition that can be proved; see Definition A.1.5 and Remark A.1.14. For example, $0 + n$ computes to n : the theory unfolds the definition of addition. By contrast: $n + 0$ does not compute to n ; one proves $\text{Id}_{\mathbb{N}}(n + 0, n)$ by induction. Inductive types allow familiar mathematical objects to be generated by constructors: natural numbers from zero and successor, finite types from explicit labels, lists from empty lists and cons, and so on. The more formal definitions can be found in Definitions A.1.16, A.1.17 and A.1.24.

The corresponding rules are recorded in Section A.1. Epistemic states are developed in the next movement as a mode theory of named states and their transitions; see Section A.2.

This concludes the philosophical and mathematical setting for the constructive part of the book. The Martin-Löf-style type theory used here is intuitionistic in its logic, epistemological in its interpretation, finite enough to avoid infinite paradoxing, and computational enough to connect with proof checking. It therefore answers Jaynes’ demand for explicit constructions: the framework tracks what can be formed, proved and computed from a given context, while leaving ontology deliberately open.

Martin-Löf-style type theory is a deliberate choice that fulfils the stated requirements. It is still worth remembering that even a careful choice is not

³Dependent function types roughly correspond to universal quantification, $\forall$, dependent pair types roughly correspond to existential quantification, $\exists$; see Figure 4.1 and Definition A.1.28.

⁴Indeed, this way of considering functions – as terms themselves – forms the basis of functional programming languages, in which functions can be passed around as terms.

4. Third movement: Adagio

foolproof. The opening quotation by Adams therefore serves as a warning: a theory is only as good as its applications. I will put this theory to use in the following movement.

CHAPTER 5

Fourth movement: Allegro maestoso

*“Scientists have calculated that the chance of anything so
patently absurd actually existing are millions to one.
But magicians have calculated that million-to-one chances crop
up nine times out of ten”*

Terry Pratchett in *Mort*

5. Fourth movement: Allegro maestoso

This movement reconstructs mathematical inference in a finite, constructive setting. Three issues have to be addressed:

1. **Subjectivity**: has to become part of the formal language rather than being a philosophical discussion on priors.
2. **Probability**: has to be reconstructed over finite, decidable objects rather than arbitrary classical propositions.
3. **Causality**: has to enter as intervention and model change, not merely as correlation.

5.1 Context manipulation in Martin-Löf-style type theory

Contexts are the quiet machinery of subjectivity. A context records what is currently available: variables, assumptions, dependencies and previously constructed terms. In ordinary prose one says “given what we know”, in type theory one writes a context. The formal definitions can be found in Definition A.1.2 and Remark A.1.3.

Several operations on contexts have immediate epistemic readings. *Extending a context by a new variable* corresponds to learning or introducing a new assumption. *Allowing a later variable to depend on an earlier one* records directed dependence. *Deleting an unused variable* corresponds to forgetting or ignoring something that nothing else depends on. If a question depends on a variable absent from the current context, however, the answer is unavailable: not in this context, not from here, not with this information. What is in scope can be used; what is out of scope cannot even be formed as a judgement. These operations give a first, austere account of subjectivity.

This also gives a first account of intervention. Intervention sets a variable and severs the dependencies that normally determine it, whereas observation leaves those dependencies intact. In a context, that means changing the dependency structure. *To hypothesise* is similar but more theatrical: one extends the current context with alternative variables, changes something there, and asks what follows under that altered arrangement.

Context manipulation gives the raw operations; modes classify the epistemic states. Ordinary type theory can already perform the operations; a mode theory marks them as movements between states of knowing, as discussed in the following section.

5.2 Modes and modalities

A mode theory records epistemic states. A mode is a named state and a modality is an admitted move from one such state to another. Several moves may be applied in succession; the composite is again a modality, and composition is associative. The definitions are in Section A.2.

With subjectivity explicitly formalised, the next step is to allow uncertainty within the epistemic states themselves, that is, uncertainty over the propositions available in a given context. This requires a new type: **probability types**. These are discussed next.

5.3 Probability types

The guiding idea is simple: a probability assignment should be an explicit object attached to a finite and, ideally, decidable domain. It should therefore be represented by a type whose inhabitants carry the required structure, as laid down in Definition A.3.12. The philosophy behind this type is phrased nicely by Pratchett in the opening quote: a probability is an assignment relative to who is counting, not a label on the world.

However, in Martin-Löf-style type theory an arbitrary proposition is not decidable, because excluded middle is not available. Probability is therefore assigned only to explicitly decidable events on finite types; when the subject matter is the proof status of a proposition, a finite inspection may tag an outcome as proved, refuted or unresolved, and those tags remain decidable even when the proposition itself is not (Definitions A.3.5 and A.3.7).

Clayton and Waddington provide the bridge from Cox-style reasoning to a finite, constructive reconstruction [CW17]. The reconstruction used here keeps four qualitative urn rules:

1. **Qualitative coherence:** equivalent descriptions receive equivalent comparisons;
2. **Count invariance:** for a fixed urn size, only the number of selected cells matters;
3. **Strict monotonicity:** selecting more cells in the same urn is strictly more plausible;
4. **Refinement invariance:** replacing every cell by equally many indistinguishable subcells does not change the represented plausibility.

These rules are formalised in Definition A.3.34. The order of use is slightly different from the original Clayton–Waddington route; see Table B.5. The route used here is summarised in Table 5.1.

5. Fourth movement: Allegro maestoso

Step	Ingredient or result	Role in the argument	Where
1	Finite resolution-status events	Replaces probability on arbitrary unresolved propositions by a decidable three-way event partition.	Proposition B.6.7 and Corollary B.6.18
2	Finite urn principles	Give the order-preserving rescaling $U_N(k) \mapsto k/N$; decidable urn complements then satisfy the finite complement rule.	Proposition B.6.9 and Corollary B.6.10
3	Constructed product urns	Cartesian products of finite cell types give multiplication for rectangular events after the ratio rescaling.	Proposition B.6.11
4	Record–urn equivalence	Shows that urn presentations and common-denominator finite rational records agree on every decidable event.	Proposition B.6.13
5	Finite event calculus	Proves unit-interval bounds, normalisation and finite additivity, with monotonicity and inclusion–exclusion as corollaries.	Remark A.3.16, Theorem 5.3.1, and Corollaries B.6.16 and B.6.17
6	Conditioning and update	Defines conditional probability, finite Bayes’ theorem and Bayesian update for decidable events with positive conditioning probability.	Definitions A.3.17 and A.3.19, Remark A.3.18, Theorem 5.3.2, Proposition B.6.20, and Corollary B.6.21

Table 5.1: Route through the finite probability reconstruction.

The two presentations in step 4 are the same finite distribution, related by the computable maps `toUrn` and `ofUrn`; see Figure 5.1.

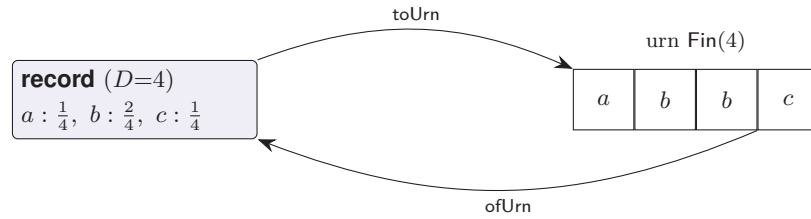


Figure 5.1: Record and urn presentations of a finite rational distribution agree on every decidable event (Proposition B.6.13); the maps `toUrn`/`ofUrn` are the computable witnesses checked in Lean.

From these rules, after the rescaling $U_N(k) \mapsto k/N$, one obtains the finite Kolmogorov laws and then Bayes’ theorem for decidable events; see

5.4. Structural causal models in Martin-Löf-style type theory

Theorems 5.3.1 and 5.3.2 below. The proofs are carried out in Section B.6.

Theorem 5.3.1 (Finite Kolmogorov laws). *Let $\mu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $E, F : X \rightarrow \text{Bool}$ be decidable events. Then:*

$$5.3.1.1. \quad 0 \leq P_{\mu}(E) \leq 1;$$

$$5.3.1.2. \quad P_{\mu}(\top_X) \sim 1, \text{ where } \top_X(x) := \text{true is the total event};$$

$$5.3.1.3. \quad \text{if } E \text{ and } F \text{ are disjoint, then}$$

$$P_{\mu}(E \cup F) \sim P_{\mu}(E) + P_{\mu}(F).$$

More generally, every finite pairwise disjoint family of decidable events is finitely additive.

Theorem 5.3.2 (Bayes' theorem for decidable finite events). *Let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $E, F : X \rightarrow \text{Bool}$ be decidable events with $P_{\mu}(E) > 0$ and $P_{\mu}(F) > 0$. With conditional probability defined by*

$$P_{\mu}(E \mid F) := \frac{P_{\mu}(E \cap F)}{P_{\mu}(F)},$$

one has

$$P_{\mu}(E \mid F) \sim \frac{P_{\mu}(F \mid E)P_{\mu}(E)}{P_{\mu}(F)}.$$

Neither theorem uses countable additivity, and neither assigns probability to an arbitrary unresolved proposition. Because the starting rules are finite and explicit, and because the final result appears in an epistemological and constructive setting, the construction captures many features desired by both Jaynes and his critics. The main victory is conceptual: probability is made into something fundamentally constructible and checkable. Representative pieces of this reconstruction are checked in Lean; see Section 5.5.

For a complete formalism, causality still needs to be explicitly introduced. In the next section, **structural causal models** will serve both as an implementation of causality and as a vehicle for epistemic states.

5.4 Structural causal models in Martin-Löf-style type theory

Pearl-style structural causal models can now be rebuilt from the same basis. A finite structural causal model consists of variables, exogenous noise, structural functions and a finite rational probability distribution over the exogenous variables [Pea09; PM18]. The formal definition can be found in Definition A.4.18. In the constructive setting, each component can be represented explicitly: variables as finite types or indexed families, structural functions as terms, exogenous uncertainty as a probability type – defined in the previous section – and interventions as transformations of the structural functions.

A structural causal model records a hypothesis about how the world is put together. That is already close to an epistemic position. What remains is to allow the belief or the model itself to change. Packaging the model in use as a named state is done in the following subsection, **epistemic modes**.

5. Fourth movement: Allegro maestoso

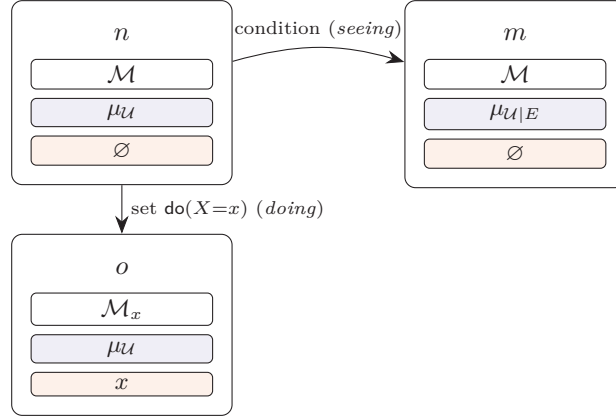


Figure 5.2: An epistemic causal mode is a name together with a record $R = (\mathcal{M}, b, x)$. Conditioning changes the belief and $\text{do}(X=x)$ changes the intervention. Conditioning is not generally invertible: recovering the previous belief would require retaining it as provenance. Modalities are these moves; see Definitions A.2.2 and A.2.3.

Epistemic modes

The modal reading gives structural causal models their epistemic interpretation. A causal model is more than an abstract graph: it represents a genuine state of knowledge, namely what variables are distinguished, what mechanisms are assumed, what interventions are meaningful, and which counterfactual questions can be asked. An epistemic causal mode packages that knowledge as a named record $R = (\mathcal{M}, b, x)$: the current structural causal model, the current latent belief and the current hard intervention; see Definition A.7.6. Modalities mark admissible transitions that may change any of these three fields; see Figure 5.2.

In mode theory, composites of modalities are again legitimate modalities; see Definition A.2.1. Causal reasoning fits this modal reading compositionally: an intervention or twin-network construction is not treated as a modal primitive. Composition therefore removes the need for a separate ‘do modality’ or ‘twin-network modality’. The point of the modal machinery introduced in Section 5.2 is precisely that smaller, causally meaningful edits can be organised into larger constructions.

Seen in this way, Pearl’s framework suggests transitions beyond ordinary causal queries. In an ordinary query, a structural causal model is usually held fixed; in scientific reasoning, however, the model itself may change: a mechanism may be discovered; a variable may be split in two; an intervention may be reclassified; a hidden confounder may be introduced; or a previous assumption may be withdrawn. Treating causal models as epistemic modes makes such changes dynamically visible as transitions between states of knowledge.

Mira’s tenure-track model

It is worth running a concrete example before presenting more general results. The natural choice is to return to Mira’s file from Example 4.2.1, now as a

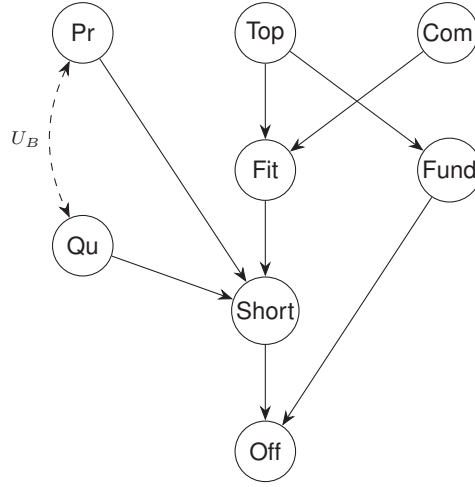


Figure 5.3: Mira's tenure-track structural causal model. The triples $\text{Top} \rightarrow \text{Fit} \leftarrow \text{Com}$, $\text{Pr} \rightarrow \text{Short} \leftarrow \text{Fit}$ and $\text{Short} \rightarrow \text{Off} \leftarrow \text{Fund}$ are colliders; the dashed edge is the latent common cause U_B inducing $\text{Pr} \leftrightarrow \text{Qu}$ (Example 5.4.1).

structural causal model. Officially, the committee claims to select on research dossier strength alone; in practice, candidates from prestigious institutions tend to do better, fashionable topics tend to attract committee members with matching expertise, and funding lines have their own seasonality. The interesting question is whether the bias visible at the output reflects a stable causal mechanism or merely a hidden common cause. The model below is small enough to inspect line by line and rich enough to exhibit the standard causal phenomena; see Figure 5.3. The type-level proposition $\text{Offer}(\text{Mira})$ will now be represented by the Boolean endogenous variable Off . Because this concrete model makes Off decidable, its events receive probabilities directly; the three-way resolution construction from Section 5.3 is needed only when the underlying proposition is not decidable.

Example 5.4.1 (Mira's tenure-track search, as a structural causal model). Returning to the committee file of Example 4.2.1, the same situation is now a stylised structural causal model. The finite probabilities below should be read as a toy model for a candidate already under serious committee consideration, not as empirical base rates for all tenure-track applications. All endogenous variables are Boolean. Let Pr denote institutional prestige, Qu research dossier strength, Top topic fashionability, Com committee expertise, Fit committee fit, Short being shortlisted, Fund funding-line availability and Off receiving the tenure-track offer; the previous proposition $\text{Offer}(\text{Mira})$ is the event $\text{Off} = 1$ on Mira's joint exogenous assignment. One finite structural causal model is given by independent Boolean exogenous variables $U_B, U_{\text{Pr}}, U_{\text{Qu}}, U_{\text{Top}}, U_{\text{Com}}, U_{\text{Fund}}$ and

5. Fourth movement: Allegro maestoso

structural equations

$$\begin{aligned} \text{Pr} &:= U_B \vee U_{\text{Pr}}, & \text{Qu} &:= U_B \vee U_{\text{Qu}}, \\ \text{Top} &:= U_{\text{Top}}, & \text{Com} &:= U_{\text{Com}}, \\ \text{Fit} &:= \text{Top} \wedge \text{Com}, & \text{Short} &:= \text{Qu} \wedge (\text{Pr} \vee \text{Fit}), \\ \text{Fund} &:= \text{Top} \vee U_{\text{Fund}}, & \text{Off} &:= \text{Short} \wedge \text{Fund}. \end{aligned}$$

The directed structure contains

$$\begin{aligned} U_B &\rightarrow \text{Pr}, & U_B &\rightarrow \text{Qu}, & \text{Top} &\rightarrow \text{Fit} \leftarrow \text{Com}, \\ \text{Pr} &\rightarrow \text{Short} \leftarrow \text{Fit}, & \text{Qu} &\rightarrow \text{Short}, & \text{Top} &\rightarrow \text{Fund}, \\ & & \text{Short} &\rightarrow \text{Off} \leftarrow \text{Fund}. \end{aligned}$$

If U_B is not observed, the observed semi-Markovian projection (in the sense of Definition A.4.23) has the bidirected edge $\text{Pr} \leftrightarrow \text{Qu}$. Thus the example contains an unobserved common cause, colliders, ordinary directed chains and the graph predicates used later for d-separation and c-components (Definitions A.4.28 and A.5.16). With the finite rational probabilities of Example A.9.1, the calculation in Example A.9.2 gives, for instance,

$$P(\text{Qu} = 1 \mid \text{Pr} = 1) = \frac{3}{4}, \quad P(\text{Qu} = 1 \mid \text{do}(\text{Pr} = 1)) = \frac{1}{2}.$$

Seeing prestige gives evidence about the hidden background variable that also affects dossier strength. Setting prestige by intervention does not change that background variable and therefore does not by itself improve the dossier.

The same finite model also supports a representative counterfactual calculation. Suppose the actual evidence is that the candidate had no prestige, had a strong dossier, and received no offer:

$$E := \{\text{Pr} = 0, \text{Qu} = 1, \text{Off} = 0\}.$$

After conditioning the exogenous distribution on E , replacing the equation for Pr by the constant value 1, and evaluating the offer variable in the intervened model, one obtains

$$P(\text{Off}_{\text{Pr}=1} = 1 \mid E) = \frac{1}{2}.$$

The calculation uses a counterfactual copy of the model: the factual and counterfactual sectors share the same exogenous background, while the intervened mechanism is replaced in the copy; see Figure 5.4. This is a genuine abduction–action–prediction calculation for the concrete finite model, worked out in Example A.9.3.

Finite adequacy

The claim is a finite correspondence: in the finite, discrete and recursive case, the ordinary ingredients of a structural causal model can be represented inside the type-theoretic setting used here. Variables become finite types; structural equations become terms; exogenous uncertainty becomes a finite rational probability type; interventions become transformations of the relevant

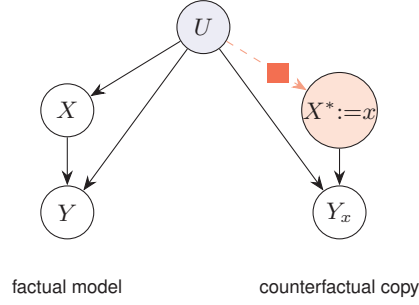


Figure 5.4: A counterfactual copy of a structural causal model. The factual and counterfactual sectors share the same exogenous background U ; the counterfactual intervention replaces the mechanism for X^* by the imposed value x , and Y_x is then evaluated in the copied model.

structural equations; and the graph tests used by do-calculus remain finite combinatorial checks.

Once this bridge is in place, the classical finite results do not need to be reinvented. The appendix proves the correspondence, records what must be supplied from the published specialisations, and checks the resulting transport; it does not reprove Pearl–Tian–Shpitser completeness or the global Markov theorem from scratch [HV06; Pea12; SP06; SP08; TS09]. The mathematical interfaces are specified in Section A.8; their Lean realisation and proof status are summarised in Section B.5.

Theorem 5.4.2 (Coded finite causal-model adequacy). *In the finite recursive source class whose observed values are explicitly coded by finite indices, the intrinsic dependent structural causal model and its finite-table presentation assign equivalent rational probabilities to every observational and hard-interventional event. Identifiability questions asked on either side are therefore the same questions.*

Proposition 5.4.3 (Finite counterfactual semantic adequacy). *The source-table and intrinsic presentations agree on the partial denotation of every represented Boolean multiworld query. Observational counterfactual identifiability and uniform support are therefore preserved and reflected between the two presentations.*

This proposition is the main-text form of Proposition B.10.8 and Corollary B.10.9. It is a semantic correspondence theorem, not a completeness theorem: completeness for the represented supported counterfactual fragment remains conditional on the explicitly supplied published interface of Theorem B.11.8.

The proof, and the more technical source–target wording, are in Section B.7. The transport argument is mostly bookkeeping, but this is precisely why it matters: probability types supply uncertainty; the mode theory supplies epistemic movement; structural causal models supply intervention and counterfactual vocabulary. Representative pieces of this correspondence are checked in Lean; see Section 5.5.

5. Fourth movement: Allegro maestoso

5.5 Lean 4 implementation

The mathematical manifesto, Section 2.4, asks for computation and explicitness. Lean 4 is used as a proof checker for the finite reconstructions of the probability types and structural causal models of Sections 5.3 and 5.4 [MU21]. Representative definitions, proofs and examples are independently checkable, and the foundational boundary of those files is recorded in Section B.5; the proof-status conventions are in Table B.2.

The development is constructive-extensional Lean: the audited declarations use Lean’s extensional infrastructure, without excluded middle or classical choice.

The flavour of the Lean code is best seen in an example, the record–urn equivalence theorem of Listing 5.1:

Listing 5.1: Record–urn extensionality (`lean/Thesis/Probability.lean`).

```
theorem urn_to_record_extensional (m : UrnProb 0) :  
  forall E : Event 0, QProb.Equiv ((ofUrn m).probVal E) (m.probVal E) :=  
    ofUrn_agrees m  
  
theorem record_to_urn_extensional (R : FiniteProbRecord 0) :  
  forall E : Event 0, QProb.Equiv (R.probVal E) (R.toUrn.probVal E) :=  
    R.toUrn_agrees
```

Read informally, the two statements say that converting an urn to a finite record, or a finite record to an urn, does not change the probability assigned to any decidable event. The proof is not deep, and that is exactly the sort of bookkeeping that prose often hides. Lean is used here to keep the finite construction honest.

The Lean implementation is best read as a test of explicitness. The theorem-facing modules check the finite probability, causal and transport constructions used in the later appendices. The module map is in Section B.5; the external theorem interfaces are in Table B.3.

5.6 Proof status

The results established in this movement have three different statuses.

First, results proved within the present finite framework include the probability reconstruction and the semantic correspondence between the intrinsic and source-table causal presentations; see Sections B.6 and B.7. Representative declarations and examples are checked in Lean 4; see Sections B.5 and 5.5.

Second, the causal soundness, completeness and latent-projection consequences are conditional transport results. The corresponding published results enter through explicitly supplied interfaces; the monograph proves what follows when those interfaces are inhabited, but does not derive their inhabitants from the cited papers. The interfaces and resulting transports are recorded in Table B.3 and Theorems B.11.4 and B.11.8.

Third, the philosophical and metatheoretic claims are supported by explicit prose arguments rather than Lean declarations; see, for example, Sections B.1 and B.2 and Proposition B.3.1. The complete proof-status conventions and foundational boundary are given in Table B.2 and Section B.5.

These distinctions determine how terms such as *proved*, *checked*, *transported* and *conditional* should be read in the following summary.

5.7 Intermediate results

Within the restricted finite setting of this monograph, this movement has constructed the following.

1. **Epistemic states.** Modes represent epistemic states, modalities record admitted transitions between them, and proof-carrying operations realise the corresponding updates; see Proposition B.9.3.
2. **Probability.** Starting from qualitative urn assumptions that are shown to be satisfiable together, the probability reconstruction derives a rational representation of urn plausibility; see Propositions B.6.6 and B.6.9. The resulting calculus proves the finite Kolmogorov laws and Bayes' theorem; see Theorems 5.3.1 and 5.3.2. Bayesian conditioning connects the modes to this probability calculus by producing an updated belief that agrees with the corresponding posterior; see Proposition B.9.5.
3. **Causality.** Recursive structural causal models represent causal structure and intervention. The source-table and intrinsic presentations agree on observational and hard-interventional probabilities and pose the same identifiability questions; see Theorem 5.4.2, Proposition B.7.8, and Corollary B.7.9. This correspondence also covers represented Boolean multiworld queries: their partial denotations agree across the two presentations, while observational counterfactual identifiability and uniform support are preserved and reflected; see Proposition 5.4.3. The two-node shared-cause example additionally identifies $P(Y \mid \text{do}(X))$ as $P(Y)$ throughout its compatible model class without using the general external completeness interfaces; see Proposition B.7.14.

Published soundness and completeness results are represented by explicitly supplied interfaces; see Table B.3. Conditional on those interfaces, the transport theorems connect identifiability to checked identifying certificates; see Theorems B.11.4 and B.11.8 and Corollary B.11.10.

A formally correct reconstruction cannot, however, rescue a scientific pipeline that selects for something other than correct inference; that question belongs to the next movement, Chapter 6.

CHAPTER 6

Fifth movement: Presto con fuoco

“For far too long economists have sought to define themselves in terms of their supposedly scientific methods. In fact, those methods rely on an immoderate use of mathematical models, which are frequently no more than an excuse for occupying the terrain and masking the vacuity of the content. Too much energy has been and still is being wasted on pure theoretical speculation without a clear specification of the economic facts one is trying to explain or the social and political problems one is trying to resolve”

Thomas Piketty in *Capital in the Twenty-First Century*

6. Fifth movement: Presto con fuoco

This movement reads Jaynes' criticism of bad scientific attitude as a criticism of a selection environment. The habits he named can emerge as adaptive responses whenever professional reward becomes too detached from intellectual responsibility.

6.1 Questionable research practices on a large scale

Jaynes' criticism of bad scientific attitude can be read as a criticism of individual character, as was seen in Section 2.3:

1. **Gamesmanship:** the desire to display technical virtuosity, rather than to clarify a problem;
2. **Cutting corners:** the focus on polished results, without explaining the painstaking work that led to these outcomes;
3. **Hidden assumptions:** the silent transfer of what should have been stated into what is merely presumed;
4. **Hyperfocus:** the habit of investigating only an incredibly specialised subfield, whilst forgetting the wider context in which this research sits;
5. **Theoretical navel-gazing:** the production of mathematics that neither explains, predicts, nor helps anyone build or test anything practical.

In his text he then continues to blame Fisher for having all these vices, while Jeffreys is presented as a model scientist. Besides the fact that this is a rather simplistic view of both quite complicated characters, the following observation by Jaynes is at least as interesting: Jaynes laments that although Jeffreys was correct in his view on probability theory¹, he only trained one Ph.D. student during his whole career². Meanwhile, Fisher, who according to Jaynes was utterly wrong on his views on probability theory, trained dozens of influential Ph.D. students [Jay03]. Although in itself completely anecdotal, this observation suggests that perhaps the mechanisms by which scientific ideas and scientific careers spread do not select primarily for 'truthfulness'.

The business language that has become central to science matters here. Discovery is naturally highly nonlinear, but is nowadays often translated into a linear framework of targets, deliverables and measurable output. This also makes the risk more precise: when funding, promotion, job security, visas and prestige all depend directly or indirectly on publication rate, publication can become the object that is optimised, rather than any 'truthfulness' as such. The phrase 'publish or perish' names a selection environment. As such, it is advisable to consider scientific inheritance, promotion and prestige as part of the machinery of inquiry as well. McElreath's reminder from the prelude, Chapter 1, is therefore central here: inference is only one component of science [McE20]. A formally correct inference engine cannot rescue a pipeline in which claims are not selected for their truthfulness, but rather for fashionability, publishability and speed.

¹Jeffreys is widely known for having resisted the theory of plate tectonics throughout his life.

²Maybe not completely coincidentally, also Jaynes only had a handful of Ph.D. students during his career, giving this observation a rather personal dimension.

6.1. Questionable research practices on a large scale

Surveys and meta-analyses of questionable research practices and misconduct already suggest that scientifically questionable behaviour is widespread, and even more so since the official record understates the problem, as (self-)reporting is naturally limited by stigma and personal risk [Fan09; JLP12; MAV05]. Retraction and paper-mill studies emphasise the same problem from another angle [Agr+25a; FSC12; Ric+25]. More recent reports on the sheer scale of industrialised fraud, fake authorship markets, pseudo-scientific journals and batch retractions make the same point even more urgently [Bri25; Nad26; NDR18; OGr25].

The scale of the consequences is difficult to explain by individual vice alone – contrary to what Jaynes posited. Two large replications make the drop visible at a glance; see Figure 6.1. The same pattern shows up across fields and measures; see Table 6.1. These findings emphasise that when published claims are put under more explicit pressure, sometimes their apparent certainty disappears. Smaldino and McElreath’s point is precisely that bad methods can spread by selection even when most individuals are not trying to corrupt the record [SM16]. Bad science is therefore not only a matter of bad people; it is also a matter of bad selection pressures.

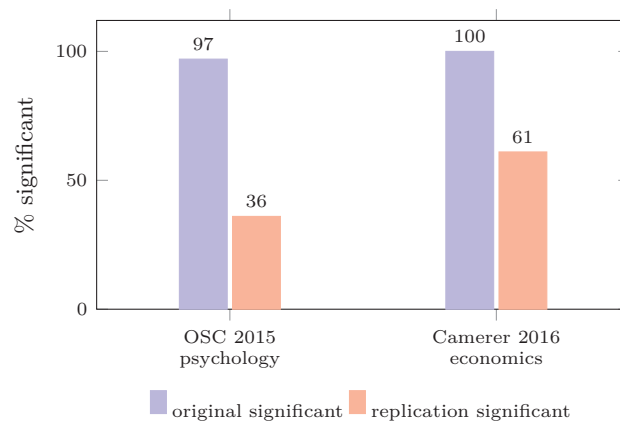


Figure 6.1: Original versus replication rates of statistically significant findings in two large projects [Cam+16; Ope15].

Explicit fraud cases are useful here, because they make the institutional pressures visible in extreme form. Stapel, Schön, Hwang, Obokata, Wakefield, the LaCour case involving Green’s coauthorship, Pandey, Raoult and Fujii are not typical in every detail, but they reveal familiar failure modes: fabricated or copied data, ethical shortcuts, failure to declare conflict of interest, salami-sliced output, hypothesising after the results are known (HARKing), reviewer manipulation, author manipulation, prestige dependence, citation games and highly inadequate correction mechanisms [BDK02; Cyr06; GSM11; Kon15; Los11; MO15; OGr24; The16; The24]. Where such incentives come from is the subject of the next sections.

6. Fifth movement: Presto con fuoco

Study	Field	What was checked	Reported signal
Open Science Collaboration [Ope15]	Psychology	100 studies from three psychology journals	97% of original studies reported significant results; 36% of replications did so, 39% were subjectively judged to replicate, and replication effects were about half the original size.
Klein et al. [Kle+14]	Social psychology	13 effects tested across 36 samples in the Many Labs project	10 effects replicated consistently, one received weak support, and two did not replicate.
Camerer et al. [Cam+16]	Economics	18 laboratory experiments from top economics journals	11 of 18 replications, or 61.1%, found a significant effect in the same direction; replicated effect sizes averaged 66% of the originals.
Begley and Ellis [BE12]	Preclinical oncology	53 landmark cancer findings checked in an industrial drug-development setting	Scientific findings were confirmed in only 6 cases, roughly 11%, well enough to continue the development programme.
Button et al. [But+13]	Neuroscience	Statistical power and its consequences for neuroscience research	The central result is not a direct replication rate: low power lowers the chance that a significant result reflects a true effect and inflates effect-size estimates.

Table 6.1: Selected reproducibility signals across fields. The table is intentionally heterogeneous: significance rates, subjective replication judgements, confirmed findings and statistical power all expose different parts of the same institutional problem.

6.2 Start of a research career

The pressure begins before a researcher has the power to resist it. A young researcher needs publications to obtain funding, funding to obtain positions, positions to obtain independence, and independence to obtain the time required for careful work. This is the catch-22 of grants and careers: one needs resources to do the work that justifies receiving resources [Ibá99; NI12]. The result favours rich institutions, already visible researchers and projects close enough to fashion that committees can recognise them; this directly results in **hyperfocus**. The inequality is institutional: if ambitious research increasingly requires expensive equipment, time, grant-writing capacity and institutional prestige, then rich entities can attempt speculative projects while poorer entities either support the existing agenda or operate with much less room for risk. As a result, a conservative funding environment has an opportunity cost: low-quality but recognisable work is generally supported while high-quality but risky work is suppressed or delayed [Agr+25b; Lan+22; PLF23].

This structure makes positive findings unusually valuable. The incentive is to produce something that can be written up as a finding. Nosek, Spies and Motyl describe the conflict directly: incentives for publishable results can pull against incentives for accurate results [NSM12]. Replication studies, negative findings and careful methodological work are all part of science, but they often carry less immediate career value than novel results. Furthermore, the penalty for an irreproducible study is also often diffuse and delayed, whereas the reward

for a publishable claim can be immediate.

The tragedy is that the system can then select against precisely the kind of slow work it later celebrates. Peter Higgs remarked that, under present productivity expectations, he would probably not have been considered productive enough to keep his post [Ait13; Al-14]. This is a careful argument against confusing accountability with throughput.

As mentioned, the pressure to publish does not need bad intent in order to create bad science. Small samples, underpowered designs, flexible stopping rules and flexible analyses allow researchers to obtain publishable results without consciously committing fraud [But+13; SNS11]. Gelman and Loken's garden of forking paths emphasises that a study can be kneaded by many unnoticed analytical choices to obtain the desired results, even when nobody is deliberately 'fishing' [GL13; GL14]. A study can be made more publishable by post-hoc hypothesising, selective reporting, choosing the positive analysis among many alternatives, cutting power because more data takes more time, or replacing careful modelling by the quickest defensible model. These are not rare choices, but ordinary research decisions that can subconsciously support a publication economy that rewards significance, clarity and speed [Ioa05; NSM12; SM16; SNS11]. **Gamesmanship, corner-cutting** and not specifying **hidden assumptions** do not have to be taught; they can emerge as adaptive responses to the environment.

Additionally, it should be noted that the aforementioned pressure can be especially sharp for researchers whose employment or residence status depends on visible output. This should be stated carefully: visa pressure does not make a researcher fraudulent, but when the right to stay in a country or the ability to support a family is linked to publication metrics, fraudulent intermediaries obtain a ready market [OGr25; Ric+25]. It can only be concluded that with the advent of more protectionist research environments in the coming years, together with intensifying competition, fraud cases and retracted studies related to visas, permits and direct livelihoods will most likely increase.

6.3 Reporting in journals

The current 'winner-takes-all' economy within science actively damages collaboration, as everyone competes for the same (limited) publishing space. When the first team to publish the positive result wins the reward, independent verification becomes a less attractive use of time. This is especially dangerous in fields where the underlying process of discovery is slow, uncertain and nonlinear. Artificial intelligence can intensify the problem by increasing the speed at which manuscripts, variations and near-duplicates can be produced [Zha26]. The journal article then turns an uncertain process into a clean product, and the market rewards the product more directly than the process that produced it, or any corrections that come after.

Journals are not merely neutral containers for such products, as they are also competing for attention, subscriptions, impact and prestige. Randy Schekman, a Nobel laureate in biology, has openly criticised prestige journals for distorting incentives [Sch13]. The justified worry in a system where journals have a conflict of interest with the truth is that surprise, novelty and controversy can be rewarded before reliability is known. As such, correction can remain

6. Fifth movement: Presto con fuoco

slow or absent even after reliability is doubted or proofs are provided of clear misconduct [Bik26a; FSC12]. In practice, investigations by journals can be pushed back onto the same institutions, laboratories and even the very same people who are under investigation [OGr24; OM23; The24]. As such, even when some of the behaviours leading to publication are formally prohibited, the consequences are often lighter, slower or less predictable than the original reward [Cam98; MAV05; Smi06].

Once publication itself becomes a currency, a market grows around that currency: salami slicing, extreme self-citation, exploitation of editorial and review roles for citation, citation brokers, ghostwritten papers, fake authorship advertisements, paper mills, predatory or pseudo-scientific journals, unusual review processes, identity manipulation and fake reviewer accounts all exploit the fact that careers, visas, rankings and institutional prestige depend on visible output [Agr+25a; Bri25; Cam98; MAV05; Nad26; NDR18; OGr25; Smi06]. Here too it remains true that **gamesmanship** and **corner-cutting** do not have to be taught as principles, but appear as responses to the environment. At this point the problem is no longer only a set of individual lapses: an entire industry has been built around the weaknesses of the current publication system.

6.4 Peer review and editorial process

Peer review is supposed to prevent many of the described behaviours, but the task assigned to it is almost impossible. Reviewers are asked to judge novelty, correctness, significance, ethics, method, relevance and sometimes fraud, usually without enough time, sufficient training, necessary data or any reward to do so [Smi06]. Furthermore, without proper training and a good definition of the process, there are many problems directly identifiable: inconsistency between reviewers; rivalry between reviewers and authors; bias, conservatism and self-interest of reviewers; and lack of clear standards for what defines a good review [Cam98].

As a result, review actively stimulates conservatism and review systems can punish risk while actually claiming to value innovation [AS14; GB24; Lan+22]. Reviewers and researchers are trained to be critical, and any paper will reveal weaknesses if pressed hard enough. Since the process is also structurally conflicted – as reviewers and authors often compete for similar publication space, funding and prestige – referees can ask for extra citations, editors can choose reviewers with predictable objections, and anonymous review can leave authors with little recourse against poor or self-interested judgement [Cam98; SR14]. In a sense, the garden of forking paths that is present for authors exists for referees and editors as well. The broader evidence that papers and patents have become less disruptive over time gives this observation a historical perspective [PLF23].

Conservatism also gives disproportionate power to senior researchers and prestigious institutes [AS14; Cam98; Lan+22]. Their names can help a paper survive review; their views can shape what is judged plausible; and their time, networks and authority make them more likely to be involved in evaluation. In extreme cases this allows for coercive citation, inappropriate authorship demands and reviewer manipulation, or even citation cartels on larger scales [Agr+25a; Ric+25]. This can slow the ordinary replacement of old ideas by

new ones considerably – a process that is natural in society and one of the cornerstones of the scientific method. A radical paper may be rejected because the shared assumptions of the review community make it hard to read fairly. **Hidden assumptions, hyperfocus and theoretical navel-gazing** can then be reproduced by peer review rather than exposed by it.

The standards for reviewing are often also surprisingly thin. Authors are usually given detailed formal obligations on how and what to publish, but reviewers and editors often receive much less training and fewer explicit standards. Proper reviewing is therefore partly learned at the expense of authors: an anonymous referee can be careless, arbitrary or self-interested while the author carries the cost [Cam98; Smi06]. As reviewers are often anonymous, often competing researchers and often exceptionally time-pressured, the mentioned consequences for not having a clear review process are not a fringe phenomenon. Additionally, the remarkable fact that reviewers are not obliged to disclose their conflicts of interest is a source of concern, especially in fields where industry and academics heavily overlap. With such a system, it should not come as a surprise if both the false-positive rate and the false-negative rate were considerable, even though both are extremely difficult to assess in practice [Ioa05; SNS11].

Finally, peer review is often expected to function as a first defence against serious offences like fraud. A reviewer working from a submitted manuscript and a tight deadline often cannot see the evidence that would expose those offences, so the guardrail is insufficient in practice. The very fact that platforms for post-publication scrutiny, such as Retraction Watch, PubPeer, Science Integrity Digest, Clear Skies, Elisabeth Bik's blog and the Black Spatula Project exist, is precisely because pre-publication review demonstrably cannot carry the whole burden [Bik26a; Bik26b; Cle26; Pub26; Ret26; The26]. As such, it can be concluded that the general expectations for the review system are unrealistically high, while the current state of affairs indicates that the results are in fact not as intended.

6.5 Career advancement

As was alluded to above, researchers are rewarded for publication counts, citation counts, grants, prestige and managerial expansion more directly than for careful reasoning. That is the same habit Piketty named, in later form: one occupies the terrain with a display of method, and the relevance of the question can wait. Salami slicing, excessive self-citation, strategic authorship, rushed review, exaggerated claims and selective reporting can then quietly turn into professional habits. In order to be successful in every successive step, mostly 'important' publications literally count, and thus quantity can quietly devour quality. In fact, when read carefully, the analysis of Martinson, Anderson and de Vries seems to imply that the occurrence of scientifically questionable behaviour increases between successive stages of a scientific career [MAV05]. If this extrapolation turns out to be correct, the system as a whole can be considered to actively support **gamesmanship, corner-cutting** and not specifying **hidden assumptions**.

For researchers the sheer growth of the scientific literature makes careful participation even harder. Keeping up with one's own field can in itself already become a full-time job, and reviewing every relevant paper with equal care

6. Fifth movement: Presto con fuoco

is downright impossible [BHM21; Lan16]; resulting in **hyperfocus**. Reports of sharply rising reviewing loads and batch retractions are therefore not side details, but a clear indication of an overstretched system [Bri25]. Artificial intelligence may help with search, drafting and checking, but it can also increase output, intensify competition and create a loop in which machine-generated manuscripts meet machine-assisted review [Zha26]. The expected result for the coming years is not more serious science, but simply more similar throughput.

By contrast, researchers at poorer institutions, or outside wealthy national systems, can be blocked from reading the literature to which they could meaningfully contribute. This directly adds a severe form of inequality to career advancement, and actively pushes such researchers towards lower-quality publication venues and fraudulent intermediaries [Ric+25; Spa18]. Furthermore, a public research system that hides publicly funded knowledge behind tolls weakens its own corrective capacity, and actively hampers potentially fruitful collaborations.

When focusing on the scientific career path itself, one sees that, rather remarkably, the role of the researcher fundamentally changes whilst advancing in successive stages. Researchers are – as the word gives away – trained to do research, but as their career progresses, they are silently turning into managers of human resources, finance, public relations, grant strategy and institutional survival. Some researchers genuinely flourish in their new role, but it is worth emphasising two things:

1. This career path is not efficient, something that might matter as often substantial public funding is involved;
2. There is often no good alternative for researchers who genuinely wish to specialise – and remain specialised – in doing research.

The mismatch between training and occupation can produce unintended or unprofessional behaviour, particularly where managerial responsibilities receive little support. Poorly supported operations can encourage **gamesmanship**, shortcutting, delegation without oversight and prestige management. If academia is to improve the quality of its output, it must also reconsider the career path of its researchers.

6.6 Reform proposals

If the diagnosis is right, funded research as it stands does not automatically inherit the name *science*. Science can be imagined in at least two noble ways: as the sincere search for truth, and as the organised attempt to solve important societal problems. A system increasingly governed by key performance indicators does not align with these noble goals [Gel14; NSM12]. That is why the current set-up fully deserves the name *funded research* instead. As it happens, some funded research is excellent science; however, most of it is merely funded research.

For funded research to fundamentally become *science* once more, several structural changes are highly recommended. The following is a starting point, not a complete programme. The focus in the following paragraphs will be about funding, peer reviewing, publication, correction and task division inside research groups.

Funding should become more pluralistic. An argument in favour of the current construction can be that it maintains a relative autonomy for researchers. Research autonomy greatly matters, but it should be emphasised that this autonomy does not move in neutral space. It is pulled by whoever has the clearest agenda and the strongest purse, which can, directly and indirectly, simply be the industry within a country [Agr+25b; SM16; Spa18]. It is therefore worthwhile, if science is to be directed towards solving important societal problems, to consider other sources of funding too:

- **Citizen-directed funding** can support local and neglected problems;
- **Democratic funding priorities**, that is, parties or democratically elected bodies that decide on funding priorities;
- **Specialist peer funding** can protect difficult work whose value is not obvious at first sight.

Such a system where funding can be obtained in more than one significant way rewards more kinds of contribution than only publishable results. Education, outreach, public problem-selection, good reviewing, replication, negative findings, careful methods, useful tools and well-curated data all create scientific and societal value. Funding and assessment should therefore be allowed to count these objects directly rather than pretending that their value only exists once it has been converted into a publication [BE12; DOR12; Mun+17; Nos+15].

Publication should be rebuilt around public access and public scrutiny. As an argument in favour of open access, it can be noted that now often considerable amounts of public spending are involved in creating publications that are publicly closed. With this argument in mind, the Council of the European Union has called for transparent, equitable and open access to scholarly publications [Cou23]. Indeed, it is justified that if public money pays for research, the public should not pay again to read it. Diamond open-access infrastructure, such as the programme described by DIAMAS, points in the right direction [DIA26]. A digital, public publication infrastructure should also make the older ‘limited space’ argument obsolete. Such a system can host well-defined, well-executed and correct work even when it is not necessarily novel.

Even more importantly, as space limitations are less stringent in the digital age, a finished paper should not be the only recognised scientific object shared. Indeed, such a platform can also host the parts of research that are usually hidden: protocols, raw data, analysis plans, code, materials, failed attempts, reviewer reports and later corrections. Public platforms such as the Open Science Framework, Wellcome Open Research and Open Research Europe show how protocols, data, methods, tools and results can all be made part of the public record [Cen26a; Eur26; Wel26]. If any specific aspect of a study, like data or a model, cannot be made fully public for ethical or proprietary reasons, the reason should itself be explicit. The repeated excuse that original data were lost should not be treated as a harmless administrative accident; in several fraud cases, missing raw data were central to the difficulty of correction [BDK02; BE12; NSM12; The16]. Registered reports, open materials, open data, better research assessment and explicit rewards for replication would make correction

6. Fifth movement: Presto con fuoco

an integral part of the ordinary scientific cycle, rather than an embarrassing afterthought [Cen26b; DOR12; Mun+17; Nat26; Nos+15; NSM12].

Naturally, the same infrastructure would be most beneficial if linking were easy. A replication should be (visibly) connected to the original study; extensions should be connected to methods and datasets they reuse; comments and post-publication discussions should remain attached to the relevant research object; and reliability signals should be able to change as evidence accumulates. Standard researcher identifiers, such as ORCID-style identifiers, would strengthen accountability by making authors, reviewers and conflicts of interest of either party harder to obscure. Templates, documentation and research-design tools could then help researchers find relevant methods, datasets and recent methodological improvements before embarking on a new study. Such an open system also changes the meaning of failure: a study that looked promising but is weakened by a later analysis, a failed replication or a better design need not disappear, but can remain in the record as a corrected step in a growing process. That is closer to how knowledge actually develops, and it lowers the burden on pre-publication review to get everything right at once. In this way, a false start does not remain there without correction, linkage or context [Nat26; NSM12].

Review should become more open, more cumulative and more clearly credited. Anonymous review may still be useful in some settings, for example in swift double-blind review that the original author can request, to quickly assess the quality of the object to be published. However, after its initial release, the system should also support signed reviews, public post-publication discussion and links between replications and originals, so that usability or credibility of scientific work can change over time. Researchers should, as such, be able to build reputations as solid reviewers and contributors. This would make review and replication itself a recognised scholarly contribution rather than invisible service work. In such a system a direct impetus would appear for institutions to offer serious training to reviewers, as good reviewers would become reputable members of the scientific community. For this to work properly, it is essential that, next to authors, also reviewers would be subjected to declare their conflicts of interest with regard to the objects they are reviewing. Such a review system would considerably lower the burden on pre-publication review, while increasing the appreciation of review as a serious scholarly contribution.

Research groups should be organised around a research-focused and an operational part. The former concerns questions, methods and intellectual direction; the latter concerns people, budgets, compliance, project support, data stewardship and institutional logistics. It would therefore be commendable if different people within a research group would be considered for these different tasks. The head of research would be responsible for the research-focused part, while the head of operations would be responsible for the operational part. With a clear separation in place, two interrelated consequences would be:

1. Researchers would be able to focus on their own research, something which they *are* properly trained for and, as such, this set-up would arguably be more efficient than the current situation;

2. As considerable public spending is involved in general research projects, it is important to ensure that the selected solution is efficient and transparent.

As a result, researchers would not have to become operations managers, something that often takes away a lot of time and attention, and for which they did *not* receive proper training. As budgets within research institutions are often tight, one could even imagine a set-up where the head of operations is shared between different groups or across an institute. Naturally, as a new role is defined, this could be a role that one specialises in during their tenure, but it should really be considered as such: a different specialisation that requires a different set of skills and knowledge.

It should be enforced by law that public research institutes make their income and expenses publicly inspectable. As public research is publicly funded, serious financial interests, contribution claims and institutional dependencies should be visible enough that failure to disclose them becomes practically and legally meaningful rather than a polite administrative courtesy. There are several fraud cases that could have been revealed much earlier if the conflict of interest were immediately disclosed, and even in these cases the conflict of interest itself was often a mere afterthought rather than the main reason for the fraud [GSM11; OGr24; OM23; The24]. It should always be clearly remembered that these institutes are *public* and, as such, hold a serious obligation towards society to generate objects that serve the general interest. Sponsorship need not automatically imply a direct mingling in the end-results; it nevertheless increases the odds for serious interference. Cases in the past where researchers accepted funding to steer the scientific debate around, e.g. the active support around harmful treatments or products, the suppression of helpful treatments, climate change causes or effects, the effects of social media on psychological well-being, or the active support for increasing financial and socioeconomic inequality, are pointers to the fact that researchers are mere human beings who can be influenced by power or money [Bol25; Gol12; OC10; PM18; Zub19]. Rather than blindly trusting institutions to self-correct when such processes take place, it is wiser and more efficient to be ahead of it.

6.7 Synthesis

The above proposals create an environment where more sources for funding will be made available to a researcher. Furthermore, the online environment in which the research would be developed would actively stimulate collaboration and sharing of a much broader set of scientific objects. In this system, rather than all becoming prolific producers of publishable papers, researchers are allowed to specialise in different roles: enthusiastic teachers, thorough reviewers, precise replicators, exceptional data-collectors, deep methodologists, gifted public speakers, creative out-of-the-box thinkers, and others. As research groups would have at least one dedicated employee focusing on operations and funding, the researcher can indeed focus on those aspects of science that the community and society need most. By ensuring that any form of interference can be readily detected, society also ensures that the interests of the researchers stay aligned with them. As a result, it can be expected that the achieved results can be faithfully judged on their current reliability or truthfulness.

6. Fifth movement: Presto con fuoco

Circling back to the habits that Jaynes mentioned as bad scientific behaviours, the new system is expected to reduce these greatly:

1. **Gamesmanship:** would lose its prize, because a single spectacular display would no longer be the main currency; linkage, review, replication and tools would count as well;
2. **Cutting corners:** would be exposed by open materials and by replications that can actually be run;
3. **Hidden assumptions:** would have to be written down, because protocols, analysis plans and data would sit in the public record;
4. **Hyperfocus:** would be harder to sustain if funding and recognition were available outside one fashionable subfield;
5. **Theoretical navel-gazing:** would be discouraged where citizen- or democratically-directed projects force the question to be named first.

By unlocking a wider view and a more society-driven approach among researchers, more society-focused results can be expected. It is likely that as a direct result the mutual trust between researchers and society will increase, to the benefit of all. Whatever one's definition of science, an activity that serves both the pursuit of truthfulness and society at large deserves to be classified as *science*. As such, the above system would rightfully deserve the name *science* once again.

Of course it cannot be expected that the above system eradicates fraud, vanity or ignorance. However, by changing what the system rewards, it makes replication, correction, negative results, careful review, public access, transparent data practices, operational competence and public problem-selection part of the normal scientific economy. It also reduces the value of a single spectacular publication by increasing the value of the surrounding process that checks, links and corrects it. That is the institutional version of the formal demand made throughout this book: if scientific bad habits are the enemy of good inference, then scientific institutions should not be stimulated to develop them.

CHAPTER 7

Postlude: Andante

“Let mathematicians and astronomers save themselves if they can with infinitely disappearing minute magnitudes, but in life itself this does not help a man to obtain his examination papers, and much less to explain spirit”

Vigilius Haufniensis in *The Concept of Anxiety*

7. Postlude: Andante

This postlude states what was shown, where the boundary sits, and what remains open.

7.1 Results

Compactly put, the aim was to represent finite Jaynes-style probabilistic updating and finite Pearl-style causal reasoning in the chosen Martin-Löf-style framework, with representative constructions checked in Lean 4, and to connect that reconstruction to the claim that good science needs both good inference and a good environment in which this inference takes place.

Within the restricted finite setting of this monograph, that reconstruction can be stated as follows.

Probability. The demand for explicit mathematical and statistical foundations stated in the first movement, Chapter 2, was refined through the consistency and reasonableness objections of the second movement, Chapter 3. The third movement, Chapter 4, supplied the constructive type-theoretic setting, and the fourth movement, Chapter 5, represented Jaynes-style probability on finite, decidable events. Four qualitative urn assumptions yield a rational representation, the finite Kolmogorov laws and Bayes' theorem; see Theorems 5.3.1 and 5.3.2. Representative constructions and proofs were checked in Lean 4.

Subjectivity. The subjectivity objection formulated in the second movement, Chapter 3, is answered in the fourth movement, Chapter 5. Modes represent epistemic states, modalities record admitted transitions between them, and Bayesian conditioning produces an updated state whose belief agrees with the corresponding posterior; see Sections 5.2 and 5.4 and Proposition B.9.5.

Causality. The limitation of probabilistic association identified in Section 3.2 is addressed in the fourth movement, Chapter 5, through finite recursive structural causal models. The intrinsic dependent and finite-table presentations agree on observational, interventional and represented Boolean multiworld queries; see Theorem 5.4.2 and Proposition 5.4.3. General completeness consequences remain conditional on the explicitly supplied published interfaces; see Section 5.6.

Scientific practice. The five scientific vices introduced in Section 2.3 are reinterpreted in the fifth movement, Chapter 6, as responses to selection pressures operating through funding, publication, review and career advancement. The proposed reforms address those pressures through pluralistic funding, open publication and review, a separation of research and operations, and financial transparency; see Section 6.7. This is an institutional interpretation and proposal, not a complete empirical sociology of science.

Taken together, these results support the claim that reliable mathematical inference requires both an explicit formal framework and an environment that rewards reliable inference.

7.2 Discussion

The five movements carried out this reconstruction while upholding high standards of mathematical rigour and academic honesty. The importance of the work could be considered with a grain of irony; see the opening quote by

Vigilius Haufniensis. However, in the context of the Cox–Jaynes programme it can be considered a necessary step. It deepens the programme, while remedying its weaknesses and honouring its philosophical stance.

In line with Jaynes’ philosophy, the work was set in an epistemological, finitary and computational setting, for which Martin-Löf-style type theory served as an ideal vehicle. However, I do not claim that this logic is the only acceptable basis for achieving this, as Brouwer–Heyting intuitionism, homotopy type theory and linear proof calculi could also have been explored [Coq25; DP24; Iem24].

Likewise, the selection of Clayton and Waddington’s rules as the basis for probability is mostly dictated by aesthetics, rather than by a conviction that this choice is the only correct one. The rules of Arnborg and Sjödin, van Horn and Snow could also have been considered [AS00; AS01; AS99; Sno98; Van03].

This monograph is not the first to consider an epistemological foundation for probability. Earlier attempts include Keynes, Ramsey, Jeffreys and Shafer, with Shafer’s work on belief functions being the most explicit [Jef61; Key21; Ram31; Sha04]. However, to the best of my knowledge, this monograph is the first to place a derivation of the finite probability calculus inside a constructive type-theoretic framework in which epistemic position is itself formally represented. Additionally, representative constructions in this framework are computationally checked from the foundational definitions onwards. That is to say, it could be interesting to explore this avenue for Shafer’s theory as well, so as to create a fair comparison between the two approaches.

The computational verification was done in Lean 4, which I do not claim to be the only acceptable proof checker. Other proof checkers, such as Agda, Rocq and Isabelle could also have been explored [DP24; MU21]. In fact, Agda sits closer to intensional Martin-Löf-style type theory than Lean 4. I selected Lean 4 mainly for its tooling and community support, accepting the constructive-extensional boundary recorded in Section B.5. Many extensional uses in the checked development have plausible intensional reformulations, but carrying them out would require restating equality-based results and has not been mechanised here. A more intensional reconstruction therefore remains possible, but it is not established by the present files.

The critical analysis of the scientific environment is based on my personal experience and the many conversations I had with researchers and professionals in the field¹. However, I do not claim that this analysis is exhaustive or definitive, not least because I am not myself part of this community as an employee. On the one hand, this limits the depth of the analysis; on the other hand, it can be argued that an outsider’s perspective provides a clearer view. This is a judgement for the reader to make.

The mathematical manifesto and the proposed reforms are my personal idealised solutions to the problems I have identified. I do not claim they are the only correct ones, nor do I claim that they are the best ones. To my surprise, there is a lack of workflow guidelines for scientific practice, which the mathematical manifesto aims to fill. I am more than happy if better ones will be put forward. Similarly, the proposed reforms are not all unique, and many have

¹Of course I have researched whether my statements could be verified and supported by numbers, but in all honesty it must be said that with the sheer amount of publications available, it is possible to always find a paper that supports a sociological claim.

7. Postlude: Andante

been proposed before. However, I did not encounter the holistic view on the environment and the sense of urgency that accompanies the current proposals in the literature. The literature often treats these problems as marginal, independent and attributable to individual researchers, with checklists and good intentions offered as sufficient remedies. I firmly disagree on each count. Like Jaynes, I experience a sense of urgency about the current state of scientific practice, and I hope that this analysis contributes to a wider discussion.

7.3 Future work

What remains open can be stated briefly. The next steps I would take are finite, and they continue the reconstructions of the fourth movement at the points I left unchecked or uncompleted.

This finite fragment could be checked in Agda, which sits closer to intensional Martin-Löf-style type theory than Lean 4 [DP24]. The aim would not be new theorems, but a second home for the same probability types, causal records and mode theory, in a checker whose syntax is nearer to the third movement. I selected Lean 4 for its tooling and community; Agda remains the more native host for the fragment used here.

The third movement set fuzzy and many-valued logics aside as accounts of degree of truth rather than degree of belief; see Section 4.1 [CFN22; Van03]. A natural continuation would be to ask how far such degree-of-truth calculi can be given the same finite, checkable treatment as the urn reconstruction, without treating a degree of truth as a degree of belief.

Shafer’s theory of belief functions is the most explicit earlier attempt at an epistemological calculus that is not Bayesian [Sha04]. A fair comparison would require representing that calculus in the same type-theoretic setting, with representative constructions checked, rather than a prose juxtaposition. My suspicion is that Shafer’s functions are a combination of ‘my’ probability functions with the ‘classical’ probability functions. However, I did not investigate this, so this remains to be seen.

On the causal side, the present work does not replace the published identification theory. More of the causal hierarchy could be represented by formalising the relevant published results. In particular, the completeness and identification arguments of Shpitser, Pearl, Tian, Huang and Valtorta [HV06; Pea12; SP06; SP08; TS09] could be encoded rather than supplied as parameters. Doing so would be a genuine extension.

As noted in the fourth movement, the current framework allows for more general causal operations than the ones presented. As the modalities themselves are rather basic, quite intricate operations, which I did not develop here, could be represented by them. This could prove a fruitful path forward for extending the mathematical machinery of causality.

The mode theory used here is a 1-category of epistemic states and admitted transitions. That is already enough to treat learning, forgetting and intervention as formal moves. It is not yet a logic of those moves: there are no formulas evaluated at a state, and no operator that reads “after this update, φ holds”. A natural continuation would be to add that apparatus, in the style of a dynamic logic [BR16]. The same states could then also be read as worlds in a Kripke model, once it is specified when a claim holds at a given state [Gar24]. A

different continuation would be to internalise modes in the type theory itself, so that they belong to the ambient syntax rather than to an algebra of records sitting beside it; multimodal dependent type theory is one such account [Gra+21; Gra23].

The book was intended as a continuation of, and homage to, Jaynes' work. I close by returning to him. To me, Jaynes' work serves as a watershed moment in my understanding of probability theory and the science of inference. I hope that the present work contributes to a broader appreciation of Jaynes' ideas.

Appendices

APPENDIX A

Definitions

“So if you are an expert and, as I said just now, have deceived me with your promise to give a demonstration about Homer, you are playing a trick on me. But if you are not an expert, but are possessed by Homer through a divine gift and, without knowledge, say many beautiful things about the poet, as I claimed is the case with you, then you are not playing a trick on me. So you need only choose how you wish us to regard you: as a man who plays tricks, or as a man governed by the gods”

Socrates in *Ion*

This appendix records the official vocabulary of the reconstruction: the judgemental fragment of Martin-Löf-style type theory, a 1-category of modes, finite probability types and finite causal models.

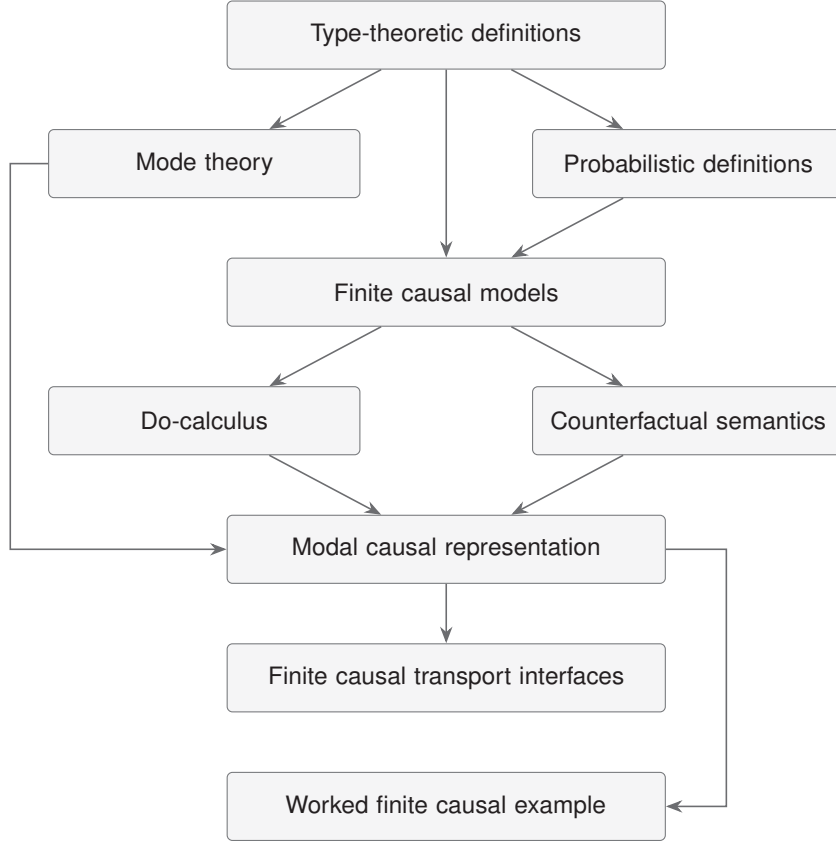


Figure A.1: Dependency map for Appendix A. Arrows show the primary conceptual dependencies rather than every technical cross-reference. The branches can be read in parallel.

A.1 Type-theoretic definitions

This section presents the judgemental syntax and rules of the Martin-Löf-style fragment used in the main text. The fragment is presented syntactically through judgemental rules. The notation follows the Martin-Löf tradition and the presentation of Nordström–Petersson–Smith [Mar84; NPS90].

Judgements, contexts and equality

Definition A.1.1 (Type-theoretic judgements). The basic judgemental forms used here are

$$\Gamma \text{ ctx}, \quad \Gamma \vdash A \text{ type}, \quad \Gamma \vdash a : A, \quad \Gamma \vdash \delta : \Delta,$$

A. Definitions

together with definitional equality judgements

$$\Gamma \vdash A \equiv B \text{ type}, \quad \Gamma \vdash a \equiv b : A, \quad \Gamma \vdash \delta \equiv \epsilon : \Delta.$$

The judgements in the first line are read as follows: Γ is a well-formed context; A is a type in context Γ , that is, a domain of possible constructions available in that context; a is a term of type A in context Γ ; and δ is a substitution from Γ to Δ . Under the later reading of propositions as types, such a term a is also evidence for the proposition represented by A .

The judgements in the second line state definitional equality: A and B are definitionally equal types in context Γ ; a and b are definitionally equal terms of type A in context Γ ; and δ and ϵ are definitionally equal substitutions from Γ to Δ . Here $\equiv$ denotes judgemental equality by computation or unfolding, not a term of an identity type. The symbol Γ is always a context; δ and ϵ are substitutions.

Definition A.1.2 (Context). A context is a finite list of typed variable declarations

$$\Gamma := x_1 : A_1, x_2 : A_2(x_1), \dots, x_n : A_n(x_1, \dots, x_{n-1}),$$

in which each type A_{i+1} may depend on the earlier variables $x_1, \dots, x_i$. Such a dependently-typed sequence is called a *telescope* in the de Bruijn–Jacobs tradition. Contexts are generated from the empty context by context extension:

$$\frac{\Gamma \text{ ctx} \quad \Gamma \vdash A \text{ type}}{\Gamma, x : A \text{ ctx}}.$$

Later declarations may depend on earlier declarations; earlier declarations may not depend on later ones.

Remark A.1.3 (Structural use of contexts). In the fragment used here, weakening, exchange and forgetting are admissible only when the dependency order permits them. Thus a fresh declaration $x : A$ may be added to a context when $\Gamma \vdash A \text{ type}$; two declarations may be exchanged only if neither dependency is broken; and a declaration $x : A$ may be forgotten from $\Gamma, x : A, \Delta$ only when neither Δ nor the target judgement contains a free occurrence of x .

Definition A.1.4 (Substitution). A context written $\Gamma, x : A, \Delta$ is one concatenated context with three consecutive parts:

$$\underbrace{\Gamma}_{\text{initial context}}, \quad \underbrace{x : A}_{\text{declaration to be replaced}}, \quad \underbrace{\Delta}_{\text{later context suffix}}$$

The prefix Γ contains the declarations available before x is introduced. The suffix Δ is a possibly dependent sequence of later declarations: its types may refer to variables from Γ , to x , and to preceding declarations within Δ .

Suppose that, using only the declarations in the initial context Γ , one can construct a term

$$\Gamma \vdash a : A.$$

Suppose also that an arbitrary judgement $\mathcal{J}$ is derivable in the complete concatenated context:

$$\Gamma, x : A, \Delta \vdash \mathcal{J}.$$

Substitution replaces x by the already available term a :

$$\frac{\Gamma \vdash a : A \quad \Gamma, x : A, \Delta \vdash \mathcal{J}}{\Gamma, \Delta[a/x] \vdash \mathcal{J}[a/x]}.$$

The term a is therefore derived from the part of the context preceding x ; it cannot depend on x or on any declaration in Δ . Every free occurrence of x is replaced by a , both in the later declarations Δ and in the judgement $\mathcal{J}$. The declaration $x : A$ disappears because its role has been taken over by the already constructed term a . No declaration $a : A$ is added: a is a term available under Γ , not a new variable.

For example, let $\Gamma = (z : A)$, let $B(x)$ be a type in context $z : A, x : A$, and consider

$$z : A, x : A, y : B(x) \vdash y : B(x).$$

Since

$$z : A \vdash z : A,$$

substituting z for x transforms this judgement into

$$z : A, y : B(z) \vdash y : B(z).$$

The notation above presents substitution metatheoretically as the operation $[a/x]$ on expressions.

In an explicit-substitution presentation, substitutions are instead included as a separate class of syntactic objects, with judgements

$$\Gamma \vdash \delta : \Delta.$$

Here Γ and Δ are complete contexts. A substitution δ supplies, using terms available in Γ , a term for every variable declared by Δ , respecting the dependencies among those declarations. It consequently sends a judgement over Δ ,

$$\Delta \vdash b : B,$$

to a judgement over Γ :

$$\Gamma \vdash b[\delta] : B[\delta].$$

For example, if

$$\Delta = (x : A, y : B(x)),$$

then a substitution $\Gamma \vdash \delta : \Delta$ may be written $\delta = (a, b)$, where

$$\Gamma \vdash a : A, \quad \Gamma \vdash b : B(a).$$

Applying δ replaces x by a and y by b simultaneously.

Explicit substitutions also have identity and composition operations, together with rules describing their effect on contexts, types and terms. This monograph uses only their ordinary action on types and terms and does not present the full explicit-substitution calculus. In the modal fragment introduced later, substitutions are restricted to one mode at a time.

A. Definitions

Definition A.1.5 (Definitional equality). Definitional equality, also called *judgemental equality*, is written $\equiv$. Its principal forms are

$$\Gamma \vdash A \equiv B \text{ type}, \quad \Gamma \vdash a \equiv b : A, \quad \Gamma \vdash \delta \equiv \epsilon : \Delta.$$

These judgements say that the type theory itself treats the displayed types, terms or substitutions as interchangeable. They do not require an additional term witnessing the equality.

Definitional equality is generated by the computation and unfolding rules of the theory, together with equations declared as judgemental computation rules. It is closed under reflexivity, symmetry and transitivity, and is preserved by substitution and by the type and term constructors. A judgemental equation is fixed in advance as part of the theory's computation rules; it cannot be introduced as an arbitrary assumption.

For example, if c is introduced as an abbreviation for t , unfolding the definition of c yields t , and hence the theory judges $c \equiv t$. Further computation equations are supplied with the individual type formers to which they belong.

Definitional equality supports conversion. If

$$\Gamma \vdash a : A \quad \text{and} \quad \Gamma \vdash A \equiv B \text{ type},$$

then the same term may be used at the definitionally equal type:

$$\Gamma \vdash a : B.$$

Definitional equality therefore does not require expressions to be written identically: expressions are also definitionally equal when the theory's computation or unfolding rules identify them.

Definitional equality is part of the judgemental structure of the type theory, not a proposition represented by a type. Equality represented within the theory is introduced separately as a type former below.

Universes and rule schemas

Definition A.1.6 (Universe). A universe $\mathcal{U}_i$ is a type whose terms are codes for the *small types* at level i . Here, *small* is a classification of type-theoretic size; it does not mean that the type has finitely many inhabitants.

The correspondence between codes and small types is given by two operations. First, a term of the universe can be decoded into the type that it represents:

$$\Gamma \vdash M : \mathcal{U}_i \quad \Rightarrow \quad \Gamma \vdash \text{El}(M) \text{ type}_i.$$

Thus M is a term serving as a code, whereas $\text{El}(M)$ is the corresponding type.

Conversely, a small type can be encoded as a term of the universe:

$$\Gamma \vdash A \text{ type}_i \quad \Rightarrow \quad \Gamma \vdash \text{Code}(A) : \mathcal{U}_i.$$

The two operations are inverse by definitional equality:

$$\text{El}(\text{Code}(A)) \equiv A, \quad \text{Code}(\text{El}(M)) \equiv M.$$

Consequently, encoding a small type and then decoding its code computes back to the original type, while decoding and then re-encoding a universe term

computes back to the original code. These are judgemental computation rules and therefore require no inhabitant of an identity type.

The restriction to small types is essential: allowing the universe to contain codes for absolutely every type, including itself, would admit Girard's paradox and thereby undermine the consistency of the theory [Coq25]. The universe itself is therefore kept outside the class of small types that it codes, or assigned a higher size in a stratified presentation. The index i records the chosen universe level. No principle of cumulativity between different universe levels is required in this monograph.

This is a Coquand-style universe presentation: types and terms of the universe are connected by the explicit operations `Code` and `El` rather than being identified silently.

Definition A.1.7 (Formation, introduction, elimination and computation). The rules for a type former are organised around four questions:

A.1.7.1. *Formation*: under which assumptions is the type well formed?

A.1.7.2. *Introduction*: how can an inhabitant of the type be constructed?

A.1.7.3. *Elimination*: how can an inhabitant of the type be used?

A.1.7.4. *Computation*: how does an elimination applied to an introduced inhabitant simplify by definitional equality?

The definitions below instantiate these four questions separately for each type former.

Remark A.1.8 (Metalinguistic status of the rule schemas). The four-part organisation in Definition A.1.7 belongs to the metalanguage used to present the type theory. It is a recurring template for describing the concrete rules of the individual type formers, not an additional type or collection of rule objects internal to Martin-Löf-style type theory. Proofs in this monograph may therefore apply the particular formation, introduction, elimination and computation rules stated for a type former, but they do not quantify over an internal type of all such rules.

Fundamental type formers

Definition A.1.9 (Dependent function type). A dependent type over A in context Γ is a family specified by a judgement $\Gamma, x : A \vdash B(x)$ **type**; it assigns a type $B(a)$ to each term $\Gamma \vdash a : A$ by substitution.

Formation. Given $\Gamma \vdash A$ **type** and $\Gamma, x : A \vdash B(x)$ **type**, the dependent function type is formed by

$$\Gamma \vdash \Pi_{x:A} B(x) \text{ type.}$$

Introduction. A term $b(x) : B(x)$ depending on $x : A$ determines a dependent function:

$$\frac{\Gamma, x : A \vdash b(x) : B(x)}{\Gamma \vdash \lambda x. b(x) : \Pi_{x:A} B(x)}.$$

Elimination. A dependent function is used by applying it to an argument:

$$\frac{\Gamma \vdash f : \Pi_{x:A} B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a)}.$$

A. Definitions

Computation. Applying an introduced function reduces by beta-reduction:

$$(\lambda x. b(x))(a) \equiv b(a).$$

Here, beta-reduction is the conventional name for the lambda-calculus computation that replaces the occurrences of the parameter x in the body b by the argument a ; in Martin-Löf-style type theory, it is stipulated as the judgemental computation rule connecting lambda abstraction and application.

When B does not depend on x , this is the ordinary non-dependent function type $\Gamma \vdash A \rightarrow B$ type, with the same introduction, elimination and computation rules and the usual beta-reduction $(\lambda x. b)(a) \equiv b[a/x]$.

Definition A.1.10 (Dependent pair type). A dependent pair packages a term $a : A$ together with a term $b : B(a)$, whose type may depend on the first component. Its rules are as follows:

Formation. If $\Gamma \vdash A$ type and $\Gamma, x : A \vdash B(x)$ type, then

$$\Gamma \vdash \Sigma_{x:A} B(x) \text{ type.}$$

Introduction. Its canonical inhabitants are pairs:

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B(a)}{\Gamma \vdash (a, b) : \Sigma_{x:A} B(x)}.$$

Elimination. Suppose

$$\Gamma, z : \Sigma_{x:A} B(x) \vdash C(z) \text{ type}$$

and

$$\Gamma, x : A, y : B(x) \vdash d(x, y) : C((x, y)).$$

For $\Gamma \vdash p : \Sigma_{x:A} B(x)$, dependent pair elimination gives

$$\Gamma \vdash \text{split}_C(d, p) : C(p).$$

In particular, the projections produce $\text{pr}_1(p) : A$ and $\text{pr}_2(p) : B(\text{pr}_1(p))$.

Computation. Elimination of an introduced pair reduces to its defining case:

$$\text{split}_C(d, (a, b)) \equiv d(a, b).$$

The projections consequently satisfy

$$\text{pr}_1(a, b) \equiv a, \quad \text{pr}_2(a, b) \equiv b.$$

When B does not depend on x , this is the ordinary non-dependent product type $\Gamma \vdash A \times B$ type, with pair inhabitants (a, b) and the same projections.

Remark A.1.11 (Finite records and named projections). A finite record type is an iterated dependent-pair type with named fields. For example, suppose a record type **Record** has fields

$$a : A, \quad b : B(a), \quad c : C(a, b).$$

If $r : \text{Record}$, its named field projections are

$$a(r) : A, \quad b(r) : B(a(r)), \quad c(r) : C(a(r), b(r)).$$

These are the corresponding dependent-pair projections with readable names; they add no new type former.

Definition A.1.12 (Sum type). A sum type records a choice between a term of A and a term of B , together with which of the two alternatives was chosen. Its rules are as follows:

Formation. If $\Gamma \vdash A$ type and $\Gamma \vdash B$ type, then

$$\Gamma \vdash A + B \text{ type.}$$

Introduction. The two injections construct inhabitants:

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl}(a) : A + B} , \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr}(b) : A + B}.$$

Elimination. Suppose

$$\Gamma, z : A + B \vdash C(z) \text{ type,}$$

together with a left case $\Gamma, x : A \vdash \ell(x) : C(\text{inl}(x))$ and a right case $\Gamma, y : B \vdash r(y) : C(\text{inr}(y))$. For $\Gamma \vdash s : A + B$, case analysis gives

$$\Gamma \vdash \text{case}_C(\ell, r, s) : C(s).$$

Computation. Case analysis reduces according to the injection used:

$$\text{case}_C(\ell, r, \text{inl}(a)) \equiv \ell(a), \quad \text{case}_C(\ell, r, \text{inr}(b)) \equiv r(b).$$

Under the later reading of propositions as types, $A + B$ represents logical disjunction. Both A and B may be inhabited, although each canonical inhabitant of $A + B$ is introduced through either inl or inr .

Definition A.1.13 (Identity type). The identity type $\text{Id}_A(a, b)$ represents equality between two terms $a, b : A$; its inhabitants are proofs of that equality. Its rules are as follows:

Formation. If $\Gamma \vdash A$ type and $\Gamma \vdash a, b : A$, then

$$\Gamma \vdash \text{Id}_A(a, b) \text{ type.}$$

Introduction. Its canonical introduction is reflexivity:

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{refl}_a : \text{Id}_A(a, a)}.$$

Elimination. Its eliminator is identity induction, or J . If

$$\Gamma, x : A, y : A, p : \text{Id}_A(x, y) \vdash C(x, y, p) \text{ type}$$

and

$$\Gamma, x : A \vdash d(x) : C(x, x, \text{refl}_x),$$

then

$$\Gamma, a, b : A, p : \text{Id}_A(a, b) \vdash J_C(d, a, b, p) : C(a, b, p),$$

Computation. Identity elimination reduces on reflexivity:

$$J_C(d, a, a, \text{refl}_a) \equiv d(a).$$

Thus it suffices to prove a property of equality proofs in the reflexivity case.

A. Definitions

Remark A.1.14 (Definitional equality and identity). Definitional equality and identity are related, but they are not the same judgement. Suppose

$$\Gamma \vdash a \equiv b : A.$$

Reflexivity gives

$$\Gamma \vdash \text{refl}_a : \text{Id}_A(a, a).$$

Because $a \equiv b$, the types $\text{Id}_A(a, a)$ and $\text{Id}_A(a, b)$ are definitionally equal. Conversion therefore allows the same term to be used as

$$\Gamma \vdash \text{refl}_a : \text{Id}_A(a, b).$$

Thus definitional equality always gives an inhabitant of the corresponding identity type.

The converse does not hold in general. In the intensional Martin-Löf-style type theory used in this monograph, a term

$$\Gamma \vdash p : \text{Id}_A(a, b)$$

does not necessarily imply

$$\Gamma \vdash a \equiv b : A.$$

Adding such an implication would amount to the equality-reflection rule

$$\frac{\Gamma \vdash p : \text{Id}_A(a, b)}{\Gamma \vdash a \equiv b : A},$$

which belongs to an extensional presentation and is not assumed in this monograph.

Definitional equality is established by the computation and unfolding rules of the theory. An identity term is instead an object within the theory and may be used through the identity eliminator J . Consequently, two terms may be propositionally identical even when computation does not make them definitionally equal.

In the remainder of Appendix A, an ordinary equation $a = b$ used as proof-carrying data abbreviates the identity type $\text{Id}_A(a, b)$, whereas $a \equiv b$ continues to denote judgemental equality. Once introduced, the relation $\sim$ is reserved for equivalence of rational presentations and its extensions.

Inductive and finite types

Definition A.1.15 (Inductive type and indexed family). The following definitions require types whose terms are generated by explicit constructors and whose inhabitants can be processed by structural recursion or induction. Rather than repeat this common pattern separately for each example, the present definition introduces inductive types and indexed inductive families in general. Concrete instances are worked out immediately below: natural numbers represent counts and sizes; finite index types provide the canonical positions used later for urn cells and for indexing finite families of variables; and lists represent the finite sequences used later in finite-support probability records.

An inductive definition specifies a type by giving canonical ways to construct its terms and a rule for defining or proving something about every term so

constructed. Constructors may use previously constructed terms recursively. The eliminator expresses that it is sufficient to handle the constructor cases, and the computation rules state how the resulting definition evaluates on each constructor.

An *ordinary inductive type* I is a single type. Given an already available indexing type

$$\Gamma \vdash K \text{ type,}$$

an *indexed inductive family*

$$\Gamma, i : K \vdash I(i) \text{ type}$$

assigns a type to each index $i : K$. A constructor of an indexed family may determine the particular index at which its result lies. An index is therefore not necessarily a natural number: it is a term of the indexing type K specified by the definition.

Formation. The formation rule declares when the type or family is well formed:

$$\Gamma \vdash I \text{ type} \quad \text{or} \quad \Gamma, i : K \vdash I(i) \text{ type.}$$

Parameters shared by the entire definition are declarations already present in the surrounding context Γ ; they remain fixed while the indices and inhabitants vary.

Introduction. The constructors give the canonical ways to build terms. A constructor of an ordinary inductive type has result type I ; a constructor of an indexed family has result type $I(i)$, where the result index $i : K$ is determined by that constructor and its arguments. Constructor arguments may themselves contain previously constructed terms of the type or family being defined.

Recursive occurrences are permitted only in *strictly positive* positions. A direct recursive constructor argument $r : I$ is permitted. A constructor argument that is itself a function consuming I , as in a hypothetical constructor $\text{bad} : (I \rightarrow B) \rightarrow I$, is not. This restriction ensures that the constructors support well-founded recursion and induction rather than unrestricted self-reference.

Elimination. For an ordinary inductive type, first choose a family

$$\Gamma, x : I \vdash P(x) \text{ type.}$$

To define a dependent function

$$\Gamma \vdash f : \Pi_{x:I} P(x),$$

it is sufficient to give one term of P for each constructor case. Whenever a constructor has a recursive argument, its case also receives the corresponding induction hypothesis.

For an indexed family, the property may depend on both the index and the inhabitant:

$$\Gamma, i : K, x : I(i) \vdash P(i, x) \text{ type.}$$

Each constructor case must establish P at the index selected by that constructor, again receiving induction hypotheses for its recursive arguments.

Computation. Applying the eliminator to a constructor reduces definitionally to the case supplied for that constructor. In a recursive case, the induction hypotheses compute by applying the same eliminator to the recursive arguments. Thus the eliminator computes constructor by constructor.

A. Definitions

Definition A.1.16 (Natural-number type). The natural-number type is the ordinary inductive type generated by zero and successor. It supplies the counts and sizes used by the finite types and probability constructions below. Its rules are as follows:

Formation. In every context Γ ,

$$\Gamma \vdash \mathbb{N} \text{ type.}$$

Introduction. Zero is a natural number, and the successor of a natural number is again a natural number:

$$\Gamma \vdash 0 : \mathbb{N}, \quad \frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{succ}(n) : \mathbb{N}}.$$

Elimination. Given a family

$$\Gamma, n : \mathbb{N} \vdash P(n) \text{ type,}$$

a zero case

$$\Gamma \vdash p_0 : P(0),$$

and a successor case

$$\Gamma, n : \mathbb{N}, q : P(n) \vdash p_s(n, q) : P(\text{succ}(n)),$$

natural-number induction gives

$$\Gamma, n : \mathbb{N} \vdash \text{natElim}_P(p_0, p_s, n) : P(n).$$

Computation. Natural-number induction reduces on zero and successor:

$$\text{natElim}_P(p_0, p_s, 0) \equiv p_0,$$

and

$$\text{natElim}_P(p_0, p_s, \text{succ}(n)) \equiv p_s(n, \text{natElim}_P(p_0, p_s, n)).$$

The numerals $1, 2, \dots$ abbreviate iterated successors of 0. Addition, multiplication and the other primitive-recursive operations on natural numbers are defined using this eliminator.

Definition A.1.17 (Finite index type). The family Fin is indexed by the natural-number type of Definition A.1.16. It assigns to each natural number n a finite type whose canonical inhabitants are the n positions $0, \dots, n - 1$. Its rules are as follows:

Formation. A natural number indexes a finite type:

$$\frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{Fin}(n) \text{ type}}.$$

Introduction. The constructors are

$$\frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{fzero}_n : \text{Fin}(\text{succ}(n))}, \quad \frac{\Gamma \vdash i : \text{Fin}(n)}{\Gamma \vdash \text{fsucc}(i) : \text{Fin}(\text{succ}(n))}.$$

For a numeral n , their iterated canonical forms are conventionally written $0, \dots, n - 1$; in particular, $\text{Fin}(0)$ has no canonical inhabitants.

Elimination. Given a family

$$\Gamma, n : \mathbb{N}, i : \text{Fin}(n) \vdash P(n, i) \text{ type,}$$

a zero case

$$\Gamma, n : \mathbb{N} \vdash p_0(n) : P(\text{succ}(n), \text{fzero}_n),$$

and a successor case

$$\Gamma, n : \mathbb{N}, i : \text{Fin}(n), q : P(n, i) \vdash p_s(n, i, q) : P(\text{succ}(n), \text{fsucc}(i)),$$

finite induction gives

$$\Gamma, n : \mathbb{N}, i : \text{Fin}(n) \vdash \text{finElim}_P(p_0, p_s, n, i) : P(n, i).$$

Computation. Finite induction reduces according to the constructor:

$$\text{finElim}_P(p_0, p_s, \text{succ}(n), \text{fzero}_n) \equiv p_0(n),$$

and

$$\text{finElim}_P(p_0, p_s, \text{succ}(n), \text{fsucc}(i)) \equiv p_s(n, i, \text{finElim}_P(p_0, p_s, n, i)).$$

Remark A.1.18 (Canonical rank and order on finite indices). The natural numbers carry the strict-order family $m <_{\mathbb{N}} n$, generated by

$$\text{zeroLess}_n : 0 <_{\mathbb{N}} \text{succ}(n)$$

and

$$\text{succLess}_{m,n} : m <_{\mathbb{N}} n \rightarrow \text{succ}(m) <_{\mathbb{N}} \text{succ}(n),$$

with no other constructors. Finite elimination defines the canonical rank map

$$\text{rank}_n : \text{Fin}(n) \rightarrow \mathbb{N}$$

by the computation rules

$$\begin{aligned} \text{rank}_{\text{succ}(n)}(\text{fzero}_n) &\equiv 0, \\ \text{rank}_{\text{succ}(n)}(\text{fsucc}(i)) &\equiv \text{succ}(\text{rank}_n(i)). \end{aligned}$$

Consequently, the canonical strict order on a finite index type is

$$i <_{\text{Fin}(n)} j \quad := \quad \text{rank}_n(i) <_{\mathbb{N}} \text{rank}_n(j).$$

Lean implementation. In Lean's structure-based presentation of **Fin**, the projection `i.val` plays the role of this rank map.

Definition A.1.19 (Finite type). Let A be a small type. A *finiteness witness* is explicit data demonstrating that A is finite. It consists of a natural number

$$n : \mathbb{N},$$

two maps

$$\text{encode} : A \rightarrow \text{Fin}(n), \quad \text{decode} : \text{Fin}(n) \rightarrow A,$$

A. Definitions

and the round-trip identity proofs

$$\eta : \Pi_{a:A} \text{Id}_A(\text{decode}(\text{encode}(a)), a)$$

and

$$\varepsilon : \Pi_{i:\text{Fin}(n)} \text{Id}_{\text{Fin}(n)}(\text{encode}(\text{decode}(i)), i).$$

The first identity says that encoding and then decoding a term of A returns the original term. The second says that decoding and then encoding a finite index returns the original index. A type A is called finite when one has specified this natural number, these two maps and these two round-trip proofs. Together, they match the inhabitants of A exactly with the n canonical positions of $\text{Fin}(n)$.

The type of these finiteness witnesses is written $\text{Finite}(A)$. Thus a term $h : \text{Finite}(A)$ carries the natural number, the two maps and both round-trip proofs displayed above.

Definition A.1.20 (Boolean type). The Boolean type is the two-valued inductive type whose canonical inhabitants are **true** and **false**. Its rules are as follows:

Formation. In every context Γ ,

$$\Gamma \vdash \text{Bool type}.$$

Introduction. It has two constructors:

$$\Gamma \vdash \text{true} : \text{Bool}, \quad \Gamma \vdash \text{false} : \text{Bool}.$$

Elimination. Given a family $\Gamma, b : \text{Bool} \vdash P(b)$ type and terms $\Gamma \vdash p_{\text{true}} : P(\text{true})$ and $\Gamma \vdash p_{\text{false}} : P(\text{false})$, Boolean case analysis defines

$$\Gamma, b : \text{Bool} \vdash \text{boolElim}_P(p_{\text{true}}, p_{\text{false}}, b) : P(b).$$

Computation. Boolean elimination reduces according to the constructor:

$$\text{boolElim}_P(p_{\text{true}}, p_{\text{false}}, \text{true}) \equiv p_{\text{true}},$$

and

$$\text{boolElim}_P(p_{\text{true}}, p_{\text{false}}, \text{false}) \equiv p_{\text{false}}.$$

Boolean conjunction is the derived operation

$$\text{and}_{\text{Bool}} : \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool},$$

defined by case analysis on its first input:

$$\text{and}_{\text{Bool}}(b, c) := \text{boolElim}_{\lambda_.\text{Bool}}(c, \text{false}, b).$$

Consequently,

$$\text{and}_{\text{Bool}}(\text{true}, c) \equiv c, \quad \text{and}_{\text{Bool}}(\text{false}, c) \equiv \text{false};$$

it returns **true** exactly when both inputs are **true**.

Boolean negation and disjunction are derived in the same way:

$$\text{not}_{\text{Bool}}(b) := \text{boolElim}_{\lambda_.\text{Bool}}(\text{false}, \text{true}, b),$$

$$\text{or}_{\text{Bool}}(b, c) := \text{boolElim}_{\lambda_.\text{Bool}}(\text{true}, c, b).$$

Thus not_{Bool} exchanges **true** and **false**, while $\text{or}_{\text{Bool}}(b, c)$ returns **true** exactly when at least one of b and c is **true**.

Remark A.1.21 (Examples of finite types). The finite index types and the Boolean type give two basic examples of finite types in the sense of Definition A.1.19:

- Every finite index type $\text{Fin}(n)$ from Definition A.1.17 is trivially finite: take both maps in Definition A.1.19 to be the identity and both round-trip proofs to be reflexivity.
- The Boolean type of Definition A.1.20 is finite with $n = 2$: encode **false** as 0 and **true** as 1, and decode those two indices back to the corresponding Boolean terms. The two round-trip identities follow by case analysis.

Remark A.1.22 (Closure of finite types). Let A and B be finite types, and let $C : A \rightarrow \mathcal{U}$ be a type family for which every $C(a)$ is finite. Then:

- the sum type $A + B$ of Definition A.1.12 is finite;
- the dependent pair type

$$\Sigma_{a:A} C(a)$$

of Definition A.1.10 is finite. In particular, the ordinary product $A \times B$, obtained by taking the constant family $C(a) := B$, is finite;

- if the inhabitants of A are enumerated as $a_0, \dots, a_{n-1}$, then the finite telescope

$$C(a_0) \times \dots \times C(a_{n-1})$$

is finite. This finite telescope is the foundational presentation used here. For readability, the same coordinate data are written pointwise as the dependent function type

$$\Pi_{a:A} C(a)$$

of Definition A.1.9. This pointwise notation does not add function extensionality to the intensional type-theoretic core.

The required encodings, decodings and round-trip proofs are constructed in Proposition B.6.1.

Lean implementation. The constructive-extensional development uses the displayed dependent function type for these finite coordinate families.

Definition A.1.23 (Optional type). The optional type adds one distinguished case representing the absence of a supplied value.

Type and constructors. For any type A , define:

$$\text{Option}(A) := \text{Fin}(1) + A.$$

Its two constructor forms are:

$$\text{none} := \text{inl}(0), \quad \text{some}(a) := \text{inr}(a).$$

Thus an inhabitant represents either the absence of a value of A or one supplied value $a : A$.

A. Definitions

Definition A.1.24 (List type). The list type is an ordinary inductive type whose terms are finite sequences. It is the list construction used later for finite-support probability records. Its rules are as follows:

Formation. If A is a type, then finite lists over A form a type:

$$\frac{\Gamma \vdash A \text{ type}}{\Gamma \vdash \text{List}(A) \text{ type}}.$$

Introduction. The empty list and the operation that places an element in front of an existing list are the constructors:

$$\frac{\Gamma \vdash A \text{ type}}{\Gamma \vdash \text{nil} : \text{List}(A)}, \quad \frac{\Gamma \vdash a : A \quad \Gamma \vdash xs : \text{List}(A)}{\Gamma \vdash \text{cons}(a, xs) : \text{List}(A)}.$$

The bracket notation used later abbreviates iterated application of these constructors:

$$[a_1, \dots, a_n] := \text{cons}(a_1, \text{cons}(a_2, \dots, \text{cons}(a_n, \text{nil}) \dots)), \quad [] := \text{nil}.$$

Elimination. Given a family

$$\Gamma, xs : \text{List}(A) \vdash P(xs) \text{ type},$$

an empty-list case

$$\Gamma \vdash p_{\text{nil}} : P(\text{nil}),$$

and a cons case

$$\Gamma, a : A, xs : \text{List}(A), q : P(xs) \vdash p_{\text{cons}}(a, xs, q) : P(\text{cons}(a, xs)),$$

list induction gives

$$\Gamma, xs : \text{List}(A) \vdash \text{listElim}_P(p_{\text{nil}}, p_{\text{cons}}, xs) : P(xs).$$

Computation. List induction reduces according to the constructor:

$$\text{listElim}_P(p_{\text{nil}}, p_{\text{cons}}, \text{nil}) \equiv p_{\text{nil}},$$

and

$$\text{listElim}_P(p_{\text{nil}}, p_{\text{cons}}, \text{cons}(a, xs)) \equiv p_{\text{cons}}(a, xs, \text{listElim}_P(p_{\text{nil}}, p_{\text{cons}}, xs)).$$

Definition A.1.25 (List concatenation and duplicate-freeness). List elimination defines the operations needed to decompose finite paths.

Concatenation. For lists $p, q : \text{List}(A)$, define:

$$\begin{aligned} \text{append}(\text{nil}, q) &:= q, \\ \text{append}(\text{cons}(a, p), q) &:= \text{cons}(a, \text{append}(p, q)). \end{aligned}$$

Occurrence and duplicate-freeness. Suppose A has decidable equality. Define $\text{occurs}_a(p) : \text{Bool}$ by list elimination, returning `true` exactly when an entry of p is identical to a . Define the duplicate-freeness type recursively:

$$\begin{aligned} \text{NoRepeat}(\text{nil}) &:= \text{Fin}(1), \\ \text{NoRepeat}(\text{cons}(a, p)) &:= \text{Id}_{\text{Bool}}(\text{occurs}_a(p), \text{false}) \times \text{NoRepeat}(p). \end{aligned}$$

An inhabitant states that no entry occurs again in the remaining suffix.

Remark A.1.26 (Optional list endpoints). The operations below must distinguish an empty list, which has no endpoint, from a nonempty list, which supplies one. The optional type of Definition A.1.23 records this distinction. List elimination defines operations returning the first and last entries when they exist:

$$\text{head?}, \text{last?} : \text{List}(A) \rightarrow \text{Option}(A),$$

with computation rules

$$\begin{aligned} \text{head?}(\text{nil}) &\equiv \text{none}, \\ \text{head?}(\text{cons}(a, p)) &\equiv \text{some}(a), \\ \text{last?}(\text{nil}) &\equiv \text{none}, \\ \text{last?}(\text{cons}(a, \text{nil})) &\equiv \text{some}(a), \\ \text{last?}(\text{cons}(a, \text{cons}(b, p))) &\equiv \text{last?}(\text{cons}(b, p)). \end{aligned}$$

Thus an identity $\text{head?}(p) = \text{some}(X)$ or $\text{last?}(p) = \text{some}(Y)$ also supplies evidence that p is nonempty.

Definition A.1.27 (Empty type). The empty type is a type with no constructors and hence no canonical inhabitants. In this book, it is obtained as the zero-index case of the finite family. Its rules are as follows:

Formation. Define

$$\perp := \text{Fin}(0).$$

Consequently,

$$\Gamma \vdash \perp \text{ type}.$$

Unfolding the definition gives $\perp \equiv \text{Fin}(0)$ judgementally. By the rules of Definition A.1.17, this is the inductive type with no constructors.

Introduction. There are no introduction rules and hence no canonical inhabitants of $\perp$.

Elimination. Given a family

$$\Gamma, x : \perp \vdash P(x) \text{ type}$$

and $\Gamma \vdash e : \perp$, elimination requires no constructor cases and yields

$$\Gamma \vdash \text{absurd}_P(e) : P(e).$$

For a family that does not depend on x , this gives the non-dependent function $\text{absurd}_P : \perp \rightarrow P$.

Computation. There are no constructor computation equations to state, because there is no introduced inhabitant on which absurd_P could compute.

Constructive logic

Definition A.1.28 (Propositions as types). The propositions-as-types interpretation is the type-theoretic instance of the Curry–Howard correspondence: propositions are represented by types, and proofs are represented by terms of those types [Mar84; NPS90]. A context Γ may consequently be read as a sequence of assumptions: a declaration $x : A$ records that x is available as evidence for A . The judgement

$$\Gamma \vdash a : A$$

A. Definitions

then says that, under the assumptions in Γ , the term a supplies constructive evidence for A . Equivalently, saying that A is proved in Γ means that a term a with judgement $\Gamma \vdash a : A$ has been constructed.

Because proofs are represented by terms, they may carry computational content. For example, a term $f : A \rightarrow B$ transforms any proof $a : A$ into a proof $f(a) : B$. The same reading extends to the remaining logical connectives through their corresponding type formers.

Remark A.1.29 (Interpretive role of propositions as types). The propositions-as-types reading is used informally throughout the book but is not invoked as a rule inside the formal transfer of Theorem B.11.4; it serves as a background interpretation of the constructive judgemental discipline.

Definition A.1.30 (Logical connectives under propositions-as-types). Under Definition A.1.28, the standard logical connectives correspond to type formers:

$$\begin{aligned} A \wedge B &:= A \times B, & A \vee B &:= A + B, \\ A \Rightarrow B &:= A \rightarrow B, & \neg A &:= A \rightarrow \perp, \\ A \iff B &:= (A \rightarrow B) \times (B \rightarrow A), & \top &:= \text{Fin}(1). \end{aligned}$$

The constructive content of a logical statement is then a term of the corresponding type. In particular, $A \wedge B$ requires both a witness of A and a witness of B ; $A \vee B$ requires a witness of one side together with evidence of which side; and $\neg A$ requires a procedure converting any witness of A into an inhabitant of the empty type.

Remark A.1.31 (Constructive negation and double negation). From $\neg A := A \rightarrow \perp$ one obtains both $\neg(A \wedge \neg A)$ and $A \rightarrow \neg\neg A$, as proved in Proposition B.4.1.

Decidability and classification

Definition A.1.32 (Decidable type). A type A is *decidable* when the sum type $A + \neg A$ is inhabited, using Definitions A.1.12 and A.1.30. A term

$$d : A + \neg A$$

is a *decision procedure* for A . Its left branch contains a witness $a : A$; its right branch contains a refutation $n : \neg A$. A negative result must therefore explain why any hypothetical witness of A would yield an inhabitant of the empty type; merely failing to find a witness does not decide A .

Remark A.1.33 (Decidability and double-negation elimination). If A is decidable in the sense of Definition A.1.32, then $\neg\neg A \rightarrow A$ is inhabited, as proved in Proposition B.4.2. The converse map $A \rightarrow \neg\neg A$ is available for every type, by Proposition B.4.1. Neither fact supplies uniform excluded middle; see Proposition B.3.1.

Definition A.1.34 (Decidable predicate). A predicate $P : X \rightarrow \mathcal{U}$ is *decidable* when it has a uniform decision procedure:

$$d_P : \prod_{x:X} (P(x) + \neg P(x)).$$

Given any $x : X$, the term $d_P(x)$ therefore returns either a witness of $P(x)$ or a refutation of $P(x)$.

Definition A.1.35 (Boolean classifier). A Boolean classifier is an equivalent Boolean-valued presentation of the decidability defined in Definition A.1.34. For a predicate $P : X \rightarrow \mathcal{U}$, it consists of a function $b : X \rightarrow \mathbf{Bool}$ together with correctness evidence for both branches:

$$\begin{aligned} \text{trueCorrect}_b &: \Pi_{x:X} ((b(x) = \text{true}) \rightarrow P(x)), \\ \text{falseCorrect}_b &: \Pi_{x:X} ((b(x) = \text{false}) \rightarrow \neg P(x)). \end{aligned}$$

A Boolean classifier and a decision procedure express the same information in different forms. Given a classifier b and an input $x : X$, inspect $b(x)$. If it returns **true**, the first correctness map supplies a witness of $P(x)$; if it returns **false**, the second supplies a refutation of $P(x)$. This gives a term of $P(x) + \neg P(x)$. Conversely, given a decision procedure for $P(x)$, return **true** when its left branch contains a witness of $P(x)$, and return **false** when its right branch contains a refutation. The contents of these branches provide the required correctness evidence. Hence a predicate is decidable in the sense of Definition A.1.34 if and only if it has a Boolean classifier.

For example, take $X := \mathbf{Fin}(3)$ and define the predicate $P(i) := \text{Id}(i, 0)$. It is decided by the classifier $b_P : \mathbf{Fin}(3) \rightarrow \mathbf{Bool}$ that returns **true** exactly at 0. When $i = 0$, the true branch supplies the witness $\text{refl}_0 : \text{Id}_{\mathbf{Fin}(3)}(0, 0)$ using the reflexivity constructor of Definition A.1.13; when $i = 1$ or $i = 2$, the false branch supplies a proof that i is not equal to 0.

Definition A.1.36 (Indicator). The indicator numerically recodes the Boolean component of a classifier from Definition A.1.35: it sends **true** to 1 and **false** to 0 so that event membership can contribute to a finite sum. The construction itself applies to any Boolean-valued function.

Data. Let X be a type and let $E : X \rightarrow \mathbf{Bool}$ be a Boolean-valued function.

Definition. Boolean elimination first defines the Boolean-to-number function

$$\text{boolToNat} : \mathbf{Bool} \rightarrow \mathbb{N}$$

by

$$\text{boolToNat}(b) := \text{boolElim}_{\lambda b. \mathbb{N}}(1, 0, b),$$

which sends **true** to 1 and **false** to 0. The indicator of E is the composite

$$\mathbb{1}_E := \text{boolToNat} \circ E : X \rightarrow \mathbb{N}.$$

Equivalently, for each $x : X$,

$$\mathbb{1}_E(x) := \text{boolElim}_{\lambda b. \mathbb{N}}(1, 0, E(x)).$$

Computation. The computation rules for the Boolean eliminator give

$$\mathbb{1}_E(x) \equiv 1 \quad \text{when} \quad E(x) \equiv \text{true}, \quad \mathbb{1}_E(x) \equiv 0 \quad \text{when} \quad E(x) \equiv \text{false}.$$

Example. For the classifier b_P from the example in Definition A.1.35, the indicator has the values

$$\mathbb{1}_{b_P}(0) \equiv 1, \quad \mathbb{1}_{b_P}(1) \equiv 0, \quad \mathbb{1}_{b_P}(2) \equiv 0.$$

Purpose. In the later probability constructions, summing $\mathbb{1}_E$ counts the cells at which E holds, while multiplying by $\mathbb{1}_E$ restricts a finite sum to those cells.

A. Definitions

Definition A.1.37 (Decidable equality). A type A has *decidable equality* when its identity predicate from Definition A.1.13 is decidable in the sense of Definition A.1.34, that is, when there is a term:

$$\text{decEq}_A : \prod_{a:A} \prod_{b:A} (\text{Id}_A(a, b) + \neg \text{Id}_A(a, b)).$$

Thus, for every $a, b : A$, the procedure determines whether they are identical: it returns either an identity $p : \text{Id}_A(a, b)$ or a refutation $n : \neg \text{Id}_A(a, b)$.

Remark A.1.38 (Finite types have decidable equality). Every finite type in the sense of Definition A.1.19 has decidable equality, as proved in Proposition B.4.3.

A.2 Mode theory

This section defines the mode theory used in the book: a 1-category whose objects are epistemic states and whose morphisms are admitted transitions between them.

Modes and modalities

Definition A.2.1 (Mode theory). A mode theory explicitly records the objects and arrows along which modal judgements may vary.

Record type. Define `ModeTheory` as the record with fields:

$$\begin{aligned} \text{Mode} &: \mathcal{U}, \\ \text{Hom} &: \text{Mode} \rightarrow \text{Mode} \rightarrow \mathcal{U}, \\ \text{id} &: \prod_{m:\text{Mode}} \text{Hom}(m, m), \\ \text{comp} &: \prod_{o,n,m:\text{Mode}} \text{Hom}(n, m) \rightarrow \text{Hom}(o, n) \rightarrow \text{Hom}(o, m). \end{aligned}$$

For composable arrows $\mu : \text{Hom}(n, m)$ and $\nu : \text{Hom}(o, n)$, write $\mu \circ \nu := \text{comp}(\mu, \nu)$ and $1_m := \text{id}(m)$.

Category laws. The record also carries identity proofs:

$$\begin{aligned} h_{\text{left}} &: \prod_{n,m:\text{Mode}} \prod_{\mu:\text{Hom}(n,m)} \text{Id}(1_m \circ \mu, \mu), \\ h_{\text{right}} &: \prod_{n,m:\text{Mode}} \prod_{\mu:\text{Hom}(n,m)} \text{Id}(\mu \circ 1_n, \mu), \\ h_{\text{assoc}} &: \prod_{o,n,p,m:\text{Mode}} \prod_{\mu:\text{Hom}(p,m)} \prod_{\nu:\text{Hom}(n,p)} \prod_{\xi:\text{Hom}(o,n)} \text{Id}((\mu \circ \nu) \circ \xi, \mu \circ (\nu \circ \xi)). \end{aligned}$$

These data make a mode theory a small 1-category presented inside the type-theoretic vocabulary of Definition A.1.13.

Definition A.2.2 (Mode). A mode is an inhabitant $m : \text{Mode}_{\mathcal{M}}$ of the object type carried by a mode theory $\mathcal{M} : \text{ModeTheory}$. In this monograph, a mode is read as an epistemic state.

Definition A.2.3 (Modality). A modality is an arrow between two modes of one mode theory.

Arrow type. For modes $n, m : \text{Mode}_{\mathcal{M}}$, a modality from n to m is an inhabitant:

$$\mu : \text{Hom}_{\mathcal{M}}(n, m),$$

written $\mu : n \rightarrow m$. The identity modality is $1_m : m \rightarrow m$. Given $\mu : n \rightarrow m$ and $\nu : o \rightarrow n$, their composite is $\mu \circ \nu : o \rightarrow m$. Generating modalities name admitted transitions between states, while composition represents a finite sequence of such transitions.

Epistemic interpretation

Remark A.2.4 (Interpretive terminology). The terms *subjectivity*, *epistemic position* and *interest* are interpretive shorthand, not additional objects of the calculus. In this monograph, an epistemic position is represented by the current mode; interest by what that state makes available; and subjectivity by the fact that inference is always from some such state. No later definition or proof treats these terms as separate formal entities.

Remark A.2.5 (Epistemic modalities mark transitions). Just as the terms in Remark A.2.4 do not add new objects to the calculus, the names learning, forgetting, conditioning, relating, unrelating, setting and unsetting do not introduce additional modal operators; they describe transitions between already specified modes. A modality $\mu : n \rightarrow m$ witnesses that the transition from n to m is admitted. Several such modalities may be applied in succession; their composite is again a modality.

A.3 Probabilistic definitions

This section defines constructive finite probability structures and operations in the type-theoretic setting of Section A.1: decidable events, urn and record presentations, conditioning and the constructive urn assumptions.

Events and resolution status

Definition A.3.1 (Finite sample type). A finite sample type is a finite type Ω , in the sense of Definition A.1.19, whose inhabitants are read as possible outcomes. Its finiteness witness enumerates those outcomes and permits finite sums over them. By Remark A.1.38, every finite sample type has decidable equality, so outcomes can be compared and singleton events can be decided.

Definition A.3.2 (Boolean event). A Boolean event is a decidable selection of terms from a type.

Event classifier. Let X be any type. A Boolean event on X is a function:

$$E : X \rightarrow \text{Bool}.$$

For each $x : X$, the value $E(x) = \text{true}$ means that x is selected, while $E(x) = \text{false}$ means that it is not. When X is a finite sample type, such a classifier is called a decidable event on that sample type.

Definition A.3.3 (Boolean event operations and disjointness). Boolean events inherit pointwise operations from the Boolean type.

A. Definitions

Event operations. Let $E, F : X \rightarrow \text{Bool}$ be Boolean events in the sense of Definition A.3.2. Intersection, union, difference and complement are the pointwise Boolean operations of Definition A.1.20:

$$\begin{aligned}(E \cap F)(x) &:= \text{and}_{\text{Bool}}(E(x), F(x)), \\ (E \cup F)(x) &:= \text{or}_{\text{Bool}}(E(x), F(x)), \\ (E \setminus F)(x) &:= \text{and}_{\text{Bool}}(E(x), \text{not}_{\text{Bool}}(F(x))), \\ E^c(x) &:= \text{not}_{\text{Bool}}(E(x)).\end{aligned}$$

The empty and full events are the constant classifiers:

$$\emptyset_X(x) := \text{false}, \quad \text{All}_X(x) := \text{true}.$$

Two events are disjoint when their intersection is pointwise equal to the empty event. This condition is represented by the type:

$$\text{Disjoint}(E, F) := \prod_{x:X} (E \cap F)(x) = \emptyset_X(x).$$

An inhabitant of this type supplies evidence that no term is selected by both classifiers. The symbols $\cap$ and $\cup$ are used only as this defined classifier notation.

For four events, write $\text{FourWayDisjoint}(E_1, E_2, E_3, E_4)$ for the product of the six types $\text{Disjoint}(E_i, E_j)$ with $i < j$. An inhabitant therefore proves pairwise disjointness of all four classifiers.

Definition A.3.4 (Singleton event). A singleton event selects exactly one presented term.

Classifier. Let X be a type with decidable equality in the sense of Definition A.1.37, and let $x : X$. Define:

$$E_x^{\text{sing}} : X \rightarrow \text{Bool}$$

to return true at $y : X$ exactly when the equality procedure returns evidence of $\text{Id}_X(y, x)$. Thus

$$E_x^{\text{sing}}(y) = \text{true} \iff \text{Id}_X(y, x).$$

Every finite sample type admits these events by Remark A.1.38.

Definition A.3.5 (Resolution status). A resolution-status package records the result of a finite inspection procedure together with evidence that every reported proof or refutation is sound. Its components are as follows:

Data. Let Ω be a finite sample type. Let

$$A : \Omega \rightarrow \mathcal{U}$$

be an arbitrary proposition-valued family on Ω . Equivalently, A is a predicate assigning to every outcome $\omega : \Omega$ a proposition $A(\omega)$. No decision procedure for $A(\omega)$ is assumed.

Status type. Define the three-constructor finite type:

Status generated by `proved`, `refuted`, `unresolved`.

Resolution package. Define the record $\text{Resolution}(A)$ with fields:

$$\begin{aligned}\text{inspect} &: \Omega \rightarrow \text{Status}, \\ \text{provedSound} &: \prod_{\omega:\Omega} (\text{inspect}(\omega) = \text{proved}) \rightarrow A(\omega), \\ \text{refutedSound} &: \prod_{\omega:\Omega} (\text{inspect}(\omega) = \text{refuted}) \rightarrow \neg A(\omega).\end{aligned}$$

Here $\neg A(\omega) := A(\omega) \rightarrow \perp$ as in Definition A.1.30. Thus a **proved** result supplies a witness of $A(\omega)$, while a **refuted** result supplies a refutation of $A(\omega)$.

Interpretation. The value **unresolved** means only that the chosen inspection procedure supplies neither kind of evidence. It does not claim that no proof or refutation exists.

The family A is not assumed to be decidable. The finite output of **inspect** is decidable instead: whether the returned status is **proved**, **refuted** or **unresolved** can always be determined.

Remark A.3.6 (Why the unresolved case is necessary). A sound resolution-status package is available for every family $A : \Omega \rightarrow \mathcal{U}$: define $\text{inspect}(\omega) := \text{unresolved}$ for every $\omega : \Omega$. The premises of the proved and refuted soundness maps are then impossible, so those maps follow by empty-type elimination.

By contrast, a sound procedure guaranteed never to return **unresolved** would make A a decidable predicate in the sense of Definition A.1.34: for every $\omega : \Omega$, its result would supply either a witness of $A(\omega)$ or a refutation of $A(\omega)$. The unresolved case is therefore what permits a total inspection procedure without assuming that the underlying family of propositions is decidable.

Definition A.3.7 (Resolution-status events). A resolution package determines three decidable events on its sample type.

Data. Let Ω and $A : \Omega \rightarrow \mathcal{U}$ be as in Definition A.3.5, let $R : \text{Resolution}(A)$ and write $\iota_R := \text{inspect}(R)$. The three resolution-status events determined by R are

$$\text{Prov}_R(\omega) := \begin{cases} \text{true}, & \iota_R(\omega) = \text{proved}, \\ \text{false}, & \text{otherwise}, \end{cases} \quad \text{Ref}_R(\omega) := \begin{cases} \text{true}, & \iota_R(\omega) = \text{refuted}, \\ \text{false}, & \text{otherwise}, \end{cases}$$

and

$$\text{Unr}_R(\omega) := \begin{cases} \text{true}, & \iota_R(\omega) = \text{unresolved}, \\ \text{false}, & \text{otherwise}. \end{cases}$$

Classification and partition. Formally, these are Boolean classifiers in the sense of Definition A.1.35 for the predicates that $\iota_R(\omega)$ equals **proved**, **refuted** or **unresolved**, respectively. They are obtained by case analysis on the finite resolution-status type, with the equality tests justified by decidable equality in the sense of Definition A.1.37. Their correctness evidence follows from the corresponding cases. The three classifiers are shown in Proposition B.6.7 to form a partition of Ω : every $\omega : \Omega$ lies in exactly one of them.

Rational values and finite aggregation

Definition A.3.8 (Nonnegative rational values). The presentation type of nonnegative rational values is

$$\mathbb{Q}_{\geq 0} := \Sigma_{a:\mathbb{N}} \Sigma_{b:\mathbb{N}} (b > 0).$$

An inhabitant $q = (a, b, h) : \mathbb{Q}_{\geq 0}$ consists of a numerator $a : \mathbb{N}$, a denominator $b : \mathbb{N}$, and a proof $h : b > 0$ that the denominator is positive. It represents the fraction a/b .

Two presentations

$$q = (a, b, h) \quad \text{and} \quad r = (c, d, k)$$

A. Definitions

represent the same rational value when

$$q \sim r \iff ad = cb.$$

The same presentations are ordered by cross multiplication:

$$\frac{a}{b} \leq \frac{c}{d} \iff ad \leq cb.$$

This is the order used throughout the finite probability constructions. Equality between values of $\mathbb{Q}_{\geq 0}$ is read extensionally through $\sim$, not as identity of their presentations. For example, with proofs $h : 5 > 0$ and $k : 10 > 0$, the presentations $(3, 5, h)$ and $(6, 10, k)$ represent the same rational value because $3 \cdot 10 = 6 \cdot 5$, although the two triples are not definitionally equal. The monograph therefore keeps the presentations together with the explicit relation $\sim$, rather than forming a quotient type.

Natural-number counts are regarded as rational values with denominator 1: the natural number m is represented by

$$(m, 1, \star) : \mathbb{Q}_{\geq 0},$$

where $\star$ is the immediate proof that $1 > 0$. This allows natural-valued indicators and cell counts to be used in rational probability expressions.

Addition, multiplication and division by a positive value use the usual fraction operations:

$$\frac{a}{b} + \frac{c}{d} := \frac{ad + cb}{bd}, \quad \frac{a}{b} \frac{c}{d} := \frac{ac}{bd}, \quad \frac{a/b}{c/d} := \frac{ad}{bc} \quad \text{when } c > 0.$$

The required proofs that the resulting denominators are positive follow from the supplied positive denominators. These operations respect $\sim$, as proved in Proposition B.6.5.

Lean implementation. The order on presentations is `QProb.LE`.

Definition A.3.9 (Finite summation). Finite summation aggregates values indexed by a finite type.

Data. Let A be a finite type in the sense of Definition A.1.19, with $n : \mathbb{N}$ and

$$\text{decode} : \text{Fin}(n) \rightarrow A$$

supplied by its finiteness witness. Let M be a type with an equivalence relation $\approx$, an addition operation and an additive identity 0. Assume that addition respects $\approx$, and that associativity, commutativity and the identity laws hold up to $\approx$.

Sum. For a function $g : A \rightarrow M$, define:

$$\sum_{a:A} g(a) := \sum_{i:\text{Fin}(n)} g(\text{decode}(i)),$$

where the sum on the right is computed recursively over the n indices, beginning with zero.

Invariance. The sum is unchanged up to $\approx$ when the indices are permuted, as proved in Proposition B.6.3. It therefore gives the same value, up to $\approx$, regardless of the order in which the terms are enumerated. Here $\approx$ expresses the

appropriate notion of equality for M : for natural-number sums it is ordinary identity, whereas for the rational presentations of Definition A.3.8 it is the cross-multiplication relation $\sim$.

Definition A.3.10 (Finite product). Finite products are the multiplicative counterpart of the finite sums of Definition A.3.9.

Data. Let A be a finite type, with $\text{decode} : \text{Fin}(n) \rightarrow A$ supplied by its finiteness witness. Let M be a type with an equivalence relation $\approx$, a multiplication operation and a multiplicative identity 1. Assume that multiplication respects $\approx$, and that associativity, commutativity and the identity laws hold up to $\approx$.

Product. For $g : A \rightarrow M$, define:

$$\prod_{a:A} g(a) := \prod_{i:\text{Fin}(n)} g(\text{decode}(i)),$$

where the product on the right is computed recursively over the n indices, beginning with 1. As with finite summation, reordering the indices leaves the result unchanged up to $\approx$, as proved in Proposition B.6.4.

Finite probability presentations

Definition A.3.11 (Urn-generated probability type). An urn-generated probability assignment is built from a nonempty finite urn whose cells are treated as equiprobable and labelled by outcomes in a chosen type. Its data and event evaluation are as follows:

Data. Let X be any type. Define

$$\text{Prob}_{\mathbb{Q}}^{\text{urn}}(X) := \sum_{N:\mathbb{N}} \sum_{h:N>0} (\text{Fin}(N) \rightarrow X).$$

An inhabitant

$$\mu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$$

consists of a number N of cells, a proof h that the urn is nonempty, and a function

$$f : \text{Fin}(N) \rightarrow X$$

assigning an outcome in X to every cell.

Event probability. Let $E : X \rightarrow \text{Bool}$ be any Boolean-valued function. First count the cells whose labels satisfy E :

$$m_E := \sum_{i:\text{Fin}(N)} \mathbb{1}_E(f(i)),$$

using the finite summation of Definition A.3.9 and the indicator of Definition A.1.36. Its probability is the nonnegative rational presentation

$$P_{\mu}(E) := (m_E, N, h) : \mathbb{Q}_{\geq 0},$$

which represents the proportion m_E/N according to Definition A.3.8. When X is instantiated by a finite sample type Ω in the sense of Definition A.3.1, the same function $E : \Omega \rightarrow \text{Bool}$ is a decidable event.

Interpretation. Every cell has probability represented by $(1, N, h)$, conventionally written $1/N$. The map f need not be injective: when several cells have

A. Definitions

the same label, their contributions add together. Nor need f be surjective: labels outside its image do not contribute to any probability sum. Consequently, X itself need not be finite. Only the source urn is finite, and every event probability is determined by the finitely many labels occurring in that urn.

This is the urn presentation of a probability assignment: outcome weights are encoded indirectly by repeated labels among equiprobable cells. The next definition introduces the direct normalised-weight presentation used for finite sample types.

Definition A.3.12 (Finite probability type). The urn presentation of Definition A.3.11 encodes probabilities indirectly through the multiplicities of labels among equiprobable cells. On a finite sample type, probabilities can instead be recorded directly by assigning a nonnegative rational weight to every outcome and proving that the weights sum to 1. This direct record presentation is defined below.

Data. Let Ω be a finite sample type. Probability values lie in the presentation type $\mathbb{Q}_{\geq 0}$ of Definition A.3.8. All sums below use the finite summation of Definition A.3.9.

The record form of a probability type over Ω is

$$\mathbf{Prob}_{\mathbb{Q}}(\Omega) := \Sigma_{p:\Omega \rightarrow \mathbb{Q}_{\geq 0}} \left(\sum_{\omega:\Omega} p(\omega) \sim 1 \right).$$

An inhabitant

$$\mu = (p, h) : \mathbf{Prob}_{\mathbb{Q}}(\Omega)$$

consists of a function

$$p : \Omega \rightarrow \mathbb{Q}_{\geq 0}$$

assigning a nonnegative rational weight to every outcome, together with a proof

$$h : \sum_{\omega:\Omega} p(\omega) \sim 1$$

that these weights are normalised.

Event probability. If $\mu = (p, h) : \mathbf{Prob}_{\mathbb{Q}}(\Omega)$ and $E : \Omega \rightarrow \mathbf{Bool}$, then

$$P_{\mu}(E) := \sum_{\omega:\Omega} \mathbf{1}_E(\omega) p(\omega),$$

using the indicator of Definition A.1.36 and the embedding of natural numbers into $\mathbb{Q}_{\geq 0}$ from Definition A.3.8. If E and F are disjoint in the sense of Definition A.3.3, then

$$P_{\mu}(E \cup F) \sim P_{\mu}(E) + P_{\mu}(F)$$

because disjointness gives the pointwise indicator identity

$$\mathbf{1}_{E \cup F}(\omega) = \mathbf{1}_E(\omega) + \mathbf{1}_F(\omega),$$

after which distributivity and finite summation separate the two contributions. This is the direct-record form of the finite additivity proved in Theorem 5.3.1; the passage from a direct record through a common-denominator record to an

urn is justified by Propositions B.6.12 and B.6.13. When only one probability assignment is in scope, the subscript is sometimes dropped: $P(E) := P_\mu(E)$.

Interpretation. The function p records outcome weights directly rather than encoding them through repeated urn labels. Finiteness of Ω makes every event sum finite, while the proof h ensures that the total weight is extensionally equal to 1. Unlike the urn presentation of Definition A.3.11, this direct presentation requires the outcome type itself to be finite.

Definition A.3.13 (Common-denominator probability record). In the direct presentation of Definition A.3.12, different outcomes may initially have different denominators. Because its sample type is finite, these fractions can be rewritten over one positive common denominator, as proved in Proposition B.6.12. Once finite support is recorded explicitly as a list, however, the codomain itself need not be finite. The common-denominator representation is structured as follows:

Data. Let X be any type. Write $\text{sumWeights}(a)$ for the natural number obtained by traversing a list $a : \text{List}(X \times \mathbb{N})$ and adding the second projection of each entry. Define the common-denominator probability type:

$$\text{Prob}_{\mathbb{Q}}^{\text{cd}}(X) := \Sigma_{a:\text{List}(X \times \mathbb{N})} \Sigma_{D:\mathbb{N}} \Sigma_{d:D>0} \text{Id}_{\mathbb{N}}(\text{sumWeights}(a), D).$$

An inhabitant is written:

$$\begin{aligned} R &= (a, D, d, h), \\ a &: \text{List}(X \times \mathbb{N}), \\ D &: \mathbb{N}, \\ d &: D > 0, \\ h &: \text{sumWeights}(a) = D. \end{aligned}$$

The list a contains finitely many explicitly supplied labels $\omega : X$, each paired with a natural weight m . No equality test on X is part of the record; if the same label occurs in several entries, each corresponding weight contributes separately. The natural number D is the common denominator, d proves that it is positive, and h proves normalisation by showing that the total weight of the list is exactly D .

Event probability. Let $E : X \rightarrow \text{Bool}$ be a Boolean-valued function. Write $\text{sumWeights}_E(a)$ for the same traversal, now adding a weight only when $E(\omega) = \text{true}$; this requires no decidable equality on X . Denote the selected weight and the resulting probability by

$$\begin{aligned} m_E &:= \text{sumWeights}_E(a), \\ P_R(E) &:= (m_E, D, d) : \mathbb{Q}_{\geq 0}. \end{aligned}$$

Here m_E is the numerator and D is the common denominator, so $P_R(E)$ represents the fraction m_E/D . Selecting entries cannot increase their total weight, hence

$$m_E \leq D \quad \text{and} \quad P_R(E) \leq 1.$$

Interpretation. The list a is a compressed finite urn: the weight m records how many unit cells carry the label ω . The total event selects every entry and therefore has weight D by h , so its probability is represented by D/D and satisfies $D/D \sim 1$.

A. Definitions

Lean implementation. This is the structure `FiniteProbRecord` in `lean/Thesis/Probability/FiniteRecord.lean`: its fields `atoms`, `den`, `den_pos` and `total_mass` correspond respectively to a , D , d and h . The functions `FiniteProbRecord.eventMass` and `FiniteProbRecord.probVal` implement m_E and $P_R(E)$.

Example A.3.14 (Colour probability presentations). The same finite rational assignment can be displayed in each of the three preceding presentations.

Let `Colour` have inhabitants `red`, `blue` and `pink`, and let E_{red} select only `red`. For the urn presentation, label five cells by:

$$[\text{red}, \text{blue}, \text{red}, \text{red}, \text{blue}].$$

For the direct presentation, assign the respective weights $3/5$, $2/5$ and $0/5$. For the common-denominator presentation, use the weighted list:

$$[(\text{red}, 3), (\text{blue}, 2), (\text{pink}, 0)]$$

with denominator 5. In every presentation the event probability computes to the rational presentation $(3, 5, d)$, where $d : 5 > 0$. Thus all three assign probability $3/5$ to `red`, $2/5$ to `blue` and 0 to `pink`.

Remark A.3.15 (Relationship between finite probability presentations). The urn, direct-weight and common-denominator presentations describe the same finite rational probability assignments, but the conversions between them use slightly different assumptions.

From an urn to direct weights. Let

$$\nu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega),$$

where Ω is a finite sample type. Here N is the number of urn cells, $h : N > 0$ proves that the urn is nonempty, and

$$f : \text{Fin}(N) \rightarrow \Omega$$

gives the outcome labelling each cell. Using decidable equality on Ω , let

$$E_{\omega} : \Omega \rightarrow \text{Bool}$$

be the singleton classifier that returns `true` exactly at ω . The number of cells labelled ω is the event count from Definition A.3.11:

$$m_{E_{\omega}} := \sum_{i : \text{Fin}(N)} \mathbb{1}_{E_{\omega}}(f(i)).$$

Define

$$p_{\nu}(\omega) := (m_{E_{\omega}}, N, h) : \mathbb{Q}_{\geq 0}.$$

Every cell has exactly one label, so summing these counts over all outcomes gives

$$\sum_{\omega : \Omega} m_{E_{\omega}} = N.$$

Consequently, the weights $p_{\nu}(\omega)$ sum extensionally to 1 and determine an inhabitant of $\text{Prob}_{\mathbb{Q}}(\Omega)$ in the sense of Definition A.3.12.

From direct weights to a common denominator. Suppose

$$\mu = (p, h) : \text{Prob}_{\mathbb{Q}}(\Omega).$$

Because Ω is finite and every $p(\omega)$ is rational, one can choose a positive denominator D , a proof $d : D > 0$, and natural weights m_ω such that

$$p(\omega) \sim (m_\omega, D, d) \quad \text{for every } \omega : \Omega$$

and

$$\sum_{\omega : \Omega} m_\omega = D.$$

These data produce a common-denominator record of Definition A.3.13. Proposition B.6.12 proves that this conversion preserves the probability of every Boolean-valued event.

Between common-denominator records and urns. An urn $\nu = (N, h, f)$ determines a common-denominator record by taking denominator N and listing its cells with unit weight:

$$[(f(i), 1)]_{i : \text{Fin}(N)}.$$

Conversely, a weighted entry (ω, m) in a common-denominator record can be expanded into m separate urn cells labelled ω . Proposition B.6.13 proves both directions and shows that the record and resulting urn assign extensionally equal probabilities to every Boolean-valued event.

Therefore, when Ω is a finite sample type, all three presentations can be converted into one another without changing the probabilities they assign to events. For an arbitrary type X , only the urn–common-denominator correspondence is available, because those presentations carry their finite support explicitly; the direct presentation requires the whole outcome type to be a finite sample type.

Remark A.3.16 (Finite probability laws). Every urn-generated probability assignment

$$\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$$

satisfies the finite counterparts of the Kolmogorov laws. For every decidable event $E : X \rightarrow \text{Bool}$:

$$0 \leq P_\mu(E) \leq 1.$$

For the full event All_X of Definition A.3.3:

$$P_\mu(\text{All}_X) \sim 1.$$

Finally, if $E, F : X \rightarrow \text{Bool}$ are disjoint decidable events, then:

$$P_\mu(E \cup F) \sim P_\mu(E) + P_\mu(F).$$

The same additivity law extends to every finite pairwise disjoint family of decidable events.

These laws are consequences of finite cell counting and the normalisation carried by the probability assignment; they are not additional assumptions or record fields. They are the content of Theorem 5.3.1, proved in Section B.6. Through the presentation equivalences of Remark A.3.15, the same laws hold for the direct-weight and common-denominator forms. No countable-additivity principle is asserted.

Conditioning and independence

Definition A.3.17 (Conditional probability). Conditioning is defined directly on the urn-generated assignments of Definition A.3.11. Its components are as follows:

Data. Let X be any type, let

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X),$$

and let $A, B : X \rightarrow \text{Bool}$ be decidable events. Write m_A and m_B for their respective selected-cell counts in μ , as in Definition A.3.11. Write $m_{A \cap B}$ for the selected-cell count of their intersection, using the Boolean event operation of Definition A.3.3.

Conditional probability. Suppose $h_B : m_B > 0$; equivalently, B has positive probability under μ . Then:

$$P_\mu(A \mid B) := (m_{A \cap B}, m_B, h_B) : \mathbb{Q}_{\geq 0}.$$

Indeed,

$$P_\mu(A \cap B) \sim \frac{m_{A \cap B}}{N}, \quad P_\mu(B) \sim \frac{m_B}{N},$$

and therefore, because $m_B > 0$,

$$\frac{P_\mu(A \cap B)}{P_\mu(B)} \sim \frac{m_{A \cap B}/N}{m_B/N} = \frac{m_{A \cap B}}{m_B}.$$

Thus $(m_{A \cap B}, m_B, h_B)$ presents the conditional probability obtained after the common urn denominator N cancels. Equivalently,

$$P_\mu(A \mid B) \sim \frac{P_\mu(A \cap B)}{P_\mu(B)}.$$

Conditional probability is partial: it is defined only when the conditioning event has positive probability under μ .

Remark A.3.18 (Finite Bayes theorem). If both A and B have positive probability, the conditional probability of Definition A.3.17 satisfies:

$$P_\mu(A \mid B) \sim \frac{P_\mu(B \mid A)P_\mu(A)}{P_\mu(B)}.$$

This is Bayes' theorem for decidable finite events. It is not an additional defining rule: it follows by expanding both conditional probabilities into finite event counts and using $A \cap B = B \cap A$. The calculation is proved in Theorem 5.3.2.

Definition A.3.19 (Bayesian update). Bayesian update filters a finite urn by newly received positive-probability evidence.

Data. Let $\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, let $E : X \rightarrow \text{Bool}$ represent the evidence, and let m_E be its selected-cell count. Suppose $h_E : m_E > 0$.

Updated assignment. Filter the ordered list of urn cells, retaining exactly those cells $i : \text{Fin}(N)$ for which $E(f(i)) = \text{true}$. Renumbering the resulting m_E retained cells in their inherited order gives an enumeration

$$f_E : \text{Fin}(m_E) \rightarrow X$$

and hence the filtered, Bayesian-updated assignment

$$\mu_E := (m_E, h_E, f_E) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X).$$

With E now taken as given evidence, let $F : X \rightarrow \text{Bool}$ be any decidable query event. The updated assignment then satisfies

$$P_{\mu_E}(F) \sim P_{\mu}(F \mid E),$$

as proved in Proposition B.6.20.

When X is a finite sample type and μ corresponds to a direct assignment $(p, h) : \text{Prob}_{\mathbb{Q}}(X)$ through Remark A.3.15, converting the filtered urn gives the familiar posterior mass function

$$p_E(x) := \frac{\mathbb{1}_E(x)p(x)}{P_{\mu}(E)}.$$

The update is defined only under the supplied positive-probability evidence h_E .

Definition A.3.20 (Independence and conditional independence of decidable events). Independence compares joint event probabilities with products of marginal event probabilities.

Data. Let X be any type, let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $A, B, C : X \rightarrow \text{Bool}$ be decidable events.

Unconditional independence. The events A and B are *independent under μ* , written $A \perp\!\!\!\perp_{\mu} B$, when

$$P_{\mu}(A \cap B) \sim P_{\mu}(A) \cdot P_{\mu}(B).$$

Equivalently, when $P_{\mu}(B) > 0$, $P_{\mu}(A \mid B) \sim P_{\mu}(A)$; and when $P_{\mu}(A) > 0$, $P_{\mu}(B \mid A) \sim P_{\mu}(B)$.

Conditional independence. The events A and B are *conditionally independent given C under μ* , written $A \perp\!\!\!\perp_{\mu} B \mid C$, when

$$P_{\mu}(A \cap B \cap C) \cdot P_{\mu}(C) \sim P_{\mu}(A \cap C) \cdot P_{\mu}(B \cap C).$$

Equivalently, when $P_{\mu}(C) > 0$, $P_{\mu}(A \cap B \mid C) \sim P_{\mu}(A \mid C) \cdot P_{\mu}(B \mid C)$; and when additionally $P_{\mu}(B \cap C) > 0$, $P_{\mu}(A \mid B \cap C) \sim P_{\mu}(A \mid C)$. These unconditional and conditional characterisations are proved in Proposition B.6.19.

The product forms are primary because they remain defined when conditioning events have zero probability, whereas the corresponding conditional probabilities do not. This is why they are used in the structural causal model proofs of Sections B.7 and B.11, including Lemma B.7.1.

Variables and event projections

Definition A.3.21 (Value type). A value-type assignment specifies the admissible values of each variable name.

Assignment family. Let I be a type of variable names. A finite value-type assignment is a family

$$\text{Val} : I \rightarrow \mathcal{U}$$

for which every $\text{Val}(i)$ is finite in the sense of Definition A.1.19. For a name $X : I$, the type $\text{Val}(X)$ is the value type of X , and its inhabitants are the

A. Definitions

admissible values that X may take. By Remark A.1.38, every value type has decidable equality in the sense of Definition A.1.37.

Example. If distinct names $C, D : I$ denote the outcomes of a coin toss and a six-sided die, one may take

$$\text{Val}(C) := \text{CoinFace}, \quad \text{Val}(D) := \text{DieFace}.$$

The two inhabitants of CoinFace are `heads` and `tails`, while the six inhabitants of DieFace are

$$\text{faceUp}(1), \dots, \text{faceUp}(6).$$

Their finiteness witnesses identify these terms with the indices of $\text{Fin}(2)$ and $\text{Fin}(6)$, respectively. The value types specify which outcomes are admissible; they do not select the outcome of a particular toss or roll.

Definition A.3.22 (Finite variable and joint assignment). A finite family of variables $\mathcal{V}$ consists of a finite type I of variable names together with a finite value-type assignment $\text{Val} : I \rightarrow \mathcal{U}$ from Definition A.3.21. Foundationally, its joint assignments form the finite telescope of Remark A.1.22. For readability, write the same coordinate data pointwise as

$$\text{Val}(\mathcal{V}) := \prod_{i:I} \text{Val}(i)$$

for its joint-assignment type. An inhabitant $v : \text{Val}(\mathcal{V})$ chooses one admissible value $v(i) : \text{Val}(i)$ for every $i : I$. By Proposition B.6.1, this type is finite and hence is a finite sample type. A joint event is therefore a decidable event

$$E : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$$

in the sense of Definition A.3.2.

Example. Let I be the two-element finite type whose variable names are C and D , and use the coin and die value types from Definition A.3.21. A joint assignment chooses one coin face and one die face. For example, $(\text{heads}, \text{faceUp}(2))$ is the assignment with $C = \text{heads}$ and $D = \text{faceUp}(2)$. Every coin-and-die pair is a possible joint assignment. A joint event may return `true` for any chosen collection of these pairs; for example, one event may return `true` exactly on

$$(\text{heads}, \text{faceUp}(2)) \quad \text{and} \quad (\text{tails}, \text{faceUp}(3)).$$

Definition A.3.23 (Coordinate equality event). A coordinate equality event selects complete assignments carrying one specified value at one specified coordinate.

Classifier. Let $\mathcal{V}$ be a finite variable family with name type I , let $i : I$ and let $x : \text{Val}(i)$. Decidable equality on the finite value type $\text{Val}(i)$ defines:

$$E_{i=x} : \text{Val}(\mathcal{V}) \rightarrow \text{Bool},$$

which returns `true` at a complete assignment v exactly when $\text{Id}_{\text{Val}(i)}(v(i), x)$ is inhabited. Thus $E_{i=x}$ is the cylinder lift of the singleton event E_x^{sing} of Definition A.3.4 on the selected coordinate i .

Notation. For a probability assignment μ on $\text{Val}(\mathcal{V})$, write $P_\mu(i = x)$ for $P_\mu(E_{i=x})$. A comma between several coordinate equations denotes the intersection of their equality events in the sense of Definition A.3.3.

Definition A.3.24 (Coordinate selection and projection). A coordinate selection gives a finite indexed presentation of a selected subfamily and its projection from complete assignments.

Selection data. Let $\mathcal{V}$ be a finite variable family with name type I and complete-assignment type $\Omega := \text{Val}(\mathcal{V})$. A coordinate selection $\mathcal{X}$ consists of $m : \mathbb{N}$ and a map:

$$j_{\mathcal{X}} : \text{Fin}(m) \rightarrow I,$$

together with injectivity evidence:

$$h_{\mathcal{X}} : \prod_{k, \ell : \text{Fin}(m)} \text{Id}_I(j_{\mathcal{X}}(k), j_{\mathcal{X}}(\ell)) \rightarrow \text{Id}_{\text{Fin}(m)}(k, \ell).$$

Its selected assignment type is:

$$\text{Val}(\mathcal{X}) := \prod_{k : \text{Fin}(m)} \text{Val}(j_{\mathcal{X}}(k)).$$

Projection. Define:

$$\pi_{\mathcal{X}} : \Omega \rightarrow \text{Val}(\mathcal{X}), \quad \pi_{\mathcal{X}}(v)(k) := v(j_{\mathcal{X}}(k)).$$

The projection retains the selected coordinates and forgets all remaining coordinates.

Definition A.3.25 (Cylinder event). A cylinder event lifts an event on selected coordinates to the full joint-assignment type.

Event lifting. Let $\mathcal{X}$ be a coordinate selection of $\mathcal{V}$ in the sense of Definition A.3.24, and let $E : \text{Val}(\mathcal{X}) \rightarrow \text{Bool}$. Define:

$$E^{\uparrow} : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}, \quad E^{\uparrow}(v) := E(\pi_{\mathcal{X}}(v)).$$

Thus $E^{\uparrow}$ tests only the selected coordinates.

Example. Continue with the coin variable C and die variable D from Definition A.3.22. Let $\mathcal{D}$ select only the die coordinate. Then:

$$\pi_{\mathcal{D}}(\text{heads}, \text{faceUp}(2)) = \text{faceUp}(2), \quad \pi_{\mathcal{D}}(\text{tails}, \text{faceUp}(2)) = \text{faceUp}(2).$$

If $E_{\mathcal{D}}$ selects only $\text{faceUp}(2)$, its lift $E_{\mathcal{D}}^{\uparrow}(c, d) := E_{\mathcal{D}}(d)$ selects both displayed complete assignments. The coin coordinate remains unrestricted.

Definition A.3.26 (Fibre event). A fibre event fixes the values of selected coordinates while leaving every remaining coordinate unrestricted.

Data. Let $\mathcal{S}$ be a coordinate selection of the finite variable family $\mathcal{V}$, and let

$$\pi_{\mathcal{S}} : \Omega \rightarrow \text{Val}(\mathcal{S})$$

be the projection of Definition A.3.24. Since $\text{Val}(\mathcal{S})$ is finite, its equality is decidable by Remark A.1.38.

Fibre over a selected assignment. Every $s : \text{Val}(\mathcal{S})$ determines the decidable event

$$\text{Fibre}_{\mathcal{S}}(s) : \Omega \rightarrow \text{Bool},$$

characterised for every complete assignment $b : \Omega$ by:

$$\text{Fibre}_{\mathcal{S}}(s)(b) = \text{true} \iff \pi_{\mathcal{S}}(b) = s.$$

A. Definitions

Here $\text{Fibre}_{\mathcal{S}}(s)(b)$ means that the Boolean classifier $\text{Fibre}_{\mathcal{S}}(s)$ is evaluated at b , using the function-application rule of Definition A.1.9.

Fibre at a complete assignment. For a complete assignment $a : \Omega$, write

$$a|_{\mathcal{S}} := \pi_{\mathcal{S}}(a)$$

for its restriction to the selected coordinates, and abbreviate

$$F_{\mathcal{S},a} := \text{Fibre}_{\mathcal{S}}(a|_{\mathcal{S}}).$$

Consequently,

$$F_{\mathcal{S},a}(b) = \text{true} \iff \pi_{\mathcal{S}}(b) = \pi_{\mathcal{S}}(a).$$

Values outside $\mathcal{S}$ may differ between a and b .

Relation to cylinder events. Let $E_{\mathcal{S},a}^{\text{sing}} : \text{Val}(\mathcal{S}) \rightarrow \text{Bool}$ be the singleton event that selects $a|_{\mathcal{S}}$. Then:

$$\Pi_{b:\Omega} F_{\mathcal{S},a}(b) = (E_{\mathcal{S},a}^{\text{sing}})^{\uparrow}(b).$$

Thus a fibre at a complete assignment is precisely the cylinder lift of its restricted singleton event.

Example. Continue with the coin and die variables of Definitions A.3.22 and A.3.25. Let $\mathcal{D}$ select only the die coordinate and take

$$a = (\text{heads}, \text{faceUp}(2)).$$

Then $a|_{\mathcal{D}} = \text{faceUp}(2)$, and $F_{\mathcal{D},a}$ selects exactly

$$(\text{heads}, \text{faceUp}(2)) \quad \text{and} \quad (\text{tails}, \text{faceUp}(2)).$$

The die value is fixed, while the unselected coin coordinate may vary.

Remark A.3.27 (Fibre partitions). Let $\mathcal{S}$ be a coordinate selection as in Definition A.3.26, with complete-assignment type Ω , and let

$$\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega)$$

be an urn-generated probability assignment in the sense of Definition A.3.11. The fibre construction of Definition A.3.26 gives one fibre event for every selected assignment $s : \text{Val}(\mathcal{S})$. These events are pairwise disjoint and jointly exhaustive: every complete assignment $b : \Omega$ is selected by exactly the fibre indexed by $\pi_{\mathcal{S}}(b)$.

Let $E : \text{Val}(\mathcal{S}) \rightarrow \text{Bool}$ be a decidable event, with cylinder lift $E^{\uparrow}$ as in Definition A.3.25. Finite additivity of Remark A.3.16 gives

$$P_{\mu}(E^{\uparrow}) \sim \sum_{s:\text{Val}(\mathcal{S})} \mathbb{1}_E(s) P_{\mu}(\text{Fibre}_{\mathcal{S}}(s)).$$

For a complete assignment $v : \Omega$, let $E_v^{\text{sing}} : \Omega \rightarrow \text{Bool}$ be the singleton event that returns **true** exactly on v . The same additivity applied inside one fibre gives

$$P_{\mu}(\text{Fibre}_{\mathcal{S}}(s)) \sim \sum_{v:\Omega} \mathbb{1}_{\text{Fibre}_{\mathcal{S}}(s)}(v) P_{\mu}(E_v^{\text{sing}}).$$

Definition A.3.28 (Local event). A local event is defined directly on the assignments to one selected family of coordinates.

Local-event type. Let $\mathcal{S}$ be a coordinate selection of $\mathcal{V}$ in the sense of Definition A.3.24. Define:

$$\text{LocalEvent}(\mathcal{S}) := \text{Val}(\mathcal{S}) \rightarrow \text{Bool}.$$

For $E : \text{LocalEvent}(\mathcal{S})$, its event on complete assignments is the cylinder lift:

$$E^\uparrow(v) := E(\pi_{\mathcal{S}}(v)).$$

Hence changing coordinates outside $\mathcal{S}$ cannot change the lifted event.

Lean implementation. Lean also supports the equivalent full-assignment presentation. For a Boolean node selection, the types **AssignmentsAgreeOn** and **EventDependsOnlyOn** implement selected-coordinate agreement and constancy on its fibres.

Definition A.3.29 (Rectangular event). A rectangular event imposes one Boolean event independently at every coordinate of a finite variable family.

Data. Let $\mathcal{V}$ be a finite family of variables with index type I and joint-assignment type $\Omega := \text{Val}(\mathcal{V})$, as in Definition A.3.22. Supply a coordinate-event family:

$$A : \Pi_{i:I}(\text{Val}(i) \rightarrow \text{Bool}).$$

Thus $A(i)$ is the event selected at coordinate i .

Classifier. Define the joint classifier:

$$\text{Rect}_A : \Omega \rightarrow \text{Bool}$$

by finite elimination over the canonical enumeration of I . In the finite-telescope presentation of Remark A.1.22, the empty family returns **true** and adjoining one final coordinate uses **and_{Bool}** from Definition A.1.20:

$$\begin{aligned} \text{Rect}_{()}(\star) &:= \text{true}, \\ \text{Rect}_{(A',A)}(\mathbf{v}', x) &:= \text{and}_{\text{Bool}}(\text{Rect}_{A'}(\mathbf{v}'), A(x)). \end{aligned}$$

Here $\star$ is the unique assignment to the empty telescope.

Characterisation. For a complete assignment $\mathbf{v} : \Omega$, the recursive definition gives:

$$\text{Id}_{\text{Bool}}(\text{Rect}_A(\mathbf{v}), \text{true}) \iff \Pi_{i:I} \text{Id}_{\text{Bool}}(A(i)(\mathbf{v}(i)), \text{true}).$$

This finite characterisation is proved in Proposition B.6.2. Equivalently, lift each $A(i)$ to a cylinder event on Ω using Definition A.3.25; then Rect_A is their finite intersection. The dependent product on the right contains coordinatewise evidence; it is not the rectangular event itself, which remains a Boolean classifier.

Example. Continue with the coin and die variables C and D of Definition A.3.22. Let the coin event accept both **heads** and **tails**, and let the die event accept only **faceUp(2)**. Their rectangular event selects exactly

$$(\text{heads}, \text{faceUp}(2)) \quad \text{and} \quad (\text{tails}, \text{faceUp}(2)).$$

This is also the cylinder event in the example of Definition A.3.25: the die coordinate is restricted while the coin coordinate remains unrestricted.

A. Definitions

By contrast, the joint event from Definition A.3.22 that selects exactly

$$(\text{heads}, \text{faceUp}(2)) \quad \text{and} \quad (\text{tails}, \text{faceUp}(3))$$

is not rectangular. Any coordinate events accepting these two assignments would have to accept both coin faces and both displayed die faces, thereby also selecting

$$(\text{heads}, \text{faceUp}(3)) \quad \text{and} \quad (\text{tails}, \text{faceUp}(2)).$$

Remark A.3.30 (Coordinate selections and events). The constructions of Definitions A.3.25, A.3.26, A.3.28 and A.3.29 are compared here. Let C and D be the coin and die variables of Definition A.3.22, let $\mathcal{S}$ select only the die coordinate, and let

$$a = (\text{heads}, \text{faceUp}(2)).$$

Write Ω for the joint coin-and-die assignment type. The third columns below use the same 2×6 board: coin rows H, T and die columns 1, ..., 6. Axes and columns mark selected-assignment data; filled cells mark complete assignments.

Selections.

Construction	Type	Coin-die example
$\text{Val}(\mathcal{S})$ Definition A.3.24	$\Pi_{k:\text{Fin}(m_{\mathcal{S}})} \text{Val}(j_{\mathcal{S}}(k))$	
$\pi_{\mathcal{S}}$ Definition A.3.24	$\Omega \rightarrow \text{Val}(\mathcal{S})$	
$\text{LocalEvent}(\mathcal{S})$ Definition A.3.28	$\text{Val}(\mathcal{S}) \rightarrow \text{Bool}$	

In the final row, the marked column depicts the cylinder lift of the local die event selecting $\text{faceUp}(2)$.

Events. All of the following are classifiers on full Ω . Let $E : \text{DieFace} \rightarrow \text{Bool}$ select only $\text{faceUp}(2)$, let $s = \text{faceUp}(2)$, and let A accept both coin faces and only that die face. Then each row selects the same two assignments:

Construction	Holds at $v : \Omega$ when	Coin-die example
$E^\uparrow$ Definition A.3.25	$E(\pi_{\mathcal{S}}(v))$	
$\text{Fibre}_{\mathcal{S}}(s)$ Definition A.3.26	$\pi_{\mathcal{S}}(v) = s$	
$F_{\mathcal{S},a}$ Definition A.3.26	$\pi_{\mathcal{S}}(v) = \pi_{\mathcal{S}}(a)$	
Rect_A Definition A.3.29	every $A_i(v_i)$	

The joint event selecting exactly $(\text{heads}, \text{faceUp}(2))$ and $(\text{tails}, \text{faceUp}(3))$ is none of these four and is not rectangular. It is also not local on $\mathcal{S}$, because it splits two die columns.

Thus

$$a : \Omega \xrightarrow{\pi} a|_{\mathcal{S}} \longrightarrow F_{\mathcal{S},a} = \text{Fibre}_{\mathcal{S}}(a|_{\mathcal{S}}) = (E^{\text{sing}})^\uparrow.$$

Every local event has a cylinder lift, and that lift is constant on fibres. A rectangular event is a finite intersection of coordinate cylinders. The probabilities of this partition are those of Remark A.3.27.

Definition A.3.31 (Independence of variable families). Independence of selected variable families requires every pair of events on those families to satisfy the corresponding event-level probability equation after cylinder lifting.

Data. Let $\mathcal{V}$ be a finite family of variables and let

$$\Omega := \text{Val}(\mathcal{V})$$

be its finite joint-assignment type from Definition A.3.22. Choose an urn-generated probability assignment

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega)$$

in the sense of Definition A.3.11. The finiteness of Ω supplies the finite domain of assignments and decidable events; the assignment μ is additional probabilistic data, whose labelling map $f : \text{Fin}(N) \rightarrow \Omega$ labels every urn cell with a full joint assignment.

Independence. Two selected variable families $\mathcal{X}$ and $\mathcal{Y}$ are independent under μ when every pair of joint events

$$E : \text{Val}(\mathcal{X}) \rightarrow \text{Bool}, \quad F : \text{Val}(\mathcal{Y}) \rightarrow \text{Bool}$$

satisfies the event-independence condition of Definition A.3.20 after cylinder lifting:

$$E^\uparrow \perp\!\!\!\perp_\mu F^\uparrow.$$

Here the arrows denote the cylinder lifts of Definition A.3.25. Unfolding the event-independence notation of Definition A.3.20, the displayed condition is

$$P_\mu(E^\uparrow \cap F^\uparrow) \sim P_\mu(E^\uparrow)P_\mu(F^\uparrow).$$

Conditional independence. The families $\mathcal{X}$ and $\mathcal{Y}$ are conditionally independent given a selected family $\mathcal{Z}$, written

$$\mathcal{X} \perp\!\!\!\perp_\mu \mathcal{Y} \mid \mathcal{Z},$$

when, for every $z : \text{Val}(\mathcal{Z})$ and every pair E, F as above,

$$E^\uparrow \perp\!\!\!\perp_\mu F^\uparrow \mid (E_{\mathcal{Z},z}^{\text{sing}})^\uparrow.$$

The event $E_{\mathcal{Z},z}^{\text{sing}} : \text{Val}(\mathcal{Z}) \rightarrow \text{Bool}$ is the singleton event that returns `true` exactly on z . It is decidable because $\text{Val}(\mathcal{Z})$ is finite, and $(E_{\mathcal{Z},z}^{\text{sing}})^\uparrow$ is its cylinder lift to Ω . Unfolding the conditional-independence notation of Definition A.3.20, the displayed condition is

$$P_\mu(E^\uparrow \cap F^\uparrow \cap (E_{\mathcal{Z},z}^{\text{sing}})^\uparrow) P_\mu((E_{\mathcal{Z},z}^{\text{sing}})^\uparrow) \sim P_\mu(E^\uparrow \cap (E_{\mathcal{Z},z}^{\text{sing}})^\uparrow) P_\mu(F^\uparrow \cap (E_{\mathcal{Z},z}^{\text{sing}})^\uparrow).$$

Example. Continue with the coin variable C and die variable D . Let μ_C be the two-cell urn containing `heads` and `tails` once each, and let μ_D be the six-cell

A. Definitions

urn containing each die face once. Under the coin-and-die identification of the full assignment type from Definition A.3.22, their product urn

$$\mu := \mu_C \otimes \mu_D : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega)$$

contains each of the twelve coin-and-die assignments once. Let H_C be the event that the coin is **heads** and let S_D be the event that the die is **faceUp(6)**. Their cylinder lifts satisfy

$$P_\mu(H_C^\uparrow) \sim \frac{1}{2}, \quad P_\mu(S_D^\uparrow) \sim \frac{1}{6},$$

while their intersection selects exactly one of the twelve assignments:

$$\begin{aligned} P_\mu(H_C^\uparrow \cap S_D^\uparrow) &\sim \frac{1}{12} \\ &\sim \frac{1}{2} \cdot \frac{1}{6} \\ &\sim P_\mu(H_C^\uparrow) P_\mu(S_D^\uparrow). \end{aligned}$$

By Definition A.3.20, this identity means that $H_C^\uparrow$ and $S_D^\uparrow$ are independent under μ . This verifies one pair of events. More generally, Proposition B.6.11 proves the same product identity for every pair of coin and die events; hence the coin and die variable families are independent under μ in the sense of this definition.

Remark A.3.32 (Independence notation). The symbol $\perp\!\!\!\perp$ has two readings in this book. A probability subscript denotes probabilistic independence: Definition A.3.20 defines it for events, and Definition A.3.31 lifts it to variable families. A graph subscript instead denotes graphical d-separation, for example $(X \perp\!\!\!\perp Y \mid Z)_{\mathcal{G}}$. The subscript therefore determines which meaning is intended. When the probability assignment is induced by a structural causal model $\mathcal{M}$, it is written $\mu_{\mathcal{M}}$.

Definition A.3.33 (Marginalisation). Marginalisation forgets selected coordinates of a finite joint assignment while retaining their contribution to the probability of the remaining coordinates. Its components are as follows:

Data. Let $\mathcal{V}$ and

$$\Omega := \text{Val}(\mathcal{V})$$

be as in Definition A.3.22, and choose an urn-generated probability assignment

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega)$$

as in Definition A.3.31. For a coordinate selection $\mathcal{Y}$ of $\mathcal{V}$, Definition A.3.24 supplies the projection $\pi_{\mathcal{Y}} : \Omega \rightarrow \text{Val}(\mathcal{Y})$.

Marginal assignment. Retain the same urn cells and relabel each one through the projection:

$$\mu_{\mathcal{Y}} := (N, h_N, \pi_{\mathcal{Y}} \circ f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{Y})).$$

Event probability. This construction is the urn pushforward of Proposition B.6.15. For every decidable event $E : \text{Val}(\mathcal{Y}) \rightarrow \text{Bool}$, both E under $\mu_{\mathcal{Y}}$ and its cylinder lift $E^\uparrow = E \circ \pi_{\mathcal{Y}}$ under μ select cell $i : \text{Fin}(N)$ exactly when

$$E(\pi_{\mathcal{Y}}(f(i))) = \text{true}.$$

That proposition therefore gives

$$P_{\mu_Y}(E) \sim P_{\mu}(E^{\uparrow}).$$

Singleton reading. In particular, if $E_{Y,y}^{\text{sing}} : \text{Val}(Y) \rightarrow \text{Bool}$ is the singleton event selecting y , then the event-probability identity above together with Definition A.3.26 gives

$$P_{\mu_Y}(E_{Y,y}^{\text{sing}}) \sim P_{\mu}(\text{Fibre}_Y(y)).$$

Constructive urn assumptions

Definition A.3.34 (Constructive Clayton–Waddington urn assumptions). Definition A.3.11 assigns the ratio k/N to an event selecting k cells from an N -cell urn. The present definition records the qualitative assumptions under which the rescaling from represented urn plausibilities to these rational ratios is well-defined and reflects strict plausibility comparisons.

In this monograph, these assumptions form a finite constructive adaptation of the urn-based argument of Clayton and Waddington [CW17]. These assumptions are not a verbatim restatement of the original argument; their main differences are summarised in Table B.5, and the corresponding finite proof is developed in Section B.6.

Data. Let Plaus be a type of qualitative plausibility values. For every $N : \mathbb{N}$, let

$$\text{plaus}_N : (\text{Fin}(N) \rightarrow \text{Bool}) \rightarrow \text{Plaus}$$

assign a qualitative value to every decidable event on the N -cell type. Equip Plaus with two type-valued binary relations

$$\approx_U, \prec_U : \text{Plaus} \rightarrow \text{Plaus} \rightarrow \mathcal{U},$$

read respectively as ‘qualitative equivalence’ and ‘strict qualitative comparison’.

Canonical events. Let

$$N, k : \mathbb{N}, \quad h_N : N > 0, \quad h_k : k \leq N.$$

Define the canonical decidable k -cell event:

$$C_{N,k} : \text{Fin}(N) \rightarrow \text{Bool}$$

to return true on the first k indices and false on the remaining indices. Define

$$U_N(k) := \text{plaus}_N(C_{N,k}) : \text{Plaus},$$

where $U_N(k)$ denotes the qualitative plausibility of the canonical event; the link with probability is established after the assumptions.

Assumptions. The constructive Clayton–Waddington assumptions used here are:

A.3.34.1. Qualitative coherence: The relation $\approx_U$ is an equivalence relation, and $\prec_U$ is irreflexive and transitive. Moreover, strict comparison respects qualitative equivalence in both arguments. More explicitly, for all $a, a', b, b' : \text{Plaus}$,

$$\begin{aligned} a \approx_U a' \wedge a \prec_U b &\implies a' \prec_U b, \\ b \approx_U b' \wedge a \prec_U b &\implies a \prec_U b'. \end{aligned}$$

A. Definitions

Since $\approx_U$ is symmetric, the same substitutions can be made in the reverse direction. Consequently, if $a \approx_U a'$ and $b \approx_U b'$, then:

$$a \prec_U b \iff a' \prec_U b'.$$

It follows that strict comparison depends only on the qualitative plausibilities represented, not on the particular descriptions chosen for them. As such, strict comparison depends only on the $\approx_U$ -equivalence classes of its arguments: replacing either argument by an equivalent representative does not change the comparison. Equivalently, $\prec_U$ induces a well-defined strict order on the qualitative-equivalence classes.

A.3.34.2. Count invariance: let $N : \mathbb{N}$ and let $E, F : \text{Fin}(N) \rightarrow \text{Bool}$ be arbitrary Boolean-valued functions. Similar to Definition A.3.11, let:

$$m_E := \sum_{i:\text{Fin}(N)} \mathbb{1}_E(i), \quad m_F := \sum_{i:\text{Fin}(N)} \mathbb{1}_F(i),$$

denote the numbers of cells selected by E and F , respectively. Events with the same selected-cell count receive qualitatively equivalent plausibilities:

$$m_E = m_F \longrightarrow \text{plaus}_N(E) \approx_U \text{plaus}_N(F).$$

Hence, for a fixed urn size N , only the number of selected cells matters, not which particular cells are selected. Consequently, replacing an event by a pointwise equal event or permuting the labels of its cells does not change its qualitative plausibility, so every event is qualitatively equivalent to its canonical same-count event; see Proposition B.6.8.

A.3.34.3. Strict monotonicity: let

$$N, k, \ell : \mathbb{N}, \quad h_N : N > 0, \quad h_k : k \leq N, \quad h_\ell : \ell \leq N.$$

If $k < \ell$, then

$$U_N(k) \prec_U U_N(\ell);$$

selecting more cells in the same urn is therefore strictly more plausible.

A.3.34.4. Refinement invariance: let

$$N, k, c : \mathbb{N}, \quad h_N : N > 0, \quad h_k : k \leq N, \quad h_c : c > 0.$$

Replacing every cell by c indistinguishable subcells preserves plausibility:

$$U_N(k) \approx_U U_{cN}(ck).$$

This permits two urn ratios to be compared after passing to a common refined urn without changing either represented plausibility.

Assumption record. The displayed data and four clauses define a record `ConstructiveUrnScale`. Its computational field is the family plaus_N ; its remaining fields are the proof-carrying coherence, count-invariance, strict-monotonicity and refinement-invariance assumptions stated above.

Lean implementation. The assumption record is `QualitativeUrnScale`. The same development defines `UrnRatio.canonicalEvent` and

`QualitativeUrnScale.ratioPlaus`. Its canonical rational inhabitant is `rationalScale`, as stated in Proposition B.6.6. The product-cell constructions are `FiniteCellProduct.equivalence` and `FiniteCellProduct.encodedRectangularEvent`; their mathematical properties are established in Proposition B.6.11.

Remark A.3.35 (Consequences of the urn assumptions). The four clauses of Definition A.3.34 are the primitive constructive Clayton–Waddington assumptions. The following consequences connect the qualitative and rational values of represented urn events.

Rational rescaling. Let the urn sizes and cell counts be:

$$N, M, k, \ell : \mathbb{N}, \quad h_N : N > 0, \quad h_M : M > 0, \quad h_k : k \leq N, \quad h_\ell : \ell \leq M.$$

The qualitative assumptions yield:

$$\begin{aligned} U_N(k) \approx_U U_M(\ell) &\iff kM = \ell N, \\ U_N(k) \prec_U U_M(\ell) &\iff kM < \ell N. \end{aligned}$$

The rational rescaling is therefore defined by:

$$\rho(U_N(k)) := (k, N, h_N) : \mathbb{Q}_{\geq 0}.$$

It is well-defined up to $\sim$ on the $\approx_U$ -equivalence classes and reflects the strict cross-multiplication order of represented ratios. These order characterisations and the well-definedness of ρ are proved in Proposition B.6.9.

Connection to labelled urns. Following Definition A.3.11, let X be any type and let μ be an urn-generated probability assignment:

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X).$$

For any Boolean-valued function $E : X \rightarrow \text{Bool}$, define the induced cell event:

$$E_f : \text{Fin}(N) \rightarrow \text{Bool}, \quad E_f(i) := E(f(i)).$$

Its selected-cell count is:

$$m_E := \sum_{i : \text{Fin}(N)} \mathbf{1}_{E_f}(i) = \sum_{i : \text{Fin}(N)} \mathbf{1}_E(f(i)).$$

Count invariance gives $\text{plaus}_N(E_f) \approx_U U_N(m_E)$, and the exact bridge to the event probability $P_\mu(E)$ from Definition A.3.11 is:

$$\boxed{P_\mu(E) \sim \rho(U_N(m_E))}.$$

It follows that for a fixed urn size N , the qualitative and rational values depend on E and f only through the selected-cell count m_E . The reduction to this count is proved in Proposition B.6.8; combining it with the rescaling of Proposition B.6.9 gives the displayed bridge.

Lean implementation. Lean represents the pullback to cells by `UrnRatio.inducedCellEvent` and packages its selected count by `UrnRatio.ofUrnEvent`. The corresponding correctness results are collected in Propositions B.6.8 and B.6.9.

A. Definitions

Scope. No finiteness or decidable equality assumption on X is required. The existence of a positive urn labelling itself implies that X is inhabited; conversely, one chosen $x : X$ yields an urn-generated probability assignment for every positive N through the trivial labelling $f(i) := x$. Naturally, trivial labelling leads to a trivial distribution; nontrivial distributions require additional labels.

Finite complements. The rescaling ρ is defined only for represented finite urn events; it does not assign a probability to an arbitrary, possibly undecidable proposition A . For a decidable urn event E , however, the complement E^c is again decidable, and:

$$P_\mu(E) + P_\mu(E^c) \sim 1.$$

The finite-complement result is proved in Corollary B.6.10.

Product urns. Multiplication requires no additional qualitative assumption. Let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$ be an M -cell urn assignment and let $\nu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(Y)$ be an N -cell urn assignment. If $E : X \rightarrow \text{Bool}$ selects k cells of μ and $F : Y \rightarrow \text{Bool}$ selects ℓ cells of ν , then the product type:

$$\text{Fin}(M) \times \text{Fin}(N)$$

serves as the cell type of the constructed product assignment $\mu \otimes \nu$. The rectangular event is defined by:

$$(E \times F)(x, y) := \text{and}_{\text{Bool}}(E(x), F(y)).$$

It selects $k\ell$ of the MN product cells, and hence:

$$P_{\mu \otimes \nu}(E \times F) \sim P_\mu(E)P_\nu(F).$$

This identity concerns the product assignment constructed from the two urns; it does not assert that an arbitrary joint probability assignment on the product type factorises. The construction of product urns and their scope are proved in Proposition B.6.11.

A.4 Finite causal models

This section fixes the finite graphical and structural objects used for causal reasoning in the book. Throughout, the standard causal phrase *probability distribution* denotes a finite rational probability assignment of Section A.3.

Nodes, edges and paths

Definition A.4.1 (Formal node type). The graph definitions below use a canonical finite presentation of the variable names.

Node type. Let $\mathcal{V}$ be a finite family of variables with finite type I of variable names and value-type assignment $\text{Val} : I \rightarrow \mathcal{U}$, as in Definitions A.3.21 and A.3.22. Choose the finiteness presentation of I from Definition A.1.19, and write

$$V := \text{Fin}(n)$$

for its index type, with maps:

$$\text{encode} : I \rightarrow V, \quad \text{decode} : V \rightarrow I$$

and the two round-trip identity proofs of that definition. The type I contains the descriptive variable names, while V is the *formal node type* used by the finite graph records. An inhabitant $v : V$ represents the variable named by $\text{decode}(v) : I$, whose value type is $\text{Val}(\text{decode}(v))$. The round-trip proofs ensure that this indexed presentation contains every variable name exactly once.

Example. Let I_{Mira} be the type of eight variable names:

Pr, Qu, Top, Com, Fit, Short, Fund, Off.

Enumerating them in the displayed order gives the formal node type $V_{\text{Mira}} := \text{Fin}(8)$. Below, the same names also abbreviate their encoded inhabitants of V_{Mira} .

Definition A.4.2 (Node selection). A node selection is a Boolean classifier on the formal node type of Definition A.4.1, together with its canonical finite indexed presentation.

Classifier. Since V is finite, a finite node selection, called a *node set* in conventional causal-graph terminology, is a decidable event on V in the sense of Definition A.3.2:

$$S : V \rightarrow \text{Bool}.$$

Membership $v \in S$ abbreviates $S(v) = \text{true}$. All Boolean operations and disjointness are inherited from Definition A.3.3, instantiated at $X := V$. In particular, $\emptyset_V$ denotes the empty selection; when V is clear, write simply $\emptyset$. For $i : V$, the singleton selection $\{i\} : V \rightarrow \text{Bool}$ is the decidable equality classifier that returns true exactly at i . For finitely many displayed nodes, braces abbreviate an iterated union of singleton selections:

$$\{i_1, \dots, i_k\} := \{i_1\} \cup \dots \cup \{i_k\}.$$

Selected-family presentation. Retain exactly those formal nodes $v : V$ for which $S(v) = \text{true}$, in their ambient index order. Filtering the canonical enumeration $0, \dots, n - 1$ gives an embedding:

$$j_S : \text{Fin}(m_S) \rightarrow V, \quad j_S(k) = j_S(\ell) \rightarrow k = \ell.$$

Here m_S is the number of nodes with $S(v) = \text{true}$. The type $\text{Fin}(m_S)$ indexes those selected positions, while j_S identifies the formal node at each position. Because the filtered list retains the ambient order, this embedding also reflects strict order:

$$h_{\text{reflect}}^S : \Pi_{k, \ell : \text{Fin}(m_S)} \text{rank}_n(j_S(k)) <_{\mathbb{N}} \text{rank}_n(j_S(\ell)) \rightarrow \text{rank}_{m_S}(k) <_{\mathbb{N}} \text{rank}_{m_S}(\ell).$$

This field allows an induced restriction to inherit the chosen topological order of its ambient graph. Composing j_S with the decoding map gives the selected variable-name embedding:

$$\bar{j}_S : \text{Fin}(m_S) \rightarrow I, \quad \bar{j}_S(k) := \text{decode}(j_S(k)),$$

which is the selected-subfamily data of Definition A.3.24. Its joint-assignment type and coordinate projection are the constructions of that definition; Definition A.3.25 supplies the corresponding lifted events. Pure graph definitions use only S and j_S . Conversely, finite search converts an embedding

A. Definitions

$j : \text{Fin}(m) \rightarrow V$ into the Boolean classifier that returns `true` exactly on its image. The constructive correspondence between these presentations is proved in Proposition B.7.4.

Running example. For the formal node type V_{Mira} of Definition A.4.1, selecting `Pr` and `Qu` gives a classifier $S_{\text{PQ}} : V_{\text{Mira}} \rightarrow \text{Bool}$ and an order-preserving embedding $j_{\text{PQ}} : \text{Fin}(2) \rightarrow V_{\text{Mira}}$.

Lean implementation. A node selection is the abbreviation `NodeSet`.

Definition A.4.3 (Proof-valued relation step). A proof-valued step records that a Boolean relation selects one ordered pair.

Data. Fix a formal node type V and a Boolean relation $R : V \times V \rightarrow \text{Bool}$. For $X, Y : V$, define:

$$\text{Step}_R(X, Y) := \text{Id}_{\text{Bool}}(R(X, Y), \text{true}).$$

An inhabitant of $\text{Step}_R(X, Y)$ is evidence that R selects the ordered pair (X, Y) . If R is irreflexive, such evidence also entails that its endpoints are distinct.

Definition A.4.4 (Directed and bidirected edge packages). Directed and bidirected edge packages record the two kinds of graphical connection used by finite causal graphs.

Data. Fix the formal node type $V = \text{Fin}(n)$ of Definition A.4.1.

Directed classifier. A directed edge relation is a Boolean classifier:

$$E^\rightarrow : V \times V \rightarrow \text{Bool}.$$

Write $X \rightarrow_E Y$ when $E^\rightarrow(X, Y) = \text{true}$. This says that there is a directed edge from X to Y ; it does not yet impose an order on the two nodes.

When an ordered directed relation is required, supply evidence of type:

$$\text{Topological}(E^\rightarrow) := \Pi_{X, Y : V} \text{Step}_{E^\rightarrow}(X, Y) \rightarrow \text{rank}_n(X) <_{\mathbb{N}} \text{rank}_n(Y),$$

where rank_n and $<_{\mathbb{N}}$ are defined in Remark A.1.18.

Ordered directed package. Define `OrderedDirectedEdges(V)` as a record type with fields:

$$\begin{aligned} E^\rightarrow &: V \times V \rightarrow \text{Bool}, \\ h_{\text{top}} &: \text{Topological}(E^\rightarrow). \end{aligned}$$

Thus h_{top} certifies that every selected directed edge points from an earlier node to a later one. A bare, directed classifier need not admit such evidence.

Bidirected package. Define `BidirectedEdges(V)` as the record type with fields:

$$\begin{aligned} E^{\leftrightarrow} &: V \times V \rightarrow \text{Bool}, \\ h_{\text{sym}} &: \Pi_{X, Y : V} \text{Step}_{E^{\leftrightarrow}}(X, Y) \rightarrow \text{Step}_{E^{\leftrightarrow}}(Y, X), \\ h_{\text{irr}} &: \Pi_{X : V} \text{Id}_{\text{Bool}}(E^{\leftrightarrow}(X, X), \text{false}). \end{aligned}$$

Write $X \leftrightarrow_E Y$ when $E^{\leftrightarrow}(X, Y) = \text{true}$.

Definition A.4.5 (Graph adjacency). Adjacency forgets the orientation of directed edges and may also include bidirected edges.

Data. Let $D : \text{OrderedDirectedEdges}(V)$ and $B : \text{BidirectedEdges}(V)$ be edge packages in the sense of Definition A.4.4.

Directed adjacency. Directed adjacency forgets arrow orientation:

$$\text{Adj}_D^{\rightarrow}(X, Y) := \text{or}_{\text{Bool}}(E_D^{\rightarrow}(X, Y), E_D^{\rightarrow}(Y, X)).$$

Full adjacency. Full adjacency additionally includes bidirected edges:

$$\text{Adj}_{D,B}(X, Y) := \text{or}_{\text{Bool}}(\text{Adj}_D^{\rightarrow}(X, Y), E_B^{\leftrightarrow}(X, Y)).$$

It returns `true` exactly when the two nodes are joined by a directed edge in either orientation or by a bidirected edge.

Running example. On V_{Mira} from Definition A.4.1, let $E_{\text{Mira}}^{\rightarrow}$ return `true` exactly on:

$$\begin{aligned} &(\text{Pr}, \text{Short}), (\text{Qu}, \text{Short}), (\text{Top}, \text{Fit}), (\text{Com}, \text{Fit}), \\ &(\text{Fit}, \text{Short}), (\text{Top}, \text{Fund}), (\text{Short}, \text{Off}), (\text{Fund}, \text{Off}). \end{aligned}$$

The displayed node order supplies $h_{\text{top}}^{\text{Mira}}$, so these data form $D_{\text{Mira}} : \text{OrderedDirectedEdges}(V_{\text{Mira}})$. Let $E_{\text{Mira}}^{\leftrightarrow}$ return `true` exactly on (Pr, Qu) and (Qu, Pr) . Its evident symmetry and irreflexivity form $B_{\text{Mira}} : \text{BidirectedEdges}(V_{\text{Mira}})$. These packages determine the directed, bidirected and adjacency relations of Mira’s finite causal graph.

Lean implementation. The ordered directed package comprises the fields `directed` and `directed_earlier` of `ObservedSignature`.

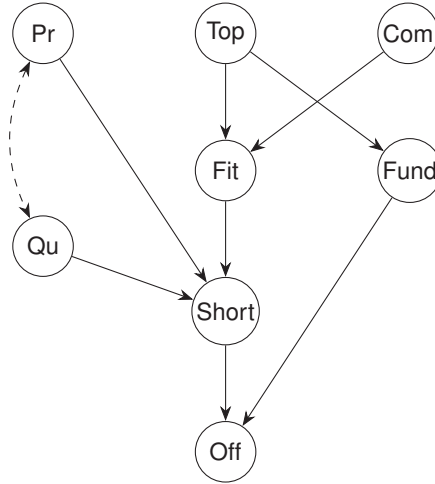


Figure A.2: Mira’s finite causal graph, shown through its directed and bidirected edge packages. The dashed line is the bidirected edge $\text{Pr} \leftrightarrow \text{Qu}$.

Definition A.4.6 (Chains, forks and colliders). Chains, forks and colliders are local configurations of three nodes in an ordered directed-edge package.

Data. Let $D : \text{OrderedDirectedEdges}(V)$ and let $A, B, C : V$. Using the

A. Definitions

proof-valued steps of Definition A.4.3, define:

$$\begin{aligned}\text{Chain}_D(A, B, C) &:= (\text{Step}_{E_D}(A, B) \times \text{Step}_{E_D}(B, C)) \\ &\quad + (\text{Step}_{E_D}(C, B) \times \text{Step}_{E_D}(B, A)), \\ \text{Fork}_D(A, B, C) &:= \text{Step}_{E_D}(B, A) \times \text{Step}_{E_D}(B, C), \\ \text{Collider}_D(A, B, C) &:= \text{Step}_{E_D}(A, B) \times \text{Step}_{E_D}(C, B).\end{aligned}$$

Their graphical readings are:

- *chain*: $A \rightarrow B \rightarrow C$ or $A \leftarrow B \leftarrow C$;
- *fork*: $A \leftarrow B \rightarrow C$; and
- *collider*: $A \rightarrow B \leftarrow C$.

These three configurations are displayed in the figure below. Chains and forks are non-collider configurations; colliders behave differently under conditioning.

Running example. For D_{Mira} , the fragments

$$\text{Top} \rightarrow \text{Fit} \rightarrow \text{Short}, \quad \text{Fit} \leftarrow \text{Top} \rightarrow \text{Fund}, \quad \text{Top} \rightarrow \text{Fit} \leftarrow \text{Com}$$

inhabit respectively a chain, a fork and a collider type.

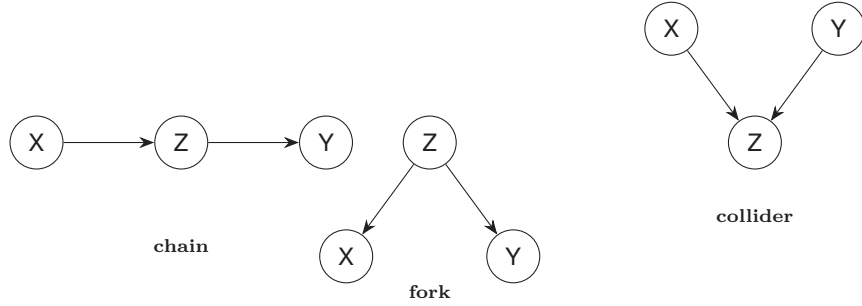


Figure A.3: The three local directed configurations of Definition A.4.6. The middle node is a non-collider in a chain or fork and a collider in the final configuration.

Definition A.4.7 (Paths). A path packages a finite node sequence together with evidence for its endpoints and every consecutive relation step.

Data. Fix the formal node type V of Definition A.4.1. Let $R : V \times V \rightarrow \text{Bool}$ be any Boolean relation, such as a directed classifier, a bidirected classifier or the full adjacency classifier of Definition A.4.5.

Consecutive-step evidence. Using list elimination from Definition A.1.24, define the type family:

$$\text{Consecutive}_R : \text{List}(V) \rightarrow \mathcal{U}$$

by:

$$\begin{aligned}\text{Consecutive}_R(\text{nil}) &:= \text{Fin}(1), \\ \text{Consecutive}_R(\text{cons}(X, \text{nil})) &:= \text{Fin}(1), \\ \text{Consecutive}_R(\text{cons}(X, \text{cons}(Y, p))) &:= \text{Step}_R(X, Y) \times \text{Consecutive}_R(\text{cons}(Y, p)).\end{aligned}$$

The empty and singleton lists contain no consecutive pair that could fail the relation, so their evidence types are inhabited. For a longer list, an inhabitant proves that the first pair passes R and that every pair in the remaining list does likewise. Each recursive call removes the first entry of that finite list. Evaluation therefore terminates for every choice of R ; no termination property of R is required.

Relation paths. For $X, Y : V$, define the record type $\text{Path}_R(X, Y)$ with fields:

$$\begin{aligned} \text{nodes} &: \text{List}(V), \\ h_{\text{start}} &: \text{head?}(\text{nodes}) = \text{some}(X), \\ h_{\text{end}} &: \text{last?}(\text{nodes}) = \text{some}(Y), \\ h_{\text{step}} &: \text{Consecutive}_R(\text{nodes}). \end{aligned}$$

The endpoint fields use the optional list endpoints of Remark A.1.26; either equation entails that the node list is nonempty. A singleton list consequently gives a trivial path from a node to itself and contains no edge. For $q : \text{Path}_R(X, Y)$, named field projection in the sense of Remark A.1.11 gives $\text{nodes}(q) : \text{List}(V)$.

Positive paths. When a path must contain at least one edge, define $\text{Path}_R^+(X, Y)$ as the record type with fields:

$$\begin{aligned} \text{path} &: \text{Path}_R(X, Y), \\ \text{first} &: V, \\ \text{second} &: V, \\ \text{rest} &: \text{List}(V), \\ h_{\text{shape}} &: \text{Id}_{\text{List}(V)} \left(\text{nodes}(\text{path}), \text{cons}(\text{first}, \text{cons}(\text{second}, \text{rest})) \right). \end{aligned}$$

Thus a positive path contains an ordinary path and evidence that its node sequence has at least two entries. The field rest is the possibly empty list of all nodes after the required first two; it is nil when the path contains exactly one edge. Transporting $h_{\text{step}}(\text{path})$ along h_{shape} exposes an inhabitant of $\text{Step}_R(\text{first}, \text{second})$ and the consecutive evidence for the remaining list.

Edge-specific paths. For $D : \text{OrderedDirectedEdges}(V)$ and $B : \text{BidirectedEdges}(V)$, write:

$$\begin{aligned} \text{Path}_D^{\rightarrow}(X, Y) &:= \text{Path}_{E_D^{\rightarrow}}(X, Y), \\ \text{Path}_B^{\leftrightarrow}(X, Y) &:= \text{Path}_{E_B^{\leftrightarrow}}(X, Y), \\ \text{Path}_{D,B}^{\text{adj}}(X, Y) &:= \text{Path}_{\text{Adj}_{D,B}}(X, Y). \end{aligned}$$

The corresponding positive forms are obtained by replacing Path with Path^+ .

Running example. The node sequence $\text{Top}, \text{Fit}, \text{Short}, \text{Off}$ underlies a positive directed path from Top to Off in D_{Mira} . The sequence Pr, Qu gives a positive bidirected path in B_{Mira} , while $\text{Pr}, \text{Qu}, \text{Short}, \text{Off}$ gives a positive full-adjacency path.

Definition A.4.8 (Parents, ancestors and descendants). Parents, ancestors and descendants depend only on ordered directed edges and their paths.

Data. Fix $D : \text{OrderedDirectedEdges}(V)$ and let $Y : V$. Each family below is a finite node selection on V in the sense of Definition A.4.2.

A. Definitions

Parents. The parent classifier selects exactly the nodes with a directed edge into Y :

$$X \in \text{Pa}_D(Y) \iff E_D^\rightarrow(X, Y) = \text{true}.$$

Ancestors and descendants. Using the directed path type of Definition A.4.7, define:

$$X \in \text{An}_D(Y) \iff \text{Path}_D^\rightarrow(X, Y) \text{ is inhabited,}$$

and:

$$X \in \text{De}_D(Y) \iff \text{Path}_D^\rightarrow(Y, X) \text{ is inhabited.}$$

Thus Y is its own ancestor and descendant because the singleton node list gives a directed path from Y to itself. Paths containing one or more edges account for all other ancestors and descendants.

Ancestors of a selection. For a node selection $W : V \rightarrow \text{Bool}$, define:

$$X \in \text{An}_D(W) \iff \Sigma_{w:V} (W(w) = \text{true}) \times \text{Path}_D^\rightarrow(X, w).$$

Thus $\text{An}_D(W)$ contains precisely the nodes that are ancestors of at least one node selected by W .

Decidability. Equality and finite directed reachability are decidable, so these conditions determine Boolean classifiers. The checked finite reachability construction is recorded in Proposition B.7.13.

Running example. For D_{Mira} :

$$\text{Pa}_{D_{\text{Mira}}}(\text{Short}) = \{\text{Pr}, \text{Qu}, \text{Fit}\}.$$

Moreover, Top is an ancestor of Off and Off is a descendant of Top .

Definition A.4.9 (Directed acyclic graph). A finite directed graph is made acyclic by requiring every selected edge to point forward in a topological ordering.

Data. For a formal node type $V = \text{Fin}(n)$, the ordered directed-edge package of Definition A.4.4 uses the canonical index rank. Define:

$$\text{DAG}(V) := \text{OrderedDirectedEdges}(V).$$

Hence an inhabitant $D = (E_D^\rightarrow, h_{\text{top}}^D) : \text{DAG}(V)$ contains both its directed classifier and its chosen topological-order evidence.

Irreflexivity and acyclicity. The topological witness determines:

$$h_{\text{irr}}^D : \Pi_{X:V} \text{Id}_{\text{Bool}}(E_D^\rightarrow(X, X), \text{false}).$$

A true self-edge would produce $\text{rank}_n(X) <_{\mathbb{N}} \text{rank}_n(X)$, contradicting irreflexivity of strict natural-number order. Similarly, a positive directed path from X to itself would make the rank strictly increase along every edge and, by transitivity, yield the same contradiction. These constructive consequences are proved in Proposition B.7.2. Because every finite cycle-free directed graph admits a topological ordering, this ordered presentation covers the usual finite directed acyclic graphs.

Running example. The package D_{Mira} constructed in Definition A.4.4 inhabits $\text{DAG}(V_{\text{Mira}})$.

Lean implementation. Lean distributes this canonical indexed presentation across the fields `count`, `directed` and `directed_earlier` of `ObservedSignature`.

Remark A.4.10 (Ranked presentation). The node indices of a finite directed graph need not themselves be its topological order. In that case, a separate rank records the required ordering. Define **RankedDAG** as the record type with fields:

$$\begin{aligned} q &: \mathbb{N}, \\ E_D &: \text{Fin}(q) \times \text{Fin}(q) \rightarrow \text{Bool}, \\ \text{rank}_D &: \text{Fin}(q) \rightarrow \mathbb{N}, \\ h_{\text{top}}^D &: \prod_{X,Y:\text{Fin}(q)} \text{Step}_{E_D}(X,Y) \rightarrow \text{rank}_D(X) <_{\mathbb{N}} \text{rank}_D(Y). \end{aligned}$$

The rank need not coincide with the natural index of a node. Its only required role is to increase along directed edges, so the irreflexivity and acyclicity argument of Definition A.4.9 applies unchanged. Conversely, the canonical presentation of that definition determines a ranked presentation by taking $q := n$, $E_D := E_{\vec{D}}$ and $\text{rank}_D := \text{rank}_n$.

Lean implementation. Lean calls this alternative presentation **RankedFinite DAG**. It is defined in `lean/Thesis/Causality/CompactHiddenDAG.lean` and is used for total graphs whose node indices are not required to be their topological order.

Definition A.4.11 (Causal graph). A finite causal graph combines an ordered directed part and a bidirected part over the same formal node type.

Data. Fix a formal node type V . Define **CausalGraph**(V) as the record type with fields:

$$\begin{aligned} \text{directed} &: \text{DAG}(V), \\ \text{bidirected} &: \text{BidirectedEdges}(V). \end{aligned}$$

For $\mathcal{G} : \text{CausalGraph}(V)$, write $D_{\mathcal{G}} := \text{directed}(\mathcal{G})$ and $B_{\mathcal{G}} := \text{bidirected}(\mathcal{G})$. Their classifier projections are denoted $E_{\mathcal{G}}^{\rightarrow} := E_{D_{\mathcal{G}}}^{\rightarrow}$ and $E_{\mathcal{G}}^{\leftrightarrow} := E_{B_{\mathcal{G}}}^{\leftrightarrow}$.

Adjacency and paths. Define:

$$\text{Adj}_{\mathcal{G}} := \text{Adj}_{D_{\mathcal{G}}, B_{\mathcal{G}}}, \quad \text{Path}_{\mathcal{G}}(X, Y) := \text{Path}_{\text{Adj}_{\mathcal{G}}}(X, Y).$$

The first is the full adjacency classifier; the second is the graph-path type of Definition A.4.7 that may traverse either kind of edge while ignoring orientation.

Graphical cases. A graph is *Markovian* when $E_{\mathcal{G}}^{\leftrightarrow}$ is constantly false. It is *semi-Markovian* when the bidirected package may select symmetric, irreflexive bidirected edges. Thus a directed acyclic graph is the directed part of either causal-graph case, while a causal graph additionally records whether bidirected connections are present.

Running example. Define $\mathcal{G}_{\text{Mira}} : \text{CausalGraph}(V_{\text{Mira}})$ by:

$$\begin{aligned} \text{directed}(\mathcal{G}_{\text{Mira}}) &:= D_{\text{Mira}}, \\ \text{bidirected}(\mathcal{G}_{\text{Mira}}) &:= B_{\text{Mira}}. \end{aligned}$$

Its nonempty bidirected classifier makes it semi-Markovian.

Lean implementation. Lean makes the same division. An **ObservedSignature** contains the formal node and value types and the ordered directed part; an **ObservedGraph** over that signature contains the bidirected part.

A. Definitions

Definition A.4.12 (Induced finite causal restriction). An induced restriction selects part of a causal graph, retains every directed or bidirected edge whose two endpoints remain selected, and ignores all other nodes and incident edges.

Data. Let $\mathcal{G} : \text{CausalGraph}(V)$. A finite node classifier $W : V \rightarrow \text{Bool}$ in the sense of Definition A.4.2 determines an embedding:

$$j_W : \text{Fin}(m_W) \rightarrow V$$

and order-reflection evidence h_{reflect}^W . The latter converts an ambient strict-rank comparison between selected nodes into the corresponding comparison between their filtered indices.

Construction. The induced restriction $\mathcal{G} \mid W$ has formal node type $\text{Fin}(m_W)$ and classifiers:

$$\begin{aligned} E_{\mathcal{G} \mid W}^{\rightarrow}(k, \ell) &:= E_{\mathcal{G}}^{\rightarrow}(j_W(k), j_W(\ell)), \\ E_{\mathcal{G} \mid W}^{\leftrightarrow}(k, \ell) &:= E_{\mathcal{G}}^{\leftrightarrow}(j_W(k), j_W(\ell)). \end{aligned}$$

Its topological evidence is:

$$h_{\text{top}}^{\mathcal{G} \mid W}(k, \ell, e) := h_{\text{reflect}}^W(h_{\text{top}}^{D_{\mathcal{G}}}(j_W(k), j_W(\ell), e)),$$

where $e : \text{Step}_{E_{\mathcal{G} \mid W}^{\rightarrow}}(k, \ell)$. Bidirected symmetry and irreflexivity are inherited by applying the corresponding fields of $B_{\mathcal{G}}$ at $j_W(k)$ and $j_W(\ell)$. These data therefore form a causal graph on $\text{Fin}(m_W)$.

Running example. The restriction of $\mathcal{G}_{\text{Mira}}$ to $\{\text{Pr}, \text{Qu}\}$ retains their bidirected edge. Its restriction to $\{\text{Top}, \text{Fit}, \text{Com}\}$ retains the two directed arrows into Fit .

Causal models and distributions

The remaining definitions build a structural causal model in stages. The finitary setting is fixed throughout: endogenous and exogenous variables receive explicit finite value types, the exogenous source carries a finite product probability assignment, and an all-directed causal skeleton records which inputs each structural mechanism may read.

Definition A.4.13 (Endogenous signature). An endogenous signature assigns an inhabited finite value type to every node of an ordered directed graph.

Node and value types. Let $V := \text{Fin}(n)$ be a formal node type in the sense of Definition A.4.1. Provide a value-type family and its finiteness evidence in the sense of Definition A.1.19:

$$\begin{aligned} \text{Val} : V &\rightarrow \mathcal{U}, \\ h_{\text{finite}} &: \Pi_{i:V} \text{Finite}(\text{Val}(i)). \end{aligned}$$

Finiteness alone does not imply inhabitation: a finite type may be empty. Therefore also provide a default value at every node:

$$\text{default} : \Pi_{i:V} \text{Val}(i).$$

Each finiteness witness supplies a complete duplicate-free enumeration and decidable equality by Definition A.1.19 and Remark A.1.38.

Directed structure. Provide a directed parent graph:

$$D : \text{DAG}(V).$$

The joint endogenous-assignment type is:

$$\text{Val}(\mathcal{V}) := \prod_{i:V} \text{Val}(i).$$

Signature record. Define $\text{EndogenousSignature}(V)$ as the record type with fields:

$$\begin{aligned} \text{Val} &: V \rightarrow \mathcal{U}, \\ h_{\text{finite}} &: \prod_{i:V} \text{Finite}(\text{Val}(i)), \\ \text{default} &: \prod_{i:V} \text{Val}(i), \\ D &: \text{DAG}(V). \end{aligned}$$

Interpretation. The graph D alone records only formal nodes, directed edges and their topological order. The endogenous signature additionally specifies what value each node may take and how it can be initialised. It is therefore a typed directed acyclic graph, not merely a directed acyclic graph. The endogenous signature records only directed mechanism dependencies. Bidirected edges are not additional inputs to an endogenous mechanism. In the theorem-facing terminology below, these endogenous nodes are the modelled coordinates retained as observed nodes after the exogenous roots are hidden. Here *observed* contrasts with hidden background variables; it does not mean that evidence has actually been received for every coordinate.

Running example. Mira’s endogenous node type is V_{Mira} from Definition A.4.1; each of its eight nodes has value type Bool . Its directed parent structure is D_{Mira} from Definition A.4.4.

Lean implementation. These data form **ObservedSignature**; Lean’s term *observed* therefore corresponds to the theorem-facing endogenous coordinates just described. For executable finite search, Lean stores a complete duplicate-free enumeration and decidable equality for each value type explicitly. The finiteness witness above constructively supplies those fields; Lean’s final two fields provide the directed classifier and topological-order evidence of D .

Definition A.4.14 (Exogenous source). An exogenous source specifies the possible values of the background variables and their joint finite probability assignment before any structural mechanism is evaluated.

Root and value types. Let $U := \text{Fin}(r)$ be a formal node type in the sense of Definition A.4.1, now used to index the exogenous roots rather than the endogenous nodes. Provide a finite value type for every root:

$$\begin{aligned} \text{Val}_{\mathcal{U}} &: U \rightarrow \mathcal{U}, \\ h_{\text{finite}}^{\mathcal{U}} &: \prod_{\ell:U} \text{Finite}(\text{Val}_{\mathcal{U}}(\ell)). \end{aligned}$$

As in the endogenous case, these finiteness witnesses supply the enumerations and equality procedures needed for finite computation. The joint exogenous-assignment type is:

$$\text{Val}(\mathcal{U}) := \prod_{\ell:U} \text{Val}_{\mathcal{U}}(\ell).$$

Coordinate factors. For every $\ell : U$, provide an urn-generated probability assignment:

$$\nu_{\ell} : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}_{\mathcal{U}}(\ell)).$$

Joint assignment. Provide an urn-generated assignment:

$$\mu_{\mathcal{U}} : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{U})).$$

A. Definitions

Rectangular events. The exogenous roots U , together with their value-type assignment $\text{Val}_{\mathcal{U}}$, form a finite family of variables in the sense of Definition A.3.22. Rectangular events on $\text{Val}(\mathcal{U})$ are those of Definition A.3.29.

Product law. Require evidence:

$$h_{\text{prod}} : \Pi_{A:\Pi_{\ell:U}(\text{Val}_{\mathcal{U}}(\ell) \rightarrow \text{Bool})} P_{\mu_{\mathcal{U}}}(\text{Rect}_A) \sim \prod_{\ell:U} P_{\nu_{\ell}}(A(\ell)).$$

It states that every rectangular event has the probability obtained by multiplying its coordinate probabilities, using the finite product of Definition A.3.10. This is the finite, proof-carrying independence condition imposed on the exogenous roots.

Source record. Define $\text{ExogenousSource}(U)$ as the record type with fields:

$$\begin{aligned} \text{Val}_{\mathcal{U}} &: U \rightarrow \mathcal{U}, \\ h_{\text{finite}}^{\mathcal{U}} &: \Pi_{\ell:U} \text{Finite}(\text{Val}_{\mathcal{U}}(\ell)), \\ \nu &: \Pi_{\ell:U} \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}_{\mathcal{U}}(\ell)), \\ \mu_{\mathcal{U}} &: \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{U})), \\ h_{\text{prod}} &: \Pi_{A:\Pi_{\ell:U}(\text{Val}_{\mathcal{U}}(\ell) \rightarrow \text{Bool})} P_{\mu_{\mathcal{U}}}(\text{Rect}_A) \sim \prod_{\ell:U} P_{\nu_{\ell}}(A(\ell)). \end{aligned}$$

The coordinate assignments ν_{ℓ} already witness that their value types are inhabited, so no separate default values are required.

Running example. Mira’s exogenous source has six Boolean roots:

$$U_B, U_{\text{Pr}}, U_{\text{Qu}}, U_{\text{Top}}, U_{\text{Com}}, U_{\text{Fund}}.$$

For each Boolean root ℓ , let E_{true}^{ℓ} be the event on $\text{Val}_{\mathcal{U}}(\ell)$ selecting true. Their coordinate factors satisfy:

$$\begin{aligned} P_{\nu_{U_B}}(E_{\text{true}}^{U_B}) &\sim \frac{1}{3}, \\ P_{\nu_{U_{\text{Pr}}}}(E_{\text{true}}^{U_{\text{Pr}}}) &\sim \frac{1}{4}, \\ P_{\nu_{U_{\text{Qu}}}}(E_{\text{true}}^{U_{\text{Qu}}}) &\sim \frac{1}{4}, \\ P_{\nu_{U_{\text{Fund}}}}(E_{\text{true}}^{U_{\text{Fund}}}) &\sim \frac{1}{4}, \\ P_{\nu_{U_{\text{Top}}}}(E_{\text{true}}^{U_{\text{Top}}}) &\sim \frac{1}{2}, \\ P_{\nu_{U_{\text{Com}}}}(E_{\text{true}}^{U_{\text{Com}}}) &\sim \frac{1}{2}. \end{aligned}$$

The joint source is their finite product urn.

Lean implementation. Lean distributes this conceptual package across **LatentExtension** and **FiniteLatentSCM**: the former stores the exogenous value family and explicit finite operations derived from its finiteness witnesses, while the latter stores **factor**, **prior** and **product_law**. The common-denominator representation used there is equivalent to the urn representation by Proposition B.6.13.

Definition A.4.15 (Exogenous incidence and input restriction). Exogenous incidence records which background coordinates each endogenous mechanism may inspect.

Data. Let $\mathcal{V}$ be an endogenous signature in the sense of Definition A.4.13, with node type V . Let $\mathcal{U}$ be an exogenous source in the sense of Definition A.4.14, with root type U and value family $\text{Val}_{\mathcal{U}}$. Supply the Boolean incidence classifier:

$$L : U \times V \rightarrow \text{Bool}.$$

It returns `true` exactly when the root $\ell : U$ is available as an input to the mechanism at $i : V$. The corresponding proof-valued incidence type is:

$$\text{Incident}_L(\ell, i) := \text{Id}_{\text{Bool}}(L(\ell, i), \text{true}).$$

Input type. For $i : V$, define:

$$\text{ExogenousInputs}_L(i) := \Pi_{\ell:U} \text{Incident}_L(\ell, i) \rightarrow \text{Val}_{\mathcal{U}}(\ell).$$

An inhabitant supplies a value for every root certified as incident to i . If $L(\ell, i) = \text{false}$, no incidence witness can be supplied, so a mechanism cannot request that coordinate.

Restriction. For a complete exogenous assignment $\mathbf{u} : \text{Val}(\mathcal{U})$, define:

$$\begin{aligned} \text{restrict}_{L,i}^{\mathcal{U}}(\mathbf{u}) &: \text{ExogenousInputs}_L(i), \\ \text{restrict}_{L,i}^{\mathcal{U}}(\mathbf{u})(\ell)(e) &:= \mathbf{u}_{\ell}. \end{aligned}$$

The proof $e : \text{Incident}_L(\ell, i)$ identifies an available coordinate; the restriction then returns that coordinate of the complete assignment.

Running example. In Mira’s model, the shared root is incident to both prestige and quality:

$$L(U_B, \text{Pr}) = \text{true}, \quad L(U_B, \text{Qu}) = \text{true}.$$

Each private root is incident only to its correspondingly named endogenous node.

Lean implementation. The incidence classifier is the `incident` field of `LatentExtension`. Its dependent input family is `LatentExtension.Inputs`; a complete latent assignment restricts to it by ordinary function application.

Definition A.4.16 (Latent causal skeleton). A latent causal skeleton combines an endogenous signature and an exogenous source with exogenous incidence. It contains no structural mechanisms yet.

Data. Let $\mathcal{V}$ be an endogenous signature in the sense of Definition A.4.13, with node type V and directed graph D . Let $\mathcal{U}$ be an exogenous source in the sense of Definition A.4.14, with root type U . Finally, let $L : U \times V \rightarrow \text{Bool}$ be an exogenous incidence classifier in the sense of Definition A.4.15.

Skeleton record. These components form the record type:

$$\text{LatentCausalSkeleton}(V, U),$$

with fields:

$$\begin{aligned} \text{endogenous} &: \text{EndogenousSignature}(V), \\ \text{exogenous} &: \text{ExogenousSource}(U), \\ \text{incident} &: U \times V \rightarrow \text{Bool}. \end{aligned}$$

A. Definitions

Thus the data introduced above determine:

$$\mathcal{K} := (\mathcal{V}, \mathcal{U}, L) : \text{LatentCausalSkeleton}(\mathcal{V}, \mathcal{U}).$$

The exogenous-input types and restriction functions of Definition A.4.15 are derived operations on this record; they are not additional fields. For $\mathcal{K} : \text{LatentCausalSkeleton}(\mathcal{V}, \mathcal{U})$, write $\text{Val}_{\mathcal{K}}$ for the value-type family projected from its endogenous signature, and write $\text{ExogenousInputs}_{\mathcal{K}}(i)$ and $\text{restrict}_{\mathcal{K},i}^{\mathcal{U}}$ for the derived constructions above.

All-directed graph. On the combined node type $U + V$, define an edge from $\text{inl}(\ell)$ to $\text{inr}(i)$ exactly when $L(\ell, i) = \text{true}$, and an edge from $\text{inr}(i)$ to $\text{inr}(j)$ exactly when $E_D^{\rightarrow}(i, j) = \text{true}$. All other cases return **false**, so no edge enters an exogenous root. Giving every exogenous root rank 0 and every endogenous node i rank $\text{rank}_n(i) + 1$ supplies an acyclicity ranking: the rank strictly increases along every edge, although distinct exogenous roots may share rank 0.

Interpretation and scope. An endogenous node may read no exogenous root, one root or several roots; conversely, one exogenous root may feed several endogenous nodes. The last case represents a shared source of variation. In the model class used here, endogenous nodes are observed and the explicit exogenous roots are unobserved. Observability and the endogenous–exogenous distinction need not coincide in more general structural-model presentations.

Running example. Mira’s endogenous and exogenous signatures, together with the incidence classifier of Definition A.4.15, give the all-directed skeleton in the figure below.

Lean implementation. The exogenous value data and incidence classifier form **LatentExtension**; its **expandedEdge** combines them with the directed part of **ObservedSignature** in exactly the two nonempty cases above. Probability fields are added by **FiniteLatentSCM**, as described in Definition A.4.14.

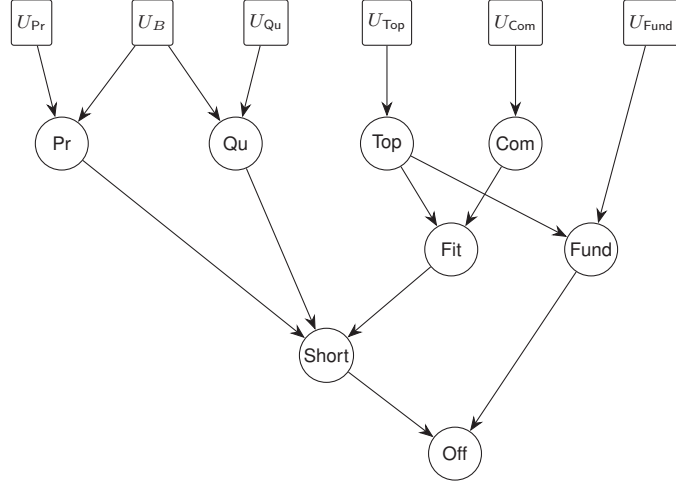


Figure A.4: The explicit all-directed graph of Mira’s latent causal skeleton. Rectangles are exogenous roots and circles are endogenous nodes. The shared root U_B is incident to both Pr and Qu ; no bidirected edge is primitive in this representation.

Definition A.4.17 (Structural mechanisms). The latent causal skeleton determines exactly which inputs each endogenous mechanism may inspect.

Data. Let

$$\mathcal{K} : \text{LatentCausalSkeleton}(V, U)$$

be a skeleton in the sense of Definition A.4.16. Write $\mathcal{V}$, $\mathcal{U}$ and L for its endogenous signature, exogenous source and incidence classifier, and write D , Val and $\text{Val}_{\mathcal{U}}$ for the corresponding projected data.

Input types. For $c : V$, define the endogenous inputs and combine them with the exogenous-input type of Definition A.4.16:

$$\begin{aligned} \text{EndogenousInputs}_{\mathcal{K}}(c) &:= \Pi_{p:V} (E_D^{\rightarrow}(p, c) = \text{true}) \rightarrow \text{Val}(p), \\ \text{MechanismInputs}_{\mathcal{K}}(c) &:= \text{EndogenousInputs}_{\mathcal{K}}(c) \times \text{ExogenousInputs}_{\mathcal{K}}(c). \end{aligned}$$

The first component supplies values only for declared endogenous parents; the second supplies values only for declared incident roots. The parent classifier $\text{Pa}_D(c)$ of Definition A.4.8 selects the first family of coordinates, while $\text{EndogenousInputs}_{\mathcal{K}}(c)$ supplies one value for every selected parent. It therefore contains the same parent-value data as $\text{Val}(\text{Pa}_D(c))$, written in the dependent function form used by the structural evaluator.

Mechanism family. A finite structural mechanism family is:

$$\mathcal{F} : \Pi_{c:V} \text{MechanismInputs}_{\mathcal{K}}(c) \rightarrow \text{Val}(c).$$

Write $f_c := \mathcal{F}(c)$ for the mechanism at c . If c has no declared parents and no incident roots, both input types are vacuous, $\text{MechanismInputs}_{\mathcal{K}}(c)$ has a unique inhabitant $\star$, and f_c is a function out of that singleton. The node still contributes a mechanism.

Running example. Mira’s complete Boolean mechanism family is:

$$\begin{aligned} f_{\text{Pr}} &:= U_B \vee U_{\text{Pr}}, & f_{\text{Qu}} &:= U_B \vee U_{\text{Qu}}, \\ f_{\text{Top}} &:= U_{\text{Top}}, & f_{\text{Com}} &:= U_{\text{Com}}, \\ f_{\text{Fit}} &:= \text{Top} \wedge \text{Com}, & f_{\text{Short}} &:= \text{Qu} \wedge (\text{Pr} \vee \text{Fit}), \\ f_{\text{Fund}} &:= \text{Top} \vee U_{\text{Fund}}, & f_{\text{Off}} &:= \text{Short} \wedge \text{Fund}. \end{aligned}$$

The first two mechanisms can read the shared root because the incidence evidence displayed in Definition A.4.16 supplies those inputs. In each of those two equations, either the shared root or the private root suffices to make the output true. The topic and committee mechanisms copy their private roots. The remaining four mechanisms combine only the endogenous parents and, where applicable, the private root selected by the skeleton. These eight equations therefore define f_c for every node $c : V_{\text{Mira}}$ used by the solution map below.

Lean implementation. In the namespaces `ObservedSignature` and `LatentExtension`, the two input types are respectively `ParentValues` and `Inputs`. The field `mechanism` of `FiniteLatentSCM` receives them in the equivalent curried form `ParentValues → Inputs → Value`; pairing and currying these inputs are mutually definable without any classical principle.

Definition A.4.18 (Structural causal model). In this monograph, a structural causal model is a finite latent-root model obtained by equipping a latent causal skeleton with one structural mechanism for every endogenous node.

A. Definitions

Model record. Define $\text{SCM}(V, U)$ as the dependent record type with fields:

$$\begin{aligned} \text{skeleton} &: \text{LatentCausalSkeleton}(V, U), \\ \text{mechanism} &: \Pi_{c:V} \text{MechanismInputs}_{\text{skeleton}}(c) \rightarrow \text{Val}_{\text{skeleton}}(c). \end{aligned}$$

Thus a skeleton $\mathcal{K}$ and a mechanism family $\mathcal{F}$ over that skeleton form the model:

$$\mathcal{M} = (\mathcal{K}, \mathcal{F}) : \text{SCM}(V, U).$$

Running example. Mira’s endogenous signature, six-root exogenous source and incidence classifier form the skeleton displayed in Figure A.4. Equipping that skeleton with the Boolean equations of Definition A.4.17 gives Mira’s structural causal model.

Lean implementation. These data form **FiniteLatentSCM**, abbreviated **ExactModel** in the transport development. Lean flattens the exogenous source, incidence classifier and mechanism family into this model and its **LatentExtension** field. Its explicit enumerations and common-denominator probability records carry information equivalent to the finiteness witnesses and urn assignments of the staged presentation above.

Definition A.4.19 (Prefix assignments). Prefix assignments represent the endogenous values already computed at an intermediate stage of topological evaluation.

Data. Let $\mathcal{V}$ be an endogenous signature in the sense of Definition A.4.13, with node type $V = \text{Fin}(n)$ and value family Val . Let $k : \mathbb{N}$ and supply $h_k : k \leq n$. Write:

$$\iota_k : \text{Fin}(k) \rightarrow V$$

for the canonical inclusion of the first k indices. Because Val is indexed by the complete node type, a prefix index $j : \text{Fin}(k)$ denotes the same node $\iota_k(j) : V$; the inclusion does not renumber it.

Prefix type. Define:

$$\text{Prefix}_{\mathcal{V}}(k) := \Pi_{j:\text{Fin}(k)} \text{Val}(\iota_k(j)).$$

The empty prefix has the unique inhabitant $\star : \text{Prefix}_{\mathcal{V}}(0)$.

Extension. Supply $h'_k : k < n$ and let $i_k : V$ be the node with index k . Define:

$$\text{extend}_k : \text{Prefix}_{\mathcal{V}}(k) \rightarrow \text{Val}(i_k) \rightarrow \text{Prefix}_{\mathcal{V}}(k+1).$$

Write $j_k : \text{Fin}(k) \rightarrow \text{Fin}(k+1)$ for the canonical inclusion and $\text{last}_k : \text{Fin}(k+1)$ for the final index. The extension retains every earlier value and assigns the supplied value to the new final coordinate:

$$\begin{aligned} \text{extend}_k(\mathbf{s}, x)(j_k(j)) &:= \mathbf{s}(j), \\ \text{extend}_k(\mathbf{s}, x)(\text{last}_k) &:= x. \end{aligned}$$

These cases exhaust $\text{Fin}(k+1)$, so finite elimination determines the complete extended prefix.

Lean implementation. Lean names the dependent prefix family and its extension operation:

$$\text{ObservedSignature.Prefix}, \quad \text{FiniteProduct.extend}.$$

Definition A.4.20 (Solution map). The ordered directed signature makes every finite latent-root model recursively evaluable.

Data. Let $\mathcal{M}$ be a model in the sense of Definition A.4.18, and write $\mathcal{K}$ for its latent causal skeleton. Its node type is $V = \text{Fin}(n)$. Use the prefix types, canonical inclusions and extension operations of Definition A.4.19 for its endogenous signature $\mathcal{V}$.

Typed evaluation step. Fix a complete exogenous assignment $\mathbf{u} : \text{Val}(\mathcal{U})$. Supply $h_k : k < n$, let $i_k : V$ be the node with index k , and let $\mathbf{s} : \text{Prefix}_{\mathcal{V}}(k)$ be an assignment to the already evaluated nodes. If

$$e : E_D^{\rightarrow}(p, i_k) = \text{true},$$

the topological evidence of D gives $\text{rank}_n(p) <_{\mathbb{N}} \text{rank}_n(i_k)$. Since $\text{rank}_n(i_k) \equiv k$, this has the form:

$$h_{\text{top}}(p, i_k, e) : \text{rank}_n(p) <_{\mathbb{N}} k.$$

Let $\bar{p}_e : \text{Fin}(k)$ be the canonical finite index characterised by:

$$\text{rank}_k(\bar{p}_e) \equiv \text{rank}_n(p).$$

It identifies the coordinate of p in the already evaluated prefix. The prefix and the complete exogenous assignment therefore determine the typed mechanism input:

$$\begin{aligned} \text{parentInput}_k(\mathbf{s}) &: \text{EndogenousInputs}_{\mathcal{K}}(i_k), \\ \text{parentInput}_k(\mathbf{s})(p)(e) &:= \mathbf{s}(\bar{p}_e), \\ \text{latentInput}_k(\mathbf{u}) &:= \text{restrict}_{\mathcal{K}, i_k}^{\mathcal{U}}(\mathbf{u}), \\ &: \text{ExogenousInputs}_{\mathcal{K}}(i_k), \\ \text{mechanismInput}_{k, \mathbf{u}}(\mathbf{s}) &:= (\text{parentInput}_k(\mathbf{s}), \text{latentInput}_k(\mathbf{u})), \\ &: \text{MechanismInputs}_{\mathcal{K}}(i_k), \\ \text{next}_{k, \mathbf{u}}(\mathbf{s}) &:= f_{i_k} \\ &\quad (\text{mechanismInput}_{k, \mathbf{u}}(\mathbf{s})). \end{aligned}$$

Recursive evaluation. Define

$$\mathbf{s}_k(\mathbf{u}) : \text{Prefix}_{\mathcal{V}}(k)$$

by natural-number recursion:

$$\begin{aligned} \mathbf{s}_0(\mathbf{u}) &:= \star, \\ \mathbf{s}_{k+1}(\mathbf{u}) &:= \text{extend}_k(\mathbf{s}_k(\mathbf{u}), \text{next}_{k, \mathbf{u}}(\mathbf{s}_k(\mathbf{u}))). \end{aligned}$$

The construction still evaluates one node at a time: the mechanism at i_k may inspect endogenous parents, whose values become available only in the preceding prefix.

Solution map. After the n successor steps, the completed prefix is a joint endogenous assignment because $\text{Prefix}_{\mathcal{V}}(n)$ and $\text{Val}(\mathcal{V})$ have the same coordinates. Define:

$$\begin{aligned} \text{sol}_{\mathcal{M}} &: \text{Val}(\mathcal{U}) \rightarrow \text{Val}(\mathcal{V}), \\ \text{sol}_{\mathcal{M}}(\mathbf{u}) &:= \mathbf{s}_n(\mathbf{u}). \end{aligned}$$

A. Definitions

Thus the solution at each node is constructed only from an earlier prefix and the fixed exogenous assignment; no later endogenous value is used during evaluation.

Interpretation. The solution map sends each complete exogenous assignment to the uniquely determined endogenous assignment.

Running example. Consider the exogenous assignment in which $U_B = \text{true}$ and every private root is **false**. Evaluation in Mira’s displayed topological order gives:

$$\text{sol}_{\mathcal{M}}(\mathbf{u}) = (\text{true}, \text{true}, \text{false}, \text{false}, \text{false}, \text{true}, \text{false}, \text{false}),$$

in the coordinate order (Pr, Qu, Top, Com, Fit, Short, Fund, Off).

Lean implementation. In the namespace `FiniteLatentSCM`, recursive evaluation is `evalPrefixUnder` and `evalUnder`. Their pointwise and complete-assignment agreement is proved in Proposition B.7.10. The independent finite-source evaluator and this dependent interpretation are related by Theorem 5.4.2.

Definition A.4.21 (Observational assignment). The solution map of Definition A.4.20 transfers the exogenous source to an observational probability assignment on the endogenous variables.

Pushforward. By Definition A.3.11, the joint exogenous urn can be presented as:

$$\begin{aligned} \mu_{\mathcal{U}} &= (N, h_N, g), \\ g &: \text{Fin}(N) \rightarrow \text{Val}(\mathcal{U}), \end{aligned}$$

where g labels every urn cell by a complete exogenous assignment. Relabelling each of its cells by the resulting endogenous assignment gives the pushforward:

$$\mu_{\mathcal{M}} := (N, h_N, \text{sol}_{\mathcal{M}} \circ g) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

Hence every decidable event $E : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$ satisfies:

$$P_{\mu_{\mathcal{M}}}(E) \sim P_{\mu_{\mathcal{U}}}(E \circ \text{sol}_{\mathcal{M}}).$$

This is the urn pushforward of Proposition B.6.15. Marginal, conditional and independence statements about the model are evaluated using $\mu_{\mathcal{M}}$.

Interpretation. Pushing the exogenous urn through the solution map transfers uncertainty over the background variables to the observational probability assignment over the endogenous variables.

Running example. Pushing Mira’s complete finite exogenous product urn through the solution map of Definition A.4.20 gives the induced observational assignment on Mira’s endogenous variables.

Lean implementation. In the namespace `FiniteLatentSCM`, the pushforward is `observationalDist`. Its agreement with recursive evaluation is proved in Proposition B.7.10. The common-denominator and urn presentations agree by Proposition B.6.13.

Definition A.4.22 (Causal Bayesian network). A causal Bayesian network combines a directed acyclic graph with local probability assignments and a joint assignment that factorises through those local assignments.

Graphical data. Let $D : \text{DAG}(V)$ be a directed acyclic graph on the finite variable family $\mathcal{V}$ of Definition A.3.22. For $i : V$, the parent family $\text{Pa}_D(i)$ is the finite selection of Definition A.4.8.

Local probability kernels. For every $i : V$, provide:

$$\kappa_i : \text{Val}(\text{Pa}_D(i)) \rightarrow \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(i)).$$

Thus every parent assignment $\mathbf{pa}_i$ selects a normalised probability assignment $\kappa_i(\mathbf{pa}_i)$ over the possible values of i . The kernel is supplied for every parent assignment; it is not obtained by conditioning a joint assignment and therefore remains defined when that parent assignment has probability zero.

If i has no parents, its parent-assignment type is the empty telescope, represented by $\text{Fin}(1)$ with unique inhabitant $\star$. Its local kernel is consequently the unconditional probability assignment:

$$\kappa_i(\star) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(i)).$$

No parent value is required in this case.

Joint assignment. Provide an urn-generated joint assignment:

$$\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

For $v : \text{Val}(\mathcal{V})$, let $v(i)$ be its value at i and let $\mathbf{pa}_i(v)$ be its restriction to the parents of i . For a parent assignment $\mathbf{pa} : \text{Val}(\text{Pa}_D(i))$ and a value $a : \text{Val}(i)$, abbreviate the probability of the singleton event selecting a by

$$q_i(a \mid \mathbf{pa}) := P_{\kappa_i(\mathbf{pa})}(E_a^{\text{sing}}).$$

The singleton event is that of Definition A.3.4; it is decidable because $\text{Val}(i)$ is finite.

Factorisation. The joint assignment must satisfy:

$$P_{\mu}(E_v^{\text{sing}}) \sim \prod_{i:V} q_i(v(i) \mid \mathbf{pa}_i(v)),$$

where E_v^{sing} is the singleton event on the finite joint assignment type.

Interpretation. The formula reconstructs the probability of a complete assignment by multiplying one local probability for every variable, evaluated at the parent values occurring in that same assignment. Consequently, the joint probability is governed by the parent-indexed local kernels rather than by one unrelated probability assignment for each variable. Its causal reading is that the parents are the direct graphical inputs to the local mechanism and that every variable is independent of its non-descendants conditional on its parents.

Lean implementation. The local-kernel part of this definition is the record `FiniteRationalCPT`, which uses an explicit finite code for each parent configuration and a finite rational row for each code. The construction `FiniteRationalCPT.toFunctionalCBN` turns those rows into finite response-function seeds, and `toSCM` produces the structural model of Definition A.4.18. The Lean record does not separately store the displayed observed joint assignment; evaluation of the resulting structural model constructs it. Both preservation of every supplied row and the displayed global factorisation of that observational assignment are established in Lemma B.8.1.

Graphical separation and probabilistic consequences

Definition A.4.23 (Latent projection). Latent projection hides the explicit exogenous roots of a latent causal skeleton and records their observable graphical consequences.

A. Definitions

Data. Let $\mathcal{K}$ be a latent causal skeleton in the sense of Definition A.4.16, with endogenous graph D , exogenous-root index type U and incidence classifier $L : U \times V \rightarrow \text{Bool}$.

Projection. Define the observed causal graph in the sense of Definition A.4.11:

$$\pi(\mathcal{K}) : \text{CausalGraph}(V)$$

by retaining the directed package:

$$D_{\pi(\mathcal{K})} := D,$$

and defining its bidirected classifier by:

$$E_{\pi(\mathcal{K})}^{\leftrightarrow}(i, j) = \text{true} \iff \neg \text{Id}_V(i, j) \wedge \Sigma_{\ell:U} \text{Incident}_L(\ell, i) \times \text{Incident}_L(\ell, j).$$

Thus two distinct endogenous nodes are joined bidirectionally exactly when they share an explicit exogenous root.

Interpretation. This operation depends only on the skeleton. It does not inspect the exogenous probabilities or structural mechanisms, and its result is an observed causal graph rather than a latent-free structural causal model. It is not an induced restriction in the sense of Definition A.4.12: hiding a shared root creates a bidirected edge between its observed children.

Running example. Mira's root U_B is incident to Pr and Qu , so its projection produces the bidirected edge $\text{Pr} \leftrightarrow \text{Qu}$ displayed in Figures A.2 and 5.3. No other root in Mira's skeleton is incident to two endogenous nodes.

Lean implementation. In the namespace `LatentExtension`, the construction is `observedGraph` and its bidirected classifier is `projectedBidirected`. Symmetry and irreflexivity are proved when the resulting `ObservedGraph` is constructed. The direct finite projection results are stated in Proposition B.8.2.

Definition A.4.24 (Canonical pair-root expansion). The canonical pair-root expansion introduces explicit latent pair-root candidates whose outgoing arrows represent every bidirected edge of a finite causal graph.

Data. Let $\mathcal{G} : \text{CausalGraph}(V)$ in the sense of Definition A.4.11. The construction accepts any such graph. In particular, $\mathcal{G}$ may be the graph $\pi(\mathcal{K})$ obtained by the latent projection of Definition A.4.23 from a skeleton $\mathcal{K}$.

Expanded node type. Define the finite type:

$$\tilde{V} := V + (V \times V).$$

It is finite by Remark A.1.22. Its left summand contains the original nodes, embedded by:

$$\text{obs}(i) := \text{inl}(i).$$

The right summand contains a candidate latent root for every ordered pair of original nodes:

$$U_{A,B} := \text{inr}(A, B).$$

Active pair roots. The candidate $U_{A,B}$ is active exactly when:

$$E_{\mathcal{G}}^{\leftrightarrow}(A, B) = \text{true}.$$

Because the bidirected classifier is symmetric, an edge between A and B activates both ordered candidates $U_{A,B}$ and $U_{B,A}$. Each active root has precisely the two outgoing arrows:

$$U_{A,B} \rightarrow \text{obs}(A), \quad U_{A,B} \rightarrow \text{obs}(B).$$

An inactive candidate has no incident arrow. Every original directed edge $A \rightarrow B$ is retained as $\text{obs}(A) \rightarrow \text{obs}(B)$, and no arrow enters a candidate root. Write $\tilde{D}_{\mathcal{G}}$ for the resulting all-directed graph. It has no bidirected edges: each bidirected edge of $\mathcal{G}$ is represented only by the outgoing arrows of its active pair-root candidates.

Acyclicity. Give every candidate root rank 0 and every original node $\text{obs}(i)$ rank $\text{rank}_n(i) + 1$. The rank strictly increases along every edge, so $\tilde{D}_{\mathcal{G}}$ is a finite directed acyclic graph.

Projection and scope. Hiding the active pair roots recovers exactly the directed and bidirected classifiers of $\mathcal{G}$, as proved in Proposition B.7.7. This is a graph-level right inverse to latent projection, not a reconstruction of whichever hidden model may have produced $\mathcal{G}$. It supplies no value types, probability assignments or structural mechanisms for the introduced roots.

The expanded node type contains a candidate root for every ordered pair, even when that candidate remains inactive. Such a candidate is isolated and therefore cannot occur on a path between original nodes or affect a separation test. In a Markovian graph there are no bidirected edges, so every candidate root remains inactive.

Lean implementation. The executable separation development in `lean/Thesis/Causality/Derivation.lean` uses the fixed candidate type above. The constructors `SeparationNode.observed` and `SeparationNode.latentPair` implement its two summands. In the namespace `ObservedGraph`, the operation `expandedMutilatedEdge` constructs the expanded directed relation together with any requested graph mutilation. The graph-only construction in `lean/Thesis/Causality/PairRoot.lean` instead filters out inactive candidates and uses increasing rank to retain only one of the two active ordered roots for each bidirected edge. These expanded node types are not definitionally identical, and no equality of their path types is asserted. Both constructions do, however, recover the same observed graph after latent projection, as recorded in Proposition B.7.7.

Remark A.4.25 (Explicit, projected and canonically expanded graphs). The constructions above distinguish three graph representations:

Representation	Exogenous roots	Bidirected edges
$\mathcal{M}, \mathcal{K}$ Definitions A.4.16 and A.4.18	explicit	none
$\mathcal{G} = \pi(\mathcal{K})$ Definition A.4.23	hidden	possibly present
$\tilde{D}_{\mathcal{G}}$ Definition A.4.24	canonical pair roots	none

Thus the relevant object flow is:

$$\mathcal{M} = (\mathcal{K}, \mathcal{F}) \longrightarrow \mathcal{K} \xrightarrow{\pi} \mathcal{G} \longrightarrow \tilde{D}_{\mathcal{G}}.$$

A. Definitions

The first arrow selects the model's skeleton, the second hides its explicit roots, and the third introduces canonical pair roots for the graphical separation test. The final arrow recovers the projected graph after hiding its new roots, but it does not recover the original skeleton or structural causal model.

Definition A.4.26 (Simple separation path). A simple separation path is an adjacency path in the canonical all-directed expansion whose node list contains no repeated entry.

Data. Let $\mathcal{G} : \text{CausalGraph}(V)$, let $x, y : V$ and use the canonical all-directed expansion $\tilde{D}_{\mathcal{G}}$ of Definition A.4.24.

Internal triples. For

$$p : \text{Path}_{\text{Adj}_{\tilde{D}_{\mathcal{G}}}}(\text{obs}(x), \text{obs}(y))$$

and $A, B, C : \tilde{V}$, define:

$$\begin{aligned} & \text{InternalTriple}(p; A, B, C) \\ & := \Sigma_{q, r : \text{List}(\tilde{V})} \text{Id}_{\text{List}(\tilde{V})} \left(\text{nodes}(p), \text{append}(q, \text{cons}(A, \text{cons}(B, \text{cons}(C, r)))) \right). \end{aligned}$$

The prefix q and suffix r locate three consecutive entries with B internal to the path. Concatenation is that of Definition A.1.25.

Path type. Define the record $\text{SimplePath}_{\mathcal{G}}(x, y)$ with fields:

$$\begin{aligned} & \text{path} : \text{Path}_{\text{Adj}_{\tilde{D}_{\mathcal{G}}}}(\text{obs}(x), \text{obs}(y)), \\ & \text{simple} : \text{NoRepeat}(\text{nodes}(\text{path})). \end{aligned}$$

The path may traverse directed arrows in either direction because it uses adjacency rather than the directed edge classifier itself.

Definition A.4.27 (Blocked separation path). A simple separation path is blocked when one of its internal triples carries the usual non-collider or collider obstruction.

Data. Let $\mathcal{G} : \text{CausalGraph}(V)$, let $x, y : V$, let $Z : V \rightarrow \text{Bool}$ and let $p : \text{SimplePath}_{\mathcal{G}}(x, y)$.

Blocking type. Define $\text{Blocked}_{\mathcal{G}, Z}(p)$ as the sum of the following two witness types:

$$\begin{aligned} & \Sigma_{A, B, C : \tilde{V}} \text{InternalTriple}(\text{path}(p); A, B, C) \\ & \quad \times (\text{Chain}_{\tilde{D}_{\mathcal{G}}}(A, B, C) + \text{Fork}_{\tilde{D}_{\mathcal{G}}}(A, B, C)) \\ & \quad \times \Sigma_{z : V} \text{Id}_{\text{Bool}}(Z(z), \text{true}) \times \text{Id}_{\tilde{V}}(B, \text{obs}(z)), \end{aligned}$$

or

$$\begin{aligned} & \Sigma_{A, B, C : \tilde{V}} \text{InternalTriple}(\text{path}(p); A, B, C) \\ & \quad \times \text{Collider}_{\tilde{D}_{\mathcal{G}}}(A, B, C) \\ & \quad \times \Pi_{z : V} \text{Id}_{\text{Bool}}(Z(z), \text{true}) \rightarrow \text{Id}_{\text{Bool}}(\text{De}_{\tilde{D}_{\mathcal{G}}}(B)(\text{obs}(z)), \text{false}). \end{aligned}$$

The first summand selects a conditioned internal non-collider. The second selects an internal collider for which neither the collider nor any descendant is selected by Z ; descendants include the collider itself by Definition A.4.8.

Definition A.4.28 (d-separation). d-separation is the graphical path-blocking criterion on the canonical finite all-directed representation of a causal graph. The same construction applies uniformly to Markovian and semi-Markovian graphs.

Data. Let $\mathcal{G} : \text{CausalGraph}(V)$ in the sense of Definition A.4.11. It may be supplied directly or obtained from a latent causal skeleton by Definition A.4.23. The separation relation below depends only on $\mathcal{G}$, not on which hidden model produced it. Write $\tilde{D}_{\mathcal{G}}$ for its canonical pair-root expansion and $\text{obs} : V \rightarrow \tilde{V}$ for the original-node embedding.

Paths. Let $X, Y, Z : V \rightarrow \text{Bool}$ be pairwise disjoint finite node selections in the sense of Definition A.4.2. Separation paths and their blocking evidence are the types of Definitions A.4.26 and A.4.27. The conditioning selection is tested only at embedded original nodes, so no candidate latent root belongs to it. The three local configurations are displayed in Figure A.3: conditioning on the middle node blocks a chain or fork, whereas a collider blocks by default and is opened by conditioning on it or one of its descendants.

Graphical independence. Define the d-separation type:

$$\begin{aligned} (X \perp\!\!\!\perp Y \mid Z)_{\mathcal{G}} \\ &:= \prod_{x,y:V} \text{ld}_{\text{Bool}}(X(x), \text{true}) \rightarrow \text{ld}_{\text{Bool}}(Y(y), \text{true}) \\ &\rightarrow \prod_{p:\text{SimplePath}_{\mathcal{G}}(x,y)} \text{Blocked}_{\mathcal{G},Z}(p). \end{aligned}$$

An inhabitant supplies blocking evidence for every simple separation path between selected endpoints. The graph subscript distinguishes this graphical relation from probabilistic conditional independence, whose subscript is a probability assignment, as specified in Remark A.3.32.

Scope. The definition above uses the pairwise-disjoint instance needed by every do-calculus rule in this monograph. Dropping the disjointness premise gives the broader graphical relation, but no result below requires that broader instance independently of a do-calculus rule.

Running example. In the directed expansion of $\mathcal{G}_{\text{Mira}}$, the path $\text{Top} \rightarrow \text{Fit} \leftarrow \text{Com}$ is blocked by its collider when Z is empty. The remaining paths are likewise blocked, so:

$$(\text{Top} \perp\!\!\!\perp \text{Com} \mid \emptyset)_{\mathcal{G}_{\text{Mira}}}.$$

Conditioning on Fit opens the displayed collider path.

Lean implementation. Lean keeps the executable test `ObservedGraph.dSeparated` and the path specification `PathSpecification.ActivePath` as separate definitions. Both accept arbitrary node classifiers; the pairwise-disjoint restriction above belongs to the do-calculus applications. Agreement between the two definitions is supplied separately as an external interface.

Definition A.4.29 (Local causal Markov condition). The causal Markov condition relates the ordered directed graph to conditional independence in its joint probability assignment.

Data. Let $D : \text{DAG}(V)$ be a directed acyclic graph on a finite variable family $\mathcal{V}$, and let:

$$\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

A. Definitions

Non-descendant selection. For every node $i : V$, let $\text{ND}_D(i) : V \rightarrow \text{Bool}$ select the nodes that are neither descendants nor parents of i :

$$\text{ND}_D(i)(j) = \text{true} \iff \text{De}_D(i)(j) = \text{false} \wedge \text{Pa}_D(i)(j) = \text{false}.$$

The parent and descendant selections are those of Definition A.4.8. Because i is its own descendant, it is not selected by $\text{ND}_D(i)$.

Condition. The assignment μ satisfies the causal Markov condition with respect to D when every $i : V$ satisfies:

$$\{i\} \perp\!\!\!\perp_{\mu} \text{ND}_D(i) \mid \text{Pa}_D(i).$$

The singleton, parent and non-descendant classifiers are interpreted as finite variable selections and lifted to cylinder events as in Definition A.3.31.

Interpretation. Thus every variable is conditionally independent of its non-descendants other than its parents once its parents are known.

Scope. The causal Markov condition is standard background connecting a directed causal graph with its probabilistic conditional independences, but it is not used as a separate assumption by any later result in this monograph. The checked correspondence theorem of Theorem 5.4.2 instead starts from deterministic structural mechanisms and a product assignment over the exogenous roots.

Definition A.4.30 (Global Markov property). The global Markov property connects the d-separation relation of a causal graph with conditional independence in a probability assignment.

Data. Let $\mathcal{G} : \text{CausalGraph}(V)$ be a causal graph on the finite variable family $\mathcal{V}$, and let:

$$\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

Condition. The pair $(\mathcal{G}, \mu)$ satisfies the global Markov property when, for every three pairwise disjoint finite variable selections $X, Y, Z : V \rightarrow \text{Bool}$:

$$(X \perp\!\!\!\perp Y \mid Z)_{\mathcal{G}} \implies X \perp\!\!\!\perp_{\mu} Y \mid Z.$$

Interpretation. This definition is stated directly for a graph–assignment pair. It therefore applies to a directed Bayesian network and, using the projected causal graph, to a semi-Markovian structural causal model. Whether a particular construction satisfies the implication is a mathematical property of that construction, not additional data hidden in the notation.

Remark A.4.31 (Faithfulness). Faithfulness requires graphical separation to capture all and only the conditional independences of a probability assignment.

Data. Let $\mathcal{G} : \text{CausalGraph}(V)$ be a causal graph on a finite variable family $\mathcal{V}$, and let:

$$\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

Condition. The assignment μ is *faithful* to $\mathcal{G}$ when, for every three pairwise disjoint finite variable selections $X, Y, Z : V \rightarrow \text{Bool}$:

$$(X \perp\!\!\!\perp Y \mid Z)_{\mathcal{G}} \iff X \perp\!\!\!\perp_{\mu} Y \mid Z.$$

Interpretation. The forward implication is exactly the global Markov property of Definition A.4.30. Faithfulness additionally requires the converse:

every probabilistic conditional independence must be graphically represented, so no additional independence arises only from accidental parameter cancellation.

Scope. Faithfulness is standard background vocabulary in causal discovery, but it is not used by any later definition, assumption or result in this monograph. In particular, the identification and do-calculus completeness results of Section B.11 work with the graph and compatible model class as given.

General hidden-DAG extensions

Definition A.4.32 (Partially observed directed acyclic graph). The direct-root projection of Definition A.4.23 suffices for the explicit structural models used in this monograph. The more general hidden-DAG layer of Proposition B.8.4 also permits hidden nodes with parents and hidden nodes lying along longer directed paths. Its graph data are separated from the projection operation below.

Total directed graph. Let $D_H : \text{RankedDAG}$ be a ranked directed acyclic graph in the sense of Remark A.4.10. Write q_H for its node count, $T_H := \text{Fin}(q_H)$ for its total node type, E_H for its edge classifier and rank_H for its rank.

Observed nodes. Let $V = \text{Fin}(n)$ be the formal node type of the graph that remains visible after projection, in the sense of Definition A.4.2. Supply an embedding of these observed nodes into the total graph:

$$\begin{aligned} o : V &\rightarrow T_H, \\ h_o : \Pi_{i,j:V} \text{Id}_{T_H}(o(i), o(j)) &\rightarrow \text{Id}_V(i, j). \end{aligned}$$

Thus $o(i)$ identifies the total-graph node represented by the observed node i . The proof h_o is necessary because a function between finite types need not be injective: finiteness and decidable equality alone do not prevent two observed names from being mapped to the same total node.

Observed image and hidden nodes. Because V and T_H are finite and have decidable equality, finite search determines a Boolean classifier $\text{Obs}_o : T_H \rightarrow \text{Bool}$ satisfying:

$$\text{Obs}_o(t) = \text{true} \iff \Sigma_{i:V} \text{Id}_{T_H}(o(i), t) \text{ is inhabited.}$$

Define the hidden classifier as the Boolean complement:

$$H_o(t) := \text{not}_{\text{Bool}}(\text{Obs}_o(t)).$$

Thus the embedding, rather than an independently supplied classifier, determines which nodes are hidden.

Compact record. Package the nonredundant data as the record type $\text{CompactFiniteHiddenDAG}(V)$. Writing T_{graph} for the total node type carried by its graph field, its fields are:

$$\begin{aligned} \text{graph} &: \text{RankedDAG}, \\ o : V &\rightarrow T_{\text{graph}}, \\ h_o : \Pi_{i,j:V} \text{Id}_{T_{\text{graph}}}(o(i), o(j)) &\rightarrow \text{Id}_V(i, j). \end{aligned}$$

For $\mathcal{C} : \text{CompactFiniteHiddenDAG}(V)$, the operation

$$\text{toFiniteHiddenDAG}(\mathcal{C}) : \text{FiniteHiddenDAG}(V)$$

A. Definitions

retains its total graph and observed embedding, installs the computed classifier H_o , and supplies the evidence:

$$\begin{aligned} h_{\text{obs}} &: \prod_{i:V} \text{Id}_{\text{Bool}}(H_o(o(i)), \text{false}), \\ h_{\text{cover}} &: \prod_{t:T_H} \text{Id}_{\text{Bool}}(H_o(t), \text{true}) + \sum_{i:V} \text{Id}_{T_H}(o(i), t). \end{aligned}$$

Conversely, every expanded $\text{FiniteHiddenDAG}(V)$ determines a compact record by forgetting this redundant classifier and evidence; re-expansion recovers its hidden classifier pointwise. This equivalence is proved in Proposition B.7.3.

For an expanded graph $\mathcal{H} : \text{FiniteHiddenDAG}(V)$, write $T_{\mathcal{H}}$, $o_{\mathcal{H}}$, $H_{\mathcal{H}}$ and $E_{\mathcal{H}}$ for the corresponding fields. A hidden node need not be a root: it may have incoming and outgoing directed edges. A *hidden root* is the special case certified by the type:

$$\text{HiddenRoot}_{\mathcal{H}}(\ell) := \text{Id}_{\text{Bool}}(H_{\mathcal{H}}(\ell), \text{true}) \times \prod_{s:T_{\mathcal{H}}} \text{Id}_{\text{Bool}}(E_{\mathcal{H}}(s, \ell), \text{false}).$$

Scope. This is a graph record, not yet a structural causal model. A model over the total graph must additionally provide finite value types, structural mechanisms and probability data for its background variation. Hiding nodes then produces an observed graph and observed probability assignment; it does not generally produce a latent-free structural causal model of the same record type. In particular, this general graph cannot be obtained merely by choosing among the exogenous coordinates of Definition A.4.18: those coordinates are roots, whereas hidden nodes here may themselves have parents and may occur internally on longer directed paths.

Lean implementation. Lean defines `RankedFiniteDAG` and `CompactFiniteHiddenDAG`. Its computed classifier is `CompactFiniteHiddenDAG.hidden`, its expansion is `CompactFiniteHiddenDAG.toFiniteHiddenDAG`, and the converse operation is `FiniteHiddenDAG.toCompact`. These declarations are in `lean/Thesis/Causality/CompactHiddenDAG.lean`. The expanded record `FiniteHiddenDAG` remains in `lean/Thesis/Causality/Reductions.lean`. The associated executable hidden-DAG SCM record `FiniteHiddenDAGSCM` is described at the proof boundary in Proposition B.8.3.

Definition A.4.33 (Hidden-internal path). A hidden-internal path is a directed path whose intermediate nodes are hidden, while its two endpoints may be either hidden or observed.

Data. Let $\mathcal{H} : \text{FiniteHiddenDAG}(V)$ be a partially observed directed acyclic graph in the sense of Definition A.4.32. Use its total node type $T_{\mathcal{H}}$, edge classifier $E_{\mathcal{H}}$ and hidden classifier $H_{\mathcal{H}}$.

Path type. The inductive type $\text{HiddenInternalPath}_{\mathcal{H}}(s, t)$ is generated by:

$$\begin{aligned} \text{direct} &: \prod_{s,t:T_{\mathcal{H}}} \text{Step}_{E_{\mathcal{H}}}(s, t) \\ &\quad \rightarrow \text{HiddenInternalPath}_{\mathcal{H}}(s, t), \\ \text{tail} &: \prod_{s,j,t:T_{\mathcal{H}}} \text{HiddenInternalPath}_{\mathcal{H}}(s, j) \rightarrow \text{Id}_{\text{Bool}}(H_{\mathcal{H}}(j), \text{true}) \\ &\quad \rightarrow \text{Step}_{E_{\mathcal{H}}}(j, t) \rightarrow \text{HiddenInternalPath}_{\mathcal{H}}(s, t). \end{aligned}$$

The first constructor supplies one directed edge. The second extends an existing path through its hidden endpoint j ; this requirement is what prevents an observed internal node from being contracted by projection. Thus a chain

$o_{\mathcal{H}}(i) \rightarrow h_1 \rightarrow h_2 \rightarrow o_{\mathcal{H}}(j)$ supplies a hidden-internal path from $o_{\mathcal{H}}(i)$ to $o_{\mathcal{H}}(j)$ when h_1 and h_2 are hidden. Write $s \rightsquigarrow_{\mathcal{H}} t$ when this path type is inhabited.

Lean implementation. Lean names this inductive type:

`FiniteHiddenDAG.HiddenInternalPath.`

Its constructors are `direct` and `tail`.

Definition A.4.34 (Projected hidden-DAG relations). Projected hidden-DAG relations state which directed and bidirected connections between observed nodes are induced by hidden-internal paths.

Data. Let $\mathcal{H} : \text{FiniteHiddenDAG}(V)$ be an expanded partially observed graph in the sense of Definition A.4.32. It may be supplied directly or obtained as `toFiniteHiddenDAG(C)` from a compact record $\mathcal{C}$. Use its total node type $T_{\mathcal{H}}$, observed embedding $o_{\mathcal{H}}$, hidden classifier $H_{\mathcal{H}}$ and edge classifier $E_{\mathcal{H}}$.

Projected directed relation. A projected directed edge is a hidden-internal directed path between two embedded observed nodes:

$$\text{ProjectedDirected}_{\mathcal{H}}(i, j) := o_{\mathcal{H}}(i) \rightsquigarrow_{\mathcal{H}} o_{\mathcal{H}}(j).$$

Projected bidirected relation. A projected bidirected edge has distinct observed endpoints and is witnessed by a hidden node with a hidden-internal path to each endpoint:

$$\begin{aligned} \text{ProjectedBidirected}_{\mathcal{H}}(i, j) &:= \neg \text{Id}_V(i, j) \times \sum_{\ell : T_{\mathcal{H}}} \text{Id}_{\text{Bool}}(H_{\mathcal{H}}(\ell), \text{true}) \\ &\quad \times (\ell \rightsquigarrow_{\mathcal{H}} o_{\mathcal{H}}(i)) \times (\ell \rightsquigarrow_{\mathcal{H}} o_{\mathcal{H}}(j)). \end{aligned}$$

Examples. A path $o_{\mathcal{H}}(X) \rightarrow h_1 \rightarrow h_2 \rightarrow o_{\mathcal{H}}(Y)$ with hidden h_1, h_2 produces the observed arrow $X \rightarrow Y$. A hidden common ancestor with paths

$$\ell \rightarrow h_1 \rightarrow o_{\mathcal{H}}(X), \quad \ell \rightarrow h_2 \rightarrow o_{\mathcal{H}}(Y)$$

produces $X \leftrightarrow Y$. The bidirected classifier records this projected result; it is not itself the hidden path.

Definition A.4.35 (General hidden-DAG projection). A general hidden-DAG projection packages a partially observed directed graph with a causal graph that records exactly its projected relations.

Data. Let $\mathcal{H} : \text{FiniteHiddenDAG}(V)$ be as in Definition A.4.34. Use the relations `ProjectedDirectedH` and `ProjectedBidirectedH` defined there.

Projection record. Define `FiniteLatentProjection(V)` as the record type with fields:

$$\begin{aligned} \text{hiddenDAG} &: \text{FiniteHiddenDAG}(V), \\ \text{graph} &: \text{CausalGraph}(V), \\ h_{\text{directed}} &: \prod_{i, j : V} \text{Id}_{\text{Bool}}(E_{\text{graph}}^{\rightarrow}(i, j), \text{true}) \iff \text{ProjectedDirected}_{\text{hiddenDAG}}(i, j), \\ h_{\text{bidirected}} &: \prod_{i, j : V} \text{Id}_{\text{Bool}}(E_{\text{graph}}^{\leftrightarrow}(i, j), \text{true}) \\ &\iff \text{ProjectedBidirected}_{\text{hiddenDAG}}(i, j). \end{aligned}$$

Interpretation. The directed field contracts an observed-to-observed path whose internal nodes are hidden into one observed arrow. The bidirected field

A. Definitions

selects two distinct observed endpoints when one hidden node has hidden-internal directed paths to both.

Relation to direct roots. For a latent causal skeleton $\mathcal{K}$, take the total nodes to be its exogenous and endogenous nodes, classify the exogenous nodes as hidden roots and the endogenous nodes as observed, and retain the incidence and endogenous directed edges of its all-directed graph. In this special case, every hidden-internal path from a root to an observed child consists of one incidence edge. The general projection therefore reduces to the direct projection $\pi(\mathcal{K})$ of Definition A.4.23. An observed intermediate node is not contracted: only paths whose internal nodes are hidden contribute new projected edges.

Purpose. The direct-root case suffices for Mira’s structural model. A selected nonempty family of executable finite hidden-DAG models, its projection and the external identifiability-preservation boundary needed by the completeness transfer are stated precisely in Proposition B.8.4 and Remark B.8.6.

Lean implementation. The graph and projection records are `FiniteHiddenDAG` and `FiniteLatentProjection`; the latter stores the matching evidence above and derives its observed graph.

A.5 Do-calculus

This section defines structural intervention, finite kernel queries, causal identification, hedge criteria and the finite do-calculus expression language.

Structural interventions

Definition A.5.1 (Graph mutilation). Graph mutilation is Pearl’s graphical representation of intervention. It is a graph operation on the endogenous/observed nodes of a causal graph, not an operation on mechanisms.

Mutilation type. For a finite node type V , define the record `Mutilation(V)` with fields:

$$\begin{aligned} \text{incoming} &: V \rightarrow \text{Bool}, \\ \text{outgoing} &: V \rightarrow \text{Bool}. \end{aligned}$$

For an all-directed edge classifier $E : V \times V \rightarrow \text{Bool}$ and $m : \text{Mutilation}(V)$, retain the arrow $i \rightarrow j$ exactly when $E(i, j) = \text{true}$, $\text{outgoing}(m)(i) = \text{false}$ and $\text{incoming}(m)(j) = \text{false}$.

Canonical operations. Let $X, Z : V \rightarrow \text{Bool}$ be finite node selections in the sense of Definition A.4.2. Write:

$$\begin{aligned} \overline{X} &:= (\text{incoming} = X, \text{outgoing} = \emptyset), \\ \underline{X} &:= (\text{incoming} = \emptyset, \text{outgoing} = X), \\ \overline{X} \underline{Z} &:= (\text{incoming} = X, \text{outgoing} = Z). \end{aligned}$$

Graph operation. The mutilated graph $\mathcal{G}_{\overline{X}}$ is obtained by deleting all directed arrows into X , because an intervention on X replaces its usual causes. The graph $\mathcal{G}_{\underline{X}}$ is obtained by deleting all directed arrows out of X . In the semi-Markovian reading of Definition A.4.11, these operations are applied after passing to the canonical finite latent-variable directed acyclic graph of Definition A.4.24; equivalently, a bidirected edge incident with an intervened variable no longer

contributes a latent arrowhead into that variable in $\mathcal{G}_{\overline{X}}$. These operations are combined when several marks occur together.

Running example. For Mira's graph, $\mathcal{G}_{\text{Mira}, \overline{\text{Pr}}}$ deletes incoming arrows into Pr. On the observed graph this deletes the latent head of $\text{Pr} \leftrightarrow \text{Qu}$. Equivalently, expand that edge to $U_B \rightarrow \text{Pr}$ and $U_B \rightarrow \text{Qu}$. Incoming arrows into Pr are then $U_{\text{Pr}} \rightarrow \text{Pr}$ and $U_B \rightarrow \text{Pr}$, as in the figure below. The edge $\text{Pr} \rightarrow \text{Short}$ and $U_B \rightarrow \text{Qu}$ remain.

Lean implementation. Incoming and outgoing deletions are the fields of `GraphMutilation`; `bar` and `barUnderline` construct the canonical operations above.

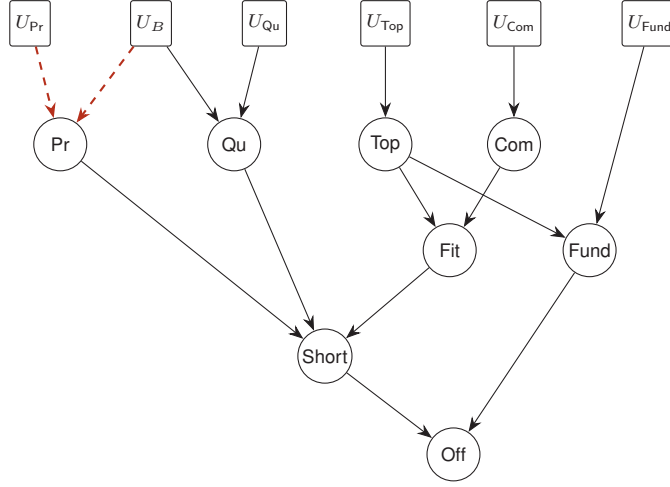


Figure A.5: Incoming arrows into Pr under $\mathcal{G}_{\text{Mira}, \overline{\text{Pr}}}$. Dashed coloured arrows $U_{\text{Pr}} \rightarrow \text{Pr}$ and $U_B \rightarrow \text{Pr}$ are marked for deletion; $U_B \rightarrow \text{Qu}$ and $\text{Pr} \rightarrow \text{Short}$ are retained.

Definition A.5.2 (Hard intervention). A hard intervention is a dependent partial assignment of replacement values to endogenous coordinates. It is not a graph operation.

Data. Let $\mathcal{V}$ be an endogenous signature in the sense of Definition A.4.13, with node type V and value family Val .

Intervention type. Define:

$$\text{Do}(\mathcal{V}) := \prod_{i:V} \text{Option}(\text{Val}(i)).$$

The optional-value type is that of Definition A.1.23. For $x : \text{Do}(\mathcal{V})$, define its target classifier:

$$\text{targets}(x)(i) := \begin{cases} \text{false}, & x(i) = \text{none}, \\ \text{true}, & x(i) = \text{some}(v). \end{cases}$$

Reference-induced intervention. Let $T : V \rightarrow \text{Bool}$ be a node selection in the sense of Definition A.4.2, and let $a : \text{Val}(\mathcal{V})$ be a complete assignment. Define:

$$\text{fromAssignment}(T, a) : \text{Do}(\mathcal{V}).$$

A. Definitions

Its value at $i : V$ is:

$$\text{fromAssignment}(T, a)(i) := \begin{cases} \text{none}, & T(i) = \text{false}, \\ \text{some}(a(i)), & T(i) = \text{true}. \end{cases}$$

Boolean elimination gives

$$\Pi_{i:V} \text{Id}_{\text{Bool}}(\text{targets}(\text{fromAssignment}(T, a))(i), T(i)).$$

Thus $\text{fromAssignment}(T, a)$ intervenes on exactly the nodes selected by T and obtains every replacement value from the corresponding coordinate of a .

Lean implementation. This dependent partial assignment is **HardIntervention**; its target classifier is **HardIntervention.targets**, node-wise extension is **HardIntervention.set**. Given a kernel whose action field is T , **Kernel.intervention** computes the underlying dependent assignment of $\text{fromAssignment}(T, a)$.

Definition A.5.3 (Intervention and interventional distribution). Applying a hard intervention to a structural causal model yields an intervened model, a mutilated graph and an interventional probability assignment.

Data. Let $\mathcal{M} = (\mathcal{K}, \mathcal{F})$ be a structural causal model in the sense of Definition A.4.18, and let $x : \text{Do}(\mathcal{V})$ be a hard intervention of Definition A.5.2 on its endogenous signature. Write

$$T_x := \text{targets}(x)$$

for its target selection, and let $\overline{T_x} : \text{Mutilation}(V)$ be the incoming mutilation of Definition A.5.1 determined by T_x , applied to the projected causal graph of Definition A.4.23.

Result record. Define the canonical record value $\text{InterventionResult}_{\mathcal{M}}(x)$ by:

$$\begin{aligned} \text{model} &:= \mathcal{M}_x := (\mathcal{K}_x, \mathcal{F}_x), \\ \text{graph} &:= \mathcal{G}_{\overline{T_x}}, \\ \text{law} &:= \text{IntDist}_{\mathcal{M}}(x). \end{aligned}$$

The following paragraphs specify the three displayed components.

Interpretation. Pearl's notation $\text{do}(x)$, or $\text{do}(X = a)$ when x is induced by a selected assignment $a : \text{Val}(X)$, names this operation. It is not the observation $X = a$: it replaces mechanisms rather than conditioning on their realised values.

Structural operation. The field **model** is the intervened model $\mathcal{M}_x = (\mathcal{K}_x, \mathcal{F}_x)$, where $\mathcal{K}_x$ removes every incoming endogenous and exogenous incidence into a node selected by T_x . For each $i : V$ and $\text{inp} : \text{MechanismInputs}_{\mathcal{K}}(i)$, the overridden mechanism is

$$f_i^x(\text{inp}) := \begin{cases} f_i(\text{inp}), & x(i) = \text{none}, \\ v, & x(i) = \text{some}(v) \text{ for } v : \text{Val}(i) \end{cases}$$

and $\mathcal{F}_x(i) := f_i^x$. Thus **none** retains the mechanism, while **some**(v) replaces it by a constant function. The type $\text{Do}(\mathcal{V})$ is a partial assignment of replacement values, not an optional copy of $\mathcal{F}$. All other declarations, mechanisms and the exogenous source $\mu_{\mathcal{U}}$ are unchanged. The intervened model is again recursive

over the same topological order. The same endogenous assignment is obtained by retaining $\mathcal{M}$ and, during the recursive evaluation of Definition A.4.20, returning the assigned constant at each selected node.

Graph. The field **graph** is the mutilated graph $\mathcal{G}_{\overline{T_x}}$ of Definition A.5.1.

Interventional assignment. If $\mu_{\mathcal{U}} = (N, h_N, g)$, the field law is

$$\text{IntDist}_{\mathcal{M}}(x) := \mu_{\mathcal{M}_x} := (N, h_N, \text{sol}_{\mathcal{M}_x} \circ g) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

Thus, for any decidable event $E : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$,

$$P_{\text{IntDist}_{\mathcal{M}}(x)}(E) \sim P_{\mu_{\mathcal{U}}}(E \circ \text{sol}_{\mathcal{M}_x}).$$

This is the same construction as Definition A.4.21, now on $\mathcal{M}_x$.

Running example. For $X = \{\text{Pr}\}$ and $a_{\text{Pr}} = \text{true}$, the intervened Mira model replaces the equation for Pr by the constant **true**. The skeleton $\mathcal{K}_x$ removes the same incoming incidence into Pr as $\mathcal{G}_{\text{Mira}, \overline{\text{Pr}}}$ in Figure A.5: arrows from U_B and U_{Pr} are removed, while $U_B \rightarrow \text{Qu}$ and $\text{Pr} \rightarrow \text{Short}$ remain.

Lean implementation. The dependent partial assignment is **HardIntervention**. The evaluator override, on the record **FiniteLatentSCM**, is **equationUnder** and **evalNodeUnder**.

Kernel queries

Definition A.5.4 (Partial rational values). Kernel evaluation uses the common result type:

$$\text{PartialQ} := \text{Option}(\mathbb{Q}_{\geq 0}),$$

formed from the optional type of Definition A.1.23 and the nonnegative rational presentations of Definition A.3.8. A value **some**(q) represents the defined rational result q , whereas **none** represents a conditional probability that is undefined because its conditioning event has probability zero. In particular, **none** is distinct from **some**(0).

Partial-value equivalence. For $r, s : \text{PartialQ}$, extend the relation $\sim$ of Definition A.3.8 by optional-value elimination:

$$\begin{aligned} \text{none} &\sim \text{none} := \text{Fin}(1), \\ \text{some}(p) &\sim \text{some}(q) := (p \sim q), \\ \text{none} &\sim \text{some}(q) := \text{Fin}(0), \\ \text{some}(p) &\sim \text{none} := \text{Fin}(0). \end{aligned}$$

Hence **none** records absence of a conditional value, not the rational number zero. Here $\text{Fin}(1)$ and $\text{Fin}(0)$ are the inhabited and empty finite types of Definition A.1.17. Two present values are compared by cross-multiplication.

Definition A.5.5 (Finite kernel queries). A kernel query records the whole interventional kernel, not one event probability.

Data. Let $\mathcal{V}$ be an endogenous signature in the sense of Definition A.4.13, with node type V . Node selections and their disjointness evidence are those of Definition A.4.2.

Joint kernel. Define the record $\text{Kernel}(\mathcal{V})$ with fields:

$$\begin{aligned} Y &: V \rightarrow \text{Bool}, \\ X &: V \rightarrow \text{Bool}, \\ h_{XY} &: \text{Disjoint}(X, Y). \end{aligned}$$

A. Definitions

It denotes the whole interventional kernel $P(Y \mid \text{do}(X))$ rather than one event probability.

Conditional kernel. Define the record $\text{CondKernel}(\mathcal{V})$ with fields:

$$\begin{aligned} Y, X, Z &: V \rightarrow \text{Bool}, \\ h_{XY} &: \text{Disjoint}(X, Y), \\ h_{XZ} &: \text{Disjoint}(X, Z), \\ h_{YZ} &: \text{Disjoint}(Y, Z). \end{aligned}$$

It denotes the whole conditional interventional kernel $P(Y \mid \text{do}(X), Z)$.

Lean implementation. The query records are `JointKernelQuery` and `ConditionalKernelQuery`; their node-selection fields are `outcome`, `action` and `condition`. The definitions `JointKernelQuery.operationKernel` and `ConditionalKernelQuery.operationKernel` place these fields in the common `Kernel` record.

Definition A.5.6 (Pointwise kernel denotation). A whole kernel is specified by its value at every complete reference assignment. The reference assignment supplies the intervention, outcome and conditioning values simultaneously.

Data. Let $\mathcal{M}$ be a structural causal model of Definition A.4.18 on an endogenous signature $\mathcal{V}$, and write

$$\Omega := \text{Val}(\mathcal{V}).$$

Let $a : \Omega$ be a complete assignment. Through the selected-family presentation of Definition A.4.2, every node selection S determines a coordinate selection. Hence Definition A.3.26 supplies both the restricted assignment $a|_S$ and the fibre event

$$F_{S,a} : \Omega \rightarrow \text{Bool}$$

selecting the complete assignments that agree with a on S . For any action selection $X : V \rightarrow \text{Bool}$, the same assignment determines the hard intervention $\text{fromAssignment}(X, a)$ of Definition A.5.2. Write

$$\mu_{\mathcal{M}}^{X,a} := \text{law}(\text{InterventionResult}_{\mathcal{M}}(\text{fromAssignment}(X, a)))$$

for the interventional assignment of Definition A.5.3, the law field of that result record. Partial values are those of Definition A.5.4.

Denotation type. A (joint or conditional) denotation is a map

$$d : \Omega \rightarrow \text{PartialQ}.$$

The two displays below inhabit this type.

Joint denotation. Let $Q_J = (Y, X, h_{XY}) : \text{Kernel}(\mathcal{V})$ in the sense of Definition A.5.5. Define

$$\llbracket Q_J \rrbracket_{\mathcal{M}}^J : \Pi_{a:\Omega} \text{PartialQ}$$

by

$$\llbracket Q_J \rrbracket_{\mathcal{M}}^J(a) := \text{some}\left(P_{\mu_{\mathcal{M}}^{X,a}}(F_{Y,a})\right).$$

The outer `some` embeds this always-defined rational value into `PartialQ`.

Conditional denotation. Let $Q_C = (Y, X, Z, h_{XY}, h_{XZ}, h_{YZ}) : \text{CondKernel}(\mathcal{V})$. Define

$$\llbracket Q_C \rrbracket_{\mathcal{M}}^C : \Pi_{a:\Omega} \text{PartialQ}$$

by

$$\llbracket Q_C \rrbracket_{\mathcal{M}}^C(a) := \begin{cases} \text{some}\left(P_{\mu_{\mathcal{M}}^{x,a}}(F_{Y,a} \mid F_{Z,a})\right), & P_{\mu_{\mathcal{M}}^{x,a}}(F_{Z,a}) > 0, \\ \text{none}, & P_{\mu_{\mathcal{M}}^{x,a}}(F_{Z,a}) \sim 0. \end{cases}$$

Thus joint kernels are always defined. A conditional value is absent exactly when its conditioning fibre has probability zero.

Interpretation. The two displays are the finite meanings of

$$P_{\mathcal{M}}(Y = a|_Y \mid \text{do}(X = a|_X))$$

and

$$P_{\mathcal{M}}(Y = a|_Y \mid \text{do}(X = a|_X), Z = a|_Z),$$

respectively. Varying a over Ω supplies every pointwise value of the kernel.

Lean implementation. The query records are converted to **Kernel**; its reference-induced intervention and fibre classifier are **Kernel.intervention** and **Kernel.agreesOn**. The partial result type is **ProbabilityResult.Result**, and evaluation is **Kernel.denote**.

Definition A.5.7 (Fixed-event kernel reading). A joint kernel records enough pointwise values to recover the probability of any fixed local outcome event.

Data. Let $x : \text{Do}(\mathcal{V})$ be a hard intervention of Definition A.5.2, and write $T_x := \text{targets}(x)$. Write $\Omega := \text{Val}(\mathcal{V})$ for the complete-assignment type of that endogenous signature. Let $Y : V \rightarrow \text{Bool}$ be a finite node selection of Definition A.4.2, with $h_{TY} : \text{Disjoint}(T_x, Y)$. Through the selected-family presentation of that definition, Y determines a coordinate selection, so **LocalEvent**(Y) of Definition A.3.28 is well-typed.

Joint event. Let $E : \text{LocalEvent}(Y)$ and let $E^\uparrow : \Omega \rightarrow \text{Bool}$ be its cylinder lift. Write

$$\nu_x := \text{law}(\text{InterventionResult}_{\mathcal{M}}(x)).$$

Its interventional value in $\mathcal{M}$ is:

$$P_{\nu_x}(E^\uparrow).$$

Associated joint kernel. Discarding the replacement values in x and the particular event E yields

$$Q = (Y, T_x, h_{TY}) : \text{Kernel}(\mathcal{V})$$

of Definition A.5.5, instantiating the record field h_{XY} by h_{TY} .

Recovery. By the fibre sum of Remark A.3.27,

$$P_{\nu_x}(E^\uparrow) \sim \sum_{s:\text{Val}(Y)} \mathbf{1}_E(s) P_{\nu_x}(\text{Fibre}_Y(s)).$$

Each fibre mass equals the joint denotation of Q in Definition A.5.6 at any complete assignment that realises s together with the action values of x .

Lean implementation. The record **InterventionalQuery** stores the fixed intervention and local event; its **kernelQuery** projection forgets their particular values.

A. Definitions

Remark A.5.8 (Conditional fixed-event scope). A conditional kernel likewise supplies pointwise conditional fibre values, but those values do not by themselves determine every conditional event probability.

Data. Retain x, T_x, Y, E, h_{TY} and ν_x from Definition A.5.7. Let $Z : V \rightarrow \text{Bool}$ carry evidence $h_{TZ} : \text{Disjoint}(T_x, Z)$ and $h_{YZ} : \text{Disjoint}(Y, Z)$, and let $C : \text{LocalEvent}(Z)$. Write $E^\uparrow$ and $C^\uparrow$ for their cylinder lifts.

Conditional value. The partial rational

$$\begin{cases} \text{some}(P_{\nu_x}(E^\uparrow \mid C^\uparrow)), & P_{\nu_x}(C^\uparrow) > 0, \\ \text{none}, & P_{\nu_x}(C^\uparrow) \sim 0 \end{cases} : \text{PartialQ},$$

is **none** exactly when $P_{\nu_x}(C^\uparrow) = 0$, and is not **some**(0) in that case.

Associated conditional kernel. Discarding the replacement values in x and the events E, C yields

$$Q_C = (Y, T_x, Z, h_{TY}, h_{TZ}, h_{YZ}) : \text{CondKernel}(\mathcal{V}),$$

denoting $P(Y \mid \text{do}(T_x), Z)$.

Conditional recovery. At each $a : \Omega$, that kernel stores $P(F_{Y,a} \mid F_{Z,a})$ as **some**(p) when $P(F_{Z,a}) > 0$. It does not store the masses $P(F_{Z,a})$. Being able to compute a particular $P(F_{Y,a} \mid F_{Z,a})$ does not make it possible to compute $P(E \mid \text{do}(x), C)$ for an arbitrary local event C on Z . In other words, identifiability of $P(Y \mid \text{do}(T_x), Z)$ does not give identifiability of an arbitrary conditional event.

Lean implementation. The record **ConditionalQuery** stores the fixed local events; its **kernelQuery** projection forgets those events and the replacement values.

Definition A.5.9 (Local kernel support). Conditional kernels require a partial value because their conditioning fibres need not have positive probability.

Definedness. Let Ω be a complete-assignment type, let $d : \Omega \rightarrow \text{PartialQ}$, and let $a : \Omega$. Define

$$\text{DefinedAt}(d, a) := \Sigma_{p : \mathbb{Q}_{\geq 0}} (d(a) \sim \text{some}(p)),$$

using the partial-value relation of Definition A.5.4. An inhabitant supplies a rational p , not a partial value: allowing $p = \text{none}$ would count an undefined result as defined.

Kernel instances. For a kernel query Q of Definition A.5.5 in a model $\mathcal{M}$, write $\text{DefinedAt}_{\mathcal{M}}(Q, a)$ for definedness of the corresponding denotation of Definition A.5.6 at a . Every joint denotation is defined at every assignment. A conditional denotation is defined exactly when its conditioning fibre has positive probability.

Lean implementation. Partial values and their comparison are **ProbabilityResult.Result** and **ProbabilityResult.Equivalent**. Pointwise definedness is **ProbabilityTerm.SupportedAt**.

Definition A.5.10 (Kernel comparison). Kernel comparison is pointwise agreement of denotations, on one shared endogenous signature.

Data. Let $\mathcal{V}$ be an endogenous signature in the sense of Definition A.4.13, with node type V and value family Val . Let $\mathcal{M}$ and $\mathcal{N}$ be structural causal models of Definition A.4.18 on that signature, and write $\Omega := \text{Val}(\mathcal{V})$.

Agreement where defined. For denotations $d, d' : \Omega \rightarrow \text{PartialQ}$, define

$$\begin{aligned} & \text{AgreesWhereDefined}(d, d') \\ & := \Pi_{a:\Omega} (\text{DefinedAt}(d, a) \rightarrow \text{DefinedAt}(d', a) \rightarrow d(a) \sim d'(a)). \end{aligned}$$

Neither premise is leading: the type constrains only those assignments at which both denotations are defined. In particular, if d is defined at a single assignment and d' is defined everywhere, and the two values agree there, the type is inhabited. The relation $\simeq$ below is stronger: it also requires matching **none/some**.

Pointwise equivalence. Write $d \simeq d'$ for the type

$$\Pi_{a:\Omega} (d(a) \sim d'(a)).$$

A mixed **some/none** comparison reduces to $\text{Fin}(0)$, so this stronger relation also requires matching definedness.

Kernels. Conditional kernels of Definition A.5.5 are compared by **AgreesWhereDefined** of the two conditional denotations of Definition A.5.6. For a joint kernel Q_J , the denotation is always defined, so the same comparison coincides with $\llbracket Q_J \rrbracket_{\mathcal{M}}^J \simeq \llbracket Q_J \rrbracket_{\mathcal{N}}^J$.

Lean implementation. Conditional queries compare by **AgreesOnCommon Support** or the stronger **ResultEquivalent**; joint queries compare by **ValueEquivalent**.

Remark A.5.11 (Intervention graphs and query laws). One intervention $x : \text{Do}(\mathcal{V})$ determines the result record $\text{InterventionResult}_{\mathcal{M}}(x)$ of Definition A.5.3. The constructions of this section are compared here.

Graphs.

Construction	Contains	Action
$\mathcal{G}$ Definition A.4.11	endogenous nodes; directed and bidirected edges	none
$\mathcal{G}_{T_x}$ Definition A.5.1	endogenous nodes; directed and bidirected edges	remove incoming arrows into T_x
$\mathcal{K}$ Definition A.4.18	endogenous nodes and edges; exogenous roots; incidence; mechanisms	none
$\mathcal{K}_x$ Definition A.5.3	endogenous nodes and edges; exogenous roots; incidence; mechanisms	remove incoming incidence into T_x ; replace mechanisms by $\mathcal{F}_x$

Remark A.5.12 (Interventional laws and query values). An intervention result also supplies the probability law from which joint and conditional kernel values are evaluated.

Queries.

Object	Meaning	Definedness
law Definition A.5.3	$P(\cdot \mid \text{do}(x))$	always
$\llbracket Q_J \rrbracket^J$ Definition A.5.6	$P(Y \mid \text{do}(X))$ at each a	always some (p)
$\llbracket Q_C \rrbracket^C$ Definition A.5.6	$P(Y \mid \text{do}(X), Z)$ at each a	some (p) when the Z -fibre has positive mass

A. Definitions

Thus

$$x \longrightarrow \text{InterventionResult}_{\mathcal{M}}(x) = (\text{model}, \text{graph}, \text{law})$$

and a joint event is recovered from `law` as in Definition A.5.7, not as a fourth kernel.

Identification and hedge criteria

Definition A.5.13 (Compatible models). Compatibility records that a structural causal model realises a given causal graph.

Data. Let $\mathcal{V}$ be an endogenous signature in the sense of Definition A.4.13, with node type V , and let $\mathcal{G}$ be a causal graph on that signature in the sense of Definition A.4.11. Let $\mathcal{M} = (\mathcal{K}, \mathcal{F}) : \text{SCM}(V, U)$ be a structural causal model of Definition A.4.18, and let L be the exogenous incidence classifier of $\mathcal{K}$ in the sense of Definition A.4.15. Write $\pi(\mathcal{K})$ for the latent projection of Definition A.4.23. The model's value types and directed classifier agree with those of $\mathcal{G}$ by construction.

Children of a root. For $\ell : U$, let $S_\ell : V \rightarrow \text{Bool}$ be the node selection $S_\ell(c) := L(\ell, c)$ of Definition A.4.2. Write m_{S_ℓ} as in that definition.

Compatibility record. Define the record $\text{Compatible}_{\mathcal{G}}(\mathcal{M})$ with fields:

$$\begin{aligned} \text{bounded} &: \Pi_{\ell:U} (m_{S_\ell} \leq 2), \\ \text{projected} &: \Pi_{i,j:V} \text{Id}_{\text{Bool}}(E_{\pi(\mathcal{K})}^{\leftrightarrow}(i, j), E_{\mathcal{G}}^{\leftrightarrow}(i, j)). \end{aligned}$$

The first field says that every exogenous root is incident to at most two endogenous nodes, using m_{S_ℓ} of Definition A.4.2. The second says that the projected bidirected classifier of Definition A.4.23 agrees with that of $\mathcal{G}$.

Model type. The type of compatible models is indexed by $\mathcal{G}$, not by a single exogenous type:

$$\text{Mod}(\mathcal{G}) := \Sigma_U \Sigma_{\mathcal{M}:\text{SCM}(V,U)} \text{Compatible}_{\mathcal{G}}(\mathcal{M}).$$

The first sum ranges over exogenous types; V is the node type of $\mathcal{G}$.

Lean implementation. The model type is `ExactModel`, which packs the exogenous signature as a field. Compatibility is `Compatible`, the product of `IsCanonicalSemiMarkovian` and `HasProjectedGraph`.

Definition A.5.14 (Observational equivalence). Two compatible models are observationally equivalent when they assign equivalent probabilities to every decidable observed event.

Data. Let $\mathcal{V}$ be the endogenous signature of Definition A.5.13, and write $\Omega := \text{Val}(\mathcal{V})$. Let $\mathcal{G}$ be a causal graph on that signature, and let $\mathcal{M}, \mathcal{N} : \text{Mod}(\mathcal{G})$. Write $\mu_{\mathcal{M}}$ and $\mu_{\mathcal{N}}$ for their observational assignments of Definition A.4.21. Both are probability assignments on this same Ω : every compatible model reuses $\mathcal{V}$.

Equivalence. Define $\mathcal{M} \equiv_{\text{obs}} \mathcal{N}$ by

$$\Pi_{E:\Omega \rightarrow \text{Bool}} (P_{\mu_{\mathcal{M}}}(E) \sim P_{\mu_{\mathcal{N}}}(E)),$$

using event probability of Definition A.3.11 and the rational equivalence $\sim$ of Definition A.3.8.

Lean implementation. Observational equivalence is `ObservationallyEquivalent`.

Definition A.5.15 (Identification). A kernel is identifiable from a graph when observationally equivalent compatible models agree on its denotation.

Joint kernels. Let $Q_J : \text{Kernel}(\mathcal{V})$ be a joint kernel of Definition A.5.5, and let $\mathcal{G}$ be as in Definition A.5.13. It is identifiable from $\mathcal{G}$ when, for every $\mathcal{M}, \mathcal{N} : \text{Mod}(\mathcal{G})$,

$$\mathcal{M} \equiv_{\text{obs}} \mathcal{N} \implies \llbracket Q_J \rrbracket_{\mathcal{M}}^J \simeq \llbracket Q_J \rrbracket_{\mathcal{N}}^J,$$

using observational equivalence of Definition A.5.14 and the denotation and comparison of Definitions A.5.6 and A.5.10.

Conditional kernels. A conditional kernel $Q_C : \text{CondKernel}(\mathcal{V})$ is identifiable when, for every $\mathcal{M}, \mathcal{N} : \text{Mod}(\mathcal{G})$,

$$\mathcal{M} \equiv_{\text{obs}} \mathcal{N} \implies \text{AgreesWhereDefined}(\llbracket Q_C \rrbracket_{\mathcal{M}}^C, \llbracket Q_C \rrbracket_{\mathcal{N}}^C).$$

This is weaker than pointwise $\simeq$: the latter would additionally force the two models to have identical definedness domains.

Events. A joint event of Definition A.5.7 is a derived application: if its associated kernel is identifiable, the event value is determined by the fibre sum of Remark A.3.27 at the action values of x . A conditional event is not recovered from the conditional kernel alone, as in that remark. The converse is not asserted: agreement on one event does not imply agreement of the whole kernel [Pea09; SP06; TS09].

Lean implementation. Identifiability is **Identifiable**; the conditional comparison is **ConditionalIdentifiable**.

Definition A.5.16 (c-component). A c-component is a nonempty node selection that is connected by bidirected paths of Definition A.4.7.

Data. Let $\mathcal{G}$ be a semi-Markovian causal graph in the sense of Definition A.4.11, with node type V , and let $S : V \rightarrow \text{Bool}$ be a finite node selection of Definition A.4.2. Write $\mathcal{G} \mid S$ for the induced restriction of Definition A.4.12, and m_S as in Definition A.4.2.

Connectedness. Define

$$\text{Connected}_{\mathcal{G}}^{\leftrightarrow}(S) := \text{Fin}(m_S) \times \prod_{k, \ell : \text{Fin}(m_S)} \text{Path}_{E_{\mathcal{G} \mid S}^{\leftrightarrow}}(k, \ell).$$

An inhabitant of $\text{Fin}(m_S)$ is a selected node, so S is nonempty. The product says every pair of selected nodes, including a node with itself, is joined by a bidirected path of Definition A.4.7 in $\mathcal{G} \mid S$. Singleton paths of that definition make every singleton selection connected.

Interpretation. The type is bidirected connectedness of S . Connectedness is not maximality: a connected selection may sit inside a larger connected one. An inhabitant is maximal when adjoining any unselected node breaks connectedness. A conventional c-component is a maximal inhabitant, which is not required in this book.

Running example. Let S_{PQ} select $\{\text{Pr}, \text{Qu}\}$, S_{Pr} select only Pr , and S_{Top} select only Top . All three inhabit $\text{Connected}_{\mathcal{G}}^{\leftrightarrow}$: S_{PQ} by the bidirected edge; the other two by a singleton path. The selection S_{Pr} is not maximal: adjoining Qu stays connected. The selection S_{PQ} is maximal: no other Mira node is joined to Pr or Qu by a bidirected path. The selection S_{Top} is maximal: Top has no bidirected edge.

A. Definitions

Lean implementation. The internal-connectedness predicate is `BidirectedComponent`, generated inductively; it is inhabited exactly when the path type above is inhabited.

Definition A.5.17 (c-forest). A c-forest is a bidirected-connected node selection in which every selected node has at most one selected directed child.

Data. Let $\mathcal{G}$ be a finite semi-Markovian causal graph in the sense of Definition A.4.11, with node type V , and let $S, R : V \rightarrow \text{Bool}$ be finite node selections of Definition A.4.2. The selection S names the forest nodes and R names its roots.

Helpers. Define

$$\text{ChildIn}_{\mathcal{G}}(S, i, j) := \text{Id}_{\text{Bool}}(S(i), \text{true}) \times \text{Id}_{\text{Bool}}(S(j), \text{true}) \times \text{Step}_{E_{\mathcal{G}}}^{\leftrightarrow}(i, j)$$

and

$$\begin{aligned} \text{AtMostOneChild}_{\mathcal{G}}(S) &:= \prod_{i, j_1, j_2 : V} \text{ChildIn}_{\mathcal{G}}(S, i, j_1) \\ &\rightarrow \text{ChildIn}_{\mathcal{G}}(S, i, j_2) \rightarrow \text{Id}_V(j_1, j_2). \end{aligned}$$

Write

$$\text{Sink}_{\mathcal{G}}(S, r) := \text{Id}_{\text{Bool}}(S(r), \text{true}) \times \prod_{j : V} \neg \text{ChildIn}_{\mathcal{G}}(S, r, j).$$

Forest type. Define the record $\text{CForest}_{\mathcal{G}}(S, R)$ with fields:

$$\begin{aligned} \text{connected} &: \text{Connected}_{\mathcal{G}}^{\leftrightarrow}(S), \\ \text{oneChild} &: \text{AtMostOneChild}_{\mathcal{G}}(S), \\ \text{roots} &: \prod_{r : V} (\text{Id}_{\text{Bool}}(R(r), \text{true}) \leftrightarrow \text{Sink}_{\mathcal{G}}(S, r)). \end{aligned}$$

The field `connected` is that of Definition A.5.16. Thus $R(r) = \text{true}$ if and only if r is a node of $\mathcal{G} \mid S$ with no outgoing directed edge in $\mathcal{G} \mid S$. The forest does not carry an independently chosen edge set.

Lean implementation. These fields are the `component`, `at_most_one_child` and `roots_exact` fields of `CForest $\mathcal{G}$  S R`.

Definition A.5.18 (Hedge). A hedge is two nested c-forests of Definition A.5.17 with common roots: the large family meets the action selection X , the small family avoids X , and every selected root reaches the outcome selection Y after incoming arrows into X are deleted.

Data. Let $Q = (Y, X, h_{XY}) : \text{Kernel}(V)$ be a joint kernel of Definition A.5.5, and let $\mathcal{G}$ be a finite semi-Markovian causal graph of Definition A.4.11 on the same node type V . A hedge for Q in $\mathcal{G}$ consists of three Boolean node selections

$$S_{\text{large}}, S_{\text{small}}, R : V \rightarrow \text{Bool}.$$

Hedge type. Define the record $\text{Hedge}_{\mathcal{G}}(Q)$ with fields:

$$\begin{aligned} \text{large} &: \text{CForest}_{\mathcal{G}}(S_{\text{large}}, R), \\ \text{small} &: \text{CForest}_{\mathcal{G}}(S_{\text{small}}, R), \\ \text{included} &: \prod_{i : V} \text{Id}_{\text{Bool}}(S_{\text{small}}(i), \text{true}) \rightarrow \text{Id}_{\text{Bool}}(S_{\text{large}}(i), \text{true}), \\ \text{meets}_X &: \prod_{i : V} \text{Id}_{\text{Bool}}(S_{\text{large}}(i), \text{true}) \times \text{Id}_{\text{Bool}}(X(i), \text{true}), \\ \text{avoids}_X &: \prod_{i : V} \text{Id}_{\text{Bool}}(S_{\text{small}}(i), \text{true}) \rightarrow \text{Id}_{\text{Bool}}(X(i), \text{false}), \\ \text{reaches}_Y &: \prod_{r : V} \text{Id}_{\text{Bool}}(R(r), \text{true}) \rightarrow \prod_{y : V} \text{Id}_{\text{Bool}}(Y(y), \text{true}) \times \text{Path}_{D_{\mathcal{G}_X}}^{\rightarrow}(r, y). \end{aligned}$$

The fields **large** and **small** are c-forests of Definition A.5.17. The last field uses a directed path of Definition A.4.7 in the mutilated graph $\mathcal{G}_{\overline{X}}$ of Definition A.5.1.

Interpretation. Inhabiting this type is the finite graphical obstruction to identifying Q [TS09]; it is not the identifiability judgement of Definition A.5.15. The fields **meets_X** and **avoids_X** together with inclusion make the small family a proper part of the large family.

Lean implementation. This dependent record is **HedgeWitness**.

Do-calculus syntax and derivations

The kernel queries of Definition A.5.5 are the meanings. The expressions below are the syntactic counterparts used for algebraic and do-calculus proofs. Their intended denotation is Definition A.5.6; identifiability is Definition A.5.15. The final definition packages proofs of equality in this syntax.

Definition A.5.19 (Finite probability-expression syntax). Finite probability expressions are the objects of do-calculus equality.

Data. Let $\mathcal{V}$ be an endogenous signature of Definition A.4.13. Node selections are those of Definition A.4.2.

Syntax. Define the type $\text{Expr}(\mathcal{V})$ inductively. A symbolic kernel constructor has the form

$$P(Y \mid \text{do}(X), W),$$

where X, Y, W are pairwise disjoint finite node selections. The other constructors generate expressions from zero, kernels of Definition A.5.5, marginalisation, evaluation at a fixed complete assignment, addition, multiplication and defined division.

Events. Evaluation at fixed assignments and finite addition make every decidable Boolean event expressible: enumerate its satisfying complete assignments and add their singleton kernel masses, as in the fibre sum of Remark A.3.27. The empty event is represented by zero.

Lean implementation. The shared syntax is **ProbabilityTerm**, with a partial finite-rational denotation.

Definition A.5.20 (Do-calculus rules). Pearl's three rules are relations on the expressions of Definition A.5.19, with side conditions in mutilated graphs of Definition A.5.1.

Reading. The displays use Pearl's conventional equality notation. In the finite rational interpretation, each equality means pointwise $\sim$ of partial denotations of Definitions A.5.4 and A.5.6 on their local positive support of Definition A.5.9; a zero denominator has no conditional value.

Rule-three action selection. For node selections X, Z, W , let $D_{\mathcal{G}_{\overline{X}}}$ be the directed part of the graph after deleting arrows into X . Define:

$$Z(W) := Z \setminus \text{An}_{D_{\mathcal{G}_{\overline{X}}}}(W),$$

using ancestors of a selection from Definition A.4.8. Thus $Z(W)$ selects exactly those nodes of Z that are not ancestors of any node selected by W in $\mathcal{G}_{\overline{X}}$.

Rules. The three transformation rules [Pea09; Pea12] are:

A.5.20.1. insertion and deletion of observations:

$$P(y \mid \text{do}(x), z, w) = P(y \mid \text{do}(x), w)$$

A. Definitions

whenever $(Y \perp\!\!\!\perp Z \mid X, W)_{\mathcal{G}_{\overline{X}}}$;

A.5.20.2. action and observation exchange:

$$P(y \mid \text{do}(x), \text{do}(z), w) = P(y \mid \text{do}(x), z, w)$$

whenever $(Y \perp\!\!\!\perp Z \mid X, W)_{\mathcal{G}_{\overline{X}, \underline{Z}}}$;

A.5.20.3. insertion and deletion of actions:

$$P(y \mid \text{do}(x), \text{do}(z), w) = P(y \mid \text{do}(x), w)$$

whenever $(Y \perp\!\!\!\perp Z \mid X, W)_{\mathcal{G}_{\overline{X}, \overline{Z(W)}}}$.

The side conditions use d-separation of Definition A.4.28 in the indicated mutilated graph. Write $\text{RuleSeparation}_k(X, Y, Z, W)$ for the d-separation type displayed for each of the three numbered rules, and use $\text{FourWayDisjoint}(X, Y, Z, W)$ from Definition A.3.3 for its pairwise-disjointness evidence.

Lean implementation. Rule applications are `DoRuleApplication`, parameterised by `RuleSeparation`.

Definition A.5.21 (Do-calculus derivation). A derivation is an inductively generated equality between expressions of Definition A.5.19.

Data. Let $\mathcal{G}$ be a finite causal graph in the Markovian or semi-Markovian sense of Definition A.4.11, and let $s, t : \text{Expr}(\mathcal{V})$.

Derivation type. Define $\text{Derive}_{\mathcal{G}}(s, t)$ as the inductively generated type whose equality constructors are:

$$\begin{aligned} \text{refl}_s &: \text{Derive}_{\mathcal{G}}(s, s), \\ \text{sym}(D) &: \text{Derive}_{\mathcal{G}}(t, s) \quad \text{for } D : \text{Derive}_{\mathcal{G}}(s, t), \\ \text{trans}(D_1, D_2) &: \text{Derive}_{\mathcal{G}}(s, r) \quad \text{for } D_1 : \text{Derive}_{\mathcal{G}}(s, t), D_2 : \text{Derive}_{\mathcal{G}}(t, r). \end{aligned}$$

The remaining constructor families are indexed by the corresponding expression constructors and comprise:

A.5.21.1. marginalisation, formal conditioning and the chain rule, with finite disjointness conditions; division is undefined at a complete assignment when the denominator evaluates to `none` or to `some(0)`;

A.5.21.2. congruence under marginalisation, fixed-assignment evaluation, addition, multiplication and division;

A.5.21.3. one of Pearl’s three rules from Definition A.5.20, carrying pairwise-disjointness evidence for X, Y, Z, W and an inhabitant of d-separation of Definition A.4.28 in the required mutilated graph of Definition A.5.1.

Interpretation. A derivation is evidence that two expressions are equal, not an evaluation of either expression. Soundness (Appendix B) says the two denotations agree on compatible models. It is not the identifiability judgement of Definition A.5.15; completeness later says an identifiable kernel has such a derivation.

Lean implementation. The inductive type is `DoCalculusDerivation`, parameterised by `RuleSeparation`; omitting that argument selects `executableRuleSeparation`, while `PathDoRuleApplication` and `PathDoCalculusDerivation` specialise the same syntax to active-path evidence. Definedness of an interpreted expression is `ProbabilityTerm.SupportedAt`.

A.6 Counterfactual semantics

This section defines finite counterfactual queries through abduction, action and prediction. Potential outcomes and finite Boolean multiworld syntax are the basic objects; the ordinary one-action query is recovered as a derived case. Twin networks and occurrence-indexed multiworlds supply two explicit semantics in which every counterfactual world shares the same exogenous assignment while carrying its own intervention.

Counterfactual queries

Definition A.6.1 (Potential outcome). A potential outcome evaluates selected endogenous coordinates after a hard intervention while retaining the same exogenous assignment.

Data. Let $\mathcal{M}$ be a structural causal model of Definition A.4.18, let $x : \text{Do}(\mathcal{V})$ be a hard intervention of Definition A.5.2, and let $\mathcal{Y}$ be a coordinate selection of its endogenous variables in the sense of Definition A.3.24. Let $u : \text{Val}(\mathcal{U})$ be an exogenous assignment.

Potential outcome. Define:

$$Y_x(u) := \pi_{\mathcal{Y}}(\text{sol}_{\mathcal{M}_x}(u)) : \text{Val}(\mathcal{Y}).$$

Here $\mathcal{M}_x$ is the intervened model of Definition A.5.3 and $\pi_{\mathcal{Y}}$ is the coordinate projection of Definition A.3.24.

Interpretation. First solve the intervened model at u ; then retain only the coordinates selected by $\mathcal{Y}$. The subscript x records the changed mechanisms. Keeping the argument u fixed is what makes the factual and counterfactual outcomes concern the same exogenous unit.

Definition A.6.2 (Counterfactual atom). A counterfactual atom states one value claim under one intervention.

Atom type. Let $\mathcal{V}$ be an endogenous signature of Definition A.4.13, with node type V . Define the record $\text{CFAtom}(\mathcal{V})$ with fields:

action : $\text{Do}(\mathcal{V})$,
 node : V ,
 value : $\text{Val}(\text{node})$.

An atom $A = (x, i, v)$ is written $[i_x = v]$.

Definition A.6.3 (Finite counterfactual event syntax). Finite counterfactual event syntax combines finitely many atoms of Definition A.6.2, which may carry different interventions.

Formation rules. Define the inductive type $\text{CFEvent}(\mathcal{V})$ by the constructors:

true, false : $\text{CFEvent}(\mathcal{V})$,
 atom : $\text{CFAtom}(\mathcal{V}) \rightarrow \text{CFEvent}(\mathcal{V})$,
 and, or : $\text{CFEvent}(\mathcal{V}) \rightarrow \text{CFEvent}(\mathcal{V}) \rightarrow \text{CFEvent}(\mathcal{V})$,
 not : $\text{CFEvent}(\mathcal{V}) \rightarrow \text{CFEvent}(\mathcal{V})$.

Evaluation. For a structural causal model $\mathcal{M}$ and an exogenous assignment u , the atomic case returns true exactly when

$$\text{sol}_{\mathcal{M}_x}(u)(i) = v.$$

A. Definitions

The remaining cases use Boolean elimination from Definition A.1.20. Recursive evaluation therefore gives a classifier:

$$\llbracket \gamma \rrbracket_{\mathcal{M}} : \text{Val}(\mathcal{U}) \rightarrow \text{Bool},$$

in which every atom is evaluated at the same u .

Interpretation. A term of $\text{CFEvent}(\mathcal{V})$ is a finite syntax tree, not yet a Boolean event in the sense of Definition A.3.2. Evaluation in $\mathcal{M}$ interprets that tree as such an event on the exogenous assignment type $\text{Val}(\mathcal{U})$. The constructors **and**, **or** and **not** are interpreted by the corresponding Boolean event operations of Definition A.3.3.

Lean implementation. The finite event syntax is implemented by `CounterfactualAtom` and `CounterfactualEvent`; event evaluation is the corresponding `holds` declaration.

Definition A.6.4 (Finite multiworld counterfactual query). A finite multiworld query pairs an outcome event with a conditioning event and evaluates their conditional probability under one shared exogenous assignment.

Data. Let $\mathcal{V}$ be an endogenous signature and let $\mathcal{M}$ be a structural causal model on that signature. Use the event syntax of Definition A.6.3.

Query type and denotation. Define the record $\text{CFQuery}(\mathcal{V})$ with fields:

$$\begin{aligned} \text{outcome} &: \text{CFEvent}(\mathcal{V}), \\ \text{condition} &: \text{CFEvent}(\mathcal{V}). \end{aligned}$$

For $Q = (\gamma, \delta)$, its model-level denotation is the partial probability value:

$$\llbracket Q \rrbracket_{\mathcal{M}} := \begin{cases} \text{some}(P_{\mu_{\mathcal{U}}}(\llbracket \gamma \rrbracket_{\mathcal{M}} \mid \llbracket \delta \rrbracket_{\mathcal{M}})), & P_{\mu_{\mathcal{U}}}(\llbracket \delta \rrbracket_{\mathcal{M}}) > 0, \\ \text{none}, & P_{\mu_{\mathcal{U}}}(\llbracket \delta \rrbracket_{\mathcal{M}}) \sim 0. \end{cases} : \text{PartialQ}.$$

The result type and its equivalence relation are those of Definition A.5.4.

Interpretation. The outcome and condition may mention several interventions, but all their atoms are evaluated at one shared exogenous assignment. The conditioning event selects the retained exogenous units; if it has zero probability, the partial query value is **none**.

Lean implementation. The query record and its evaluator are `CounterfactualQuery` and its `denote` declaration.

Definition A.6.5 (Factual evidence pullback). Factual evidence on endogenous assignments induces an event on the exogenous assignments that generate them.

Data. Let $\mathcal{M}$ be a structural causal model and let $E : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$ be a factual event.

Pullback event. Define:

$$E_{\mathcal{M}}^{\mathcal{U}}(u) := E(\text{sol}_{\mathcal{M}}(u)).$$

This Boolean event selects exactly those exogenous assignments whose factual solution satisfies E .

Interpretation. The construction evaluates the factual model first and then tests its endogenous result. It therefore transfers an event from the endogenous assignment type back to the exogenous assignment type without changing the model.

Definition A.6.6 (One-action counterfactual query). An ordinary abduction–action–prediction query is a derived case of the finite syntax of Definition A.6.4.

Data. Let $\mathcal{M}$ be a structural causal model, let $E, F : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$ be factual-evidence and target events, and let $x : \text{Do}(\mathcal{V})$ be a hard intervention.

Finite compilation. Finite elimination over complete assignments constructs counterfactual events $\gamma_{x,F}$ and δ_E , together with correctness terms:

$$\begin{aligned} h_\gamma &: \Pi_{u:\text{Val}(\mathcal{U})} \text{Id}_{\text{Bool}} \left(\llbracket \gamma_{x,F} \rrbracket_{\mathcal{M}}(u), F(\text{sol}_{\mathcal{M}_x}(u)) \right), \\ h_\delta &: \Pi_{u:\text{Val}(\mathcal{U})} \text{Id}_{\text{Bool}} \left(\llbracket \delta_E \rrbracket_{\mathcal{M}}(u), E(\text{sol}_{\mathcal{M}}(u)) \right). \end{aligned}$$

The compiler therefore proves the two Boolean values identical inside the type theory. It does not require them to become the same expression merely by unfolding their definitions. Define the one-action query:

$$\text{singleAction}(E, x, F) := (\gamma_{x,F}, \delta_E) : \text{CFQuery}(\mathcal{V}).$$

Abduction. Use the factual evidence pullback $E_{\mathcal{M}}^{\mathcal{U}}$ of Definition A.6.5. If this event has positive probability, conditioning the exogenous urn as in Definition A.3.19 gives:

$$\mu_{\mathcal{U}|E} := (\mu_{\mathcal{U}})_{E_{\mathcal{M}}^{\mathcal{U}}},$$

and the present value of the query is conventionally written:

$$P_{\mathcal{M}}(F_x \mid E) := P_{\mu_{\mathcal{U}|E}}(F \circ \text{sol}_{\mathcal{M}_x}).$$

Without positive evidence, its denotation is **none**. This is a calculation inside $\mathcal{M}$, not an identifiability judgement.

Interpretation. Abduction restricts the exogenous belief to units whose factual outcome satisfies E . Action evaluates those same units in the intervened model $\mathcal{M}_x$. Prediction then tests whether the resulting endogenous assignment satisfies F .

Running example. In Mira’s model, take E to assert $\text{Pr} = 0$, $\text{Qu} = 1$ and $\text{Off} = 0$, let x set Pr to 1, and let F select $\text{Off} = 1$. The query is written $P_{\mathcal{M}_{\text{Mira}}}(\text{Off}_{\text{Pr}=1} = 1 \mid E)$.

Lean implementation. The derived constructor is **CounterfactualQuery**. **singleAction**. Its agreement with the direct, twin-network and occurrence-indexed semantics is established in Propositions B.10.1 and B.10.2.

Counterfactual identification and support

Definition A.6.7 (Counterfactual identifiability). Counterfactual identifiability compares the partial denotation of one query across the compatible models of a causal graph.

Data. Let $\mathcal{G}$ be a causal graph of Definition A.4.11 and let $Q : \text{CFQuery}(\mathcal{V})$.

Identifiability. Define $\text{CFIdentifiable}_{\mathcal{G}}(Q)$ to be the type:

$$\Pi_{\mathcal{M}, \mathcal{N} : \text{Mod}(\mathcal{G})} (\mathcal{M} \equiv_{\text{obs}} \mathcal{N}) \rightarrow (\llbracket Q \rrbracket_{\mathcal{M}} \sim \llbracket Q \rrbracket_{\mathcal{N}}),$$

using compatible models and observational equivalence from Definitions A.5.13 and A.5.14.

A. Definitions

Interpretation. This is a comparison across models, not the calculation of Definition A.6.6 inside one selected model. It states that the counterfactual value is fixed by the shared observational assignment and causal graph throughout the compatible model family.

Lean implementation. The corresponding predicate is **Counterfactual Identifiable**.

Definition A.6.8 (Uniform counterfactual support). Uniform support requires one counterfactual query to have a present value throughout the compatible model family.

Data. Let $\mathcal{G}$ be a causal graph and let $Q : \text{CFQuery}(\mathcal{V})$ be as in Definition A.6.7.

Support type. Define $\text{CFSupported}_{\mathcal{G}}(Q)$ by:

$$\Pi_{\mathcal{M}:\text{Mod}(\mathcal{G})} \Sigma_{p:\mathbb{Q}_{\geq 0}} (\llbracket Q \rrbracket_{\mathcal{M}} \sim \text{some}(p)).$$

Interpretation. This is stronger than support in one chosen model: it requires the conditioning event to have positive probability throughout the compatible model family.

Lean implementation. The displayed constructive type retains a chosen witness for every compatible model. The predicate **CounterfactualSupported** instead stores each pointwise witness proof-irrelevantly under **Nonempty**; it is the truncated interface consumed by Lean, not a definitionally identical record.

Twin and multiworld semantics

Definition A.6.9 (Finite twin network). A finite twin network represents the factual and one counterfactual world in a single structural causal model.

Data and nodes. Let $\mathcal{M}$ be a structural causal model on node type V and let $x : \text{Do}(\mathcal{V})$. Define the two-world tag type $W_2 := \text{Bool}$, reading **false** as factual and **true** as counterfactual. The twin node type and value family are:

$$V^{\text{tw}} := W_2 \times V, \quad \text{Val}^{\text{tw}}(w, i) := \text{Val}(i).$$

Edges and mechanisms. Directed arrows are copied only between nodes carrying the same world tag. The factual copy retains the source mechanisms. In the counterfactual copy, every incoming endogenous and exogenous input at a target of x is removed and its mechanism is replaced by the selected constant. Both copies retain the same exogenous source and prior; its incidence is copied to each world except at a counterfactual target of x .

Evaluation. Evaluation at one latent assignment is therefore

$$u \longmapsto (\text{sol}_{\mathcal{M}}(u), \text{sol}_{\mathcal{M}_x}(u)).$$

Applying the latent projection of Definition A.4.23 derives bidirected edges between any two tagged nodes sharing an exogenous root, including nodes in different worlds.

Interpretation. The twin network is one finite structural causal model containing two linked worlds, not two independently sampled models. Their common exogenous assignment preserves the dependence needed to compare factual and counterfactual outcomes for the same unit.

Running example. For the intervention $\text{Pr} := 1$, the Mira twin network contains a factual and a counterfactual copy of every observed node. Both copies use the same U_B , but the counterfactual equation for Pr^c is constant, where the superscript c denotes the copy tagged by `true`. Its incoming uses of U_B and U_{Pr} are cut. The equation for Qu^c still uses the shared root U_B , which is what preserves the relevant cross-world dependence.

Lean implementation. This construction is `TwinNetwork`; its generic constructor is `FiniteLatentSCM.twinNetwork`. The dependent tagged-node network is independently encoded as an ordinary finite-indexed structural causal model by `TwinNetwork.Encoding.model`. The two presentations agree pointwise and on event probabilities, as established in Proposition B.10.1.

Definition A.6.10 (Occurrence-indexed multiworld). An occurrence-indexed multiworld retains a separate world for every syntactic atom occurrence of a finite counterfactual query.

Occurrence type. Let $Q : \text{CFQuery}(\mathcal{V})$ contain k atom occurrences in left-to-right order. Define

$$\text{Occ}(Q) := \text{Fin}(k), \quad W_Q := \text{Fin}(1) + \text{Occ}(Q),$$

where the left inhabitant is the factual world and the right summand contains the occurrence worlds. Its node type and value family are:

$$V^Q := W_Q \times V, \quad \text{Val}^Q(w, i) := \text{Val}(i).$$

Construction. Every world shares the source exogenous assignment. Directed arrows occur only within one world. The factual world retains the source mechanisms; an occurrence world uses the intervention stored by its atom and removes the corresponding incoming endogenous and exogenous inputs. Because occurrences, rather than extensionally equal interventions, index the worlds, repeated atoms remain distinct without deciding equality of dependent intervention functions. For a one-action query, its denotation agrees with the twin-network semantics of Definition A.6.9, although its occurrence-indexed representation may contain more than two worlds. Write $\mathcal{M}^Q$ for the resulting occurrence-indexed structural causal model.

Running example. A compound Mira event containing both $[\text{Off}_{\text{Pr}:=1} = 1]$ and $[\text{Off}_{\text{Fit}:=1} = 1]$ receives two occurrence worlds in addition to the factual world. The worlds share the same exogenous assignment but carry their two different settings. Repeating either atom syntactically would create another occurrence world even though its intervention data are extensionally equal.

Lean implementation. The construction is `OccurrenceMultiworld`; `OccurrenceMultiworld.Encoding.model` materialises it as an ordinary finite-indexed structural causal model. Its semantic adequacy is established in Propositions B.10.2, B.10.6 and B.10.8.

Remark A.6.11 (Counterfactual scope). Interventional queries alter mechanisms and then evaluate a resulting distribution. Counterfactual queries may additionally condition the shared exogenous assignment on factual evidence before evaluating one or more intervention worlds; an unconditional query uses the constantly true condition. They therefore occupy Pearl’s third level of inquiry. Appendix B states separately which external completeness assumption is needed for the supported finite fragment in Theorem B.11.8. The algorithms conventionally named ID^* and IDC^* are not reproduced here [SP08].

A.7 Modal causal representation

This section gives the finite causal constructions their modal interpretation. A causal mode carries a structural model, a current finite belief and a current intervention; modalities mark admitted transitions, while separately defined proof-carrying operations construct the corresponding record changes. This is the modal reading of the finite causal records, as in Section A.2.

Modal do-calculus traces

Definition A.7.1 (Causal query mode). A causal query mode records the three node selections determining a symbolic kernel.

Mode type. Let $\mathcal{V}$ be an endogenous signature with node type V . Define the record $\text{QueryMode}(\mathcal{V})$ with fields:

$$\begin{aligned} \text{outcome} &: V \rightarrow \text{Bool}, \\ \text{action} &: V \rightarrow \text{Bool}, \\ \text{condition} &: V \rightarrow \text{Bool}. \end{aligned}$$

Query updates. For node selections $X, W : V \rightarrow \text{Bool}$, define the endomaps:

$$\begin{aligned} \text{intervene}_X &: \text{QueryMode}(\mathcal{V}) \rightarrow \text{QueryMode}(\mathcal{V}), \\ \text{observe}_W &: \text{QueryMode}(\mathcal{V}) \rightarrow \text{QueryMode}(\mathcal{V}). \end{aligned}$$

The first leaves the outcome and condition fields unchanged and takes the union of X with the current action selection. The second leaves the outcome and action fields unchanged and takes the union of W with the current condition selection. Starting from $(Y, \emptyset, \emptyset)$ therefore constructs the mode (Y, X, W) of the kernel $P(Y \mid \text{do}(X), W)$.

Interpretation. A query mode annotates a kernel. It is distinct from a causal epistemic record, which stores a structural model, a belief and an intervention. The displayed maps update symbolic query annotations; they are not themselves modalities of Definition A.2.3 until they are placed in a specified mode theory.

Lean implementation. The modality and record types are `CausalQueryModality` and `CausalQueryMode`; the action and condition updates are `CausalQueryMode.lock`.

Definition A.7.2 (Modal do-calculus rule cell). A modal rule cell records one proof-carrying change between two causal query modes.

Cell type. Let $q, q' : \text{QueryMode}(\mathcal{V})$. Define $\text{ModalRuleCell}(q, q')$ as the type generated by three constructors corresponding to the rules of Definition A.5.20. For rule number k , write $q_k^L(X, Y, Z, W)$ and $q_k^R(X, Y, Z, W)$ for the query modes of its left- and right-hand kernels. The corresponding constructor has the dependent type:

$$\begin{aligned} \text{cell}_k &: \Pi_{X, Y, Z, W : V \rightarrow \text{Bool}} \\ &\quad \Pi_{h_{\text{disj}} : \text{FourWayDisjoint}(X, Y, Z, W)} \\ &\quad \Pi_{h_{\text{sep}} : \text{RuleSeparation}_k(X, Y, Z, W)} \\ &\quad \text{ModalRuleCell}(q_k^L(X, Y, Z, W), q_k^R(X, Y, Z, W)). \end{aligned}$$

Here `FourWayDisjoint` is the type of Definition A.3.3, and `RuleSeparationk` is the d-separation type introduced for rule k in Definition A.5.20.

Interpretation. An inhabitant is not merely a pair of query modes: it contains the disjointness and d-separation evidence that justifies one do-calculus rewrite between them. The first constructor relates kernels that differ by an observation, the second exchanges action and observation forms, and the third relates kernels that differ by an action. Each permitted symbolic change is therefore retained together with its graphical justification.

Lean implementation. The indexed cell type is `ModalRuleCell`.

Definition A.7.3 (Modal do-calculus trace). A modal trace annotates a do-calculus derivation of Definition A.5.21 by causal query modes and modal rule cells.

Trace type. Let D be a do-calculus derivation. Define the type `ModalTrace( $D$ )` recursively on D . A rule leaf carries an inhabitant of the corresponding cell type from Definition A.7.2; composition carries traces of both constituent derivations; and each probability-algebra constructor carries traces of its immediate subderivations. The source and target query modes at every stage are therefore determined by the annotated derivation.

Scope. A trace is an annotation of a derivation, not an operation that rewrites a structural model. The operations of this section realise those action and condition selections.

Running example. The Mira kernel $P(\text{Qu} \mid \text{do}(\text{Pr} = \text{true}))$ has action selection Pr and outcome selection Qu . A derivation involving that kernel records any change of action or condition selections as modal rule cells; the cells do not by themselves alter $\mathcal{M}_{\text{Mira}}$.

Lean implementation. Recursive traces are `ModalDerivationTrace`.

Causal epistemic records and modes

Definition A.7.4 (Causal signature). A causal signature packages the endogenous and exogenous coordinates on which a causal epistemic record is typed.

Signature record. Define `CausalSignature` as the record with fields:

```

endogenousType :  $\mathcal{U}$ ,
exogenousType :  $\mathcal{U}$ ,
endogenous : EndogenousSignature(endogenousType),
exogenous : ExogenousSource(exogenousType).
```

For $\mathcal{S} : \text{CausalSignature}$, write $V_{\mathcal{S}}$ and $U_{\mathcal{S}}$ for its two coordinate types, and write $\mathcal{V}_{\mathcal{S}}$ and $\mathcal{U}_{\mathcal{S}}$ for the corresponding signatures.

Interpretation. The record fixes the dependent coordinate systems on which a causal model, belief and intervention can be stored together. It does not itself contain a causal model or an epistemic state.

Definition A.7.5 (Finite causal epistemic record). A finite causal epistemic record stores a structural model together with the belief and intervention currently in force.

Record type. Let $\mathcal{S}$ be a causal signature of Definition A.7.4. Define the

A. Definitions

record $\text{CausalRecord}(\mathcal{S})$ with fields:

$$\begin{aligned} \text{model} &: \text{SCM}(\mathcal{V}_{\mathcal{S}}, \mathcal{U}_{\mathcal{S}}), \\ \text{belief} &: \text{Prob}_{\mathbb{Q}}^{\text{cd}}(\text{Val}(\mathcal{U}_{\mathcal{S}})), \\ \text{intervention} &: \text{Do}(\mathcal{V}_{\mathcal{S}}). \end{aligned}$$

The belief uses the common-denominator presentation of Definition A.3.13.

Interpretation. For $R : \text{CausalRecord}(\mathcal{S})$, write $R = (\mathcal{M}, b, x)$ for these three fields. These fields jointly contain the current causal and epistemic information. In particular, no separate relation graph is maintained: directed and latent links are part of the structural causal model, the belief records the presently available uncertainty, and the intervention records the structural settings currently in force.

Running example. Mira’s initial record is

$$R_{\text{Mira}} = (\mathcal{M}_{\text{Mira}}, \mu_{\mathcal{U}}, x_{\emptyset}),$$

where $x_{\emptyset}(i) := \text{none}$ is the empty hard intervention. Receiving evidence changes the second field; performing a setting changes the third; structural editing may change the first.

Lean implementation. The displayed record is the current-state projection **ModeTheory.CurrentState**. Lean’s operational **CausalEpistemicRecord** extends these three fields with a **ModalStack** that records announced and committed edit steps; **CurrentState.ofRecord** forgets that bookkeeping field. Lean indexes both records by the endogenous signature, while the model field packages the exogenous source.

Definition A.7.6 (Epistemic causal mode). An epistemic causal mode gives a name to one finite causal epistemic record.

Mode type. Let **Name** be a type of endpoint names and let $\mathcal{S}$ be a causal signature. Define the record $\text{CausalMode}(\text{Name}, \mathcal{S})$ with fields:

$$\begin{aligned} \text{name} &: \text{Name}, \\ \text{record} &: \text{CausalRecord}(\mathcal{S}). \end{aligned}$$

Interpretation. The name distinguishes endpoints in a transition but has no causal or probabilistic effect. The record carries the structural model, current belief and intervention whose changes are tracked by proof-carrying edit transitions.

Example. For the running example, a name such as m_{Mira} paired with R_{Mira} forms the initial mode. A posterior mode after receiving evidence has a different record and may have a different name, but the name itself contributes no information to the posterior.

Lean implementation. The pair of a name and a record is **CausalMode**; Lean specialises the endpoint-name type to **String**.

Proof-carrying transitions

Definition A.7.7 (Fixed-signature epistemic transition). The fixed-signature layer separates executable operations from their proof-carrying transitions.

Transition type. Let Name be an endpoint-name type and let $\mathcal{S}$ be a causal signature. Define the label type `RecordLabel` with constructors `conditioning`, `setting` and `unsetting`. For records R, R' on the same signature, define `RecordStep`(ℓ, R, R') inductively. Its constructors are:

A.7.7.1. `conditioning` the latent belief on a Boolean event supplied with positive-probability evidence;

A.7.7.2. `setting` one endogenous coordinate to a supplied value;

A.7.7.3. simultaneously `setting` a selected family of coordinates to the values of a supplied complete assignment; and

A.7.7.4. `removing` one supplied setting.

Define the record `CausalTransition`($\text{Name}, \mathcal{S}$) with fields:

```

label : RecordLabel,
source : CausalMode(Name,  $\mathcal{S}$ ),
target : CausalMode(Name,  $\mathcal{S}$ ),
valid : RecordStep(label, record(source), record(target)).

```

Model preservation. Every constructor changes only the current belief or intervention. The structural model and its endogenous and exogenous signatures remain fixed. A selected-node observation is the corresponding finite conditioning event.

Running example. Observing the event

$$E := E_{\text{Pr}=0} \cap E_{\text{Qu}=1} \cap E_{\text{Off}=0}$$

of Definition A.3.23 conditions Mira's current belief, while setting `Pr` to 1 inserts that value into the intervention field. Both transformations leave the stored structural model unchanged at this fixed-signature layer.

Proof-carrying transitions. Each corresponding transition contains source and target modes together with a proof that the stated record transformation produced the target. The transition label names the move; the step proof records that the source and target differ by the corresponding belief or intervention update. A realisation witness records that these operations realise the corresponding action and condition selections of Definition A.7.1; from an empty intervention, the multi-node operation is pointwise equal to the intervention used by the kernel semantics.

Lean implementation. The transition record is `CausalTransition`; the step proof is `CausalRecordStep`; realisation is `CausalTransition.Realizes`.

Remark A.7.8 (Identification under fixed-signature transitions). The transitions of Definition A.7.7 preserve the structural model. Static joint and conditional identifiability are consequently invariant along them, as stated in Proposition B.9.4. Conditioning and setting may still change the current numerical belief or evaluation state; the invariance concerns identification relative to the unchanged model and graph.

A. Definitions

Definition A.7.9 (Causal link). A causal link names one typed input of an endogenous mechanism.

Link type. For endogenous node type V and exogenous root type U , define $\text{CausalLink}(V, U)$ as the sum of:

A.7.9.1. an endogenous link (p, c, h_{top}) , where $p, c : V$ and h_{top} proves that p occurs earlier than c ; and

A.7.9.2. an exogenous link (ℓ, c) , where $\ell : U$ and $c : V$.

Interpretation. Relating or unrelating such a link changes the directed classifier or incidence classifier and therefore requires a replacement mechanism with the resulting input type.

Lean implementation. This sum type is `CausalLink`.

Definition A.7.10 (Typed causal edit interface). A typed causal edit interface specifies an indexed family of admissible operations and the result computed by every such operation.

Interface data. Supply the dependent family:

$$\text{EditOperation} : \Pi_{\mathcal{S} : \text{CausalSignature}} \text{CausalRecord}(\mathcal{S}) \rightarrow \Pi_{\mathcal{T} : \text{CausalSignature}} \mathcal{U}.$$

For signatures $\mathcal{S}, \mathcal{T}$ and a record $R : \text{CausalRecord}(\mathcal{S})$, supply its result map:

$$\text{result}_{\mathcal{S}, \mathcal{T}, R} : \text{EditOperation}(R, \mathcal{T}) \rightarrow \text{CausalRecord}(\mathcal{T}).$$

When the signatures and source record are determined by the operation, write simply $\text{result}(e)$. An inhabitant of $\text{EditOperation}(R, \mathcal{T})$ carries its operation-specific inputs, while result computes the uniquely determined target record.

Admissibility. Admissibility is expressed by inhabitation of the indexed family. An operation must carry all data required at its source and target signatures, and its target is computed by result . Thus neither an arbitrary result nor an independently selected label can be attached to an operation.

Lean implementation. The closed indexed family is `CausalEditOperation`; its eliminator is `result`.

Definition A.7.11 (Typed causal edit transition). A typed causal edit transition names source and target modes and certifies that the target record is the result of an admissible operation.

Transition type. Let `Name` be an endpoint-name type, let $\mathcal{S}, \mathcal{T}$ be causal signatures and use the edit interface of Definition A.7.10. Define the record $\text{CausalEditTransition}(\text{Name}, \mathcal{S}, \mathcal{T})$ with fields:

$$\begin{aligned} &\text{source} : \text{CausalMode}(\text{Name}, \mathcal{S}), \\ &\text{target} : \text{CausalMode}(\text{Name}, \mathcal{T}), \\ &\text{operation} : \text{EditOperation}(\text{record}(\text{source}), \mathcal{T}), \\ &\text{realised} : \text{Id}_{\text{CausalRecord}(\mathcal{T})}(\text{record}(\text{target}), \text{result}(\text{operation})). \end{aligned}$$

The equality field certifies that the named target is the record computed from the source and operation.

Correctness. The operation typing and structural realisation properties are established in Corollary B.9.6 and Proposition B.9.9.

Running example. The structural realisation of $\text{do}(\text{Pr} = 1)$ replaces Mira’s equation for Pr by a constant and unrelates its incoming latent links from U_B and U_{Pr} . It retains the outgoing link $\text{Pr} \rightarrow \text{Short}$ and the separate link $U_B \rightarrow \text{Qu}$. The replacement mechanism and each removed link are explicit operation inputs; the edit theorems prove that the other links are preserved.

Lean implementation. The dependent-signature transition is `CausalEditTransition`.

Definition A.7.12 (Causal edit path). A causal edit path follows a finite chain of proof-carrying edits.

Path type. Fix an endpoint-name type `Name`. Define `CausalEditPath(m, n)` inductively for causal modes m and n , which may carry different causal signatures. The empty path inhabits `CausalEditPath(m, m)`. A transition of Definition A.7.11 gives a one-step path, and composition joins two paths whose intermediate modes agree. Every inhabitant therefore carries its literal source and target modes together with the proof-carrying operations between them.

Lean implementation. Finite edit paths are `CausalEditPath`.

Definition A.7.13 (Causal edit mode theory). Causal modes and finite edit paths form a mode theory in the sense of Definition A.2.1.

Objects. For an endpoint-name type `Name`, define the record `CausalModeObject(Name)` with fields:

signature : CausalSignature,
endpoint : CausalMode(Name, signature).

Arrows. For objects a, b , define:

$$\text{Hom}_{\text{causal}}(a, b) := \text{CausalEditPath}(\text{endpoint}(a), \text{endpoint}(b)).$$

The empty path is the identity arrow, and path composition is arrow composition. Induction on finite paths supplies the left-identity, right-identity and associativity proofs required by Definition A.2.1. Thus each causal mode object is a mode in the sense of Definition A.2.2, and each causal edit path is a modality in the sense of Definition A.2.3.

Definition A.7.14 (Causal belief origin). A causal belief origin identifies the model and finite probability assignment from which a reindexed belief is obtained.

Origin type. For a causal signature $\mathcal{O}$, define:

$$\text{BeliefOrigin}(\mathcal{O}) := \text{CausalRecord}(\mathcal{O}) + \text{SCM}(\mathcal{V}_{\mathcal{O}}, \mathcal{U}_{\mathcal{O}}).$$

Interpretation. The left summand contributes the current record, its structural model and its current belief. The right summand contributes a structural model and its prior. For $O : \text{BeliefOrigin}(\mathcal{O})$, write $\text{model}(O)$ and $\text{belief}(O)$ for these derived data.

Lean implementation. This sum type is `BeliefReindexing.Origin`; its projections are `Origin.model` and `Origin.belief`.

Definition A.7.15 (Certified reindexing policy). A certified reindexing policy states how the intervention stored in a target record is handled during belief reindexing.

A. Definitions

Policy type. Let $R : \text{CausalRecord}(\mathcal{S})$, let $O : \text{BeliefOrigin}(\mathcal{O})$, and let

$$\begin{aligned} a &: \text{Val}(\mathcal{U}_{\mathcal{O}}) \rightarrow \text{Val}(\mathcal{U}_{\mathcal{S}}), \\ r &: \text{Val}(\mathcal{U}_{\mathcal{O}}) \rightarrow \text{Val}(\mathcal{V}_{\mathcal{S}}). \end{aligned}$$

Write $R = (\mathcal{M}, b, x)$. Define $\text{ReindexingPolicy}(R, O, a, r)$ with two constructors:

$$\begin{aligned} \text{preserve} &: (\Pi_{u:\text{Val}(\mathcal{U}_{\mathcal{O}})} \text{Id}(\text{sol}_{\mathcal{M}_x}(a(u)), r(u))) \rightarrow \text{ReindexingPolicy}(R, O, a, r), \\ \text{clear} &: (\Pi_{u:\text{Val}(\mathcal{U}_{\mathcal{O}})} \text{Id}(\text{sol}_{\mathcal{M}_x}(a(u)), \text{sol}_{\mathcal{M}}(a(u)))) \\ &\rightarrow (\Pi_{u:\text{Val}(\mathcal{U}_{\mathcal{O}})} \text{Id}(\text{sol}_{\mathcal{M}}(a(u)), r(u))) \\ &\rightarrow \text{ReindexingPolicy}(R, O, a, r). \end{aligned}$$

Interpretation. The preserving constructor retains the intervention x . The clearing constructor may remove it only after the first proof shows that its effect has already been absorbed into the structural equations; the second proof then identifies ordinary structural evaluation with the reference map.

Lean implementation. The policy type is **CertifiedInterventionPolicy**.

Definition A.7.16 (Certified belief reindexing). Certified belief reindexing derives a new belief on a fixed target record from a causal origin and certifies its endpoint evaluation.

Reindexing type. Let $R : \text{CausalRecord}(\mathcal{S})$ be the target record. Define the record $\text{BeliefReindexing}(R)$ with fields:

$$\begin{aligned} \text{originSignature} &: \text{CausalSignature}, \\ \text{origin} &: \text{BeliefOrigin}(\text{originSignature}), \\ \text{assignment} &: \text{Val}(\mathcal{U}_{\text{originSignature}}) \rightarrow \text{Val}(\mathcal{U}_{\mathcal{S}}), \\ \text{reference} &: \text{Val}(\mathcal{U}_{\text{originSignature}}) \rightarrow \text{Val}(\mathcal{V}_{\mathcal{S}}), \\ \text{policy} &: \text{ReindexingPolicy}(R, \text{origin}, \text{assignment}, \text{reference}). \end{aligned}$$

The reindexed belief is the finite pushforward of $\text{belief}(\text{origin})$ along assignment . It is therefore derived from causal data rather than supplied independently.

Intervention policy. The final field uses the policy of Definition A.7.15. Clearing is admitted only after the retained override has been shown semantically absorbed.

Correctness. The provenance, realisation and clearing obligations are established in Propositions B.9.7 and B.9.15.

Lean implementation. The corresponding record is **CertifiedOperation** in the **BeliefReindexing** namespace. Its policies are constructors of **CertifiedInterventionPolicy**, and the typed edit constructor is **reindexingBelief**.

Definition A.7.17 (Operation realisation of modal rule cells). A modal rule cell is realised by realising both endpoint kernels and equating their values.

Realisation type. For a rule cell $c : \text{ModalRuleCell}(q, q')$ of Definition A.7.2, define $\text{RealisesCell}(c)$ as a realisation of the source kernel q , a realisation of the target kernel q' , and equality of the two values. Each kernel realisation carries the setting and observation transitions that execute that kernel's action and condition selections, positive evidence for the observation, and equality with the kernel denotation.

Interpretation. The cell itself records a symbolic do-calculus step. Its realisation supplies concrete epistemic transitions for both endpoint kernels and verifies that their evaluated values agree.

Lean implementation. Rule-cell realisation is `OperationRealizedModalRuleCell`; kernel realisations are `KernelOperationRealization`.

Definition A.7.18 (Operation realisation of modal traces). A complete modal trace is realised by realising each of its constituent rule cells.

Realisation type. Let τ be a modal do-calculus trace of Definition A.7.3. Define `Realises`(τ) recursively on the trace. A rule-cell leaf contributes `RealisesCell`(c) from Definition A.7.17. Composition and probability-algebra constructors contribute the products of the realisation types of their immediate subtraces.

Interpretation. Rule-cell leaves are realised as in Definition A.7.17. Composition and probability-algebra constructors assemble those realisations; they do not add further record operations.

Lean implementation. Recursive realisation is `ModalDerivationTrace.OperationRealizations`. At rule leaves it is `OperationRealizedModalRuleCell`; the transitions inside each kernel realisation are witnessed by `CausalTransition.Realizes`.

Endpoint and intervention semantics

Definition A.7.19 (Endpoint query). An endpoint query keeps its numerator and denominator events explicit.

Query type. For a complete endogenous assignment type Ω , define the record `EndpointQuery`(Ω) with fields:

$$\begin{aligned} \text{numerator} &: \Omega \rightarrow \text{Bool}, \\ \text{denominator} &: \Omega \rightarrow \text{Bool}. \end{aligned}$$

Interpretation. A joint or unconditional query uses the constantly true denominator. A conditional query places the conjunction of its outcome and condition in the numerator and the condition alone in the denominator.

Lean implementation. This record is `EndpointQuery`.

Definition A.7.20 (Intervention overlay). Intervention overlay combines accumulated settings with a later hard intervention.

Operation. For hard interventions $x, y : \text{Do}(\mathcal{V})$, define $x \triangleright y$ coordinatewise: it returns the value selected by y when that value is present, and otherwise returns the value selected by x . Thus the right-hand intervention takes precedence.

Lean implementation. Intervention overlay is `HardIntervention.overlay`.

Definition A.7.21 (Current-state event probability). Current-state event probability reads a decidable event from the literal current causal record.

Probability. Let $R = (\mathcal{M}, b, x)$ be a finite causal epistemic record, where b is its current latent belief and x its accumulated hard intervention. Its current observed assignment is obtained by the map

$$u \mapsto \text{sol}_{\mathcal{M}_x}(u),$$

A. Definitions

so a decidable observed event E has current-state probability

$$P_R(E) := P_b(E \circ \text{sol}_{\mathcal{M}_x}).$$

This construction uses the current belief b and accumulated intervention x , rather than the original prior of $\mathcal{M}$.

Lean implementation. Current event probability is `CausalEpistemicRecord.observedValue`.

Definition A.7.22 (Current-state endpoint semantics). Endpoint semantics applies the probability assignment of Definition A.7.21 to an endpoint query.

Endpoint denotation. Let $Q = (N, D) : \text{EndpointQuery}(\Omega)$ be an endpoint query of Definition A.7.19. Its partial denotation is:

$$\llbracket Q \rrbracket_R := \begin{cases} \text{some}(P_R(N) / P_R(D)), & P_R(D) > 0, \\ \text{none}, & P_R(D) \sim 0. \end{cases} : \text{PartialQ}.$$

Thus endpoint semantics reads the current belief and accumulated intervention rather than silently returning to the model's original product prior.

Kernels and paths. For an ordinary kernel, endpoint evaluation first forms $x \triangleright y$, where x is the record's accumulated intervention and y is the kernel action, using Definition A.7.20, and then evaluates the kernel numerator and conditioning events under the retained current belief. Path-level endpoint evaluation evaluates an endpoint query at the literal target record of a causal edit path of Definition A.7.12. At an initial record, this agrees with the existing model-level kernel denotation. These agreements are established in Propositions B.9.15 and B.9.17.

Scope. This is an executable semantics for the concrete endpoint histories constructed below. It is not a definition of identifiability after an arbitrary epistemic construction and does not assert a new completeness theorem for arbitrary interleavings of observation, intervention, learning and forgetting.

Running example. At the initial Mira record, $P_R(E)$ is the ordinary observational probability of E . After conditioning the belief on the evidence from Definition A.6.6 and setting Pr to 1, endpoint evaluation of $\text{Off} = 1$ gives the corresponding abduction–action–prediction value.

Lean implementation. Partial endpoint evaluation is `EndpointQuery.denote`.

Definition A.7.23 (Atomic structural intervention). Atomic structural intervention compiles a hard intervention into an ordered path of primitive causal edits.

Execution type. Let $m : \text{CausalMode}(\text{Name}, \mathcal{S})$, write $\text{record}(m) = (\mathcal{M}, b, x_0)$, and let $x : \text{Do}(\mathcal{V}_S)$. Define $\text{AtomicExecution}_m(x)$ as the record containing a target causal signature $\mathcal{T}$, a target mode n , a path

$$p : \text{CausalEditPath}(m, n),$$

and two-sided transports between the source and target endogenous coordinates and between their exogenous coordinates. Atomic compilation constructs a canonical inhabitant $\text{compile}_m(x) : \text{AtomicExecution}_m(x)$ by replacing each selected structural equation by its constant and then removing every incoming directed and exogenous input at that node. For an execution e and a

source exogenous assignment u , write $\text{endpointEval}(e, u)$ for evaluation at the transported target assignment, returned in the source endogenous coordinates.

Lean implementation. The execution record is `AtomicIntervention.Execution`; atomic compilation is `AtomicIntervention.compile`.

Definition A.7.24 (Atomic structural plan). An atomic structural plan records the complete graph and mechanism effect of an atomic execution.

Plan type. Let m and x be as in Definition A.7.23. Write V , U , $E^\rightarrow$ and L for the source node type, root type, directed classifier and incidence classifier. Define $e := \text{compile}_m(x)$, and use its two-sided coordinate transports to read endpoint nodes and roots in the source coordinates. Write $E_e^\rightarrow$, L_e and f_i^e for the transported endpoint directed classifier, incidence classifier and mechanism at i , and write $\text{MechanismInputs}_e(i)$ for the input type of f_i^e . The record $\text{AtomicPlan}_m(x)$ has fields:

$$\begin{aligned} h_{\text{fixed}} &: \Pi_{i:V} \Pi_{v:\text{Val}(i)} (x(i) = \text{some}(v)) \rightarrow \Pi_{z:\text{MechanismInputs}_e(i)} \text{Id}_{\text{Val}(i)}(f_i^e(z), v), \\ h_{\text{removed}}^\rightarrow &: \Pi_{p,c:V} \Pi_{v:\text{Val}(c)} (x(c) = \text{some}(v)) \rightarrow E_e^\rightarrow(p, c) = \text{false}, \\ h_{\text{removed}}^L &: \Pi_{\ell:U} \Pi_{c:V} \Pi_{v:\text{Val}(c)} (x(c) = \text{some}(v)) \rightarrow L_e(\ell, c) = \text{false}, \\ h_{\text{retained}}^\rightarrow &: \Pi_{p,c:V} (x(c) = \text{none}) \rightarrow E_e^\rightarrow(p, c) = E^\rightarrow(p, c), \\ h_{\text{retained}}^L &: \Pi_{\ell:U} \Pi_{c:V} (x(c) = \text{none}) \rightarrow L_e(\ell, c) = L(\ell, c). \end{aligned}$$

Interpretation. Thus every selected mechanism is constant, all of its incoming inputs are removed, and the complete input classifiers of every unselected child are unchanged.

Lean implementation. The proof-carrying plan is `AtomicIntervention.Plan`.

Definition A.7.25 (Deterministic atomic realisation). Deterministic atomic realisation compares an atomic endpoint with direct evaluation under the compiled hard intervention.

Realisation type. Let m and x be as in Definition A.7.23, write $\text{record}(m) = (\mathcal{M}, b, x_0)$, and let $e : \text{AtomicExecution}_m(x)$. A deterministic realisation is a record $\text{DeterministicRealisation}(e, x)$ with fields:

$$\begin{aligned} \text{assignmentMap} &: \text{Val}(\mathcal{U}_S) \rightarrow \text{Val}(\mathcal{U}_{\text{signature}(e)}), \\ \text{agreement} &: \Pi_{u:\text{Val}(\mathcal{U}_S)} \text{Id}(\text{endpointEval}(e, u), \text{sol}_{\mathcal{M}_x}(u)). \end{aligned}$$

Pushing the current belief through `assignmentMap` gives the endpoint belief.

Running example. Compiling $\text{Pr} := 1$ in Mira's model first installs the constant equation and then cuts the incoming latent uses of U_B and U_{Pr} . The compiled endpoint therefore has the same intervention targets as the hard intervention of Definition A.5.2.

Lean implementation. Deterministic realisation is `AtomicIntervention.Execution.DeterministicRealizesEvaluation`.

Definition A.7.26 (Compact surgical intervention). Compact surgery constructs in one step the structural model represented by the atomic path of Definition A.7.23.

Construction. For a model $\mathcal{M}$ and hard intervention x , replace each selected mechanism by its constant value and remove every incoming directed and exogenous input at its target. Retain all other mechanisms, edges, exogenous

A. Definitions

factors and the prior. The resulting model is the intervened model $\mathcal{M}_x$ of Definition A.5.3 in a compact presentation.

Transition. For a source mode m and intervention x , write $\text{CompactTransition}_m(x)$ for the type of the one-step typed edit transition from m to this compact target, and write $\text{compactTransition}_m(x)$ for its canonical inhabitant.

Agreement. For every source exogenous assignment, the atomic endpoint and compact model produce identical endogenous assignments after the canonical coordinate transport. Their event probabilities therefore agree, as established in Propositions B.9.9 and B.9.15.

Lean implementation. The compact construction and its plan are `SurgicalIntervention` and `SurgicalIntervention.Plan`.

Counterfactual execution

Definition A.7.27 (Modal counterfactual atom trace). A modal counterfactual atom trace attaches the canonical structural execution and its compact reference transition to one atom.

Trace type. Let m be an epistemic causal mode. For an atom $A : \text{CFAtom}(\mathcal{V})$, define the record $\text{AtomTrace}_m(A)$ with fields:

$$\begin{aligned} \text{execution} &: \text{AtomicExecution}_m(\text{action}(A)), \\ h_{\text{canonical}} &: \text{Id}(\text{execution}, \text{compile}_m(\text{action}(A))), \\ h_{\text{det}} &: \text{DeterministicRealisation}(\text{execution}, \text{action}(A)), \\ \text{referenceTransition} &: \text{CompactTransition}_m(\text{action}(A)), \\ h_{\text{reference}} &: \text{Id}(\text{referenceTransition}, \text{compactTransition}_m(\text{action}(A))). \end{aligned}$$

Here `DeterministicRealisation` is the type of Definition A.7.25, and $\text{CompactTransition}_m(x)$ is the type of the one-step compact transition constructed by Definition A.7.26; the atomic execution is the proof-carrying edit path of Definition A.7.23.

Definition A.7.28 (Modal counterfactual event trace). An event trace collects atom traces according to the finite Boolean syntax of a counterfactual event.

Trace type. Let m be an epistemic causal mode. Define $\text{EventTrace}_m(\gamma)$ recursively over the constructors of `CFEvent` from Definition A.6.3. The true and false constructors contribute `Fin(1)`; an atom contributes its atom-trace type from Definition A.7.27; binary Boolean constructors contribute the product of their two subtraces; and negation contributes the trace of its subevent.

Definition A.7.29 (Modal counterfactual query trace). A modal counterfactual query trace supplies event traces and retains a separately indexed trace for every syntactic atom occurrence.

Occurrences. Let m be an epistemic causal mode and let Q be a finite counterfactual query. The canonical finite enumeration of atom occurrences in Q determines a projection:

$$\text{atomAt}_Q : \text{Occ}(Q) \rightarrow \text{CFAtom}(\mathcal{V}).$$

The occurrence type is that of Definition A.6.10.

Query trace. Define the record $\text{QueryTrace}_m(Q)$ with fields:

$\text{outcome} : \text{EventTrace}_m(\text{outcome}(Q)),$
 $\text{condition} : \text{EventTrace}_m(\text{condition}(Q)),$
 $\text{occurrences} : \prod_{o: \text{Occ}(Q)} \text{AtomTrace}_m(\text{atomAt}_Q(o)).$

The occurrence field prevents repeated syntax from being collapsed extensionally.

Lean implementation. These types are `ModalCounterfactualAtomTrace`, `ModalCounterfactualEventTrace` and `ModalCounterfactualQueryTrace`.

Definition A.7.30 (Factual-base occurrence execution). Factual-base occurrence execution constructs the finite multiworld required by a query from a fresh factual copy of the source model.

Construction. For a finite counterfactual query Q , append one endogenous copy for every occurrence in $\text{Occ}(Q)$. Install the within-world directed links, shared exogenous incidences and copied mechanisms; reindex the source prior as the endpoint belief; and compile the hard intervention attached to every occurrence. Each stage is a typed causal edit of Definition A.7.11, so their composite is a causal edit path. Define $\text{FactualExecution}_m(Q)$ as the type of these proof-carrying constructions from mode m , and write $\text{canonicalFactual}_m(Q)$ for the construction just described.

Endpoint. The resulting record provides the numerator and denominator events used by the endpoint semantics of Definition A.7.22.

Lean implementation. This construction is `ExecutedOccurrenceConstruction`.

Definition A.7.31 (Empty-base occurrence execution). Empty-base occurrence execution constructs the same finite occurrence model while making every signature extension explicit.

Construction. Begin with no endogenous nodes and no exogenous roots. Create every source root and every factual and counterfactual node, then install the links, mechanisms, belief reindexing and occurrence interventions of Definition A.7.30. At the endpoint, the transported directed and exogenous-incidence classifiers agree pointwise with those of the direct occurrence model. Hence no cross-world edge or other unintended input is introduced. Define $\text{EmptyExecution}_m(Q)$ as the type of these empty-base constructions, and write $\text{canonicalEmpty}_m(Q)$ for the construction just described.

Lean implementation. This construction is `FromEmptyExecutedOccurrenceConstruction`.

Definition A.7.32 (Executed modal counterfactual trace). An executed modal counterfactual trace combines the recursive trace of Definition A.7.29 with both occurrence executions.

Trace type. For a mode m and counterfactual query Q , define $\text{ExecutedQueryTrace}_m(Q)$ as a record with fields:

$\text{trace} : \text{QueryTrace}_m(Q),$
 $\text{fromFactual} : \text{FactualExecution}_m(Q),$
 $h_{\text{factual}} : \text{Id}(\text{fromFactual}, \text{canonicalFactual}_m(Q)),$
 $\text{fromEmpty} : \text{EmptyExecution}_m(Q),$
 $h_{\text{empty}} : \text{Id}(\text{fromEmpty}, \text{canonicalEmpty}_m(Q)).$

A. Definitions

The two execution types are those of Definitions A.7.30 and A.7.31.

Endpoint agreement. The endpoints may have different dependent signatures, so they are compared after transport to the common occurrence coordinates. Every occurrence event then has equivalent probability, and the partial query denotations agree in both the supported and unsupported cases. These statements are proved in Propositions B.10.2 to B.10.4 and B.10.8.

Lean implementation. The combined record is `ModalCombinedCounterfactualConstruction`; its two routes are `ExecutedOccurrenceConstruction` and `FromEmptyExecutedOccurrenceConstruction`. Endpoint evaluation is `endpointDenote`.

Remark A.7.33 (Modal abduction–action–prediction). The one-action construction has an executable sequence and an agreement result.

Execution. For a supported one-action query, the concrete abduction–action–prediction sequence starts at the model’s prior, conditions the current latent belief on factual evidence, compiles the action into atomic edits, transports the posterior belief to the actual endpoint record, and evaluates the transported target event there. The action is represented by this atomic execution, and prediction is its endpoint probability.

Agreement. A compact one-shot action is used only as an independently checked reference. The endpoint probability is identified with the model-level counterfactual value, with the compact probability and with the direct twin-network value. These equalities are established in Propositions B.9.17 and B.10.1.

Running example. For Mira’s evidence E and action $\text{Pr} := 1$, abduction changes the latent belief, atomic action changes the causal evaluation state, and prediction evaluates $\text{Off} = 1$ at the resulting endpoint. The resulting probability agrees with the twin-network value.

Lean implementation. The namespace `AbductionActionPrediction` groups the action, prediction, compact reference and model-level comparison constructions; it is not a separate record type.

Learning and forgetting

Definition A.7.34 (Endogenous causal learning). Endogenous learning appends one terminal variable and the mechanism that computes it.

Specification. Let $R : \text{CausalRecord}(\mathcal{S})$ be a current causal record, with endogenous node type V and exogenous root type U . Let $P : V \rightarrow \text{Bool}$ and $L : U \rightarrow \text{Bool}$ be selections equipped with the finite indexed presentations of Definition A.4.2. Their selected assignment types are the coordinate products of Definition A.3.24; write them as $\text{Val}(P)$ and $\text{Val}_{\mathcal{U}}(L)$. Define:

$$\begin{aligned} \text{NewInputs}(P, L) &:= \text{Val}(P) \\ &\quad \times \text{Val}_{\mathcal{U}}(L). \end{aligned}$$

The two factors provide values for the selected endogenous parents and exogenous

roots, respectively. Define the record $\text{EndogenousVariableSpec}(R)$ with fields:

$$\begin{aligned} &\text{valueType} : \mathcal{U}, \\ &\quad \text{finite} : \text{Finite}(\text{valueType}), \\ &\quad \text{default} : \text{valueType}, \\ &\quad \text{parents} : V \rightarrow \text{Bool}, \\ &\quad \text{roots} : U \rightarrow \text{Bool}, \\ &\text{mechanism} : \text{NewInputs}(\text{parents}, \text{roots}) \rightarrow \text{valueType}. \end{aligned}$$

Operation. The learning operation appends the new variable at the end of the topological order, preserves every existing variable, equation and link, adds exactly the selected incoming directed and exogenous links, and installs the supplied mechanism. Because the dependent signature changes, this operation inhabits the typed edit interface of Definition A.7.10.

Lean implementation. The specification is $\text{EndogenousVariableSpec}$; its indexed learning constructor is part of $\text{CausalEditOperation}$.

Definition A.7.35 (Exogenous causal learning). Exogenous learning appends one independent root to the current source of uncertainty.

Specification. Define the record $\text{ExogenousVariableSpec}$ with fields:

$$\begin{aligned} &\text{valueType} : \mathcal{U}, \\ &\quad \text{finite} : \text{Finite}(\text{valueType}), \\ &\quad \text{factor} : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{valueType}). \end{aligned}$$

Operation. The operation appends a fresh root, initially incident to no endogenous mechanism. It extends both the model prior and the current belief by the independent product with factor . A later causal-link edit of Definition A.7.9 may connect the root to selected mechanisms, provided that each affected mechanism is replaced at its new input type.

Lean implementation. The specification is $\text{ExogenousVariableSpec}$; its learning constructor belongs to $\text{CausalEditOperation}$.

Definition A.7.36 (Unused endogenous forgetting). Endogenous forgetting removes a variable only after its absence from all retained causal data has been proved.

Data. Let $\mathcal{K}$ be a latent causal skeleton with endogenous node type V , directed classifier $E^{\rightarrow}$, and current causal record R . Let $d : V$ be the node proposed for deletion.

Criterion. A removable endogenous node $d : V$ must satisfy:

$$\Pi_{c:V} \text{Id}_{\text{Bool}}(E^{\rightarrow}(d, c), \text{false}).$$

Incoming arrows and incident exogenous roots are permitted because they feed only the discarded mechanism.

Operation. Decidable equality defines the retained node selection $S_d : V \rightarrow \text{Bool}$, which returns false exactly at d . The operation uses the induced finite restriction of Definition A.4.12, reindexes the dependent value and parent families, and retains the same exogenous probability space. The current intervention is restricted to the retained coordinates, and retained events and queries are lifted along the corresponding projection. Evaluation and these lifted

A. Definitions

query denotations are preserved; the resulting two-way identification statement is proved in Proposition B.9.12 and Corollary B.9.13.

Lean implementation. The proof-carrying specification is `ObservedDeletionSpec`; the indexed operation is `forgettingUnusedEndogenous`.

Definition A.7.37 (Unused exogenous forgetting). Exogenous forgetting removes a root that is absent from every retained mechanism.

Data. Let $\mathcal{K}$ be a latent causal skeleton with endogenous node type V , exogenous root type U , incidence classifier L , and current causal record R . Let $u : U$ be the root proposed for deletion.

Criterion. The required evidence is:

$$\Pi_{c:V} \text{Id}_{\text{Bool}}(L(u, c), \text{false}),$$

using the incidence type of Definition A.4.15.

Operation. The operation removes the coordinate u , marginalises the prior and current belief along the retained-coordinate projection, and reindexes the remaining factors and exogenous inputs. It derives the product law for the retained roots. Structural evaluation and observational, interventional and current-belief event values are preserved, as proved in Proposition B.9.14.

Lean implementation. The proof-carrying specification is `ExogenousDeletionSpec`; the indexed operation is `forgettingUnusedExogenous`.

Remark A.7.38 (Admissibility, modality and inverse). Incoming arrows at a deleted endogenous sink are admissible because their only use is in the discarded equation; an outgoing arrow would retain a use of the variable. This is why “unused” is the precise criterion and “isolated” or “unconnected” would be unnecessarily strong.

Deletion is a context-sensitive causal edit justified by its proof of unusedness. Its modal label announces forgetting. Unlike provenance-guided forgetting, a general deletion has no canonical inverse unless the removed type, mechanisms and probability data are retained as provenance. By contrast, the paired inverse of either learning operation restores its recorded source exactly.

Definition A.7.39 (Conservative terminal learning). The conservative subcase is specified by its data and inverse transitions.

Data. A conservative terminal specification is an endogenous-variable specification of Definition A.7.34 whose root selection is empty. Its parent field is a node selection $P : V \rightarrow \text{Bool}$, and its mechanism depends only on the selected parent values. Write $\mathcal{V}^+$ for the endogenous signature that appends the new variable c_{new} as the last variable in the topological order. It retains every old directed edge, permits only nodes selected by P to point into c_{new} , and gives c_{new} no incident exogenous root. The extension therefore reuses the old exogenous source and prior, retains every old mechanism and adds only the specified terminal mechanism.

Transitions. The learning operation performs this extension and lifts the current intervention along the canonical inclusion $\mathcal{V} \rightarrow \mathcal{V}^+$. It is packaged as the typed causal edit transition of Definition A.7.11. Its provenance-guided inverse is available only with the original mode and the terminal specification, and restores that original record. The separate unused-variable constructors

above implement general legal deletion but do not claim to recover the discarded record.

Lean implementation. The specification is `TerminalVariableSpec`. The learning operation is `learnRecord`; the packaged transitions are `learnTerminal` and `forgetTerminal`. The transition record is `CausalEditTransition`; the operation family is `CausalEditOperation`.

Remark A.7.40 (Preservation under conservative terminal learning). Old events, interventions and queries are lifted without selecting the new coordinate. For these lifts, old observational, interventional and current-belief probabilities are preserved. Moreover, joint and locally supported conditional identifiability of an old query imply identifiability of its lift to $\mathcal{V}^+$. This is a conservative one-way preservation result, not a claim that arbitrary causal learning preserves identifiability or that every signature extension has this form. These properties are proved in Proposition B.9.10 and Corollary B.9.11.

A.8 Finite causal transport interfaces

This section specifies the finite coded source, its intrinsic dependent interpretation, derivation certificates with local support, and the external theorem interfaces. The interfaces are explicit hypotheses: citing a published theorem does not construct an inhabitant.

Source and target presentations

Definition A.8.1 (Finite table signature). A finite table signature gives every endogenous node and value an explicit finite code.

Signature record. Define `FiniteTableSignature` as the record with fields:

$$\begin{aligned} \text{count} &: \mathbb{N}, \\ \text{arity} &: \text{Fin}(\text{count}) \rightarrow \mathbb{N}, \\ \text{defaultCode} &: \prod_{i: \text{Fin}(\text{count})} \text{Fin}(\text{arity}(i)), \\ \text{directed} &: \text{OrderedDirectedEdges}(\text{Fin}(\text{count})). \end{aligned}$$

Interpretation. For a signature T with the displayed data, its node type and value family are:

$$\begin{aligned} V_T &:= \text{Fin}(\text{count}), \\ \text{Val}_T(i) &:= \text{Fin}(\text{arity}(i)), \\ \text{Assignment}_T &:= \prod_{i: V_T} \text{Val}_T(i). \end{aligned}$$

The default code makes every value type inhabited. Each value type is traversed by the canonical enumeration of the finite index type from Definition A.1.17, and `directed` gives the ordered directed parent relation of Definition A.4.4. Thus T is the coded special case of the endogenous signature in Definition A.4.13.

Lean implementation. The record is `FiniteTableSignature`. Lean stores a chosen code enumeration together with completeness and duplicate-freeness evidence; its interpretation as an endogenous signature is `FiniteTableSignature.toObserved`.

Definition A.8.2 (Finite table exogenous coordinates). Finite table exogenous coordinates give every background coordinate an explicitly enumerable carrier.

A. Definitions

Probability factors and their joint law are supplied by a finite table structural causal model over these coordinates.

Coordinate record. Let T be a finite table signature. Define the record $\text{FiniteTableLatent}(T)$ with fields:

$$\begin{aligned} \text{count} &: \mathbb{N}, \\ \text{Value} &: \text{Fin}(\text{count}) \rightarrow \mathcal{U}, \\ \text{finite} &: \prod_{\ell: \text{Fin}(\text{count})} \text{Finite}(\text{Value}(\ell)), \\ \text{incident} &: \text{Fin}(\text{count}) \rightarrow V_T \rightarrow \text{Bool}. \end{aligned}$$

Its complete assignment type is:

$$\text{Assignment}_{\mathcal{L}} := \prod_{\ell: \text{Fin}(\text{count})} \text{Value}(\ell).$$

Interpretation. The finiteness witnesses provide the enumerations and equality procedures required for finite sums. The incidence classifier identifies the exogenous inputs of each endogenous mechanism, as in Definition A.4.15.

Lean implementation. The record is **FiniteTableLatent**. It stores an explicit enumeration, completeness evidence and decidable equality for every coordinate.

Definition A.8.3 (Finite table structural causal model). A finite table structural causal model is the coded model presentation used on the source side of transport. It is not a second mathematical model class.

Mechanism inputs. Let T be a finite table signature and let $\mathcal{L} : \text{FiniteTableLatent}(T)$. For $i : V_T$, define:

$$\begin{aligned} \text{ParentValues}_T(i) &:= \prod_{p: V_T} (\text{directed}_T(p, i) = \text{true}) \rightarrow \text{Val}_T(p), \\ \text{LatentInputs}_{\mathcal{L}}(i) &:= \prod_{\ell: \text{Fin}(\text{count}_{\mathcal{L}})} (\text{incident}_{\mathcal{L}}(\ell, i) = \text{true}) \rightarrow \text{Value}_{\mathcal{L}}(\ell), \\ \text{TableInputs}_{T, \mathcal{L}}(i) &:= \text{ParentValues}_T(i) \times \text{LatentInputs}_{\mathcal{L}}(i). \end{aligned}$$

Model record. Define $\text{FiniteTableSCM}(T)$ as the record with fields:

$$\begin{aligned} \text{latent} &: \text{FiniteTableLatent}(T), \\ \text{factor} &: \prod_{\ell: \text{Fin}(\text{count}_{\text{latent}})} \text{Prob}_{\mathbb{Q}}^{\text{cd}}(\text{Value}_{\text{latent}}(\ell)), \\ \text{prior} &: \text{Prob}_{\mathbb{Q}}^{\text{cd}}(\text{Assignment}_{\text{latent}}), \\ \text{productLaw} &: \prod_{A: \prod_{\ell} (\text{Value}_{\text{latent}}(\ell) \rightarrow \text{Bool})} P_{\text{prior}}(\text{Rect}_A) \sim \prod_{\ell} P_{\text{factor}(\ell)}(A(\ell)), \\ \text{mechanism} &: \prod_{i: V_T} \text{TableInputs}_{T, \text{latent}}(i) \rightarrow \text{Val}_T(i). \end{aligned}$$

The probability fields use the common-denominator presentation of Definition A.3.13; its equivalence with the urn presentation is proved in Proposition B.6.13.

Mechanisms. Each mechanism is a finite coded function from its parent and incident-root values to an output code. Its domain is explicitly enumerable, but the record need not store the function as a literal list of rows.

Semantics. Observational and interventional event values are finite sums of the exogenous prior over the recursively evaluated table. Agreement of that source evaluation with the intrinsic solution map is proved in Proposition B.7.10.

Lean implementation. The model is `FiniteTableSCM`. Its mechanism family is stored in the field `table`; the other fields are `latent`, `factor`, `prior` and `product_law`.

Definition A.8.4 (Finite table graph). A finite table graph supplies the bidirected part of the causal graph associated with a table model.

Data. Let T be a finite table signature of Definition A.8.1, with node type V_T .

Graph record. Supply

$$B : \text{BidirectedEdges}(V_T)$$

in the sense of Definition A.4.4. The directed part already belongs to the endogenous signature determined by T . Together, the directed part and B form a causal graph of Definition A.4.11 on V_T ; write this interpreted graph as $\overline{G}$.

Interpretation. The table graph is kept separate from the table structural causal models over T . Agreement of B with the classifier induced by one such model's latent incidence is compatibility evidence, not a field of either presentation.

Lean implementation. These data form `FiniteTableGraph`. Its field `bidirected` carries the classifier together with its symmetry and irreflexivity evidence.

Definition A.8.5 (Finite table source presentation). A finite table source presentation is the named bundle over which the external source theorems quantify.

Package. Define the record `FiniteTableSource` with fields:

$$\begin{aligned} \text{signature} &: \text{FiniteTableSignature}, \\ \text{graph} &: \text{FiniteTableGraph}(\text{signature}). \end{aligned}$$

Write $(T, G) : \text{FiniteTableSource}$ for an inhabitant.

Lean implementation. The two fields are represented by `FiniteTableSignature` and `FiniteTableGraph`.

Definition A.8.6 (Compatible finite source models). Compatible source models share one finite table signature and its projected causal graph.

Data. Let $(T, G) : \text{FiniteTableSource}$ be a source presentation of Definition A.8.5.

Model family. The source models associated with (T, G) are the finite table structural causal models over T whose latent incidence induces the bidirected classifier supplied by G , in the sense of Definition A.5.13. Write

$$\text{SourceMod}(T, G)$$

for this type of compatible source models.

Source equivalence. For $\mathcal{M} : \text{SourceMod}(T, G)$, write $P_{\mathcal{M}}^{\text{src}}(E)$ for the finite-sum probability of an event $E : \text{Assignment}_T \rightarrow \text{Bool}$ and write $\llbracket Q \rrbracket_{\mathcal{M}}^{\text{src}}$ for the source denotation of a kernel query. For $\mathcal{M}, \mathcal{N} : \text{SourceMod}(T, G)$, define:

$$\mathcal{M} \equiv_{\text{obs}}^{\text{src}} \mathcal{N} \quad := \quad \prod_{E : \text{Assignment}_T \rightarrow \text{Bool}} (P_{\mathcal{M}}^{\text{src}}(E) \sim P_{\mathcal{N}}^{\text{src}}(E)).$$

Lean implementation. Compatibility and observational equivalence are `FiniteSourceCompatible` and `FiniteSourceObservationallyEquivalent`.

A. Definitions

Definition A.8.7 (Finite source kernel equivalence). Source kernel equivalence compares two independently evaluated finite table models at one joint or conditional query.

Data. Let $(T, G) : \text{FiniteTableSource}$ and let $\mathcal{M}, \mathcal{N} : \text{SourceMod}(T, G)$.

Joint-kernel equivalence. For a joint query Q_J , define:

$$\text{SourceJointEquivalent}(\mathcal{M}, \mathcal{N}, Q_J) := (\llbracket Q_J \rrbracket_{\mathcal{M}}^{\text{src}} \simeq \llbracket Q_J \rrbracket_{\mathcal{N}}^{\text{src}}).$$

Conditional-kernel equivalence. For a conditional query Q_C , define:

$$\begin{aligned} \text{SourceConditionalEquivalent}(\mathcal{M}, \mathcal{N}, Q_C) \\ := \text{AgreesWhereDefined}(\llbracket Q_C \rrbracket_{\mathcal{M}}^{\text{src}}, \llbracket Q_C \rrbracket_{\mathcal{N}}^{\text{src}}), \end{aligned}$$

using the common-support comparison of Definition A.5.10.

Interpretation. Joint equivalence compares the complete denotations. Conditional equivalence compares values only where both source models define them.

Definition A.8.8 (Finite source identifiability). Source identifiability requires observationally equivalent finite table models to have equivalent kernel denotations.

Data. Let $(T, G) : \text{FiniteTableSource}$. Use the model family and source observational equivalence of Definition A.8.6, and the querywise equivalences of Definition A.8.7.

Identifiability. Define the two source-identifiability types:

$$\begin{aligned} \text{SourceJointIdentifiable}_G(Q_J) := \Pi_{\mathcal{M}, \mathcal{N} : \text{SourceMod}(T, G)} (\mathcal{M} \equiv_{\text{obs}}^{\text{src}} \mathcal{N}) \rightarrow \\ \text{SourceJointEquivalent}(\mathcal{M}, \mathcal{N}, Q_J), \end{aligned}$$

$$\begin{aligned} \text{SourceConditionalIdentifiable}_G(Q_C) := \Pi_{\mathcal{M}, \mathcal{N} : \text{SourceMod}(T, G)} (\mathcal{M} \equiv_{\text{obs}}^{\text{src}} \mathcal{N}) \rightarrow \\ \text{SourceConditionalEquivalent}(\mathcal{M}, \mathcal{N}, Q_C). \end{aligned}$$

These are the source analogues of Definitions A.5.14 and A.5.15.

Lean implementation. Joint and conditional source identifiability are `FiniteSourceIdentifiable` and `FiniteSourceConditionalIdentifiable`.

Definition A.8.9 (Intrinsic dependent causal target). The dependent target interprets a finite table source as an intrinsic latent-root model.

Data. Let T be a finite table signature and let $\mathcal{M}$ be a finite table structural causal model over T , in the sense of Definitions A.8.1 and A.8.3.

Signature interpretation. Its coded endogenous signature is an endogenous signature $\overline{T}$ of Definition A.4.13, with $\text{Val}(i) = \text{Fin}(\text{arity}_T(i))$. Its assignment type is $\Pi_{i:V_T} \text{Fin}(\text{arity}_T(i))$.

Model interpretation. Interpreting $\mathcal{M}$ produces a structural causal model $\overline{\mathcal{M}}$ of Definition A.4.18 on $\overline{T}$. It retains the incidence, coded mechanisms and product-law evidence of the source. This is another presentation of the same finite model data, not another model class.

Adequacy. The interpreted model is Markovian when every latent root is incident to at most one endogenous mechanism and canonical semi-Markovian when every root is incident to at most two. Shared roots induce the bidirected edges of Definition A.4.23. Source finite-sum semantics and target pushforward semantics agree pointwise up to the rational equivalence $\sim$ of

Definition A.3.8. Observational, interventional and partial-kernel agreement is proved in Section B.7.

Scope. The target is an intrinsic dependent representation of the finite type-theoretic structure used here. It is not a syntax-and-semantics theorem for every presentation of Martin-Löf-style type theory. Its constructive and intensional boundary is explained in Section B.5.

Lean implementation. `FiniteTableSignature.toObserved` interprets the signature. `FiniteTableSCM.interpret` interprets the model and produces an `ExactModel`.

Definition A.8.10 (Transport query). A transport query is either a joint or a conditional kernel on the interpreted source signature.

Query type. For $S = (T, G) : \text{FiniteTableSource}$, define:

$$\text{TransportQuery}(S) := \text{Kernel}(\overline{T}) + \text{CondKernel}(\overline{T}).$$

Interpretation. This sum distinguishes the two kernel syntaxes. It does not contain a support witness; local support for a partial conditional value is supplied separately in the sense of Definition A.5.9.

Definition A.8.11 (Restricted finite correspondence fragment). The restricted fragment packages a source presentation, one compatible model and one query compared by the transport theorems.

Fragment type. Define:

$$\begin{aligned} \text{CorrespondenceFragment} := \Sigma_{S:\text{FiniteTableSource}} \\ \left(\text{SourceMod}(\text{signature}(S), \text{graph}(S)) \right. \\ \left. \times \text{TransportQuery}(S) \right). \end{aligned}$$

Interpretation. This is a restriction of the theorem domain, not a fourth model class. The source table and interpreted model are two presentations of the same finite data. Transport quantifies over this package rather than over unrestricted classical or type-theoretic models.

Running example. Mira's eight Boolean nodes determine a coded signature T_{Mira} and graph G_{Mira} . Together they form a finite table source presentation. Its six-root table model interprets as $\mathcal{M}_{\text{Mira}}$. The joint kernel

$$P(\text{Qu} \mid \text{do}(\text{Pr}))$$

is a query in this fragment. Evaluating it at $\text{Pr} = \text{true}$ and $\text{Qu} = \text{true}$ gives

$$P(\text{Qu} = \text{true} \mid \text{do}(\text{Pr} = \text{true})).$$

A conditional-kernel query belongs to the fragment when its conditioning selection has local support. Its checked realisation is summarised in Corollary B.10.11.

Certificates and support

Definition A.8.12 (Action-free probability expression). Action-freeness identifies expressions that can be evaluated from an observational assignment alone.

A. Definitions

Predicate. Let $t : \text{Expr}(\mathcal{V})$ be an expression of Definition A.5.19. Define $\text{ActionFree}(t)$ recursively: every primitive kernel occurring in t must have the empty action selection, and every immediate subexpression must itself be action-free. The zero expression is action-free directly; a kernel is action-free exactly when its action selection is empty.

Interpretation. An action-free expression can be evaluated from the observational assignment alone. It is therefore the required endpoint of an identification derivation, rather than merely an expression in which intervention notation has been hidden.

Lean implementation. The predicate is `ProbabilityTerm.ActionFree`.

Definition A.8.13 (Local derivation support). Local derivation support records that every partial operation used by a concrete derivation is defined at one assignment.

Support type. Let $\mathcal{V}$ be an endogenous signature, let $\mathcal{G} : \text{CausalGraph}(\mathcal{V})$, let $\mathcal{M} : \text{Mod}(\mathcal{G})$, and let $t, t' : \text{Expr}(\mathcal{V})$. For $D : \text{Derive}_{\mathcal{G}}(t, t')$ in the sense of Definition A.5.21 and $a : \text{Val}(\mathcal{V})$, define $\text{DerivationSupport}_{\mathcal{M}}(a, D)$ recursively on D . Its fields state that the denotations of t and t' are present values of `PartialQ` and supply the corresponding support evidence for every immediate subderivation. In particular, every denominator introduced by conditioning or division must be positive.

Interpretation. Support belongs to the chosen derivation at the chosen assignment. It verifies every partial intermediate step, not only the initial and final expressions.

Lean implementation. The recursive support type is `LocalDerivationSupport`.

Definition A.8.14 (Identification certificate). An identification certificate reduces one source expression to an observational expression and records where that derivation is defined.

Data. Let $\mathcal{G}$ be a finite causal graph, let $t : \text{Expr}(\mathcal{V})$ be an expression of Definition A.5.19, and write $\Omega := \text{Val}(\mathcal{V})$. Write $\text{Derive}_{\mathcal{G}}(t, t')$ for the type of derivations of Definition A.5.21 from t to t' . Compatible models are those of Definition A.5.13.

For an expression s and assignment a , abbreviate

$$\text{DefinedAt}_{\mathcal{M}}(s, a) := \text{DefinedAt}(\lambda b. \llbracket s \rrbracket_{\mathcal{M}}(b), a),$$

using the definedness type of Definition A.5.9.

Certificate record. Define the record `IdentCertG(t)` with fields:

$$\begin{aligned} &\text{formula} : \text{Expr}(\mathcal{V}), \\ &h_{\text{free}} : \text{ActionFree}(\text{formula}), \\ &\text{derivation} : \text{Derive}_{\mathcal{G}}(t, \text{formula}), \\ &\text{support} : \prod_{\mathcal{M} : \text{Mod}(\mathcal{G})} \prod_{a : \Omega} \text{DefinedAt}_{\mathcal{M}}(t, a) \rightarrow \\ &\quad \text{DerivationSupport}_{\mathcal{M}}(a, \text{derivation}). \end{aligned}$$

Interpretation. The formula is the observational endpoint, the derivation records how the source expression reaches it, and the support field certifies that every partial step is defined wherever the source expression is defined. These

are separate obligations: an action-free formula alone is not an identification certificate.

For a joint or conditional kernel Q of Definition A.5.5, its certificate is obtained by taking t to be the corresponding kernel expression t_Q . The definedness premise is then the local support of Definition A.5.9.

Lean implementation. The generic cores are `IdentificationCertificate` and `PublishedIdentificationCertificate`; the public joint and conditional wrappers retain their query indices.

Definition A.8.15 (Finite source derivation support). Source derivation support specialises local derivation support to the finite-sum semantics of a table source.

Support type. Let (T, G) be a finite table source and write $\text{SourceDerivationSupport}_{\mathcal{M}}(a, D)$ for the source version of the support type of Definition A.8.13, evaluated by the finite-sum semantics of Definition A.8.3. For a source expression s , abbreviate its pointwise support by:

$$\text{DefinedAt}_{\mathcal{M}}^{\text{src}}(s, a) := \text{DefinedAt}(\lambda b. \llbracket s \rrbracket_{\mathcal{M}}^{\text{src}}(b), a).$$

Interpretation. An inhabitant records support for both endpoints of the derivation and recursively for every intermediate denominator. It is therefore stronger than support of the initial expression alone.

Lean implementation. `FiniteTableSCM.TermSupportedAt` is the source term-support predicate, and `FiniteSourceLocalDerivationSupport` is the recursive support tree.

Definition A.8.16 (Finite source identification certificates). Source certificates specialise the identification certificates of Definition A.8.14 to a finite table source. They use the shared do-calculus syntax and derivation type, but their support is interpreted by the finite-sum semantics of the source presentation.

Data. Let (T, G) be a finite table source. Use the source support types of Definition A.8.15.

Joint certificate. For a joint kernel Q_J , define the record $\text{SourceJointCert}_G(Q_J)$ with fields:

formula : $\text{Expr}(\overline{T})$,
 $h_{\text{free}} : \text{ActionFree}(\text{formula})$,
derivation : $\text{Derive}_{\overline{G}}(t_{Q_J}, \text{formula})$,
support : $\prod_{\mathcal{M} : \text{SourceMod}(T, G)} \prod_{a : \text{Assignment}_T} \text{DefinedAt}_{\mathcal{M}}^{\text{src}}(t_{Q_J}, a) \rightarrow \text{SourceDerivationSupport}_{\mathcal{M}}(a, \text{derivation})$.

Conditional certificate. For a conditional kernel Q_C , define the record $\text{SourceConditionalCert}_G(Q_C)$ by the same display with Q_J replaced by Q_C . In both records, the recursive support field includes every endpoint and intermediate denominator reached by the derivation. The conditional wrapper does not add a second, independent positivity field.

Scope. How source certificates interpret as target certificates is not part of this record; it is stated in Appendix B.

Lean implementation. `FiniteSourceJointCertificate` and `FiniteSourceConditionalCertificate` are the joint and conditional certificate records, respectively.

A. Definitions

Definition A.8.17 (Finite source counterexample). A finite source counterexample is explicit evidence that one query is not determined by the source observational assignment.

Counterexample record. For a finite table source (T, G) and a joint kernel query Q , define $\text{SourceCounterexample}_G(Q)$ as the record with fields:

$$\begin{aligned} \text{left, right} &: \text{SourceMod}(T, G), \\ \text{observationalAgreement} &: \text{left} \equiv_{\text{obs}}^{\text{src}} \text{right}, \\ \text{queryDifference} &: \neg \text{SourceJointEquivalent}(\text{left}, \text{right}, Q). \end{aligned}$$

Interpretation. The final field refutes whole-kernel equivalence; it need not choose a particular assignment at which the denotations differ.

Lean implementation. The record is `FiniteSourceCounterexample`.

Definition A.8.18 (Mode-indexed identification certificate). A mode-indexed certificate combines an observational reduction with the modal trace that realises its action and condition selections.

Data. Let $\mathcal{G}$ be a finite causal graph, let m be an epistemic causal mode on the same endogenous signature, let Q be a finite kernel query and let t_Q be its probability expression. Define:

$$\text{ModeCompatible}_{\mathcal{G}}(m) := \text{Compatible}_{\mathcal{G}}(\text{model}(\text{record}(m))).$$

Write $\text{Trace}_m(D)$ for the modal traces of Definition A.7.3 whose underlying derivation is D .

Certificate record. Define the record $\text{ModalIdentCert}_m(Q)$ with fields:

$$\begin{aligned} \text{compatible} &: \text{ModeCompatible}_{\mathcal{G}}(m), \\ \text{certificate} &: \text{IdentCert}_{\mathcal{G}}(t_Q), \\ \text{trace} &: \text{Trace}_m(\text{derivation}(\text{certificate})), \\ \text{realisation} &: \text{Realises}(\text{trace}). \end{aligned}$$

Interpretation. The first field proves compatibility of the current model with $\mathcal{G}$. The certificate supplies the observational reduction; the final two fields record how its action and condition selections are executed between causal modes, using Definition A.7.18.

Lean implementation. The records are indexed by the supplied soundness package. Their joint and conditional forms are `ModeIndexedOperationRealizedJointCertificate` and `ModeIndexedOperationRealizedConditionalCertificate`.

Definition A.8.19 (Finite source counterfactual identifiability). Source counterfactual identifiability compares a multiworld query across observationally equivalent finite table models.

Identifiability type. Let (T, G) be a finite table source and let $Q : \text{CFQuery}(\bar{T})$. Define:

$$\begin{aligned} \text{SourceCFIdentifiable}_G(Q) &:= \Pi_{\mathcal{M}, \mathcal{N} : \text{SourceMod}(T, G)} (\mathcal{M} \equiv_{\text{obs}}^{\text{src}} \mathcal{N}) \\ &\rightarrow ([Q]_{\mathcal{M}}^{\text{src}} \sim [Q]_{\mathcal{N}}^{\text{src}}). \end{aligned}$$

Interpretation. The type is the source-table analogue of counterfactual identifiability in Definition A.6.7. It requires one query value to be invariant across observationally equivalent compatible table models.

Lean implementation. This predicate is `FiniteSourceCounterfactualIdentifiable`.

Definition A.8.20 (Uniform finite source counterfactual support). Uniform support states that a source counterfactual query has a present rational value in every compatible finite table model.

Support type. Let (T, G) be a finite table source and let $Q : \text{CFQuery}(\overline{T})$. Define:

$$\text{SourceCFSupported}_G(Q) := \prod_{\mathcal{M} : \text{SourceMod}(T, G)} \sum_{p : \mathbb{Q}_{\geq 0}} (\llbracket Q \rrbracket_{\mathcal{M}}^{\text{src}} \sim \text{some}(p)).$$

Interpretation. The chosen rational may depend on the model, but a present value must exist in every compatible source model. This is the source-table analogue of the uniform support condition of Definition A.6.8.

Lean implementation. The predicate is `FiniteSourceCounterfactualSupported`. Lean stores each pointwise support witness under `Nonempty`; the displayed constructive presentation retains a chosen witness p .

Definition A.8.21 (Finite source counterfactual reduction). A source counterfactual reduction rewrites one supported multiworld query as an ordinary finite probability expression.

Reduction record. Let (T, G) be a finite table source and let Q be a counterfactual query of Definition A.6.4. Define the record $\text{CFReduction}_G(Q)$ with fields:

$$\begin{aligned} \text{reference} &: \text{Assignment}_T, \\ \text{reducedFormula} &: \text{Expr}(\overline{T}), \\ \text{agreement} &: \prod_{\mathcal{M} : \text{SourceMod}(T, G)} (\llbracket Q \rrbracket_{\mathcal{M}}^{\text{src}} \sim \llbracket \text{reducedFormula} \rrbracket_{\mathcal{M}}^{\text{src}}(\text{reference})). \end{aligned}$$

Interpretation. The reference assignment selects the point at which the ordinary expression is evaluated. The agreement field says that this value reproduces the original multiworld query in every compatible source model. Interventional kernels may remain; only the counterfactual syntax has been eliminated.

Lean implementation. The record is `FiniteSourceCounterfactualReduction`.

Definition A.8.22 (Observationalisation). Finite source counterfactual observationalisation continues a counterfactual reduction to an action-free source expression.

Observationalisation record. For $R : \text{CFReduction}_G(Q)$, define $\text{CFObservationalisation}(R)$ with fields:

$$\begin{aligned} \text{observationalFormula} &: \text{Expr}(\overline{T}), \\ \text{actionFree} &: \text{ActionFree}(\text{observationalFormula}), \\ \text{derivation} &: \text{Derive}_{\overline{G}}(\text{reducedFormula}(R), \text{observationalFormula}), \\ \text{support} &: \prod_{\mathcal{M} : \text{SourceMod}(T, G)} \text{SourceDerivationSupport}_{\mathcal{M}}(\text{reference}(R), \text{derivation}). \end{aligned}$$

Interpretation. The preceding reduction has translated the multiworld query into an ordinary probability expression, but that expression may still contain

A. Definitions

interventions. An observationalisation supplies a supported do-calculus derivation that removes those remaining actions. Its action-free endpoint can therefore be evaluated from the observational assignment alone.

Lean implementation. The record is `FiniteSourceCounterfactualObservationalization`. Its Lean presentation is additionally indexed by the supplied finite source soundness package because its stored path derivation is compiled through that package's d-separation interface.

External theorem interfaces

Definition A.8.23 (Executable d-separation test). The executable graphical side condition is a Boolean test on one mutilated finite causal graph.

Test type. Let $\mathcal{G}$ be a causal graph on finite node type V . Finite ancestral moralisation and reachability define:

$$\begin{aligned} \text{dSepTest}_{\mathcal{G}} : \text{Mutilation}(V) \\ &\rightarrow (V \rightarrow \text{Bool}) \rightarrow (V \rightarrow \text{Bool}) \\ &\rightarrow (V \rightarrow \text{Bool}) \rightarrow \text{Bool}. \end{aligned}$$

Its arguments are a graph mutilation and the two endpoint selections and conditioning selection. This definition supplies a decidable classifier; agreement with the path specification is separate evidence.

Interpretation. The Boolean result is an executable answer to one finite graphical query. It is not by itself evidence that the answer agrees with the path-blocking definition; that evidence is the separate interface below.

Lean implementation. The Boolean test is `ObservedGraph.dSeparated`, using the finite ancestral moralisation and reachability construction.

Definition A.8.24 (Executable d-separation correctness interface). The path definition of d-separation and its executable finite test are connected by an explicit correctness interface.

Data. Let $\mathcal{G}$ be a causal graph of Definition A.4.11, let $m : \text{Mutilation}(V)$ and let $X, Y, Z : V \rightarrow \text{Bool}$. The Boolean test is that of Definition A.8.23.

Write

$$\begin{aligned} \text{TestSeparates}_{\mathcal{G}}(m, X, Y, Z) &:= \text{Id}_{\text{Bool}}(\text{dSepTest}_{\mathcal{G}}(m, X, Y, Z), \text{true}), \\ \text{PathSeparates}_{\mathcal{G}}(m, X, Y, Z) &:= (X \perp\!\!\!\perp Y \mid Z)_{\mathcal{G}_m}. \end{aligned}$$

Interface. The correctness interface carries, for every m, X, Y, Z , both implications between the test returning true and the path-blocking judgement

$$(X \perp\!\!\!\perp Y \mid Z)_{\mathcal{G}_m}$$

of Definition A.4.28. More explicitly, define the record `DSeparationCorrect( $\mathcal{G}$ )` with fields:

$$\begin{aligned} \text{testToPath} : \prod_{m:\text{Mutilation}(V)} \prod_{X,Y,Z:V \rightarrow \text{Bool}} &\text{TestSeparates}_{\mathcal{G}}(m, X, Y, Z) \rightarrow \text{PathSeparates}_{\mathcal{G}}(m, X, Y, Z), \\ \text{pathToTest} : \prod_{m:\text{Mutilation}(V)} \prod_{X,Y,Z:V \rightarrow \text{Bool}} &\text{PathSeparates}_{\mathcal{G}}(m, X, Y, Z) \rightarrow \text{TestSeparates}_{\mathcal{G}}(m, X, Y, Z). \end{aligned}$$

Thus an executable side condition may be used exactly when the corresponding path specification is inhabited.

Scope. The interface does not assume a probabilistic Markov property. It states only that two finite graphical formulations of d-separation agree.

Lean implementation. The interface is `DSeparationCorrectness`; the two compared specifications are `ObservedGraph.dSeparated` and `PathSpecification.PathDSeparated`.

Definition A.8.25 (Finite source primitive soundness). Primitive source soundness states the local denotational laws from which soundness of a complete derivation is obtained by induction.

Data. Let (T, G) be a finite table source and let $\mathcal{M} : \text{SourceMod}(T, G)$. For source expressions s, t and an assignment $a : \text{Assignment}_T$, write $\text{EquivalentAt}_{\mathcal{M}}^{\text{src}}(s, t, a)$ when their two present source denotations are equivalent under $\sim$.

Soundness record. Define $\text{PrimitiveSoundness}(\mathcal{M}, G)$ as a record with four fields:

A.8.25.1. *do-rule soundness*, which turns any application of a rule from Definition A.5.20, together with support of both sides at a , into $\text{EquivalentAt}_{\mathcal{M}}^{\text{src}}(s, t, a)$;

A.8.25.2. *marginalisation soundness*, which gives the same result for the finite marginalisation constructor;

A.8.25.3. *conditioning soundness*, which gives it for the definition of conditional probability; and

A.8.25.4. *chain-rule soundness*, which gives it for the finite chain-rule constructor.

Each field quantifies over the node selections and complete assignment used by its operation, and requires the corresponding disjointness, graphical and local-support evidence.

Lean implementation. The path-facing record is `FiniteSourcePathPrimitiveSoundness`; compiling its graphical conditions gives `FiniteSourceLocalPrimitiveSoundness`.

Definition A.8.26 (External finite source soundness interface). Source denotational soundness is supplied as a proof-carrying interface, rather than inferred from a bibliographic reference.

Data. Let (T, G) be a finite table source presentation in the sense of Definition A.8.5, and write $\overline{G}$ for its intrinsic graph. For a compatible source model $\mathcal{M}$, use the primitive soundness type of Definition A.8.25.

Interface. Define the record $\text{SourceSoundness}(T, G)$ with fields:

$$\begin{aligned} \text{dSeparation} &: \text{DSeparationCorrect}(\overline{G}), \\ \text{primitiveSoundness} &: \prod_{\mathcal{M} : \text{SourceMod}(T, G)} \text{PrimitiveSoundness}(\mathcal{M}, G). \end{aligned}$$

The second field states that marginalisation, conditioning, the chain rule and each of Pearl's three rules preserve the finite source denotation on their stated local support. Induction on a source derivation may use precisely these primitive fields.

A. Definitions

Scope. An inhabitant is the required finite specialisation of the published soundness results. The transport development consumes this inhabitant but does not construct it from the cited papers.

Lean implementation. The corresponding package is `PublishedFiniteSourceSoundness`. Its path-facing primitive component has type `FiniteSourcePathPrimitiveSoundness`.

Definition A.8.27 (External finite source completeness interface). Completeness is a separate interface over the same finite table source.

Data. Let T and G be as in Definition A.8.26.

Interface. Define the record `SourceCompleteness( $T, G$ )` with fields:

$$\begin{aligned}
 & \text{dSeparation} : \text{DSeparationCorrect}(\overline{G}), \\
 & \text{jointComplete} : \Pi_{Q:\text{Kernel}(\overline{T})} \\
 & \quad \text{SourceJointIdentifiable}_G(Q) \\
 & \quad \rightarrow \text{SourceJointCert}_G(Q), \\
 & \text{conditionalComplete} : \Pi_{Q:\text{CondKernel}(\overline{T})} \\
 & \quad \text{SourceConditionalIdentifiable}_G(Q) \\
 & \quad \rightarrow \text{SourceConditionalCert}_G(Q), \\
 & \text{hedgeCounterexample} : \Pi_{Q:\text{Kernel}(\overline{T})} \text{Hedge}_{\overline{G}}(Q) \\
 & \quad \rightarrow \text{SourceCounterexample}_G(Q).
 \end{aligned}$$

The final field takes a hedge witness of Definition A.5.18 and returns a source counterexample of Definition A.8.17. The witness is an input to this construction, not a field of the resulting counterexample.

Scope. The package specifies the exact finite source theorem needed by transport. It does not assert that every published completeness theorem automatically inhabits this specialised record.

Lean implementation. `PublishedFiniteSourceCompleteness` is the corresponding package.

Definition A.8.28 (External finite counterfactual-completeness interface). Counterfactual completeness requires a source-native multiworld reduction in addition to ordinary source soundness.

Data. Let (T, G) be a finite table source presentation and let `sound` : `SourceSoundness( $T, G$ )` be a soundness package of Definition A.8.26. Use the source-native identifiability and support conditions of Definitions A.8.19 and A.8.20.

Interface. Define the record `SourceCFCompleteness( $T, G, \text{sound}$ )` with the field:

$$\begin{aligned}
 & \text{complete} : \Pi_{Q:\text{CFQuery}(\overline{T})} \text{SourceCFIdentifiable}_G(Q) \rightarrow \text{SourceCFSupported}_G(Q) \\
 & \quad \rightarrow \Sigma_{R:\text{CFReduction}_G(Q)} \text{CFObservationalisation}(R).
 \end{aligned}$$

Thus every source-identifiable and uniformly supported query receives both a multiworld reduction and an action-free continuation, each with its local support evidence.

Scope. This interface specifies the output required from the finite specialisation of the counterfactual-identification algorithms. It does not implement their branch selection or termination argument.

Lean implementation. `PublishedFiniteCounterfactualCompleteness` is the corresponding package. Its component types are `FiniteSourceCounterfactualReduction` and `FiniteSourceCounterfactualObservationalization`.

Definition A.8.29 (Hidden model family). A hidden model family selects an inhabited collection of executable models on one finite hidden directed acyclic graph.

Data. Let P be a finite hidden-DAG projection of Definition A.4.35. Write $\text{HiddenSCM}(P)$ for the type of executable finite structural causal models on its full hidden directed acyclic graph, whose finite semantics are described in Proposition B.8.3. Define the record $\text{HiddenModelFamily}(P)$ with fields:

$$\begin{aligned} \text{Model}_{\mathcal{H}} &: \mathcal{U}, \\ \text{model}_0 &: \text{Model}_{\mathcal{H}}, \\ \text{semantics} &: \text{Model}_{\mathcal{H}} \rightarrow \text{HiddenSCM}(P). \end{aligned}$$

Interpretation. The term model_0 witnesses that the selected model-index type is inhabited. It is used only to rule out an empty model family; no construction depends on which inhabitant was chosen. The index type need not itself be finite. Every indexed model nevertheless has executable finite semantics. Joint and common-support conditional comparisons within this family use the observational-equivalence criterion of Definition A.5.15.

Lean implementation. The selected family is `HiddenDAGModelFamily`. Lean stores its inhabitation as `Nonempty Model`; the displayed constructive presentation retains the chosen inhabitant model_0 .

Definition A.8.30 (Hidden-family identifiability). Hidden-family identifiability restricts the ordinary model comparison to a selected hidden model family.

Family identifiability. Let P be as in Definition A.8.29 and let $\mathcal{H} : \text{HiddenModelFamily}(P)$. For model indices $m, n : \text{Model}_{\mathcal{H}}$, abbreviate $\mathcal{M}_m := \text{semantics}(m)$ and $\mathcal{M}_n := \text{semantics}(n)$. For a joint query Q_J , define:

$$\begin{aligned} \text{HiddenJointIdentifiable}_{\mathcal{H}}(Q_J) \\ := \prod_{m, n : \text{Model}_{\mathcal{H}}} (\mathcal{M}_m \equiv_{\text{obs}} \mathcal{M}_n) \rightarrow \left(\llbracket Q_J \rrbracket_{\mathcal{M}_m}^J \simeq \llbracket Q_J \rrbracket_{\mathcal{M}_n}^J \right). \end{aligned}$$

For a conditional query Q_C , define:

$$\begin{aligned} \text{HiddenConditionalIdentifiable}_{\mathcal{H}}(Q_C) \\ := \prod_{m, n : \text{Model}_{\mathcal{H}}} (\mathcal{M}_m \equiv_{\text{obs}} \mathcal{M}_n) \rightarrow \text{AgreesWhereDefined}(\llbracket Q_C \rrbracket_{\mathcal{M}_m}^C, \llbracket Q_C \rrbracket_{\mathcal{M}_n}^C). \end{aligned}$$

Interpretation. These conditions ask whether observational agreement determines the query within the selected hidden model family, rather than among all models compatible with the projected graph. Joint queries require agreement of the complete denotations, whereas conditional queries compare values only at assignments supported by both models. Because the selected family need not contain every compatible model, family-relative identifiability does not by itself establish graph-wide identifiability.

Projected comparison. For the projected graph, write:

$$\begin{aligned} \text{Identifiable}_{\text{graph}(P)}(Q) &\quad \text{for the joint condition,} \\ \text{ConditionalIdentifiable}_{\text{graph}(P)}(Q) &\quad \text{for the conditional condition,} \end{aligned}$$

A. Definitions

in the sense of Definition A.5.15.

Definition A.8.31 (External latent-projection interface). The general hidden-DAG transport connects identifiability in a selected hidden family with identifiability on its projected causal graph.

Interface record. Let P be a finite hidden-DAG projection and let $\mathcal{H} : \text{HiddenModelFamily}(P)$ as in Definitions A.8.29 and A.8.30. Define $\text{LatentProjectionInterface}(P, \mathcal{H})$ with fields:

$$\begin{aligned} \text{joint} &: \Pi_{Q:\text{Kernel}(\mathcal{V}_P)} \\ &\quad \text{HiddenJointIdentifiable}_{\mathcal{H}}(Q) \\ &\quad \leftrightarrow \text{Identifiable}_{\text{graph}(P)}(Q), \\ \text{conditional} &: \Pi_{Q:\text{CondKernel}(\mathcal{V}_P)} \\ &\quad \text{HiddenConditionalIdentifiable}_{\mathcal{H}}(Q) \\ &\quad \leftrightarrow \text{ConditionalIdentifiable}_{\text{graph}(P)}(Q). \end{aligned}$$

Here $\mathcal{V}_P$ and $\text{graph}(P)$ are the observed signature and projected causal graph carried by P .

Scope. The interface does not quantify over every imaginable hidden model. Its model family is explicit data, and every member retains internally executable finite semantics.

Lean implementation. `PublishedLatentProjection` is the corresponding interface.

A.9 Worked finite causal example

This section applies the finite definitions to Mira’s tenure-track model. Exact rational calculations distinguish observation from intervention, including the collider arithmetic, and carry out a model-level counterfactual by abduction, action and prediction.

Example A.9.1 (Mira’s finite model specification). Mira’s tenure-track example is assembled as a fully finite model from Definitions A.4.13, A.4.14, A.4.16 to A.4.18, A.4.20, A.4.21, A.5.3 and A.6.6.

Endogenous signature. All endogenous value types are the two-element Boolean type. In the numerical formulas below, 1 abbreviates `true` and 0 abbreviates `false`.

The endogenous node type is $V_{\text{Mira}} := \text{Fin}(8)$, with names

Pr, Qu, Top, Com, Fit, Short, Fund, Off

in that order.

Probability notation. Write $\mu_{\mathcal{U}}$ for the exogenous product assignment and $\mu_{\mathcal{M}}$ for its observational pushforward through the solution map. For a hard intervention x , write $\mu_{\mathcal{M}_x}$ for the law field of $\text{InterventionResult}_{\mathcal{M}}(x)$ from Definition A.5.3. These subscripts keep the probability-bearing assignment explicit in the calculations below. Coordinate equations use the equality events and probability notation of Definition A.3.23; a comma denotes their Boolean intersection in the sense of Definition A.3.3. Every displayed comparison between nonnegative rational probability values is read through the equivalence $\sim$ of Definition A.3.8.

A.9. Worked finite causal example

Exogenous source. The exogenous variables are independent Boolean variables

$$U_B, U_{Pr}, U_{Qu}, U_{Top}, U_{Com}, U_{Fund},$$

with finite rational masses

$$P_{\mu_U}(U_B = 1) \sim \frac{1}{3}, \quad P_{\mu_U}(U_{Pr} = 1) \sim P_{\mu_U}(U_{Qu} = 1) \sim P_{\mu_U}(U_{Fund} = 1) \sim \frac{1}{4},$$

and

$$P_{\mu_U}(U_{Top} = 1) \sim P_{\mu_U}(U_{Com} = 1) \sim \frac{1}{2}.$$

Their joint assignment satisfies the exogenous product law of Definition A.4.14; this is the precise meaning of independence in the displayed model.

Structural mechanisms. The structural functions are

$$\begin{aligned} Pr &:= U_B \vee U_{Pr}, & Qu &:= U_B \vee U_{Qu}, \\ Top &:= U_{Top}, & Com &:= U_{Com}, \\ Fit &:= Top \wedge Com, & Short &:= Qu \wedge (Pr \vee Fit), \\ Fund &:= Top \vee U_{Fund}, & Off &:= Short \wedge Fund. \end{aligned}$$

Here $\wedge$ and $\vee$ abbreviate the Boolean conjunction and disjunction derived in Definition A.1.20.

Graph. In the full finite directed acyclic graph, U_B is a common cause of Pr and Qu . If U_B is hidden, the projected semi-Markovian graph has the bidirected edge $Pr \leftrightarrow Qu$. The node selection returning true exactly at Pr and Qu is therefore a c-component in the sense of Definition A.5.16. The graph also contains the colliders

$$Top \rightarrow Fit \leftarrow Com, \quad Pr \rightarrow Short \leftarrow Fit, \quad Short \rightarrow Off \leftarrow Fund.$$

Example A.9.2 (Mira's observational and interventional calculations). The finite model of Example A.9.1 distinguishes graphical conditioning, observation and intervention by exact rational calculation.

Collider calculation. The nodes Top and Com are d-separated by the empty conditioning selection in the sense of Definition A.4.28, but the path through Fit is opened when one conditions on Fit or one of its descendants. Numerically, the deterministic equation $Fit := Top \wedge Com$ gives

$$P_{\mu_M}(Top = 1 \mid Com = 1) \sim \frac{1}{2} \sim P_{\mu_M}(Top = 1),$$

while conditioning on the collider event $Fit = 0$ changes the relation:

$$P_{\mu_M}(Top = 1 \mid Fit = 0) \sim \frac{1}{3}, \quad P_{\mu_M}(Top = 1 \mid Com = 1, Fit = 0) \sim 0.$$

This is the collider phenomenon in a finite deterministic form.

Observation and intervention. The distinction between observation and intervention is also visible by finite summation. First,

$$P_{\mu_M}(Pr = 1) \sim 1 - P_{\mu_U}(U_B = 0, U_{Pr} = 0) \sim 1 - \frac{2}{3} \cdot \frac{3}{4} \sim \frac{1}{2}.$$

A. Definitions

Similarly $P_{\mu_{\mathcal{M}}}(\text{Qu} = 1) \sim 1/2$, and

$$P_{\mu_{\mathcal{M}}}(\text{Pr} = 1, \text{Qu} = 1) \sim 1 - \frac{1}{2} - \frac{1}{2} + \frac{2}{3} \cdot \frac{3}{4} \cdot \frac{3}{4} \sim \frac{3}{8}.$$

Hence

$$P_{\mu_{\mathcal{M}}}(\text{Qu} = 1 \mid \text{Pr} = 1) \sim \frac{P_{\mu_{\mathcal{M}}}(\text{Pr} = 1, \text{Qu} = 1)}{P_{\mu_{\mathcal{M}}}(\text{Pr} = 1)} \sim \frac{3/8}{1/2} \sim \frac{3}{4}.$$

But the intervention $\text{do}(\text{Pr} = 1)$ replaces the equation for Pr by the constant value 1 and leaves the mechanism for Qu unchanged, so

$$P_{\mu_{\mathcal{M}_{\text{Pr}=1}}}(\text{Qu} = 1) \sim P_{\mu_{\mathcal{M}}}(\text{Qu} = 1) \sim \frac{1}{2}.$$

Committee fit. A second calculation shows why observing committee fit is not the same as forcing committee fit. Since $\text{Fit} = 1$ implies $\text{Top} = 1$ and $\text{Com} = 1$, it also implies $\text{Fund} = 1$. Under $\text{Fit} = 1$, the shortlist equation reduces to $\text{Short} = \text{Qu}$, and therefore

$$P_{\mu_{\mathcal{M}}}(\text{Off} = 1 \mid \text{Fit} = 1) \sim P_{\mu_{\mathcal{M}}}(\text{Qu} = 1) \sim \frac{1}{2}.$$

By contrast, in the intervened model $\text{do}(\text{Fit} = 1)$, the variable Top is not forced. The offer requires both $\text{Qu} = 1$ and $\text{Fund} = 1$, and these are independent because they depend on disjoint exogenous variables. Since

$$P_{\mu_{\mathcal{M}}}(\text{Fund} = 1) \sim 1 - P_{\mu_{\mathcal{M}}}(U_{\text{Top}} = 0, U_{\text{Fund}} = 0) \sim 1 - \frac{1}{2} \cdot \frac{3}{4} \sim \frac{5}{8},$$

one gets

$$P_{\mu_{\mathcal{M}_{\text{Fit}=1}}}(\text{Off} = 1) \sim \frac{1}{2} \cdot \frac{5}{8} \sim \frac{5}{16}.$$

Example A.9.3 (Mira's counterfactual and modal calculation). The same finite model supports a model-level counterfactual query and its modal execution.

Counterfactual calculation. Finally, the same finite model permits a model-level counterfactual calculation. Define the Boolean classifier

$$E : \text{Val}(\mathcal{V}_{\text{Mira}}) \rightarrow \text{Bool}$$

to return true exactly on complete assignments satisfying $\text{Pr} = 0$, $\text{Qu} = 1$ and $\text{Off} = 0$. It is the conjunction, using and_{Bool} , of the three decidable equality classifiers. Under the structural equations, $\text{Pr} = 0$ forces $U_B = 0$ and $U_{\text{Pr}} = 0$. Together with $\text{Qu} = 1$, this forces $U_{\text{Qu}} = 1$. Moreover, when $\text{Pr} = 0$ and $\text{Qu} = 1$, the equation for Short reduces to $\text{Short} = \text{Fit}$. Since $\text{Fit} = 1$ implies $\text{Top} = 1$, hence $\text{Fund} = 1$, the additional evidence $\text{Off} = 0$ forces $\text{Fit} = 0$. Thus

$$P_{\mu_{\mathcal{M}}}(E) \sim \frac{2}{3} \cdot \frac{3}{4} \cdot \frac{1}{4} \cdot \frac{3}{4} \sim \frac{3}{32},$$

so the abduction step is well defined.

Condition on E , then perform the action $\text{do}(\text{Pr} = 1)$. In the intervened model, $\text{Short} = 1$, because $\text{Qu} = 1$ is retained from the posterior evidence and Pr has been set to 1. Therefore the counterfactual offer depends only on

$$\text{Fund} = \text{Top} \vee U_{\text{Fund}}.$$

A.9. Worked finite causal example

The evidence about **Pr** and **Qu** is independent of $U_{\text{Top}}, U_{\text{Com}}, U_{\text{Fund}}$, and the remaining relevant information is $\text{Fit} = 0$, i.e. not both **Top** and **Com**. Hence

$$P_{\mu_{\mathcal{M}}}(\text{Fund} = 1 \mid E) \sim P_{\mu_{\mathcal{M}}}(\text{Fund} = 1 \mid \text{Fit} = 0) \sim \frac{1/4 + 1/8}{3/4} \sim \frac{1}{2}.$$

Consequently

$$P_{\mathcal{M}}(\text{Off}_{\text{Pr}:=1} = 1 \mid E) \sim \frac{1}{2}.$$

The same posterior also gives

$$P_{\mathcal{M}}(\text{Off}_{\text{Fit}:=1} = 1 \mid E) \sim \frac{1}{2},$$

because setting $\text{Fit} = 1$ likewise makes **Short** = 1 while leaving the same posterior uncertainty about **Fund**. This is Pearl’s abduction–action–prediction procedure in the finite model: first condition the exogenous distribution, then change the structural equation, then evaluate the target. It is not a proof that arbitrary counterfactuals are identifiable from the observational distribution alone.

Type-theoretic interpretation. Internally to the present framework, this example is a finite structural causal model, packaged as an epistemic causal mode. The model is a structural causal model of Definition A.4.18 on the endogenous signature of the example. The executable instance also belongs to the restricted source–target fragment of Definition A.8.11. The structural equations are terms in Boolean value types; the exogenous assignment carries a finite rational probability assignment; and every displayed probability is computed by a finite sum or a quotient with positive denominator.

Lean implementation. The executable model is `TenureTrack.model` in `lean/Thesis/Examples/TenureTrack.lean`. It is an `ExactModel`, so its observational, interventional, conditional, counterfactual and modal calculations use the same records and recursive evaluator as the correspondence development. Its principal checked values and the agreements with the twin network, atomic abduction–action–prediction and both occurrence constructions are collected in Corollary B.10.11.

APPENDIX B

Proofs and arguments

“The important thing is life — life and nothing else! What is any ‘discovery’ whatever compared with the incessant, eternal discovery of life?”

Hippolyte in *Idiot*

This appendix collects the supporting arguments and proofs: philosophical motivation, the constructive boundary of the type theory, the Lean verification conventions, the finite probability reconstruction and the finite causal transport.

The foundational route is summarised in Table B.1. Section B.5 then explains how the Lean-supported claims are to be read and provides the detailed dependency map in Table B.4 for the formal proofs and transports that follow.

Section	Establishes	Depends on	Used by
Causality argument	Why causal distinctions belong in mathematical inference.	Scientific explanation	Causal framework
Physics argument	Why the formal setting is finite, constructive and epistemic.	Finite measurement	Constructive setting
Excluded middle	Why global excluded middle is not derivable in the bare fragment.	Kripke/presheaf semantics	Constructive facts; probability
Constructive facts	Negation, decidability and equality results for finite types.	Excluded-middle boundary; finite types	Finite constructions
Lean boundary	How checked, conditional and cited claims are to be read.	Modules; axiom audit	Sections B.6 to B.11

Table B.1: Foundational route through the opening sections of Appendix B.

B.1 Causality argument

This section is a pragmatic argument for including causal models in a theory of mathematical inference. Scientific questions distinguish seeing from doing, and a formal language has to mark that distinction.

Human beings reason causally almost before they reason formally. We ask what made the glass fall, why the patient recovered, what caused the model to fail, which policy changed the outcome, and what would have happened had the intervention been different. Causal talk of this kind is an ordinary grammar of human orientation, not an exotic philosophical habit.

Science can be described, at least in part, as the disciplined extension of this causal grammar. It constructs, tests, rejects and stabilises causal models of material, biological and social situations. A science without such models would not merely be technically incomplete: it would fail to speak the language in which many scientific questions are actually asked.

None of which is to say that any causal model will do. Scientific inquiry is communal, historical and bounded. With Huizinga, one may say that it contains a playful and creative element: scientists construct models, concepts and experimental situations together [Hui49]. With Erasmus, one may add that such constructions retain an ironic or provisional character: they are humanly made, and therefore should not be confused with reality itself [Era93]. With Kierkegaard, one may add that irony is not limitless: it is bounded by what has already been inherited [Kie80; Kie83; Kie89]. Fleck sharpens this social point: causal judgement is stabilised inside communities, inherited styles of thought, shared techniques and historically formed expectations [Fle79]. Hacking sharpens the practical point: science is not only representational, but interventionist, because scientists isolate, manipulate, produce, measure

B. Proofs and arguments

and stabilise phenomena [Hac83]. To this I add that scientific construction is bounded by intersubjective accountability: claims must pass through other people, practices, instruments, institutions and traditions of criticism.

The result is a middle position about those models. Against a crude Feyerabendian reading, not any causal story is acceptable: a model still has to pass through shared techniques, instruments and criticism [Fey75]. Against a crude Humean scepticism, one need not first prove that causation is a feature of the world itself in order to treat causal models as scientifically indispensable [Hum75; Hum78]. Against a purely Kantian framing, whether causality originates in the world or in human cognition is not the question that decides the formal language: once science is read as a practice of shared orientation, intervention and explanation, what matters is that the models can mark those acts [Kan98].

That philosophical demand does not yet select a calculus. A formal theory of mathematical inference for science needs a language in which a causal model can distinguish seeing from doing and association from intervention, and can ask a counterfactual question without treating it as ordinary conditioning. Pearl's framework is the language used here, justified pragmatically rather than metaphysically [Pea09; PM18]. It is chosen because it gives those distinctions a home, not because it is deduced from the argument above, and not because it solves every philosophical problem about causality.

B.2 Physics argument

This section is a pragmatic argument for a constructive, epistemological reading of physical measurement.

A measured value is a finite construction – with precision, procedure, instrument and context – rather than an actually completed real number handed down from the heavens. Gisin's work is valuable because it presses this point against the habitual use of completed real numbers in physics [Bar26; BDG25; DG19; Gis20].

The difficulty is that Gisin's proposed numbers still risk importing infinity through the back door. He treats an observed number as an infinite sequence of digit-wise propensities – probability-like weights for each later digit – so the number may look constructive while still depending on an indefinitely extended object. That is not the constructivist instinct taken seriously enough. A genuinely constructive reading should say that measurement constructs a value up to the precision of the measurement, not that it reveals part of a hidden infinite propensity sequence.

Gisin's finite-information picture is not offered as a reconstruction of quantum mechanics: it can suggest indeterminism and related tensions, but it does not recover the quantum theory that is actually used.

Spekkens' toy theory is the link to quantum physics as it is now practised [HS10; Spe07]. It is a deliberately simple model in which an observer has only restricted knowledge of an underlying ontic state. Several features often advertised as distinctively quantum – interference-like behaviour, noncommuting measurements, no-cloning and epistemic update – then arise as effects of that restriction. Spekkens' toy theory is not quantum mechanics; it matters here because contemporary quantum discussion already contains a

live epistemological reading, in which some of what looks like ontology can be a restriction on knowledge. The toy theory still falls short of the quantum theory that is actually used, however, and Pusey, Barrett and Rudolph then show that, under their stated assumptions, the quantum state cannot be treated as merely epistemic if one also keeps a definite underlying ontic state [PBR12]. That combination is therefore unavailable: a fully definite hidden world in which present-day quantum mechanics is only ignorance does not work.

Two directions remain open, and Spekkens stays in place as a method in both. The pragmatic stance of this monograph is the first: leave the definite ontic premise unassumed, so that the theorem does not get started, and keep the epistemological habit of treating some structure as restricted knowledge. The second is that a later model of finite observations might remediate the blockage, even for a reader who wants an ontic account. Pusey, Barrett and Rudolph assume the predictions of current quantum mechanics; a successor theory need not take today's quantum state as its ontic state, and relative to a deeper ontic structure the present state could again be epistemic. Neither direction is carried out here; both are reasons not to freeze today's quantum state as ontology.

The natural conclusion is therefore agnostic and constructive. Theories should be treated as epistemological vehicles for finite observations. There is no need, for the purposes of this book, to decide what the ultimate ontology of physics is. Measurements are finite; formal reasoning about them should therefore begin constructively. Spekkens shows why that epistemological reading is not alien to quantum physics as it is now discussed; Gisin shows why the finite-information point should not be implemented by an infinite hidden sequence.

B.3 Excluded middle is not derivable

The claim used in the main text is metatheoretic: global excluded middle is not derivable in the bare constructive fragment. If one wants to use it, one must add it as an additional classical principle.

Proposition B.3.1 (Global excluded middle is not derivable). *In the Martin-Löf-style fragment used in this monograph, the uniform principle, read over small propositions or proposition-like types in the universe,*

$$\Pi_{A:\mathcal{U}}(A + \neg A)$$

is not derivable without adding a classical axiom or restricting A to a decidable class of types.

Proof. The argument is by soundness with respect to standard Kripke/presheaf semantics for dependent type theory [AR11; Gra23]. The model used here validates the underlying non-modal fragment needed in this argument: contexts and substitution, dependent function types, finite product and sum types, the empty type, negation as function into the empty type and a small universe containing the proposition-like type used below. It is not a Boolean model, and it does not validate uniform excluded middle.

The countermodel is used only for the propositional fragment of the argument, so the intensional/extensional distinction for identity types does not affect the failure of uniform excluded middle.

B. Proofs and arguments

Consider the two-stage Kripke frame

$$w_0 \leq w_1,$$

and let P be an atomic proposition which is not forced at w_0 but is forced at w_1 . Forcing is persistent along extensions: if a proposition is forced at an earlier stage, it remains forced at later stages. Thus information may grow from w_0 to w_1 , so it is coherent for P not to be forced at w_0 while being forced at w_1 .

At w_0 , P is not forced by construction. Moreover, $\neg P$ is not forced at w_0 , because forcing $\neg P$ would mean that no extension of w_0 could force P ; but w_1 is an extension of w_0 where P is forced. Hence neither disjunct of

$$P + \neg P$$

is forced at w_0 . Thus excluded middle is not valid in this sound model of the constructive propositional fragment.

If the displayed uniform excluded-middle term were derivable in bare Martin-Löf-style type theory for a universe containing this small propositional type P , its interpretation would give, at every stage and for every such proposition, either a witness of the proposition or a witness of its negation. Instantiating it with P at w_0 would force $P + \neg P$, contradicting the model above. Therefore no such term is derivable in the base theory. *Formal status.* This is a metatheoretic countermodel argument supported by the cited Kripke/presheaf soundness results; it is deliberately not presented as a new Lean theorem inside the object theory. ■

The same warning applies to double-negation elimination as a global principle. Constructively one always has

$$A \rightarrow \neg\neg A,$$

witnessed directly by $\lambda a.\lambda n.n(a)$. The converse

$$\neg\neg A \rightarrow A$$

is a classical principle and is not available uniformly in the base theory.

This does not conflict with the finite probability constructions used in this book. For particular finite types, equality, membership and Boolean predicates are decidable by construction. Thus one may use $E + \neg E$ for a finite decidable event E , and one may take complements inside finite event algebras. What is rejected is the global move from an arbitrary proposition A to $A + \neg A$ without a decision procedure or an explicit classical assumption.

B.4 Constructive type-theoretic facts

The three facts recorded here are used by the type-theoretic definitions of Section A.1. They are elementary consequences of the empty type, negation, decidability and finiteness.

Proposition B.4.1 (Constructive negation and double negation). *Let A be a type, and write $\neg A := A \rightarrow \perp$ as in Definition A.1.30. Then both*

$$\neg(A \wedge \neg A) \quad \text{and} \quad A \rightarrow \neg\neg A$$

are inhabited.

Proof. A term $n : \neg A$ may be used as a function $n : A \rightarrow \perp$. Given also a term $a : A$, applying n to a yields $n(a) : \perp$. Consequently A and $\neg A$ cannot both be inhabited in a consistent theory: a pair $(a, n) : A \times \neg A$ would produce an inhabitant of the empty type. Equivalently, the theory proves $\neg(A \wedge \neg A)$.

The same application gives $A \rightarrow \neg\neg A$. The function-introduction rule of Definition A.1.9 is

$$\frac{\Gamma, p : P \vdash q : Q}{\Gamma \vdash \lambda p. q : P \rightarrow Q}.$$

The assumption $p : P$ is thereby removed from the surrounding context and retained as a bound function input; this does not assert that P is inhabited.

Work temporarily under the assumptions $a : A$ and $n : A \rightarrow \perp$. Function application gives the body $n(a) : \perp$. First bind n as the input of a function:

$$a : A \vdash \lambda n. n(a) : (A \rightarrow \perp) \rightarrow \perp.$$

By the definition of negation, $(A \rightarrow \perp) \rightarrow \perp \equiv \neg\neg A$, so

$$a : A \vdash \lambda n. n(a) : \neg\neg A.$$

Binding a as the input of an outer function then gives

$$\lambda a. \lambda n. n(a) : A \rightarrow \neg\neg A.$$

Constructing the function does not claim that either input has actually been supplied. ■

Proposition B.4.2 (Decidable double-negation elimination). *If A is decidable in the sense of Definition A.1.32, then $\neg\neg A \rightarrow A$ is inhabited. The construction depends on a supplied decision procedure $d : A + \neg A$ and does not provide double-negation elimination for arbitrary types.*

Proof. By Proposition B.4.1, the map $A \rightarrow \neg\neg A$ is available for every type. For the converse, fix a decision procedure $d : A + \neg A$ and an input

$$q : \neg\neg A \equiv (A \rightarrow \perp) \rightarrow \perp.$$

Case analysis on d produces a term of A :

1. if d contains a witness $a : A$, return a ;
2. if d contains a refutation $n : \neg A$, then $q(n) : \perp$, and empty-type elimination produces a term of A .

Write $r_d(q) : A$ for the term so obtained. Binding q then gives

$$\lambda q. r_d(q) : \neg\neg A \rightarrow A.$$

The construction does not assert that $\neg\neg A$ is already inhabited; it describes how any supplied term $q : \neg\neg A$ is converted into a term of A .

Uniform double-negation elimination would require such a decision for every proposition. That principle is equivalent to the law of excluded middle, and neither is assumed; see Proposition B.3.1. ■

B. Proofs and arguments

Proposition B.4.3 (Finite types have decidable equality). *Every finite type A in the sense of Definition A.1.19 has decidable equality in the sense of Definition A.1.37.*

Proof. Let $a, b : A$, and let n , `encode` and `decode` be the data of a finiteness witness for A . Encode both terms:

$$i := \text{encode}(a), \quad j := \text{encode}(b).$$

Decidable equality on $\text{Fin}(n)$ compares i and j , because the constructors of $\text{Fin}(n)$ can be compared recursively.

1. If i and j are identical, decode their common index. The first round-trip proof η says that decoding i returns a and decoding j returns b . Because i and j are identical, these decoded terms are identical, and hence a and b are identical.
2. If i and j differ, then a and b must differ as well. Otherwise, because `encode` is a function, identical inputs a and b would have identical outputs i and j , contradicting their comparison.

The procedure therefore returns either an identity between a and b or a refutation of such an identity.

The equality procedure uses only the first round-trip proof η . The second round-trip proof ε remains part of the finiteness witness because it ensures that every index represents an inhabitant of A . ■

B.5 Lean verification and foundational boundary

This section fixes how Lean citations in the proofs are to be read: what Lean checks internally, what it checks conditionally from explicit inputs, and what remains a cited mathematical or metatheoretic result. The excluded-middle argument of Section B.3 is metatheoretic and is not formalised as a Lean theorem.

Scope and proof status

Lean is used here as a proof checker for the finite constructions: probability records and urns, structural causal models, source–target correspondence, modal record transformations, finite derivation certificates, counterfactual execution and the tenure-track calculations. A declaration accepted by Lean has been checked from its definitions, imported declarations and explicit parameters. This does not by itself show that every parameter has been constructed in the development. In particular, structures packaging published completeness, soundness, latent-projection or counterfactual-reduction results are deliberately supplied as theorem parameters where the monograph transports rather than reproves those results.

The status expressions in Table B.2 have the following meanings.

B.5. Lean verification and foundational boundary

Status	What is established	What is not thereby established
Lean-checked declaration	Lean has accepted the stated type from the cited definitions, prior declarations and displayed parameters.	That every parameter was independently derived in this project.
Checked conditional transport	Lean proves that an explicitly packaged source theorem yields the stated target certificate, equivalence or counterexample.	That the package is inhabited merely because the original paper is accepted; its specialisation to the exact finite source interface remains explicit input.
Executable checked example	A closed finite instance is built from the theorem-facing probability and causal library, and Lean verifies the displayed computation or equality.	A theorem about arbitrary models beyond the quantified declaration being instantiated.
Mathematical or metatheoretic argument	The prose proof is supported by the displayed reasoning and cited mathematical results.	A new internal Lean formalisation, unless a separate checked declaration is also named.

Table B.2: Proof-status conventions for the Lean-supported arguments.

External interfaces and Lean module map

The transport development isolates the mathematical results imported from the causal-identification literature as ordinary structure parameters. These interfaces are obligations at the boundary: Lean checks every use of an inhabitant, but the codebase does not manufacture the inhabitant from a citation. Their mathematical content is specified in Definitions A.8.24, A.8.26 to A.8.28 and A.8.31; Table B.3 records the corresponding Lean packages and their checked uses.

Interface	Externally supplied content	Lean-checked use
DSeparationCorrectness	Equivalence between the path specification and executable ancestral moralisation.	Compiles path side conditions into checked Boolean tests.
PublishedFiniteSource Soundness	Finite source-native primitive probability laws, global-Markov consequences and do-rule semantics on explicit support.	Induction over source derivations and transport to target semantics.
PublishedFiniteSource Completeness	Joint and conditional source certificates for identifiable kernels, plus finite source countermodels for represented hedges.	Theorem B.11.4 and Corollary B.11.10.
PublishedFinite CounterfactualCompleteness	A supported source-native ID*/IDC*-style multiworld reduction and ordinary observationalisation.	Theorem B.11.8 and Corollary B.11.9.
PublishedLatentProjection	Preservation of the relevant source identifiability statement under the chosen hidden-DAG latent projection.	Relates a selected externally quantified hidden-DAG model family, whose members have internally executable semantics, to the projected graph class.

Table B.3: External theorem interfaces used by the checked transports.

B. Proofs and arguments

The theorem-facing modules are grouped by responsibility:

Thesis/Probability/ finite rational values, records, urn reconstruction, dependent finite products, constructive permutation reconstruction, reindexing, conditioning and Bayesian update;

Thesis/Causality/ intrinsic dependent SCMs, graph and query semantics, modalities, structural edits, learning, constructive unused-variable deletion, twin and occurrence-indexed counterfactual execution; in particular, **Causality/Forgetting/Basic.lean** contains executable observed-sink and unused-exogenous deletion and their preservation results, **Causality/Forgetting/Identification.lean** contains the two-way observed identifiability transport, **Causality/Forgetting.lean** re-exports both layers, and **Causality/ModeTheory.lean** names the one-shots and edit paths as the 1-category of modes;

Thesis/CausalTransport/ independent finite-table source semantics, source-target correspondence, derivation support, certificate transport, hidden-DAG model semantics and external-interface adapters;

Thesis/Examples/ the executable tenure-track model and its observational, interventional and counterfactual calculations;

Thesis/AxiomAudit.lean the enforced kernel-axiom policy for every declaration owned by an imported **Thesis.*** module.

A source reference therefore has two parts. A path such as **lean/Thesis/CausalTransport/Correspondence.lean** identifies the module, while a name such as **finiteSource_completeness_transport** identifies the checked declaration. Phrases such as “external package”, “explicit parameter” or “published specialisation” identify premises of a checked implication, not conclusions independently proved by Lean. Declaration names are used instead of source line numbers because they remain stable when surrounding documentation changes.

Intensional kernel and constructive audit

Lean’s judgemental equality is intensional. If Lean has a proof $h : a = b$, the kernel does not thereby treat **a** and **b** as definitionally equal, and Lean does not validate equality reflection. In this respect, it shares the central intensional distinction stated in Definitions A.1.5 and A.1.13. Lean is nevertheless not raw Martin-Löf-style type theory: its standard theory has a proof-irrelevant and impredicative proposition sort, quotient types and extensional principles used by ordinary library infrastructure.

The theorem-facing source uses function extensionality when replacing pointwise equality by equality of functions. In Lean’s standard environment, the axiom audit for **funext** reduces to **Quot.sound**; it does not introduce a third reported axiom. The build-time audit in **lean/Thesis/AxiomAudit.lean** enumerates every declaration owned by an imported **Thesis.*** module, computes each declaration’s transitive kernel-axiom dependencies, and rejects the build if any dependency lies outside

{propext, Quot.sound}.

Continuous integration explicitly builds the `Thesis.AxiomAudit` target. Thus the allowlist is enforced rather than merely reported: a dependency on `Classical.choice`, `sorryAx`, `Lean.ofReduceBool`, or any additional assumed axiom fails elaboration of the audit module. Finite case distinctions use explicit decidability data or structurally recursive Boolean procedures. The claim is scoped to every declaration in the imported project library modules and its transitive dependencies; it is not a claim that every declaration in Lean’s imported libraries is constructive.

Consequently, “constructive” here means that the checked declarations do not obtain witnesses by excluded middle or classical choice. It does not mean “accepted by Lean with an empty axiom report”: the two allowlisted extensional principles can still occur transitively.

Relation to Martin-Löf-style type theory

The formal development is therefore described as *constructive-extensional Lean*. The adjective “constructive” records the absence of excluded middle and classical choice from the audited results. The adjective “extensional” records the use of propositional and functional extensional reasoning and quotient soundness. It does not mean that Lean has extensional judgemental equality or equality reflection.

The target records are intrinsic dependent Lean structures, not an encoding of the raw syntax and derivation judgements of Martin-Löf-style type theory. Their finite types, dependent fields, inductive certificates and proof-carrying transformations follow the intended constructive discipline, but the present development does not prove a syntactic interpretation theorem from Lean into that raw theory. The extensional uses identified by the audit have standard intensional reformulations: equality of functions can be retained pointwise, predicates and finite probability values can be compared by explicit equivalence relations, and quotient representatives can be carried together with setoid-respect proofs. These observations provide a route to a stricter intensional reconstruction, but that reconstruction has not been mechanised here.

The resulting claim is deliberately limited. Lean checks the finite constructions and the stated transports under their explicit inputs within its constructive-extensional foundation. The book does not claim that the files themselves constitute a literal formalisation in raw intensional Martin-Löf-style syntax. Conversely, the presence of `propext` or `Quot.sound` must not be misreported as a hidden use of excluded middle: the audit distinguishes those principles explicitly.

The finite joint-assignment construction gives a concrete example of this translation boundary. Lean represents an assignment by the dependent function type `FiniteProduct.Assignment`; its enumeration and equality proofs use function extensionality only when pointwise equal coordinates are assembled into an equality of functions. An intensional reconstruction may instead represent the same finite assignment by the corresponding finite telescope, built recursively from dependent pairs. The recursive enumeration, encoding, decoding and coordinate computations are unchanged, while final equalities of dependent functions are retained as pointwise identities or as an explicit setoid relation. Thus the shorter Lean representation has a direct intensional recipe even though that alternative syntax is not reproduced in the files.

B. Proofs and arguments

With the proof-status conventions and foundational boundaries fixed, the Lean-supported proofs and transports that follow can be read in the order summarised in Table B.4.

Proof or transport	Establishes	Main inputs	Feeds into
Clayton–Waddington	Finite probability laws from the urn assumptions.	Sections A.3 and B.4	Sections B.7 to B.11
Causal-model correspondence	Agreement of source and target semantics.	Sections A.4 and B.6	Sections B.8 to B.11
SCM construction and projection	Construction from finite tables and latent projection.	Sections A.8 and B.7	Sections B.9 to B.11
Modal execution	Structural edits realise epistemic transitions.	Sections A.7 and B.8	Sections B.10 to B.11
Counterfactual execution	Finite abduction–action–prediction and multiworld semantics.	Sections A.6 and B.9	Section B.11
Identification transport	Published results yield target certificates under explicit interfaces.	Sections A.8 and B.10	Final causal claims

Table B.4: Dependency map for the Lean-supported proofs and transports.

B.6 Clayton–Waddington probability proof

This proof recovers the ordinary finite probability calculus from urn-generated probability types. Complements appear only inside finite decidable event algebras, and no global law of excluded middle is used. The corresponding Lean 4 formalisation is in the module `Thesis.Probability`, in `lean/Thesis/Probability.lean`.

Finite events, urn rescaling and records

Proposition B.6.1 (Finite-type closure and joint assignments). *Finite types are closed under finite sums, dependent sums and ordinary products. Consequently, if I is finite and $A : I \rightarrow \mathcal{U}$ assigns a finite type $A(i)$ to every $i : I$, the finite telescope obtained by listing one coordinate of type $A(i)$ for every $i : I$ is finite. With pointwise or function-extensional equality, its familiar dependent-function presentation*

$$\prod_{i:I} A(i)$$

is finite as well.

Proof. Finite types stay finite when they are placed side by side or paired. A binary sum is one finite list followed by another. An ordinary product is a pair of finite indices. A dependent sum concatenates the fibres and keeps the fibre index with its local position. Each of these encodings has an inverse by finite case analysis, so the round-trip proofs of Definition A.1.19 are available.

If I is listed as $X_0, \dots, X_{n-1}$, a joint assignment is the ordinary product of the fibres $A(X_k)$, built one coordinate at a time from the one-element empty

assignment. That telescope is finite by induction on n . The shorter dependent-function presentation needs one extra step: pointwise equal coordinates become equal functions. The telescope presentation avoids that step.

Lean checks the function presentation as `FiniteWitness` and `FiniteProduct.finiteWitness`. Both declarations are in `lean/Thesis/Probability/Construction.lean`. ■

Proposition B.6.2 (Finite rectangular-event characterisation). *Let $U = \text{Fin}(n)$, let $\text{Val} : U \rightarrow \mathcal{U}$ be a finite value-type family, let*

$$A : \prod_{\ell:U} (\text{Val}(\ell) \rightarrow \text{Bool})$$

be a family of coordinate events, and let $\mathbf{u} : \prod_{\ell:U} \text{Val}(\ell)$ be a complete assignment. The rectangular classifier of Definition A.3.29 satisfies:

$$\text{Id}_{\text{Bool}}(\text{Rect}_A(\mathbf{u}), \text{true}) \iff \prod_{\ell:U} \text{Id}_{\text{Bool}}(A(\ell)(\mathbf{u}(\ell)), \text{true}).$$

Proof. A rectangular event holds exactly when every coordinate event holds. Induct on n . The empty family is true, and there is nothing to check. At a successor step the classifier is the Boolean conjunction of the last coordinate with the preceding rectangle. Conjunction is true exactly when both parts are true, and the induction hypothesis reads the second part as the remaining coordinates.

Lean checks this induction as `FiniteProduct.rectangularEvent_eq_true_iff` in `lean/Thesis/Probability/Construction.lean`. ■

Proposition B.6.3 (Finite-sum reindexing). *Let $n : \mathbb{N}$. Let M be a type with an equivalence relation $\approx$, an addition operation and an additive identity 0, as in Definition A.3.9. Let*

$$u : \text{Fin}(n) \rightarrow M$$

and let

$$\sigma : \text{Fin}(n) \rightarrow \text{Fin}(n)$$

be a permutation, meaning that σ is equipped with an inverse and the corresponding round-trip identity proofs. Then

$$\sum_{i:\text{Fin}(n)} u(\sigma(i)) \approx \sum_{i:\text{Fin}(n)} u(i).$$

Proof. A finite sum does not depend on the order of its terms. From the inverse of σ , Lean reconstructs a list permutation of the canonical enumeration by comparing occurrence counts. It then inducts on that permutation: the empty and cons cases are immediate, an adjacent swap replaces a fragment $\cdots + x + y + \cdots$ by $\cdots + y + x + \cdots$, and transitivity assembles a finite sequence of such steps. Associativity isolates the two summands, and commutativity gives $x + y \approx y + x$. Addition respects $\approx$, so each swap preserves the total, and the whole permutation does as well.

Lean reconstructs the list permutation with `ConstructivePermutation.perm_of_count_eq`. Its proof is in `lean/Thesis/Probability/ConstructivePermutation.lean`. Lean checks the generic sum fact as `AdditiveCarrier.finSum_reindex` in `lean/Thesis/Probability/Reindexing.lean`. The later urn count argument uses the separate list-count form `count_reindex` in `lean/Thesis/Probability/Core.lean`. ■

B. Proofs and arguments

Proposition B.6.4 (Finite-product reindexing). *Let $n : \mathbb{N}$. Let M be a type with an equivalence relation $\approx$, a multiplication operation and a multiplicative identity 1, as in Definition A.3.10. Let*

$$u : \text{Fin}(n) \rightarrow M$$

and let $\sigma : \text{Fin}(n) \rightarrow \text{Fin}(n)$ be equipped with an inverse and the two round-trip identity proofs. Then

$$\prod_{i:\text{Fin}(n)} u(\sigma(i)) \approx \prod_{i:\text{Fin}(n)} u(i).$$

Proof. The argument is the multiplicative instance of Proposition B.6.3. Reconstruct the permutation of the canonical finite enumeration and induct over its adjacent swaps. Associativity isolates the two exchanged factors, commutativity swaps them, multiplication respects $\approx$, and the multiplicative identity handles the empty product. ■

Proposition B.6.5 (Lean correspondence for nonnegative rational values). *Under the fieldwise correspondence*

$$(a, b, h) \mapsto \{\text{num} := a, \text{den} := b, \text{den_pos} := h\},$$

Lean's QProb is the presentation type $\mathbb{Q}_{\geq 0}$ of Definition A.3.8, and

$$\text{QProb.Equiv}(q, r) \iff q \sim r.$$

Proof. Lean's QProb is exactly the triple of a numerator, a positive denominator and the proof that the denominator is positive. Two such triples are QProb.Equiv when their cross-multiplications agree, which is the relation $\sim$ of Definition A.3.8. The same cross-multiplication comparison, with $\leq$ in place of $=$, is QProb.LE . Addition, multiplication and division by a positive value respect $\sim$, and positivity of a numerator is independent of the chosen presentation. No quotient type is formed.

Lean checks the equivalence and the operations in `lean/Thesis/Probability/Core.lean`. ■

Proposition B.6.6 (Joint satisfiability of the qualitative urn assumptions). *The four clauses of Definition A.3.34 can be satisfied simultaneously. For a positive N -cell urn, assign an event selecting m_E cells the non-negative rational presentation m_E/N . Take qualitative equivalence to be equality by cross multiplication and strict qualitative comparison to be strict inequality by cross multiplication. For the empty cell type, assign the unique event the value zero.*

Proof. Cross-multiplication equality is an equivalence relation, while strict cross-multiplication comparison is irreflexive, transitive and invariant under replacement by equivalent presentations. Equal cell counts give identical fractions. If $k < \ell \leq N$ and $N > 0$, then $k/N < \ell/N$. Finally, replacing every cell by $c > 0$ copies changes k/N to ck/cN , which is cross-multiplication equivalent to the original presentation. The empty-urn convention is needed only because the event-level scale is defined for every natural N ; the ratio, refinement and strict-monotonicity clauses already carry positive-size witnesses.

B.6. Clayton–Waddington probability proof

Classical Cox / Van Horn route	This monograph (finite, constructive)
desiderata: real-valued, common sense, consistency	finite cell types constructed; four qualitative assumptions stated explicitly
associativity equation $\Rightarrow$ product rule first	urn rescaling $U_N(k) \mapsto k/N \Rightarrow$ ratio and sum rule first
negation function $S \Rightarrow$ sum rule second	Cartesian product urns $\Rightarrow$ product rule second
<i>universality</i> (R4) density assumed/derived	<i>universality eliminated</i> : values are k/N by construction
logical equivalence and structural symmetries stated separately	pointwise and relabelling invariance derived from finite counts
continuity on $[0, 1]$	no continuum; complements only on decidable events
construct rescaling p	record–urn equivalence $\rightarrow$ Kolmogorov laws $\rightarrow$ Bayes

Table B.5: Where this book’s finite derivation departs from the Cox–Jaynes–Van Horn argument refined by Clayton and Waddington [CW17]: only qualitative coherence, count invariance, strict monotonicity and refinement invariance remain primitive; logical equivalence, relabelling and product urns are derived or constructed.

Lean defines the event assignment as `rationalEventPlaus`, packages all four proofs in `rationalScale`, and checks the joint inhabitation statement as `qualitativeUrnScale_nonempty` in `lean/Thesis/Probability/QualitativeUrn.lean`. Strict rational comparison and its equivalence-respect law are `QProb.LT` and `QProb.lt_congr` in `lean/Thesis/Probability/Core.lean`. ■

Proposition B.6.7 (Resolution-status partition). *Let Ω be a finite sample type, let $A : \Omega \rightarrow \mathcal{U}$ and let $R : \text{Resolution}(A)$ be a resolution package of Definition A.3.5. Then the events Prov_A , Ref_A and Unr_A associated with R by Definition A.3.7 are decidable, pairwise disjoint and jointly exhaustive.*

Proof. Each outcome receives exactly one of the three statuses `proved`, `refuted` or `unresolved`. Equality on that three-element type is decidable, so the three events are decidable. If the status is `proved`, the outcome lies in Prov_A and in neither of the other two events. The other two cases are the same. The three events are therefore pairwise disjoint and cover Ω .

Lean checks the partition as `Inspection.status_count_partition` and the probability form as `UrnProb.status_partition`. ■

Proposition B.6.8 (Derived urn-event invariances). *Assume count invariance from Definition A.3.34. Let $N : \mathbb{N}$ and let $E, F : \text{Fin}(N) \rightarrow \text{Bool}$. Then:*

B.6.8.1. *if $E(i) = F(i)$ for every $i : \text{Fin}(N)$, then*

$$\text{plaus}_N(E) \approx_U \text{plaus}_N(F);$$

B.6.8.2. *if $\sigma : \text{Fin}(N) \rightarrow \text{Fin}(N)$ is a permutation, then*

$$\text{plaus}_N(E \circ \sigma) \approx_U \text{plaus}_N(E);$$

B. Proofs and arguments

B.6.8.3. if m_E is the number of cells selected by E , then

$$\text{plaus}_N(E) \approx_U U_N(m_E).$$

Proof. If E and F agree pointwise, their indicators agree pointwise. Summing those equal natural numbers gives $m_E = m_F$, so count invariance gives the first conclusion.

For a permutation σ , apply Proposition B.6.3 to the indicator function $i \mapsto \mathbb{1}_E(i)$. This gives

$$\sum_{i:\text{Fin}(N)} \mathbb{1}_E(\sigma(i)) = \sum_{i:\text{Fin}(N)} \mathbb{1}_E(i).$$

The left side is the count of $E \circ \sigma$, so count invariance gives the second conclusion. Finally, the canonical event C_{N,m_E} selects its first m_E cells and therefore has count m_E . Applying count invariance to E and C_{N,m_E} proves the third conclusion.

Lean checks the pointwise and permutation count steps as `count_congr` and `count_reindex` in `lean/Thesis/Probability/Core.lean`. Their event-level qualitative consequences are `plaus_eqv_of_pointwise`, `plaus_reindex_eqv` and `event_eqv_ratio` in the `QualitativeUrnScale` namespace. The abstract finite-sum reindexing of Proposition B.6.3 is a separate theorem; the urn argument uses the count form directly. ■

Proposition B.6.9 (Qualitative urn rescaling and event comparison). *Assume the four clauses of Definition A.3.34. Let*

$$\begin{aligned} N, M, k, \ell &: \mathbb{N}, \\ h_N &: N > 0, \quad h_M : M > 0, \\ h_k &: k \leq N, \quad h_\ell : \ell \leq M. \end{aligned}$$

The qualitative equivalence and strict order of the represented urn plausibilities are characterised exactly by cross multiplication:

$$\begin{aligned} U_N(k) \approx_U U_M(\ell) &\iff kM = \ell N, \\ U_N(k) \prec_U U_M(\ell) &\iff kM < \ell N. \end{aligned}$$

More generally, let $E, F : \text{Fin}(N) \rightarrow \text{Bool}$ be arbitrary events with selected-cell counts m_E and m_F . Then

$$\begin{aligned} \text{plaus}_N(E) \approx_U \text{plaus}_N(F) &\iff m_E = m_F, \\ \text{plaus}_N(E) \prec_U \text{plaus}_N(F) &\iff m_E < m_F. \end{aligned}$$

Consequently, the assignment

$$p(U_N(k)) := (k, N, h_N) : \mathbb{Q}_{\geq 0}$$

is well-defined on qualitative-equivalence classes and both preserves and reflects the strict cross-multiplication order of the represented ratios.

Proof. Cross multiplication decides when two urn ratios are equal or strictly ordered. First suppose that $kM < \ell N$. Refining the N -cell urn by the positive factor M gives

$$U_N(k) \approx_U U_{MN}(Mk),$$

while refining the M -cell urn by the positive factor N gives

$$U_M(\ell) \approx_U U_{NM}(N\ell).$$

The two refined urns have the same size because $MN = NM$. Their selected-cell counts satisfy

$$Mk = kM < \ell N = N\ell.$$

Strict monotonicity in this common urn, followed by compatibility of $\prec_U$ with $\approx_U$, therefore gives

$$U_N(k) \prec_U U_M(\ell).$$

The equality argument is analogous. If $kM = \ell N$, the same two refinements have equal urn sizes and equal selected-cell counts. Same-count invariance and refinement invariance consequently give

$$U_N(k) \approx_U U_M(\ell).$$

Conversely, suppose $U_N(k) \approx_U U_M(\ell)$. Natural-number comparison is decidable, so exactly one of

$$kM < \ell N, \quad kM = \ell N, \quad \ell N < kM$$

holds. The first and third cases would, by the strict comparison just established, make one of the equivalent plausibilities strictly smaller than the other. Qualitative coherence excludes this. Hence $kM = \ell N$.

Finally, suppose $U_N(k) \prec_U U_M(\ell)$. The equality case is impossible because equal ratios give qualitatively equivalent values, and equivalent values cannot be strictly ordered. The reverse inequality $\ell N < kM$ is also impossible: the first part of the proof would give $U_M(\ell) \prec_U U_N(k)$, contradicting asymmetry of the strict order. Therefore $kM < \ell N$. This proves both displayed equivalences.

Every event is qualitatively equivalent to its canonical same-count event by Proposition B.6.8. Applying the two ratio characterisations with the common positive denominator N and cancelling that denominator proves the displayed arbitrary-event equivalences.

The first ratio equivalence says precisely that replacing one representative of a qualitative-equivalence class by another replaces (k, N, h_N) by a $\sim$ -equivalent rational presentation. The second equivalence proves both preservation and reflection of the strict cross-multiplication order.

Lean checks the two-way comparison by the `eqv_iff_sameRatio` and `lt_iff_ratio_lt` declarations in the `QualitativeUrnScale` namespace of `lean/Thesis/Probability/QualitativeUrn.lean`; the arbitrary-event forms are `event_eqv_iff_count_eq` and `event_lt_iff_count_lt`. ■

Corollary B.6.10 (Urn-event complement law). *Let X be any type, let*

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X),$$

and let a decidable event $E : X \rightarrow \text{Bool}$ select k cells of μ . Its complement selects $N - k$ cells and

$$P_{\mu}(E) + P_{\mu}(E^{\complement}) \sim 1.$$

B. Proofs and arguments

Proof. The two selected-cell counts add to $k + (N - k) = N$. The fraction arithmetic of Definition A.3.8 therefore gives

$$P_\mu(E) + P_\mu(E^c) \sim (N, N, h_N) \sim 1.$$

Lean checks this event-level calculation as `UrnProb.complement_rule` in `lean/Thesis/Probability/Urn.lean`. The scope remains decidable finite urn events, not arbitrary propositions. ■

Proposition B.6.11 (Product urns after rescaling). *Let X, Y be types and let*

$$\mu = (M, h_M, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X), \quad \nu = (N, h_N, g) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(Y).$$

Let $k, \ell : \mathbb{N}$ with $h_k : k \leq M$ and $h_\ell : \ell \leq N$. If $E : X \rightarrow \text{Bool}$ selects k cells of μ and $F : Y \rightarrow \text{Bool}$ selects ℓ cells of ν , their independently constructed rectangular product satisfies

$$P_{\mu \otimes \nu}(E \times F) \sim P_\mu(E)P_\nu(F).$$

Proof. Construct the product cell type $\text{Fin}(M) \times \text{Fin}(N)$. Its row-major encoding is

$$\text{encode}(i, j) := iN + j : \text{Fin}(MN).$$

Conversely, for $q : \text{Fin}(MN)$, define

$$\text{decode}(q) := (\lfloor q/N \rfloor, q \bmod N) : \text{Fin}(M) \times \text{Fin}(N).$$

The bound $q < MN$ and positivity of N ensure that $\lfloor q/N \rfloor < M$ and $q \bmod N < N$. The quotient-remainder identities give

$$\text{decode}(\text{encode}(i, j)) = (i, j) \quad \text{and} \quad \text{encode}(\text{decode}(q)) = q.$$

These maps therefore exhibit the required equivalence

$$\text{Fin}(M) \times \text{Fin}(N) \simeq \text{Fin}(MN).$$

The labelled events induce cell events

$$E_f(i) := E(f(i)), \quad F_g(j) := F(g(j)).$$

Their rectangular cell event is the Boolean classifier

$$(E_f \times F_g)(i, j) := \text{and}_{\text{Bool}}(E_f(i), F_g(j)).$$

Enumerating the product cells row by row, every cell selected by E_f is paired with each of the ℓ cells selected by F_g . Since E_f selects k cells, the rectangular event consequently selects

$$\sum_{i : \text{Fin}(M)} \mathbf{1}_{E_f}(i) \ell = k \ell$$

cells. Transport through the displayed equivalence preserves this count and gives a Boolean event on $\text{Fin}(MN)$ selecting the same $k\ell$ cells. The proofs

h_M and h_N determine a proof $h_{MN} : MN > 0$. Therefore the rescaling of Proposition B.6.9 gives

$$P_{\mu \otimes \nu}(E \times F) \sim (k\ell, MN, h_{MN}) \sim P_\mu(E)P_\nu(F).$$

The second equivalence follows directly by cross multiplication. This constructs the product of the two given urn assignments; it does not assert that an arbitrary joint assignment on the product type is independent.

Lean checks the cell count as `selectedCells_product` in the `FiniteCellProduct` namespace and the probability equation as `UrnProb.product_probVal`. This constructs the product of two given urns; it does not say that every joint assignment is independent. ■

Proposition B.6.12 (Finite rational records admit a common denominator). *Let Ω be a finite sample type and let*

$$\mu = (p, h) : \text{Prob}_{\mathbb{Q}}(\Omega)$$

be a finite rational probability record in the sense of Definition A.3.12. Then μ has an extensionally equal presentation in the common-denominator form of Definition A.3.13: there are a natural number D , a proof $h_D : D > 0$, and natural numbers m_ω indexed by $\omega : \Omega$ such that

$$p(\omega) \sim (m_\omega, D, h_D) \quad \text{for every } \omega : \Omega,$$

and

$$\sum_{\omega : \Omega} m_\omega = D.$$

Proof. Enumerate Ω , multiply the finitely many positive denominators, and scale each numerator by the other denominator factors. Each original presentation is then equivalent by cross multiplication to the resulting common-denominator presentation. Cross-multiplying the original normalisation relation against that common denominator gives $\sum_{\omega : \Omega} m_\omega = D$.

Lean constructs the record as `toRecord`, proves its total-mass equation as `common_total_mass`, and checks eventwise preservation as `toRecord_preserves`. The declarations are in the `CommonDenominator.FiniteQMass` namespace of the probability construction module. ■

Proposition B.6.13 (Common-denominator record–urn equivalence). *Let X be any type. For weighted finite-support records in the common-denominator form of Definition A.3.13, urn presentations and record presentations are extensionally equivalent. More precisely:*

B.6.13.1. *every urn-generated assignment $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$ determines a weighted record R_μ such that*

$$P_{R_\mu}(E) \sim P_\mu(E)$$

for every decidable event $E : X \rightarrow \text{Bool}$;

B. Proofs and arguments

B.6.13.2. *every weighted common-denominator record R determines an urn-generated assignment μ_R such that*

$$P_R(E) \sim P_{\mu_R}(E)$$

for every decidable event $E : X \rightarrow \text{Bool}$.

Consequently, after choosing a common denominator, finite rational records on finite sample types may be treated as urn presentations without changing any event probabilities. The two presentations are displayed in Figure 5.1.

Proof. An urn becomes a record by giving each labelled cell unit weight. A record becomes an urn by repeating each label according to its natural weight. Both constructions count the same selected cells, so event probabilities agree even though the presentations are not the same object.

Lean checks the two directions as `urn_to_record_extensional` and `record_to_urn_extensional` in the `FiniteProbRecord` namespace. ■

Corollary B.6.14 (Finite rational record laws). *Every common-denominator finite rational probability record satisfies the finite Kolmogorov laws. In particular, event probabilities lie between zero and one, the full event has probability one, and an arbitrary finite pairwise-disjoint family has probability equal to the finite sum of its members' probabilities. If two events E and F have positive probability, positive conditioning defines posterior records and satisfies Bayes' theorem:*

$$P(E \mid F) \sim \frac{P(F \mid E)P(E)}{P(F)}.$$

Proof. The common-denominator record is extensionally equivalent to its unit-cell urn by Proposition B.6.13, so the urn laws transport eventwise. The record implementation also checks them directly: bounds and normalisation are `FiniteProbRecord.probVal_bounds` and `FiniteProbRecord.normalization`; arbitrary finite additivity is `FiniteProbRecord.finite_additivity_family`; posterior agreement is `FiniteProbRecord.conditionOn_probVal`; and Bayes' theorem is `FiniteProbRecord.bayes_formula`. ■

Proposition B.6.15 (Pushforward of an urn assignment). *Let $\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$ and let $g : X \rightarrow Y$. Relabelling every cell by g gives the pushforward urn*

$$g_*\mu := (N, h_N, g \circ f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(Y).$$

For every decidable event $E : Y \rightarrow \text{Bool}$,

$$P_{g_*\mu}(E) \sim P_{\mu}(E \circ g).$$

Proof. Both sides inspect the same N cells. A cell i is selected on the left exactly when $E(g(f(i)))$ is true, which is the condition used on the right. The selected-cell counts and denominators are therefore identical.

Lean checks the event equation as `UrnProb.probVal_map`. ■

Finite probability laws

The finite Kolmogorov laws are stated in Theorem 5.3.1. The argument here is the cell-count calculation.

Proof of Theorem 5.3.1. Event probability is a natural cell count divided by the positive urn size, so it is nonnegative; the count is at most the number of cells, so the value is at most 1. The total event counts every cell, hence has probability 1. If E and F are disjoint, their indicators add pointwise, and so do their counts. The finite-family statement is the same argument by induction. No countable additivity is claimed.

Lean checks non-negativity and the unit-interval bounds as `UrnProb.probVal_nonneg` and `UrnProb.probVal_le_one`, normalisation as `UrnProb.normalization`, two-event additivity as `UrnProb.finite_additivity`, and arbitrary finite pairwise-disjoint additivity as `UrnProb.finite_additivity_family`. ■

Corollary B.6.16 (Probability monotonicity). *Let $\mu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $E, F : X \rightarrow \text{Bool}$ be decidable events with $E \subseteq F$, that is, $E(x) = \text{true}$ implies $F(x) = \text{true}$ for every $x : X$. Then*

$$P_{\mu}(E) \leq P_{\mu}(F).$$

Proof. If E is contained in F , every selected cell of E is a selected cell of F . The finite counts are therefore ordered, and division by the same positive denominator preserves that order.

Lean checks the rational inequality as `UrnProb.monotonicity_probVal`, using `QProb.LE`. ■

Corollary B.6.17 (Two-event inclusion–exclusion). *Let $\mu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $E, F : X \rightarrow \text{Bool}$ be decidable events. Then*

$$P_{\mu}(E \cup F) + P_{\mu}(E \cap F) \sim P_{\mu}(E) + P_{\mu}(F).$$

Proof. For every urn cell the Boolean indicators satisfy $\mathbb{1}_{E \cup F} + \mathbb{1}_{E \cap F} = \mathbb{1}_E + \mathbb{1}_F$. Summing that pointwise identity gives the displayed equation.

Lean checks this as `UrnProb.inclusion_exclusion` in `lean/Thesis/Probability/Urn.lean`. ■

Corollary B.6.18 (Resolution-status probabilities). *Let Ω be a finite sample type, let $A : \Omega \rightarrow \mathcal{U}$, let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega)$ and let $R : \text{Resolution}(A)$. Then*

$$P_{\mu}(\text{Prov}_A) + P_{\mu}(\text{Ref}_A) + P_{\mu}(\text{Unr}_A) \sim 1.$$

Proof. By Proposition B.6.7, the three resolution-status events are pairwise disjoint and jointly exhaustive. Finite additivity from Theorem 5.3.1 gives

$$P_{\mu}(\text{Prov}_A \cup \text{Ref}_A \cup \text{Unr}_A) \sim P_{\mu}(\text{Prov}_A) + P_{\mu}(\text{Ref}_A) + P_{\mu}(\text{Unr}_A).$$

The union is the full event All_{Ω} of Definition A.3.3, whose probability is 1 by normalisation.

Lean checks this as `UrnProb.status_partition`. ■

Conditioning and Bayesian update

Bayes' theorem for decidable finite events is stated in Theorem 5.3.2. The argument here is the intersection-count calculation.

Proof of Theorem 5.3.2. Expand both conditionals as intersection counts. Intersection is commutative, so the two numerators agree. The displayed positive denominators may then be cancelled.

Lean checks the quotient form as `UrnProb.bayes_formula`. ■

Proposition B.6.19 (Product and conditional characterisations of independence). *Let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$ and let $A, B, C : X \rightarrow \text{Bool}$ be decidable events. Write m_A, m_B, m_C and $m_{B \cap C}$ for the selected-cell counts of A, B, C and $B \cap C$, respectively. Whenever the displayed positivity witnesses are supplied, the following equivalences hold:*

B.6.19.1. *if $h_B : m_B > 0$, then*

$$P_{\mu}(A \cap B) \sim P_{\mu}(A)P_{\mu}(B) \iff P_{\mu}(A \mid B) \sim P_{\mu}(A);$$

B.6.19.2. *if $h_A : m_A > 0$, then*

$$P_{\mu}(A \cap B) \sim P_{\mu}(A)P_{\mu}(B) \iff P_{\mu}(B \mid A) \sim P_{\mu}(B);$$

B.6.19.3. *if $h_C : m_C > 0$, then*

$$\begin{aligned} P_{\mu}(A \cap B \cap C)P_{\mu}(C) &\sim P_{\mu}(A \cap C)P_{\mu}(B \cap C) \\ \iff P_{\mu}(A \cap B \mid C) &\sim P_{\mu}(A \mid C)P_{\mu}(B \mid C); \end{aligned}$$

B.6.19.4. *if $h_C : m_C > 0$ and $h_{B \cap C} : m_{B \cap C} > 0$, then*

$$\begin{aligned} P_{\mu}(A \cap B \cap C)P_{\mu}(C) &\sim P_{\mu}(A \cap C)P_{\mu}(B \cap C) \\ \iff P_{\mu}(A \mid B \cap C) &\sim P_{\mu}(A \mid C). \end{aligned}$$

Proof. Expand every probability into its selected-cell count and positive urn denominator. In the first two cases, the product equation and the conditional equation both reduce, after cancelling the positive urn size or conditioning count, to

$$m_{A \cap B}N = m_A m_B.$$

In the third and fourth cases, both sides reduce to

$$m_{A \cap B \cap C}m_C = m_{A \cap C}m_{B \cap C}.$$

Boolean associativity identifies the two parenthesisations of $A \cap B \cap C$. All cancellations use the displayed positivity witnesses.

Lean checks the four equivalences in `lean/Thesis/Probability/Urn.lean`, beginning with `UrnProb.independent_iff_condVal_left`. ■

Proposition B.6.20 (Bayesian update is conditional probability). *Let $\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, let $E : X \rightarrow \text{Bool}$ be a decidable event with selected-cell count m_E , and suppose $h_E : m_E > 0$. Let μ_E be the filtered urn of Definition A.3.19. Then, for every decidable event $F : X \rightarrow \text{Bool}$,*

$$P_{\mu_E}(F) \sim P_{\mu}(F \mid E).$$

Proof. The posterior urn keeps exactly the original cells that satisfy E , so its denominator is m_E . Among those cells, F selects the cells that satisfy both E and F . Both sides are therefore the ratio $m_{F \cap E}/m_E$.

Lean checks the eventwise equation as `UrnProb.conditionOn_probVal`. The common-denominator form used later is the declaration with the same final name in the `FiniteProbRecord` namespace. ■

Corollary B.6.21 (Bayesian update is a probability assignment). *Under the assumptions of Proposition B.6.20, the filtered assignment*

$$\mu_E = (m_E, h_E, f_E) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$$

is a nonempty urn-generated probability assignment and is normalised:

$$P_{\mu_E}(\top_X) \sim 1.$$

Proof. The proof $h_E : m_E > 0$ makes the filtered cell type nonempty. Every one of those m_E cells satisfies the total event, so the posterior probability is $m_E/m_E \sim 1$.

Lean checks this as `UrnProb.conditionOn_normalization`. ■

The theorem applies immediately to the resolution-status events Prov_A , Ref_A and Unr_A , provided the conditioning event has positive probability. It does not turn an arbitrary proposition A into a decidable proposition. In particular, there is no general constructive theorem

$$P(\neg A \mid I) = 1 - P(A \mid I).$$

What is available is the finite resolution-status decomposition:

$$P(\text{Prov}_A \mid B) + P(\text{Ref}_A \mid B) + P(\text{Unr}_A \mid B) \sim 1$$

whenever B is a decidable conditioning event of positive probability. Thus the probability of proof and refutation may sum to less than 1; the remaining mass is exactly $P(\text{Unr}_A \mid B)$, the probability assigned to unresolved cases under the chosen finite inspection procedure.

B.7 Finite causal-model correspondence

This section records the finite source–target correspondence checked by Lean. The finite model correspondence is internal to the Lean development; the classical completeness theorem is supplied to the final transport theorem as an explicit parameter.

The source and target fragments used below are defined in Definition A.8.11: the finite table source of Definition A.8.5 and the intrinsic dependent target of Definition A.8.9. Direct-root projection is defined in Definition A.4.23; its Lean constructions are `LatentExtension.projectedBidirected` and `LatentExtension.observedGraph` in `lean/Thesis/Causality/Model.lean`. General hidden-DAG data and their projection are defined in Definitions A.4.32 and A.4.35. Their compact Lean construction is `CompactFiniteHiddenDAG` in `lean/Thesis/Causality/CompactHiddenDAG.lean`; the expanded theorem-facing records `FiniteHiddenDAG` and

B. Proofs and arguments

`FiniteLatentProjection` are in `lean/Thesis/Causality/Reductions.lean`.

The principal declarations for the finite model correspondence are `FiniteTableSignature`, `FiniteTableLatent`, `FiniteTableSCM`, `FiniteTableSCM.interpret`, `ObservedSignature` and `ExactModel`. The independent source interpretation and its pointwise preservation theorems are in `lean/Thesis/CausalTransport/FiniteSource.lean`; the intrinsic target model is in `lean/Thesis/Causality/Model.lean`, with `ExactModel` and hard interventions (Definition A.5.2) in `lean/Thesis/Causality/Core.lean`.

Lemma B.7.1 (Disjoint exogenous independence). *In an exact finite latent-root model, every rectangular event has joint probability equal to the product of its source-wise probabilities. More generally, in the finite dependent-product record constructed from a coordinate-indexed family of factors, the probability of any event on one selected coordinate equals that coordinate’s factor probability; the coordinate value types need not be homogeneous.*

Proof. Independent roots have a product law. A rectangular event on the exogenous assignment is the product of its coordinate events, and the model carries that product law as evidence rather than inferring it from separate normalisations.

Lean checks the model field as `FiniteLatentSCM.product_law`, the generic rectangular law as `FiniteProduct.record_rectangular_probVal`, and the dependent coordinate marginal as `FiniteProduct.record_coordinate_probVal_dependent`. ■

Proposition B.7.2 (Finite-index order, directed irreflexivity and ordered acyclicity). *The strict natural-number order of Remark A.1.18 is transitive and irreflexive. Consequently, the directed classifier of a topologically ordered graph is irreflexive, and a directed acyclic graph of Definition A.4.9 has no positive directed path from a node to itself.*

Proof. The usual order of natural numbers is transitive and has no loops. Transitivity is induction on the second witness, with a case split on the first. Irreflexivity is induction on the number: nothing proves $0 < 0$, and a successor loop would shrink to a smaller loop.

If a directed edge is required to raise rank, it cannot run from a node to itself. A longer directed path would then raise rank at every step, so a path from X back to X would give $\text{rank}_n(X) <_{\mathbb{N}} \text{rank}_n(X)$, which is impossible.

Lean records the rank constraint as `directed_earlier` on an observed signature. On a hidden DAG the same fact is the field `edge_rank_lt`, and `HiddenInternalPath.irreflexive` says that a hidden-internal path cannot begin and end at the same node. ■

Proposition B.7.3 (Compact and expanded hidden-DAG presentations). *Let $\mathcal{C} : \text{CompactFiniteHiddenDAG}(V)$ be as in Definition A.4.32. Finite search computes a hidden classifier H_o such that:*

$$H_o(t) = \text{true} \iff \neg (\Sigma_{i:V} \text{Id}_{T_H}(o(i), t)).$$

The expansion to `FiniteHiddenDAG(C)` preserves the node count, observed embedding, directed classifier and rank, and its computed classifier satisfies h_{obs} and h_{cover} from Definition A.4.32.

Conversely, let $\mathcal{H} : \text{FiniteHiddenDAG}(V)$ satisfy those two fields. Forgetting its explicit classifier and re-expanding the resulting compact record preserves that classifier pointwise. Hence the compact and expanded presentations carry the same finite graph and node classification data.

Proof. Enumerate V and compare each $o(i)$ with t . If one comparison succeeds, the observed-image classifier is true and its Boolean complement H_o is false. In particular, the witness i shows that $H_o(o(i)) = \text{false}$, which proves h_{obs} . If no comparison succeeds, $H_o(t)$ is true; if one succeeds, the successful index supplies $\Sigma_{i:V} \text{Id}_{T_H}(o(i), t)$. These two cases prove h_{cover} . All remaining graph fields are copied directly by the expansion.

For the converse, suppose first that the supplied classifier of $\mathcal{H}$ marks t as hidden. The field h_{obs} prevents t from being an embedded observed node. Conversely, if t lies outside the observed image, the observed alternative in h_{cover} is impossible, so its hidden alternative must hold. Thus the supplied classifier is exactly the Boolean complement computed from the observed image.

Lean checks the construction as `CompactFiniteHiddenDAG.toFiniteHiddenDAG`, the converse operation as `FiniteHiddenDAG.toCompact`, and the pointwise comparison as `FiniteHiddenDAG.toCompact_toFiniteHiddenDAG_hidden` in `lean/Thesis/Causality/CompactHiddenDAG.lean`. ■

Proposition B.7.4 (Canonical selected-node subfamily). *Let $V = \text{Fin}(n)$ be a finite node type in the sense of Definition A.4.2. Every Boolean node classifier $S : V \rightarrow \text{Bool}$ determines a natural number m_S and an embedding $j_S : \text{Fin}(m_S) \rightarrow V$ whose image consists exactly of the nodes classified as true. The canonical construction enumerates this image in ambient index order and therefore reflects strict order:*

$$\text{rank}_n(j_S(k)) <_{\mathbb{N}} \text{rank}_n(j_S(\ell)) \implies \text{rank}_{m_S}(k) <_{\mathbb{N}} \text{rank}_{m_S}(\ell).$$

Consequently, pulling a topologically ordered directed edge classifier back along j_S supplies a topologically ordered classifier on $\text{Fin}(m_S)$.

Proof. A Boolean node classifier determines a finite list of the nodes it selects, written in index order. Use the canonical enumeration $0, \dots, n-1$ of V and filter it by S . Let m_S be the length of the resulting list. Its k th entry defines $j_S(k)$; duplicate-freeness makes this map injective, and the filter law gives

$$(\exists k : \text{Fin}(m_S), j_S(k) = v) \iff S(v) = \text{true}.$$

Filtering does not change the relative order of the retained entries. Hence, if $j_S(k)$ occurs earlier than $j_S(\ell)$ in the ambient enumeration, then k occurs earlier than ℓ in the filtered enumeration. This is precisely the displayed order-reflection map. If a directed edge from $j_S(k)$ to $j_S(\ell)$ carries ambient topological-order evidence, applying this map gives the required inequality between k and ℓ .

Lean checks the length, embedding, injectivity, image and order-reflection facts as `selectedNodeCount`, `selectedEmbed`, `selectedEmbed_inj`, `mem_iff_exists_selectedEmbed` and `selectedEmbed_lt_reflect` in `lean/Thesis/Causality/Modalities.lean`. ■

B. Proofs and arguments

Remark B.7.5 (Equivalent presentations of a finite subfamily). Every embedding $j : \text{Fin}(m) \rightarrow \text{Fin}(n)$ determines a Boolean classifier selecting exactly its image. Two embeddings with the same image have equivalent index types and hence equivalent dependent joint-assignment types by coordinate transport. If $\text{Fin}(n)$ presents a finite variable-name type I , then $\text{decode} \circ j$ is also an embedding into I .

Proof. Finite search decides whether $j(k) = v$ for some k , because finite indices have decidable equality. If two embeddings have the same image, finite search and injectivity give mutually inverse index maps; precomposition transports dependent assignments coordinate by coordinate. Finally, if $\text{decode}(j(k)) = \text{decode}(j(\ell))$, applying encode and the round-trip laws of Definition A.1.19 gives $j(k) = j(\ell)$, and injectivity gives $k = \ell$.

This is an elementary finite reindexing argument. The canonical classifier-to-embedding direction is the part checked by the Lean declarations in Proposition B.7.4. ■

Remark B.7.6 (Scope of value-carrier reindexing). A finiteness witness identifies each explicitly finite value carrier with some finite-index type. That fact alone is not the full causal correspondence: a general carrier-level extension would also transport every dependent parent tuple, mechanism, intervention, assignment, event, query and compatible-model quantifier through those equivalences. The present Lean correspondence starts after this coding choice, with `FiniteTableSignature` values of the form $\text{Fin}(\text{arity}(i))$. The adequacy theorem is therefore stated for the coded finite source class rather than claiming this additional model-level reindexing layer. There is no constructive or modal obstruction to adding that layer, but its central proof would be a dependent model reindexing theorem, followed by preservation of evaluation, intervention, probability, compatibility and identifiability. It is therefore a substantial extension rather than a routine consequence of finite-type enumeration.

Proposition B.7.7 (Canonical pair-root expansion). *Let $\mathcal{G} : \text{CausalGraph}(V)$ be finite. Introduce one hidden root $U_{i,j}$ for every bidirected edge whose endpoints satisfy $\text{rank}_n(i) <_{\mathbb{N}} \text{rank}_n(j)$, and give that root precisely the two outgoing edges:*

$$U_{i,j} \rightarrow i, \quad U_{i,j} \rightarrow j.$$

Retain the directed part of $\mathcal{G}$. The resulting graph is a finite all-directed acyclic graph, and hiding the introduced roots recovers exactly the directed and bidirected classifiers of $\mathcal{G}$. The fixed candidate type $V + (V \times V)$ of Definition A.4.24 gives another presentation: every ordered pair whose bidirected test succeeds is active and every remaining candidate is isolated. Hiding these roots also recovers exactly $\mathcal{G}$.

Proof. A bidirected edge is represented by a single hidden root. Enumerate the finite type $V \times V$ and retain exactly those pairs satisfying the decidable rank and bidirected-edge tests displayed in the proposition. Indexing this filtered list by $\text{Fin}(m)$ gives the finite type of introduced roots. Put every such root before every observed node; the retained observed arrows follow the existing topological order, so the expanded graph is acyclic.

Its directed observed classifier is unchanged by construction. For the bidirected classifier, suppose first that $E_{\mathcal{G}}^{\leftrightarrow}(i, j) = \text{true}$. Bidirected irreflexivity

gives $i \neq j$, and comparison in the finite total order gives either $\text{rank}_n(i) <_{\mathbb{N}} \text{rank}_n(j)$ or the reverse inequality. In the first case $U_{i,j}$ is present; in the second, symmetry of the classifier makes $U_{j,i}$ present. Either root has arrows into both i and j , so latent projection selects the pair.

Conversely, a projected bidirected edge must be witnessed by one of the introduced roots. Such a root exists only for an originally selected bidirected pair and is incident only to its two endpoints. The queried pair is therefore that edge, possibly with its endpoints reversed; in the latter case symmetry supplies the required orientation. Hence no bidirected edge is lost or added. This proposition concerns graph structure only and supplies no probability assignment or structural mechanism for the introduced roots.

For the fixed candidate presentation, each ordered pair selected by the bidirected classifier has arrows to its two endpoints. Symmetry means that the reverse ordered pair has the same two children. Hiding either root therefore recreates the same bidirected edge, while hiding an inactive isolated candidate creates no edge. The fixed and compact expansions consequently have the same latent projection. Their node types and path types are not claimed to be identical.

Lean checks the compact graph-only expansion as `pairRootExtension_projected`. Its hidden graph form is checked by `pairRootHiddenDAG_directed` and `pairRootHiddenDAG_bidirected` in `lean/Thesis/Causality/PairRoot.lean`. The executable separation graph retains all ordered candidates through `SeparationNode.latentPair` in `lean/Thesis/Causality/Derivation.lean`. Its fixed-candidate projection argument is the elementary finite graph argument above, rather than a separate Lean declaration. No probability or mechanism data are supplied. ■

The compressed statement is Theorem 5.4.2. Its semantic source–target content is the following proposition.

Proposition B.7.8 (Finite source–target semantic adequacy). *Every finite table SCM of Definition A.8.11 has an intrinsic dependent target interpretation. Source-table evaluation and target evaluation agree at every node, and the interpretation preserves every observational and hard-interventional event value pointwise. On an interpreted finite-table signature, every target exact model can in turn be presented as a finite table SCM.*

Proof. A finite table SCM can be read as an intrinsic dependent model. The two independently defined well-founded evaluators agree node by node, so their complete assignments agree and pushforward of the same latent prior preserves both observational and hard-interventional event values. Conversely, an exact target model on an interpreted table signature can be repackaged using the same finite latent data, mechanisms and probability record.

In the `FiniteTableSCM` namespace of `lean/Thesis/CausalTransport/FiniteSource.lean`, Lean checks the evaluator comparisons as `evalNodeUnder_preserved` and `evalUnder_preserved`, the probability comparisons as `observational_preserved` and `interventional_preserved`, and the reverse presentation as `ofExact`. The two roundtrip agreement results are `interpret_ofExact_agrees` and `ofExact_interpret_agrees`. This is correspondence for the coded finite source class, not a syntax theorem for every classical SCM presentation.

■

Corollary B.7.9 (Finite-source identifiability correspondence). *Let T be a finite table signature and G a finite table graph. Interpretation preserves and reflects compatibility with G and observational equivalence. Consequently, a joint distributional kernel query is identifiable over all source models compatible with G if and only if it is identifiable over all intrinsic exact models compatible with the interpretation of G . The same equivalence holds for conditional distributional kernels, with local support retained on both sides.*

Proof. Every source model interprets as an exact target model, and every exact model on the interpreted signature has the reverse finite-table presentation of Proposition B.7.8. Compatibility and observational equivalence agree in both directions, so the universal model-class quantifiers in joint and conditional identifiability range over equivalent presentations of the same class.

Lean checks compatibility and observational agreement as `finiteSourceCompatible_iff` and `finiteSource_observational_iff`. Joint and conditional identification are checked as `finiteSource_identifiable_iff` and `finiteSource_conditionalIdentifiable_iff` in `lean/Thesis/CausalTransport/FiniteSource.lean`.

■

Proof of Theorem 5.4.2. Observational and hard-interventional event agreement is Proposition B.7.8; the corresponding joint and conditional identifiability equivalences are Corollary B.7.9. ■

Proposition B.7.10 (Agreement of prefix and well-founded structural evaluation). *Let $\mathcal{M}$ be a structural causal model in the sense of Definition A.4.18, let*

$$\tau : \Pi_{i:V} \text{Option}(\text{Val}(i))$$

be an equation override, and let $\mathbf{u} : \text{Val}(\mathcal{U})$ be a complete exogenous assignment. Write $\mathbf{s}_k^\tau$ for the prefix recursion of Definition A.4.20 with the equation override τ applied at each newly evaluated node. For every $k \leq n$ and $j : \text{Fin}(k)$, this recursion agrees with direct well-founded evaluation at the corresponding node:

$$\mathbf{s}_k^\tau(\mathbf{u})(j) = \text{evalNodeUnder}_{\mathcal{M},\tau}(\mathbf{u}, \iota_k(j)).$$

Consequently, the complete assignments agree:

$$\mathbf{s}_n^\tau(\mathbf{u}) = \text{evalUnder}_{\mathcal{M},\tau}(\mathbf{u}).$$

Mapping the exogenous prior through either evaluator therefore gives the same interventional probability record of Definition A.5.3. For the empty override, both constructions give the observational record `observationalDistM` of Definition A.4.21.

Proof. Induct on k . The empty prefix has no coordinate. At a successor stage, the topological-order field sends every parent of the new final node to an index in the already constructed prefix. The induction hypothesis identifies each of those stored parent values with direct well-founded evaluation. Both evaluators read the same exogenous coordinates and apply the same equation override and structural mechanism, so they agree at the new final coordinate. Every earlier

coordinate is retained unchanged by the constructive finite-prefix extension, where the induction hypothesis applies directly.

Taking $k = n$ gives pointwise agreement on the complete assignment. Function extensionality packages those coordinatewise equalities as equality of the two Lean assignment functions. Congruence of `FiniteProbRecord.map` then gives equality of the corresponding pushforward records, both for an arbitrary override and for the empty observational override.

Lean checks the pointwise induction as `evalPrefixUnder_eq_evalNodeUnder` in the `FiniteLatentSCM` namespace of `lean/Thesis/Causality/Model.lean`. ■

Proposition B.7.11 (Pointwise kernel-denotation agreement). *Let $\mathcal{M}$ be a finite structural causal model, let Q be a joint or conditional kernel query of Definition A.5.5, and let a be a complete reference assignment. The common finite kernel record obtained from Q evaluates at a to a partial value equivalent, in the sense of Definition A.5.9, to $\llbracket Q \rrbracket_{\mathcal{M}}(a)$ of Definition A.5.6.*

In the joint case this is the always-defined cylinder probability

$$\text{some}\left(P_{\mu_{\mathcal{M}}^{X,a}}(F_{Y,a})\right).$$

In the conditional case its numerator is the probability of $F_{Y,a} \cap F_{Z,a}$, its denominator is the probability of $F_{Z,a}$, and the result is absent exactly when that denominator is zero.

Proof. The reference-induced intervention used by the kernel record is `fromAssignment( $X, a$ )` of Definition A.5.2. For every node selection S , its agreement classifier returns true on a complete assignment b exactly when a and b agree at every node selected by S . Equivalently, their compact selected-coordinate projections of Definition A.4.2 are identical, so Definition A.3.26 identifies this classifier with $F_{S,a}$.

If X selects at least one node, the kernel evaluator uses the interventional assignment induced by `fromAssignment( $X, a$ )`. If X is empty, it instead selects the observational branch. In that case `fromAssignment( $X, a$ )` is the no-intervention assignment, and observational evaluation is definitionally evaluation under that assignment. The two branches therefore give the same probability record used in Definition A.5.6.

For a joint query, the condition selection is empty. Its denominator is consequently the full event, whose probability is equivalent to one by normalization. Dividing the outcome-fibre probability by that denominator gives an equivalent presentation of the same rational value. For a conditional query, Boolean conjunction makes the numerator event $F_{Y,a} \cap F_{Z,a}$ and the condition classifier is $F_{Z,a}$. The checked partial division returns the corresponding finite conditional probability when the denominator is positive and returns `none` when it is zero. These are exactly the two branches of Definition A.5.6.

Lean’s `ObservedSignature.project` retains the full assignment shape by inserting the same default values at every unselected coordinate. Equality of those projections is equivalent to equality of the compact selected-coordinate projections above. Lean checks the resulting fibre identification as `Kernel.agreesOn_iff_project_eq` and the unconditional comparison as `Kernel.unconditionalDenote` in `lean/Thesis/Causality/`

B. Proofs and arguments

`Semantics.lean`. The conditional comparison reduces through the definitions `Kernel.denote`, `Kernel.numeratorEvent`, `Kernel.conditionEvent` and `ProbabilityResult.divide` in the same module. ■

Proposition B.7.12 (Finite probability-expression transport). *Observational and interventional event probabilities are preserved by the encoding. Consequently, any expression formed from those values by the same finite rational operations on both sides has the same value whenever its displayed denominators are positive.*

Proof. Source and target evaluators agree at every node, so the same finite rational expression has the same value on both sides whenever its displayed denominators are positive. A zero denominator is reported as undefined, not replaced by an arbitrary rational. Support is checked along the source derivation and then transported.

Lean checks the composite expressions as `termDenote_preserved` in the `FiniteTableSCM` namespace. ■

Proposition B.7.13 (Checked graph-predicate transport). *Interpreting a source graph preserves the observed directed–bidirected graph and therefore preserves predicates and computations built directly from the graph objects formalised in the theorem-facing modules: directed reachability, bidirected connectivity, internally connected induced node restrictions (Definition A.5.16), c-forests (Definition A.5.17), graph mutilation (Definition A.5.1), the finite ancestral-moralisation d-separation test (Definition A.4.28) and hedge witnesses (Definition A.5.18).*

Proof. Interpreting a source graph does not change the observed directed or bidirected edges. Predicates built from those edges—reachability, bidirected components, c-forests, mutilation, the finite d-separation test and hedge witnesses—are therefore the same predicates after interpretation.

Lean defines those graph objects in `lean/Thesis/Causality/Identification.lean` and `lean/Thesis/Causality/Derivation.lean`. The two shared-cause calculations `shared_cause_is_d_connected` and `intervention_cut_d_separates_shared_cause` are two-node examples in `lean/Thesis/Examples/TenureTrack.lean`, not general theorems. Equivalence of the executable test with the external active-path specification remains the field of `DSeparationCorrectness`. ■

Proposition B.7.14 (Internal shared-cause identification). *Consider two Boolean observed nodes X and Y with no directed edge and with a projected shared-cause edge $X \leftrightarrow Y$. For every exact model over this observed signature and every value x ,*

$$P(Y \mid \text{do}(X = x)) \sim P(Y)$$

as distributional kernels. Consequently, $P(Y \mid \text{do}(X))$ is identifiable throughout the compatible shared-cause model class. This equivalence has both an executable Rule 3 derivation and a direct semantic proof; the latter does not use the general external soundness or completeness interfaces.

B.8. Finite structural-causal-model construction and latent projection

In the concrete model in which one fair latent bit is copied to both nodes, the observational and interventional calculations distinguish confounding from intervention:

$$P(Y = 1 \mid X = 1) \sim 1, \quad P(Y = 1 \mid \text{do}(X = 1)) \sim \frac{1}{2}.$$

Proof. Because the observed signature has no directed parents, replacing the equation for X cannot change the recursive evaluation of Y at any latent assignment. Pushing the common latent belief through those pointwise-equal Y evaluations gives rational equivalence of the two kernels. Across observationally equivalent compatible models, each interventional kernel therefore reduces to the corresponding observational Y kernel, which proves identifiability. In the concrete model, observing $X = 1$ selects the true latent bit, whereas setting $X = 1$ leaves that fair bit unchanged.

Lean checks the Rule 3 object as `secondAfterFirstDerivation`, the pointwise structural equality as `evalSecond_interveneFirst_eq`, the semantic kernel reduction as `secondAfterFirst_semantic_reduction`, and identifiability as `secondAfterFirst_identifiable`. The two displayed concrete values are `shared_observational_conditional_secondTrue` and `shared_interventional_secondTrue` in `lean/Thesis/Examples/TenureTrack.lean`. ■

B.8 Finite structural-causal-model construction and latent projection

This section records two complementary ways models enter the compatible finite source and target class: constructive functionalisation of finite rational conditional probability tables, and structural projection of explicit hidden DAGs.

Lemma B.8.1 (Finite rational causal-Bayesian-network functionalisation). *Every finite rational conditional probability table of a causal Bayesian network of Definition A.4.22, with explicitly enumerated parent configurations, has a functional representation with one independent finite response-function seed per observed node. The deterministic mechanism selects the response-table coordinate indexed by the realised parent configuration. Every original row is preserved, the observational probability of every complete assignment factorises as the product of its selected conditional-table entries, and the resulting latent-root structural causal model is Markovian.*

Proof. Each rational conditional-probability-table row becomes a common-denominator record. For one node, a complete response table is the product of those rows; selecting the coordinate indexed by the realised parents recovers the original row. Doing this independently at every node gives a Markovian latent-root structural causal model whose observational assignment probabilities factorise as the product of the selected table entries.

Lean checks row preservation as `toSCM_preserves_row` and the factorisation as `toSCM_observational_singleton_factorizes` in the `FiniteRationalCPT` namespace. ■

B. Proofs and arguments

Proposition B.8.2 (Finite semi-Markovian projection). *The direct-root semi-Markovian projection of Definition A.4.23 is represented and checked. A Markovian model has no projected bidirected edges. A shared latent root can induce a bidirected edge, as the tenure-track instance demonstrates. This is a graph-level projection theorem; it neither removes latent variables from the structural record nor proves preservation of identifiability under arbitrary hidden-DAG projection.*

Proof. In the direct-root projection, two observed nodes are bidirected exactly when they are distinct and share an incident exogenous root. If every exogenous root is incident to at most one observed node, this condition is impossible. Conversely, the explicit shared root in the tenure-track model is incident to both selected observed nodes.

Lean checks the general Markovian fact as `markovian_projection_has_no_bidirected`. The shared-root calculation `shared_source_projects_to_bidirected` is a checked instance in `lean/Thesis/Examples/TenureTrack.lean`. ■

Proposition B.8.3 (Executable finite hidden-DAG semantics). *For the partially observed hidden graph and general projection of Definitions A.4.32 and A.4.35, `FiniteHiddenDAGSCM` requires an enumerated value type and an independent private-noise factor at every full-DAG node, together with a mechanism typed only over that node's declared parents. Its observational and interventional distributions are defined by well-founded evaluation and finite pushforward.*

Proof. The rank field makes recursion over the full DAG well founded. Product noise supplies a finite assignment distribution, evaluation gives a total-node assignment, and pushforward gives the displayed observed distributions.

Lean checks the record as `FiniteHiddenDAGSCM`, with operations `evalNode`, `observationalDist` and `interventionalDist`. They are defined in `lean/Thesis/CausalTransport/HiddenDAGModel.lean`. ■

Proposition B.8.4 (Finite hidden-DAG graph projection). *The general finite hidden-DAG projection records directed hidden-internal paths and common-hidden-ancestor paths. Its projected directed arrows agree with the declared observed signature, and its bidirected arrows are exactly the pairs of distinct observed nodes reached from a common hidden ancestor by hidden-internal paths.*

Proof. The projection predicates are computed from the finite path data rather than from model probabilities. Lean checks their exact observed-graph characterisations as `FiniteLatentProjection.observedGraph_directed_iff` and `FiniteLatentProjection.observedGraph_bidirected_iff`; the hidden path in the tenure-track example is checked by `hidden_path_projects_to_bidirected`. ■

Proposition B.8.5 (Finite hidden-DAG conditional support). *For a selected finite hidden-DAG model family, `UniformConditionalIdentifiable` is equivalent to pointwise support at every assignment together with common-support conditional identifiability. Because the observed assignment type is explicitly finite, pointwise truncated support is constructively equivalent to one truncated function supplying support witnesses for all assignments.*

Proof. Under support at every assignment, common-support agreement applies to both models everywhere and therefore gives full partial-result equivalence. Conversely, uniform identifiability retains the support component and weakens full result equivalence to common-support agreement. A finite enumeration of observed assignments assembles the pointwise truncated witnesses into one truncated dependent function without invoking choice.

Lean checks the first equivalence as `uniformConditionalIdentifiable_iff` and the finite witness assembly as `kernelSupportedEverywhere_iff_globalWitness` in `lean/Thesis/CausalTransport/HiddenDAG.lean`. ■

Remark B.8.6 (External latent-projection interface). The mathematical interface is specified in Definition A.8.31. To inhabit it, the identifiability bridge must quantify over a selected nonempty `HiddenDAGModelFamily` of executable `FiniteHiddenDAGSCMs` and requires an explicit `PublishedLatentProjection` witness. That witness relates the family’s joint and common-support conditional identifiability to the corresponding identifiability notions on the projected observed graph. Lean exposes these two supplied equivalences as `hiddenDAG_joint_transport` and `hiddenDAG_conditional_transport` in `lean/Thesis/CausalTransport/HiddenDAG.lean`.

The selected family is not claimed to exhaust every executable record over the projection. Nor does this interface convert every hidden-DAG model to independent pairwise roots while preserving its probability distribution. It is an external theorem boundary, not a consequence of the executable semantics or graph projection alone.

B.9 Modal and structural causal execution

This section concerns executable, proof-carrying changes of finite causal records. The modality marks a valid source–target transition; a separately defined operation constructs the target record; and the transition witness certifies that the operation was performed.

Proposition B.9.1 (Modal separation of transitions and operations). *The finite correspondence at a causal mode is evaluated using the exact causal record stored at that mode. A transition label does not inspect or rewrite that record. A separately typed operation computes the target record, and the transition carries a proof that its named target is that computed result. Fixed-signature transitions preserve the stored exact model; signature-changing edits state their source and target signatures explicitly.*

Proof. This is immediate from the indexed transition records. A fixed-signature transition contains a `CausalRecordStep`. A signature-changing transition contains an indexed `CausalEditOperation` and its result equality. Neither transition obtains its target merely from a label.

Lean checks fixed-signature model preservation as `CausalRecordStep.model_eq`; the computed target of an edit is the `realized` field of `CausalEditTransition`. One-shot action is `act` and composite action is `run / run_single` in `lean/Thesis/Causality/ModeTheory.lean`. ■

B. Proofs and arguments

Proposition B.9.2 (Modal annotation of causal derivations). *Every symbolic kernel determines its action and condition selections, and every checked do-calculus derivation of Definition A.5.21 determines a modal trace between the corresponding query modes. Erasing this annotation recovers the original derivation. This statement adds modal structure to a derivation; it does not yet execute those selections as causal record operations.*

Proof. Construct the source and target query modes from the kernel's node sets. At each Pearl-rule leaf, attach the corresponding rule cell; the remaining constructors copy the derivation's algebra. Structural recursion preserves the underlying derivation when the annotations are erased.

Lean checks the query-mode construction as `CausalQueryMode.kernel_built_from_locks`, the annotation as `DoCalculusDerivation.toModalTrace`, and erasure as `ModalDoCalculusDerivation.erase_toModal`. ■

Concrete operation realisation of supported do-calculus leaves is stated later in Proposition B.11.6. Counterfactual atomic execution is separated into the results of Section B.10.

The proof-carrying record transformations used here are defined in Definitions A.7.5 to A.7.7, A.7.11 and A.7.18.

Proposition B.9.3 (Finite modal transition preservation). *Each model-preserving fixed-signature finite epistemic transition preserves every record component it is not specified to change, and its target is definitionally the output of the corresponding record operation. Conditioning, setting and unsetting preserve the exact structural model. Simultaneous finite intervention and selected-node observation are the multi-node specialisations used to realise action and condition selections. Simultaneous intervention is extensionally the duplicate-free sequential fold of its selected single-node settings; hence the multi-node operation is a convenience macro, not an additional causal primitive. This proposition does not apply to `CausalEditTransition`, whose purpose is precisely to permit an explicit model or signature change.*

Proof. A fixed-signature epistemic transition changes only the components it is supposed to change. Conditioning updates belief and leaves the model unchanged. Setting or unsetting a node updates the intervention and leaves the belief unchanged. A simultaneous node-set intervention is the selected single-node settings, folded in index order. It is a convenience, not a new causal primitive.

Lean checks the fold as `interveneNodes_eq_sequential` in the `CausalEpistemicRecord` namespace, using `selectedNodeList`. ■

Proposition B.9.4 (Modal identification invariance). *Let a proof-carrying fixed-signature finite epistemic transition connect two causal modes. A joint or conditional kernel is identifiable at the source mode exactly when it is identifiable at the target mode. Compatibility with any fixed observed graph is preserved as well. This invariance concerns fixed-signature transitions; certificate existence additionally needs the explicitly supplied completeness and soundness interfaces and is not part of this proposition.*

Proof. A fixed-signature transition keeps the exact structural model, so the observed graph and compatible-model class do not move. Joint and conditional identifiability therefore quantify over the same models and compare the same kernels at both endpoints.

In the `CausalTransition` namespace, Lean checks joint and conditional invariance by `jointIdentifiable_iff` and `conditionalIdentifiable_iff`. Compatibility is checked separately by `compatibleWith_iff`. ■

Proposition B.9.5 (Epistemic-update adequacy for conditioning). *Let a causal epistemic state consist of a structural model, a current finite belief and a current intervention. For positive-probability latent or observed evidence, the proof-carrying conditioning operation produces a target state whose belief agrees, event by event and up to rational equivalence, with the corresponding finite Bayesian posterior. The structural model, its independent-root prior and the current intervention are unchanged.*

Proof. Conditioning on positive-probability latent evidence replaces the belief record by the filtered and renormalised record of Proposition B.6.20. Observed evidence is first pulled back along the evaluator determined by the current intervention, after which the same latent conditioning operation is applied. Both constructions retain the model and intervention fields definitionally; the independent-root prior belongs to the unchanged model.

Lean checks latent conditioning as `conditionLatent_probVal`. Observed conditioning is the definition `conditionObservation`: it pulls observed evidence back along the current evaluator to `nodeObservationEvidence` and then invokes the same latent-conditioning construction. ■

Corollary B.9.6 (No arbitrary causal-edit endpoints). *Every causal-edit operation computes its result from its indexed source and operation-specific input data. Its constructor determines the edit label, and every checked causal-edit transition identifies its named target with that computed result. In particular, no transition can certify an arbitrary target merely by assigning it a causal-edit label.*

Proof. This is the signature-changing endpoint consequence of Proposition B.9.1. The operation family is indexed by its source record and target signature. Pattern matching on its constructor defines both `result` and `label`; the transition’s `realized` field equates its target with that result.

Lean checks the fixed-transition embedding as `CausalEditOperation.ofRecordStep_result`; the general endpoint equality is carried by `CausalEditTransition.realized`. ■

Proposition B.9.7 (Certified causal-belief reindexing). *Public belief reindexing derives its finite distribution from a certified epistemic-record belief or SCM prior. Its coordinate map is accompanied by a pointwise endpoint-realisation proof. Clearing an override requires both equality of evaluation before and after clearing and a proof that the structural evaluator realises the declared endpoint.*

Proof. The origin is an indexed constructor, not an arbitrary probability record. Applying the operation pushes that origin belief through the supplied coordinate map. The preserving policy retains the current intervention; the clearing policy carries the two displayed semantic obligations.

B. Proofs and arguments

Lean checks these clauses as `BeliefReindexing.CertifiedOperation.apply_belief`, `apply_realizes`, `apply_preserves_evaluation` and `source_realizes_reference`. ■

Proposition B.9.8 (Typed link editing and provenance round trips). *Directed relating inserts one rank-respecting endogenous-parent link; latent relating inserts one exogenous-input link. Their unrelating counterparts remove the selected link, and every edit carries the replacement mechanism required by the affected input type. Endogenous or exogenous learning followed immediately by its paired, provenance-guided forgetting restores the source record.*

Proof. The directed and latent operation constructors compute the changed signature or incidence relation and install the supplied replacement mechanism. The forgetting constructors are indexed specifically by the output of their paired learning constructor and return its recorded source.

In `DirectedLink`, Lean checks the directed changes as `addedEdge` and `removedEdge`. In `LatentLink`, it checks the latent changes as `addedIncident` and `removedIncident`. The paired round-trip declaration is named `learn_forget_record_roundTrip`; variants occur in the `CausalEditTransition`, `EndogenousVariableSpec` and `ExogenousVariableSpec` namespaces. ■

Proposition B.9.9 (Compiled and compact structural intervention). *For a hard intervention x , `AtomicIntervention.compile` replaces the selected mechanisms and removes their incoming directed and latent links by primitive typed edits. At the transported endpoint, a directed input is present exactly when the corresponding source edge is present and its child is not selected; a latent input is present exactly when the corresponding source incidence is present and its child is not selected. Its final `AtomicIntervention.Plan` certifies these complete classifiers as well as all selected constant mechanisms. The compact `SurgicalIntervention` performs direct graph mutilation in one constructor. For every source latent assignment, the canonical target assignment supplied by the deterministic compiler evaluates exactly as the compact surgical model and hence as the ordinary hard intervention.*

Proof. The compiler first installs constant mechanisms and then removes every selected node's incoming links. Well-founded evaluation shows that the compiled model returns exactly the source model's hard-intervention evaluator. The endpoint plan separately proves that the selected equations are constant. The two classifier equalities show both that every selected child's incoming directed and latent inputs have been removed and that every input of an unselected child has been preserved; hence the compiler creates no unintended structural link. The compact constructor supplies the direct reference evaluator.

Lean checks the complete structural certificate as `AtomicIntervention.plan`, the exact classifiers as `AtomicIntervention.compile_directed_eq` and `AtomicIntervention.compile_latent_incident_eq`, and their edge-existence forms as `AtomicIntervention.compile_directed_eq_true_iff` and `AtomicIntervention.compile_latent_incident_eq_true_iff`. Lean checks hard-evaluator agreement as `AtomicIntervention.compileDeterministicRealizes`, and the canonical compact comparison as `AtomicIntervention.compileDeterministicRealizes_evaluate_surgery`. The existential theorem `AtomicIntervention.compile`

`semanticallyEquivalent_surgery` is a wrapper around that specified target assignment.

Unsetting is different: it removes the current compact override at one target and does not restore an earlier setting. ■

Proposition B.9.10 (Conservative terminal learning). *The conservative terminal observed extension of Definition A.7.39 preserves every old observational, hard-interventional and current-belief event value. Every compatible model over the extended graph restricts to a compatible old model. No corresponding preservation claim is made for an arbitrary causal extension.*

Proof. The old mechanisms ignore the appended terminal coordinate, so restricting an extended evaluation recovers the old evaluation. Pushforward then gives observational, interventional and current-belief preservation. Restricting a compatible extended model preserves canonical semi-Markovianity and the old observed graph.

Lean checks these claims in `lean/Thesis/Causality/Learning.lean`. The principal declarations are `extendModel_observationalValue_old`, `extendModel_interventionalValue_old`, `learnRecord_observedValue_old` and `restrictModel_compatible`. General endogenous and exogenous learning and their provenance-guided forgetting roundtrips were already established in Proposition B.9.8. ■

Corollary B.9.11 (Conservative terminal identifiability transport). *If an old joint distributional kernel is identifiable, then its lift to a conservative terminal extension is identifiable. The same one-way preservation holds for locally supported conditional distributional kernels.*

Proof. Given two compatible extended models, restrict both along the terminal coordinate. By Proposition B.9.10, the restricted models are compatible old models and observational agreement restricts to the old signature. Old identifiability compares the restricted query denotations, which agree with the lifted denotations.

In the `TerminalVariableSpec` namespace, Lean checks the two implications as `liftJointQuery_identifiable` and `liftConditionalQuery_identifiable` in `lean/Thesis/CausalTransport/Learning.lean`. ■

Proposition B.9.12 (Constructive observed-sink forgetting). *Let $\mathcal{M}$ be an exact finite latent-root structural causal model. If an observed node d has no outgoing directed edge, then the induced model on $S \setminus \{d\}$ preserves every retained recursive evaluation and every retained observational and interventional event value. Deleting d from a current epistemic record also preserves every retained current-belief event value, and compatibility restricts to the induced graph. This deletion is a constructor of the indexed causal-edit family carrying the generic forgetting label.*

Proof. Filtering the ordered finite index gives an order-preserving skip embedding. Since d has no outgoing arrow, every parent of a retained equation is itself retained. Well-founded induction on the child index then identifies the old and restricted evaluations; pushforward of the unchanged latent prior and current belief gives the probability equalities. Restricting latent incidence preserves canonical semi-Markovianity and the induced bidirected graph.

B. Proofs and arguments

Lean checks these statements in `lean/Thesis/Causality/Forgetting/Basic.lean`; representative declarations are `restrictModel_evalNodeUnder`, `deleteRecord_observedValue` and `restrictModel_compatible`. The operation and transition builders are `forgettingUnusedEndogenous` and the endogenous declaration named `forgetUnusedTransition` in `lean/Thesis/Causality/Structural/Core.lean`. ■

Corollary B.9.13 (Observed-sink identifiability equivalence). *Let d be an observed node of S with no outgoing directed edge. For every observed graph G on S and every joint or common-support conditional kernel q over the induced signature $S \setminus \{d\}$, identifiability of the lift $q^\uparrow$ over G is equivalent to identifiability of q over the induced graph $G \setminus d$.*

Proof. For the implication from identifiability over G to identifiability over $G \setminus d$, each reduced model is extended to G by making d a constant default-valued sink, retaining its old latent roots on retained nodes, and adding deterministic unit-valued pair roots for the bidirected edges required by G . These pair roots have probability one and therefore do not change any event probability. The resulting model has observed graph exactly G ; its complete observational distribution is the pushforward of the reduced distribution along default-value insertion. Lifted query denotations agree in both models.

For the reverse implication from the reduced graph to the full graph, any pair of compatible, observationally equivalent full models is restricted along the sink deletion. This is the direction in which the semantic preservation results for restriction are used.

Lean checks the joint and conditional equivalences as `liftJointQuery_identifiable_iff` and `liftConditionalQuery_identifiable_iff` in `lean/Thesis/Causality/Forgetting/Identification.lean`. ■

Proposition B.9.14 (Constructive unused-exogenous-root forgetting). *Let $\mathcal{M}$ be an exact finite latent-root structural causal model. If an exogenous root u is incident to no observed mechanism, then deleting its finite coordinate and marginalising the prior and current belief preserves every recursive evaluation, observational and interventional event value, current-belief event value, projected observed graph and compatibility proof. The retained prior still satisfies the exact finite product law. This deletion is a constructor of the indexed causal-edit family carrying the same generic forgetting label as observed deletion.*

Proof. Structurally recursive deletion of one finite coordinate supplies both the skip embedding and its constructive inverse on every retained index. The deleted coordinate contributes the sure event to every rectangular latent event. Its factor therefore contributes multiplicative unit, so the original product law derives the product law after marginalisation. Unusedness makes the deleted branch of every mechanism input impossible. The evaluator and probability equalities follow, and finite existential elimination shows that projected bidirected incidence is unchanged.

Lean checks these statements in `lean/Thesis/Causality/Forgetting/Basic.lean`; representative declarations are `reducedProductLaw`, `reduceModel_interventionalValue` and `reduceModel_compatible`. The operation and transition builders are `forgettingUnusedExogenous`

and the exogenous declaration named `forgetUnusedTransition` in `lean/Thesis/Causality/Structural/Core.lean`. ■

Proposition B.9.15 (Canonical endpoint-record agreement). *Every deterministically realised structural execution whose endpoint intervention is proved absorbed on its transported latent assignments has a canonical one-step closure whose literal target record is its probability-bearing endpoint record. The canonical ordinary-kernel, from-factual and from-empty executions discharge this condition by proving that their fresh empty intervention remains empty through every construction and atomic-compilation step.*

Proof. Once the endpoint intervention is absorbed, the canonical closing transition installs the probability-bearing record as its literal target. The ordinary-kernel, from-factual and from-empty routes discharge the premise by keeping a fresh empty intervention empty throughout.

Lean checks the generic equality as `canonicalRealizes_endpointRecord` in `lean/Thesis/Causality/Structural/Atomic.lean`. The ordinary-kernel absorption witness is `Kernel.atomicBaseEndpointInterventionAbsorbed`. Each of the from-factual and from-empty execution modules supplies its own `endpointInterventionAbsorbed` declaration. ■

Corollary B.9.16 (Eventwise endpoint-record agreement). *Before canonical closure, a deterministically realised execution's probability-bearing endpoint record and a separately specified literal target assign equivalent probabilities to every endpoint event whenever the source belief has been pushed through the endpoint assignment and the literal target has no residual intervention. Record equality is not required.*

Proof. The belief-transport hypothesis compares the two latent event probabilities. Empty target intervention makes their structural evaluators agree on those events, so transitivity of rational equivalence gives the result.

Lean packages the two hypotheses as `TargetRecordAgreement` and checks the result as `endpointRecord_target_observedValue` in `lean/Thesis/Causality/Structural/Atomic.lean`. ■

Proposition B.9.17 (Current-state endpoint semantics). *Every proof-carrying causal edit path admits the endpoint-query semantics of Definition A.7.22. It evaluates the query at the path's literal target record and therefore retains that record's accumulated finite belief and hard intervention. A later kernel action overrides only its selected intervention coordinates. At an initial mode this semantics is equivalent to the ordinary model-level joint or conditional kernel denotation.*

The concrete histories used in this book agree with this common endpoint reading: supported do-calculus kernels, modal abduction-action-prediction of Remark A.7.33 and the occurrence-indexed counterfactual construction all reduce to their previously stated semantics. This proposition does not assert identifiability or completeness for arbitrary sequential epistemic programs.

Proof. A query at the end of an edit path is evaluated on the path's literal target record, with that record's accumulated belief and intervention. At an initial

B. Proofs and arguments

mode this is the ordinary model-level kernel denotation. The concrete histories used later—supported do-calculus kernels, abduction–action–prediction and the occurrence-indexed construction—agree with this same reading.

Lean checks the initial-mode case as `Kernel.endpointDenote_initial` and the bridges in `lean/Thesis/Causality/EndpointAgreement.lean`. ■

B.10 Counterfactual and multiworld execution

This section concerns shared-latent two-world and arbitrary finite multiworld constructions. It uses the structural operations of Section B.9.

Proposition B.10.1 (Finite twin-network adequacy). *The construction of Definition A.6.9 is finite and acyclic. Its factual marginal is the observational distribution of the original model, its counterfactual marginal is the distribution under the selected hard intervention, and its joint events are evaluated by the same latent assignment in both worlds. After conditioning on positive factual evidence, its counterfactual-event value is equivalent to the model-level value of Definition A.6.6.*

Proof. The twin duplicates the observed signature while retaining one latent assignment. Its copied mechanisms use the original equations in the factual world and the hard-intervention equations in the counterfactual world. Therefore the two projections evaluate as the original observational and intervened models. Filtering the common latent prior by factual evidence gives the same numerator and denominator as model-level abduction.

Lean checks the projections as `TwinNetwork.factual_marginal` and `TwinNetwork.counterfactual_marginal`, the finite-SCM encoding as `TwinNetwork.Encoding.model_eval_eq_encodeAssignment`, and the conditioned value as `TwinNetwork.abductedCounterfactualValue_eq_counterfactualValue`. ■

Proposition B.10.2 (Occurrence-indexed multiworld adequacy). *The occurrence-indexed multiworld of Definition A.6.10 gives each atom occurrence in a finite Boolean counterfactual event a separate world, all sharing the same latent assignment. Its dependent evaluator and ordinary finite-SCM encoding agree pointwise and on event probabilities. For a complete query, the encoded numerator and denominator are equivalent to the direct multiworld query semantics.*

Proof. Each world copies the original topological order and uses the action attached to its atom. Encoding the finite dependent world coordinates as one ordinary finite signature changes neither node evaluation nor the Boolean fold defining the event. Applying this to the condition and numerator events gives the query result.

Lean checks pointwise evaluation as `OccurrenceMultiworld.Encoding.model_eval`, event probabilities as `OccurrenceMultiworld.Encoding.model_eventProbability`, and the complete-query comparison as `CounterfactualQuery.combinedDenote_equivalent`. ■

Proposition B.10.3 (Exact from-empty multiworld structure). *The from-empty route creates every source root and occurrence-world node. Its final atomic*

target, and hence its canonical probability endpoint, has exactly the directed and latent-incidence classifiers of the reference occurrence model after coordinate transport. No additional cross-world edge, root or incidence remains possible.

Proof. Every root and node is introduced with an explicit coordinate map, every source link is recreated within its designated world, and atomic intervention removes precisely the inputs of selected children. The resulting pointwise classifier equalities and surjectivity of the root map exclude any omitted or additional structural input.

In the `FromEmptyExecutedOccurrenceConstruction` namespace, Lean checks the atomic equalities as `target_directed_eq_reference` and `target_incident_eq_reference`, their canonical-endpoint forms as `probabilityTarget_directed_eq_reference` and `probabilityTarget_incident_eq_reference`, and root exhaustiveness as `sourceRoot_surjective`. ■

Proposition B.10.4 (Agreement of executed multiworld routes). *The occurrence-indexed model can be executed atomically either by extending a fresh factual mode with the missing worlds or by creating all roots and worlds from an empty mode. For every source latent assignment, the canonical assignments constructed at the two dependent endpoints evaluate alike after observed-coordinate transport, and every occurrence-world event has equivalent probability. Consequently their numerator, denominator and complete partial conditional-query results are equivalent, including the unsupported case. The literal endpoint record of the factual-extension route supplies the probability used by subsequent mode-indexed transport.*

Proof. Both constructions install the latent coordinates, world nodes, source links and copied mechanisms required to obtain the same reference evaluator, but in a different order. Their explicit coordinate transports and canonical latent-assignment maps identify the two resulting observed assignments for each source latent assignment. Pushforward along that pointwise equality preserves the event probabilities and hence both parts of the partial conditional query.

Lean checks these comparisons as `executedRoutes_endpoint_eval_agree`, `executedRoutes_eventProbability_equiv`, `executedRoutes_numerator_equiv` and `executedRoutes_denominator_equiv`, and packages the full partial-result comparison as `executedRoutes_denote_equivalent` in `lean/Thesis/Causality/ExecutedMultiworld/Agreement.lean`. ■

Corollary B.10.5 (Route-independent multiworld endpoint agreement). *Any two endpoints satisfying the shared reference-event law over the same occurrence multiworld agree on the coordinate-transported image of every reference event and therefore on the numerator, denominator and complete partial result of every query over that reference multiworld.*

Proof. Each endpoint probability is equivalent to the same reference-event probability. Transitivity gives eventwise agreement, and applying it to the numerator and denominator predicates gives equivalence of the complete partial result, including the unsupported case.

Lean checks the four statements as `eventProbability_equiv`, `pair_numerator_equiv`, `pair_denominator_equiv` and `pair_denote_`

B. Proofs and arguments

equivalent in the `MultiworldEndpoint` namespace of `lean/Thesis/Causality/ExecutedMultiworld/Endpoint.lean`. ■

Proposition B.10.6 (One-action modal counterfactual agreement). *For a one-action query, modal abduction-action-prediction transports the posterior belief through the atomic structural endpoint and agrees with the model-level helper, compact surgery and twin-network value. The query is supported at a model exactly when its factual evidence has positive probability there.*

Proof. Abduction filters the latent belief by factual evidence. Atomic action compiles the selected hard intervention into the copied mechanisms, and prediction reads the outcome event from that endpoint. The structural intervention theorem identifies this evaluator with compact surgery; the twin theorem identifies the same shared-latent conditional ratio.

Lean checks the specified canonical-assignment equality as `AbductionActionPrediction.atomicActionDeterministicRealizes_evaluate_compactAction`; the existential `AbductionActionPrediction.action_semanticallyEquivalent_compactAction` is its wrapper. Lean checks the model and twin values as `prediction_eq_counterfactualValue` and `prediction_eq_twinNetwork`, and support as `CounterfactualQuery.singleAction_supportedAt_iff`. ■

Proposition B.10.7 (Finite unit-level counterfactual laws). *Let u be a latent assignment of a finite SCM. Evaluation satisfies: no intervention returns the factual assignment; an intervened variable takes its selected value (effectiveness); setting variables to the values they already have factually leaves the complete assignment unchanged (consistency); and extending a base intervention only by values already produced under that base intervention leaves the complete assignment unchanged, provided no base setting is removed (composition).*

Proof. These are the usual unit-level intervention laws. With no override, evaluation returns the factual assignment. An intervened variable takes the value it was set to. Setting a variable to the value it already has leaves the complete assignment unchanged. Extending an intervention only by values it has already produced leaves the complete assignment unchanged, provided no earlier setting is removed. The last fact is well-founded induction along the node order; consistency is the empty-override instance.

Lean checks the four laws as `evalUnder_noIntervention`, `evalUnder_effectiveness`, `evalUnder_consistency` and `evalUnder_composition`. ■

Proposition B.10.8 (Finite counterfactual semantic correspondence). *Every finite Boolean multiworld event has the same truth value at each latent assignment in a finite source table and its intrinsic target interpretation. Hence every partial conditional counterfactual query has equivalent source and target denotations.*

Proof. A finite Boolean multiworld event is true of a latent assignment on the source table exactly when it is true of the same assignment in the interpreted model. Applying this pointwise fact to the numerator and denominator events gives equivalence of the complete partial result.

Lean checks the event step as `finiteSourceHolds_preserved` in the `CounterfactualEvent` namespace and the query step as `finiteSourceDenote_preserved` in the `CounterfactualQuery` namespace of `lean/Thesis/CausalTransport/Counterfactual.lean`. ■

Corollary B.10.9 (Finite counterfactual model-class correspondence). *Interpretation preserves and reflects source compatibility and observational equivalence. Consequently, observational counterfactual identifiability and uniform query support are each preserved and reflected between the finite source class and its intrinsic target interpretation.*

Proof. Compatibility and observational equivalence are the source–target equivalences of Corollary B.7.9. The semantic comparison of Proposition B.10.8 then transports the universal model-class quantifiers and their support witnesses in both directions.

In `lean/Thesis/CausalTransport/Counterfactual.lean`, Lean checks the counterfactual conclusions with the common prefix `finiteSource_` counterfactual and suffixes `Identifiable_iff` and `Supported_iff`. The compatibility and observational-equivalence bridges are `finiteSourceCompatible_iff` and `finiteSource_observational_iff`. ■

Corollary B.10.10 (Interventional-query adequacy). *Every local finite ordinary interventional query has the same rational value in a finite source model and its intrinsic target interpretation. Moreover, identifiability of the whole distributional kernel implies identifiability of every decidable fixed event local to its outcome nodes.*

Proof. An ordinary interventional query is a special case of the encoding already used above, so source and target values agree. If a whole kernel is identifiable, every decidable event on its outcome nodes is identifiable as well: singleton masses determine every event, and the cylinder cells may be summed. The converse is not claimed.

Lean checks the event aggregation as `kernel_identifiable_implies_event_identifiable`. ■

Corollary B.10.11 (Executable tenure-track realisation). *The finite model of Example A.9.1 is an inhabitant of the intrinsic target presentation and uses the same probability records, recursive evaluator and structural operations as the correspondence development. The calculations of Examples A.9.2 and A.9.3 are checked by the generic finite machinery. The evidence event has probability $3/32$, and the two displayed one-action counterfactual offer probabilities are both $1/2$. For the prestige action, the direct model calculation, finite twin network, atomic modal abduction–action–prediction sequence and the occurrence-indexed from-factual and from-empty executions have equivalent denotations.*

Proof. Mira’s model is built from the same finite records, evaluator and structural operations as the rest of the development. The displayed observational, conditional and interventional values are ordinary finite calculations. The evidence event has probability $3/32$, and both one-action counterfactual offer probabilities are $1/2$. For the prestige action, the direct calculation, the twin, abduction–action–prediction and the two occurrence-indexed executions agree.

B. Proofs and arguments

Lean checks the example in `lean/Thesis/Examples/TenureTrack.lean`, including `prestigeModalCombinedConstruction_value`. ■

B.11 Transport of published identification

This section transports the Shpitser–Pearl, Huang–Valtorta and Tian–Shpitser soundness and completeness results through the finite interfaces of Section A.8 [HV06; Pea12; SP06; SP08; TS09]. Lean represents the required finite specialisations as explicit packages; inhabitants must be supplied when the corresponding transport theorem is applied. The same packages compile conservative-learning certificates and finite multiworld counterfactual queries; when both completeness and soundness are supplied they yield two-way derivability. The last subsection states the boundary of the core checked finite causal class.

On the finite source, completeness yields concrete path-based derivation trees. The `DSeparationCorrectness` field compiles every active-path side condition into the executable ancestral-moralisation test used by the concrete derivation syntax. Soundness supplies the primitive finite SCM laws, while each supplied completeness certificate carries recursive support evidence for every endpoint and subderivation. Lean then proves internally, first for source tables and then by interpretation, that these laws and witnesses make the derivation denotationally sound and that an action-free certificate yields identifiability. The generic `PublishedCompleteness` and `PublishedSoundness` structures in the lower-level correspondence module remain explicit interfaces. The finite-source `toPublished` maps construct their target-side inhabitants from source-native packages; the primary finite-source interfaces do not assume an already transported conclusion. Their identifiability, expression denotation, conditional support, derivation-support trees, primitive semantic laws and counterexamples are source-native. The counterfactual interface is source-native as well: it returns a finite multiworld reduction, followed by a path-based ordinary do-calculus derivation to an action-free observational term and a support tree for that derivation. The probability-expression and derivation syntax is a shared model-independent intermediate language, rather than a shared evaluator. `FiniteTableSCM.termDenote_preserved`, `FiniteSourceLocalDerivationSupport.toTarget` and `FiniteSourceCounterexample.toTarget` are the ordinary checked bridges; `CounterfactualQuery.finiteSourceDenote_preserved` and `FiniteSourceCounterfactualCertificate.toTarget` are their multiworld counterparts.

Constructing an inhabitant of the finite-source interfaces from the cited papers requires a precise metatheoretic specialisation. One must verify that the papers’ recursive semi-Markovian model class contains the finite coded table models used here; identify their graphical criterion with `PathDSeparated`; translate every ID/IDC or do-calculus output into `PathDoCalculusDerivation`; prove local support for each division used by that output; supply the finite global-Markov and primitive probability laws; and show that every represented hedge has the required finite rational countermodels. The published binary fair-coin hedge construction and finite-domain statements make this specialisation mathematically plausible, but Lean does not formalise the papers’ ID/IDC

algorithms or their metaproofs. The counterfactual interface additionally requires translating the published **ID*/IDC*** reduction into the shared-latent finite source semantics and observationalising every resulting interventional term by a supported ordinary derivation. For the Boolean event syntax of Definition A.6.4, the specialisation must also justify the finite reduction from Boolean combinations to the query fragment covered by the published algorithm. The shared ordinary expression language can now represent every resulting finite assignment event through **ProbabilityTerm.eventTerm**; this does not by itself prove that the published multiworld algorithm performs the required reduction. Accepting the papers as correct is therefore not, by itself, a kernel term of these structures; the finite specialisation is the deliberately explicit external theorem boundary. Once those source-native semantic structures over the shared finite syntax are supplied, the corresponding interpretation, certificate compilation, source and target soundness induction and completeness transports are checked internally. Modal annotation and compatible-mode indexing are additionally checked for the ordinary do-calculus certificates described below. For counterfactuals, every potential-outcome atom receives its structural-intervention trace and the resulting query trace is packaged with both executed occurrence-indexed construction routes. The probability is read from the epistemic record produced by the factual-extension endpoint, and **ModeIndexedCounterfactualDerivation.endpointReduction** composes that endpoint's checked denotation with the transported observational reduction. The external source-native reduction itself remains supplied data rather than an internally generated modal algorithm.

External hypotheses used by the transports

The interfaces themselves are defined in Definitions A.8.15, A.8.16, A.8.24, A.8.26 and A.8.27. The next three remarks record the remaining inhabitation and support obligations before the checked transport theorems can be applied.

Remark B.11.1 (External finite do-calculus soundness obligation). An inhabitant of the soundness interface of Definition A.8.26 must connect the path and executable d-separation formulations through Definition A.8.24. It must also supply the primitive marginalisation, conditioning, chain and do-rule identities for every compatible finite table model on their stated support.

To read a derivation as an equality of probabilities, the external interface must supply both implications and the primitive finite probability laws on their stated support. Once those fields are given, Lean inducts on the derivation and checks equality of source denotations; target soundness is then obtained by transport. Lean checks this conditional induction as **DoCalculusDerivation.finiteSource_denotational_soundAt**. It does not construct the external interface.

Remark B.11.2 (External finite-rational hedge obligation). To inhabit the completeness interface of Definition A.8.27 for the intended class, every accepted hedge of Definition A.5.18 must yield two compatible finite source models with equal source observational distributions and unequal source-kernel values. The familiar binary fair-coin construction explains why a rational specialisation is expected, but the development does not uniformly construct those two models

B. Proofs and arguments

from an arbitrary hedge. Once supplied, Lean transports such a counterexample by `FiniteSourceCounterexample.toTarget`.

Remark B.11.3 (Conditional support boundary). The source support data are those of Definition A.8.15. Conditional distributional kernels have a partial finite-rational denotation. Their correspondence compares a valuation only when the source kernel is supported at that valuation in each of the two models. A supplied certificate must additionally produce local support for every intermediate denominator reached by its derivation at that valuation. This is a whole-kernel condition, not an arbitrary outcome-and-condition event query. The target comparison is `ConditionalKernelQuery.AgreesOnCommonSupport`; `ConditionalKernelQuery.ResultEquivalent` is the stronger relation that also matches support status. Lean checks the source–target equivalence as `finiteSource_conditionalIdentifiable_iff`.

Internally checked transports

Theorem B.11.4 (Conditional finite causal-completeness transport). *Let T be a finite table signature and G a finite table graph. If an explicit package*

$$P : \text{PublishedFiniteSourceCompleteness } T \ G$$

is supplied, then Lean proves all of the following consequences:

- (i) *every type-theoretically identifiable joint distributional kernel has the encoded joint derivation certificate supplied by P ;*
- (ii) *every locally supported type-theoretically identifiable conditional distributional kernel has the encoded conditional derivation certificate supplied by P ;*
- (iii) *every hedge represented by P yields a target counterexample and therefore failure of type-theoretic identifiability.*

These are consequences of the supplied finite specialisation, not an internal proof that the published completeness theorem inhabits P .

Proof. Source–target identifiability correspondence first turns each target premise into its finite-source counterpart. The appropriate field of P then returns a source derivation certificate or source counterexample. The checked interpretation maps carry that object back to the intrinsic target without changing its denotation, support data or observational comparison.

Lean packages the three consequences in `finiteSource_completeness_transport`; hedge counterexamples are mapped by `transport_hedge_failure`. The parameter P remains explicit. ■

Remark B.11.5 (Identification traces). The formalisation transports an inspectable derivation tree, but does not implement the branch trace or termination proof of the ID or conditional-ID algorithm that produces it. Finite d-separation search itself is implemented; branch selection and certificate generation remain part of the supplied completeness theorem.

Proposition B.11.6 (Operation-sensitive modal certificate transport). *Given an epistemic causal mode, an interpreted finite source graph, finite-source completeness and primitive-soundness packages, mode compatibility together with joint identifiability is equivalent to the existence of an `ModeIndexedOperationRealizedJointCertificate`; the analogous equivalence combines mode compatibility with identifiability for locally supported conditional queries. Each certificate retains the compatibility proof, compiled derivation and modal trace, and realises every supported kernel and Pearl-rule leaf by the checked record operations.*

Proof. Completeness supplies the encoded certificate. Compatible-mode indexing annotates its causal steps, and primitive soundness realises its kernel and rule leaves. Conversely, the realised certificate contains an encoded certificate whose supplied soundness proof gives identifiability, and its mode index supplies compatibility. All external packages remain parameters.

Lean checks the two equivalences as the declarations ending in `joint_iff` and `conditional_iff` under the common prefix `finiteSource_modeIndexedOperationRealized_`. ■

Conservative-learning certificate transport

This subsection compiles conservative-learning certificates on the finite source: given completeness and primitive-soundness packages for the extended graph, an identifiable lifted query receives an operation-realised modal certificate. The intrinsic learning and forgetting construction is in Proposition B.9.10 and Corollary B.9.11; only the compilation of those certificates belongs here.

Corollary B.11.7 (Conservative-learning certificate transport). *If completeness and primitive-soundness packages are supplied for the conservatively extended signature and graph, an identifiable lifted joint or locally supported conditional query receives an operation-realised modal certificate. The package retains the exact unified learning and forgetting transitions, compatibility of the source, learned and restored modes, and current-belief preservation.*

Proof. If completeness and soundness packages are supplied for the extended graph, an identifiable lifted query receives a modal certificate that still records the learning and forgetting steps. The packages remain inputs for the new graph.

Lean checks the two transports as `terminalLearningJointTransport` and `terminalLearningConditionalTransport` in the `CausalEditTransition` namespace. ■

Counterfactual completeness transport

This subsection transports finite multiworld counterfactual queries. Its external source component is the interface of Definition A.8.28.

Theorem B.11.8 (Finite counterfactual-completeness transport). *Let T be a finite table signature, G a finite table graph and $Q : \text{PublishedFiniteSourceSoundness } T \ G$. If an explicit source-native specialisation*

B. Proofs and arguments

P : PublishedFiniteCounterfactualCompleteness T G Q

*is supplied, then a finite multiworld query in the intrinsic target is observationally identifiable and uniformly supported over the compatible model class if and only if it has an **EncodedCounterfactualDerivation**. That encoded object retains a source-native multiworld reduction, a path-based do-calculus derivation from its interventional expression to an action-free observational expression and recursive local support for the derivation; its **toTarget** map yields the intrinsic target certificate.*

Proof. Source and target multiworld queries agree on truth values, support and identifiability. The external package supplies a source-native reduction and an ordinary do-calculus observationalisation. Lean compiles that derivation and checks it with the supplied soundness package.

Lean checks the two-way fact as `finiteSource_transported_counterfactual_iff`. The qualification by *P* is essential: Lean does not inhabit the published `ID*/IDC*` package. ■

Corollary B.11.9 (Mode-indexed counterfactual-completeness transport). *Under the hypotheses of Theorem B.11.8, compatibility of an epistemic causal mode with the interpreted graph, identifiability and uniform support are jointly equivalent to the existence of a **ModeIndexedCounterfactualDerivation**. This package retains the encoded source derivation, compatibility witness, structural modal trace and the executed occurrence-indexed construction from both a fresh factual mode and an empty mode. Its **endpointReduction** starts from the partial probability read from the factual-extension endpoint record and ends at the transported action-free observational formula.*

Proof. The preceding theorem supplies the ordinary encoded derivation. The compatible-mode construction annotates each potential-outcome atom by its structural-intervention trace, packages both checked execution routes and composes the endpoint denotation with the transported observational reduction. Conversely, the package exposes its compatibility witness, while erasing its modal and executable data recovers the ordinary encoded derivation and therefore identifiability with uniform support.

Lean checks this equivalence as `finiteSource_modeIndexed_transported_counterfactual_iff`. ■

Completeness consequences and boundary

This subsection records the two-way derivability equivalence obtained when both a finite-source completeness package and a finite-source soundness package are supplied, and states the boundary of the core checked finite causal class.

Corollary B.11.10 (Finite two-way derivability transport). *Given both a finite-source completeness package *P* and a finite-source soundness package *Q* for *T* and *G*, a finite joint distributional kernel is type-theoretically identifiable exactly when it has an encoded action-free derivation certificate. The same equivalence holds for locally supported conditional distributional kernels.*

Proof. Completeness gives the forward implication and soundness the reverse. Source–target agreement then transports both. The packages P and Q remain external inputs.

Lean checks the two equivalences as `finiteSource_transported_joint_iff` and `finiteSource_transported_conditional_iff`. ■

The core checked model class is finite, rational, recursive and canonical semi-Markovian. It covers joint distributional interventional kernels, locally supported conditional kernels, derived local fixed-event queries, source-native and target partial semantics for their probability expressions, modal annotations of do-calculus derivations, executable realisations of finite action and condition selections, typed directed and latent-input edits, endogenous and exogenous terminal creation, provenance-guided round trips, constructive deletion of arbitrary unused observed sinks and exogenous roots, atomic structural intervention with certified selected constants and exact directed and latent classifiers that remove inputs precisely at selected children while preserving those of unselected children, its canonical hard-evaluator equivalence, generic finite rational CPT functionalisation and conversion to independently seeded Markovian SCMs. It also covers an explicit shared-latent two-world construction, arbitrary finite Boolean multiworld query semantics, modal action-world traces, from-factual and literal from-empty occurrence-indexed execution, a generic structural abduction–action–prediction theorem, their finite-table to intrinsic correspondence, and the mode-indexed conditional counterfactual-completeness transport whose reduction starts at the executed atomic endpoint record, as stated in Theorem B.11.8. The conservative learning theorem preserves lifted old queries in one direction; the separate observed-sink deletion theorem gives a two-way identifiability correspondence for its induced restriction, but neither result asserts invariance under arbitrary signature change. Individual finite hidden-DAG models and their graph-level latent projection have internal executable semantics. The selected family over which the projection theorem quantifies, and the theorem that projection preserves its joint and common-support conditional identifiability, remain explicit external inputs. The development does not cover cyclic structural causal models, an internal construction of the published counterfactual-completeness package, transportability, path-specific effects, mechanised **ID**, **IDC**, **ID*** or **IDC*** algorithms, an internal proof of the active-path–moralisation equivalence, an internal finite global Markov theorem and the semantic validity of all three do-calculus rules, or a probability-preserving individual-model reduction from every arbitrary hidden directed-acyclic-graph structural causal model to independent pairwise latent roots. These are not harmless omissions: each requires a separate formal object or reduction theorem.

Bibliography/sources

“Contrarily, it is characteristic of a truly wise man that, being just a mortal, he doesn’t want to transgress any boundaries with his wisdom and together with humanity either turns a blind eye, or, out of courtesy, wanders along”

Folly in *The Praise of Folly*

B. Proofs and arguments

- [Ada92] Adams, D. *Mostly Harmless*. London: Heinemann, 1992.
- [Agr+25a] Agricola, I. et al. ‘Fraudulent Publishing in the Mathematical Sciences’. In: *Notices of the American Mathematical Society* vol. 72, no. 9 (Oct. 2025), pp. 1038–1049.
- [Agr+25b] Agricola, I. et al. ‘How to Fight Fraudulent Publishing in the Mathematical Sciences: Joint Recommendations of the IMU and the ICIAM’. In: *Mitteilungen der Deutschen Mathematiker-Vereinigung* vol. 33, no. 4 (1st Dec. 2025), pp. 249–252.
- [Ait13] Aitkenhead, D. *Peter Higgs: I Wouldn’t Be Productive Enough for Today’s Academic System*. The Guardian. 6th Dec. 2013. URL: <https://www.theguardian.com/science/2013/dec/06/peter-higgs-boson-academic-system> (visited on 12/05/2026).
- [AK91] Allis, V. and Koetsier, T. ‘On Some Paradoxes of the Infinite’. In: *The British Journal for the Philosophy of Science* vol. 42, no. 2 (1st June 1991), pp. 187–194. URL: <https://www.jstor.org/stable/688052>.
- [AK95] Allis, V. and Koetsier, T. ‘On Some Paradoxes of the Infinite II’. In: *The British Journal for the Philosophy of Science* vol. 46, no. 2 (1st June 1995), pp. 235–247. URL: <https://www.jstor.org/stable/687814>.
- [Al-14] Al-Khalili, J. *Higgs Would Not Find His Boson in Today’s ‘Publish or Perish’ Research Culture*. The Guardian. 14th Feb. 2014. URL: <https://www.theguardian.com/commentisfree/2014/feb/14/higgs-boson-publish-or-perish-science-culture> (visited on 12/05/2026).
- [Alp+26] Alper, J. et al. *Leiden Declaration on Artificial Intelligence and Mathematics*. Leiden Declaration on Artificial Intelligence and Mathematics. 2nd June 2026. URL: <https://leidendeclaration.ai/> (visited on 24/07/2026).
- [AR11] Awodey, S. and Rabe, F. ‘Kripke Semantics for Martin-Löf’s Extensional Type Theory’. In: *Logical Methods in Computer Science* vol. 7, no. 3, 18 (2011), pp. 1–25. URL: <https://lmcs.episciences.org/1184>.
- [AS00] Arnborg, S. and Sjödin, G. ‘Bayes Rules in Finite Models’. In: *Proceedings of the 14th European Conference on Artificial Intelligence*. Berlin, 2000, pp. 571–575. URL: <https://frontiersinai.com/ecai/ecai2000/pdf/p0571.pdf>.
- [AS01] Arnborg, S. and Sjödin, G. ‘On the Foundations of Bayesianism’. In: *AIP Conference Proceedings*. Vol. 568. AIP Publishing, 2001, pp. 61–71.
- [AS14] Alvesson, M. and Sandberg, J. ‘Habitat and Habitus: Boxed-in versus Box-Breaking Research’. In: *Organization Studies* vol. 35, no. 7 (2014), pp. 967–987.
- [AS99] Arnborg, S. and Sjödin, G. *A Note on Foundations of Bayesianism*. Tech. rep. Royal Institute of Technology, 1999.

- [Bar26] Barber, G. *What Can We Gain by Losing Infinity?* Quanta Magazine. 29th Apr. 2026. URL: <https://www.quantamagazine.org/what-can-we-gain-by-losing-infinity-20260429/> (visited on 12/05/2026).
- [BDG25] Bentzen, B., Del Santo, F. and Gisin, N. *Naturalistic Intuitionism for Physics*. Preprint, PhilSci-Archive. 2025. URL: <https://philsci-archive.pitt.edu/26743/>.
- [BDK02] Beasley, M. R., Datta, S. and Kinyoun, H. H. *Report of the Investigation Committee on the Possibility of Scientific Misconduct in the Work of Hendrik Schön and Coauthors*. Tech. rep. Bell Labs, Lucent Technologies, 2002. URL: <https://s3.amazonaws.com/org.aps-content-assets//671ef6a6-c78d-4436-9e1c-e8135315bf30/lucentrep.pdf>.
- [BE12] Begley, C. G. and Ellis, L. M. ‘Drug Development: Raise Standards for Preclinical Cancer Research’. In: *Nature* vol. 483 (2012), pp. 531–533.
- [Bec+01] Beck, K. et al. *Manifesto for Agile Software Development*. 2001. URL: <https://agilemanifesto.org/> (visited on 26/05/2026).
- [BHM21] Bornmann, L., Haunschild, R. and Mutz, R. ‘Growth Rates of Modern Science: A Latent Piecewise Growth Curve Approach to Model Publication Numbers from Established and New Literature Databases’. In: *Humanities and Social Sciences Communications* vol. 8 (2021), p. 224.
- [Bik26a] Bik, E. *Is Science Self-Correcting? Not in This Elsevier Journal*. Science Integrity Digest. 24th June 2026. URL: <https://scienceintegritydigest.com/2026/06/24/is-science-self-correcting-not-in-this-elsevier-journal/> (visited on 01/07/2026).
- [Bik26b] Bik, E. *Science Integrity Digest*. Science Integrity Digest. 2nd July 2026. URL: <https://scienceintegritydigest.com/> (visited on 02/07/2026).
- [Bol25] Bollen, T. *Geld genoeg, maar niet voor jou: Hoe heroveren we de macht over ons geld?* Amsterdam: Follow the Money Publishing B.V., 2025.
- [Bot+20] Botvinik-Nezer, R. et al. ‘Variability in the Analysis of a Single Neuroimaging Dataset by Many Teams’. In: *Nature* vol. 582 (2020), pp. 84–88.
- [BR16] Baltag, A. and Renne, B. ‘Dynamic Epistemic Logic’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. Winter 2016. Metaphysics Research Lab, Stanford University, 2016. URL: <https://plato.stanford.edu/archives/win2016/entries/dynamic-epistemic/>.
- [Bre+22] Breznau, N. et al. ‘Observing Many Researchers Using the Same Data and Hypothesis Reveals a Hidden Universe of Uncertainty’. In: *Proceedings of the National Academy of Sciences of the United States of America* vol. 119, no. 44 (2022), e2203150119.

B. Proofs and arguments

- [Bri25] Brierley, O. *How Bad Science Is Becoming Big Business*. Phys.org. 18th Aug. 2025. URL: <https://phys.org/news/2025-08-bad-science-big-business.html> (visited on 12/05/2026).
- [BT24] Banach, S. and Tarski, A. ‘Sur la décomposition des ensembles de points en parties respectivement congruentes’. In: *Fundamenta Mathematicae* vol. 6, no. 1 (1924), pp. 244–277. URL: <https://eudml.org/doc/214280>.
- [But+13] Button, K. S. et al. ‘Power Failure: Why Small Sample Size Undermines the Reliability of Neuroscience’. In: *Nature Reviews Neuroscience* vol. 14 (2013), pp. 365–376.
- [Cam+16] Camerer, C. F. et al. ‘Evaluating Replicability of Laboratory Experiments in Economics’. In: *Science* vol. 351, no. 6280 (2016), pp. 1433–1436.
- [Cam98] Campanario, J. M. ‘Peer Review for Journals as It Stands Today—Part 2’. In: *Science Communication* vol. 19, no. 4 (1998), pp. 277–306.
- [Cas26] Castelvechi, D. ‘“It is incredible”: How AI is transforming mathematics’. In: *Nature* vol. 653, no. 8115 (19th May 2026), pp. 664–665. URL: <https://www.nature.com/articles/d41586-026-01553-1> (visited on 29/05/2026).
- [Cen26a] Center for Open Science. *Open Science Framework*. Center for Open Science. 2nd July 2026. URL: <https://osf.io/> (visited on 02/07/2026).
- [Cen26b] Center for Open Science. *Registered Reports*. Center for Open Science. 2nd July 2026. URL: <https://www.cos.io/initiatives/registered-reports> (visited on 02/07/2026).
- [CFN22] Cintula, P., Fermüller, C. G. and Noguera, C. ‘Fuzzy Logic’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. Spring 2022. Metaphysics Research Lab, Stanford University, 2022. URL: <https://plato.stanford.edu/archives/spr2022/entries/logic-fuzzy/>.
- [Cle26] Clear Skies Ltd. *Clear Skies: Your Partner in Research Integrity*. Clear Skies Ltd. 2nd July 2026. URL: <https://clear-skies.co.uk/> (visited on 02/07/2026).
- [Col04] Colyvan, M. ‘The Philosophical Significance of Cox’s Theorem’. In: *International Journal of Approximate Reasoning* vol. 37, no. 1 (2004), pp. 71–85.
- [Coq25] Coquand, T. ‘Type Theory’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Fall 2025. Metaphysics Research Lab, Stanford University, 2025. URL: <https://plato.stanford.edu/archives/fall2025/entries/type-theory/>.
- [Cou23] Council of the European Union. *Council Calls for Transparent, Equitable, and Open Access to Scholarly Publications*. Council of the European Union. 23rd May 2023. URL: <https://www.consilium.europa.eu/en/press/press-releases/2023/05/23/council-calls-for-transparent-equitable-and-open-access-to-scholarly-publications/> (visited on 12/05/2026).

- [Cox46] Cox, R. T. ‘Probability, Frequency and Reasonable Expectation’. In: *American Journal of Physics* vol. 14, no. 1 (1946), pp. 1–13.
- [Cox61] Cox, R. T. *The Algebra of Probable Inference*. Baltimore: Johns Hopkins Press, 1961. URL: <https://www.press.jhu.edu/books/title/1089/algebra-probable-inference>.
- [CW17] Clayton, A. and Waddington, T. ‘Bridging the Intuition Gap in Cox’s Theorem: A Jaynesian Argument for Universality’. In: *International Journal of Approximate Reasoning* vol. 80 (2017), pp. 36–51.
- [Cyr06] Cyranoski, D. ‘Verdict: Hwang’s Human Stem Cells Were All Fakes’. In: *Nature* vol. 439, no. 7073 (2006), p. 122.
- [DG19] Del Santo, F. and Gisin, N. ‘Physics Without Determinism: Alternative Interpretations of Classical Physics’. In: *Physical Review A* vol. 100 (2019), p. 062107.
- [DIA26] DIAMAS Project. *Developing Institutional Open Access Publishing Models to Advance Scholarly Communication*. DIAMAS. 2nd July 2026. URL: <https://diamasproject.eu/> (visited on 02/07/2026).
- [DOR12] DORA. *San Francisco Declaration on Research Assessment*. DORA. 16th Dec. 2012. URL: <https://sfdora.org/read/> (visited on 12/05/2026).
- [DP24] Dybjer, P. and Palmgren, E. ‘Intuitionistic Type Theory’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Winter 2024. Metaphysics Research Lab, Stanford University, 2024. URL: <https://plato.stanford.edu/archives/win2024/entries/type-theory-intuitionistic/>.
- [Era93] Erasmus, D. *Praise of Folly and Letter to Maarten van Dorp*. Ed. by Levi, A. H. T. Trans. by Radice, B. Penguin Classics. Latin original *Moriae Encomium*, Paris: Gilles de Gourmont, 1511. Harmondsworth: Penguin, 1993.
- [Eur26] European Commission. *Open Research Europe*. European Commission. 2nd July 2026. URL: <https://open-research-europe.ec.europa.eu/> (visited on 02/07/2026).
- [Fan09] Fanelli, D. ‘How Many Scientists Fabricate and Falsify Research? A Systematic Review and Meta-Analysis of Survey Data’. In: *PLOS ONE* vol. 4, no. 5 (2009), e5738.
- [Fey75] Feyerabend, P. *Against Method: Outline of an Anarchistic Theory of Knowledge*. London: New Left Books, 1975.
- [Fle79] Fleck, L. *Genesis and Development of a Scientific Fact*. Ed. by Trepp, T. J. and Merton, R. K. Trans. by Bradley, F. and Trepp, T. J. Foreword by Thomas S. Kuhn. German original *Entstehung und Entwicklung einer wissenschaftlichen Tatsache*, Schwabe, Basel, 1935. Chicago: University of Chicago Press, 1979.
- [FSC12] Fang, F. C., Steen, R. G. and Casadevall, A. ‘Misconduct Accounts for the Majority of Retracted Scientific Publications’. In: *Proceedings of the National Academy of Sciences of the United States of America* vol. 109, no. 42 (2012), pp. 17028–17033.

B. Proofs and arguments

- [Gar24] Garson, J. ‘Modal Logic’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Spring 2024. Metaphysics Research Lab, Stanford University, 2024. URL: <https://plato.stanford.edu/archives/spr2024/entries/logic-modal/>.
- [GB24] Gross, K. and Bergstrom, C. T. ‘Rationalizing Risk Aversion in Science: Why Incentives to Work Hard Clash with Incentives to Take Risks’. In: *PLOS Biology* vol. 22, no. 8 (15th Aug. 2024), e3002750.
- [Gel14] Gelman, A. *Scientists Behaving Badly*. Statistical Modeling, Causal Inference, and Social Science. 7th Nov. 2014. URL: <https://statmodeling.stat.columbia.edu/2014/11/07/scientists-behaving-badly/> (visited on 12/05/2026).
- [GHR17] Gyenis, Z., Hofer-Szabó, G. and Rédei, M. ‘Conditioning Using Conditional Expectations: The Borel–Kolmogorov Paradox’. In: *Synthese* vol. 194, no. 7 (2017), pp. 2595–2630.
- [Gis20] Gisin, N. ‘Mathematical Languages Shape Our Understanding of Time in Physics’. In: *Nature Physics* vol. 16 (2020), pp. 114–116.
- [GL13] Gelman, A. and Loken, E. *The Garden of Forking Paths: Why Multiple Comparisons Can Be a Problem, Even When There Is No “Fishing Expedition” or “p-Hacking” and the Research Hypothesis Was Posited Ahead of Time*. Tech. rep. Department of Statistics, Columbia University, 2013. URL: https://www.stat.columbia.edu/~gelman/research/unpublished/p_hacking.pdf.
- [GL14] Gelman, A. and Loken, E. ‘The Statistical Crisis in Science’. In: *American Scientist* vol. 102, no. 6 (2014), pp. 460–465.
- [Gol12] Goldacre, B. *Bad Pharma: How Drug Companies Mislead Doctors and Harm Patients*. London: Fourth Estate, 2012.
- [GR25] Goranko, V. and Rumberg, A. ‘Temporal Logic’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Summer 2025. Metaphysics Research Lab, Stanford University, 2025. URL: <https://plato.stanford.edu/archives/sum2025/entries/logic-temporal/>.
- [Gra+21] Gratzner, D. et al. ‘Multimodal Dependent Type Theory’. In: *Logical Methods in Computer Science* vol. 17, no. 3 (28th July 2021), 11:1–11:67. URL: <https://lmcs.episciences.org/7571>.
- [Gra23] Gratzner, D. ‘Syntax and Semantics of Modal Type Theory’. PhD thesis. Aarhus University, 2023. URL: <https://iris-project.org/pdfs/2023-phd-gratzner.pdf>.
- [GSM11] Godlee, F., Smith, J. and Marcovitch, H. ‘Wakefield’s Article Linking MMR Vaccine and Autism Was Fraudulent’. In: *BMJ* vol. 342 (2011), p. c7452. URL: <https://www.bmj.com/content/342/bmj.c7452>.
- [Hac83] Hacking, I. *Representing and Intervening: Introductory Topics in the Philosophy of Natural Science*. Cambridge: Cambridge University Press, 1983.

- [Hal99a] Halpern, J. Y. ‘A Counterexample to Theorems of Cox and Fine’. In: *Journal of Artificial Intelligence Research* vol. 10 (1999), pp. 67–85. URL: <https://www.jair.org/index.php/jair/article/view/10223>.
- [Hal99b] Halpern, J. Y. ‘Cox’s Theorem Revisited’. In: *Journal of Artificial Intelligence Research* vol. 11 (1999), pp. 429–435. URL: <https://arxiv.org/abs/cs/9911012>.
- [Hof23] Hofweber, T. ‘Logic and Ontology’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Summer 2023. Metaphysics Research Lab, Stanford University, 2023. URL: <https://plato.stanford.edu/archives/sum2023/entries/logic-ontology/>.
- [HS10] Harrigan, N. and Spekkens, R. W. ‘Einstein, Incompleteness, and the Epistemic View of Quantum States’. In: *Foundations of Physics* vol. 40, no. 2 (2010), pp. 125–157.
- [Hui49] Huizinga, J. *Homo Ludens: A Study of the Play-Element in Culture*. London: Routledge & Kegan Paul, 1949.
- [Hum75] Hume, D. *An Enquiry concerning Human Understanding*. Ed. by Selby-Bigge, L. A. and Nidditch, P. H. 3rd ed. Originally published as *Philosophical Essays concerning Human Understanding*, London: A. Millar, 1748. Oxford: Clarendon Press, 1975.
- [Hum78] Hume, D. *A Treatise of Human Nature: Being an Attempt to Introduce the Experimental Method of Reasoning into Moral Subjects*. Ed. by Selby-Bigge, L. A. and Nidditch, P. H. 2nd ed. Originally published London: John Noon, 1739–1740. Oxford: Clarendon Press, 1978.
- [HV06] Huang, Y. and Valtorta, M. ‘Pearl’s Calculus of Intervention Is Complete’. In: *Proceedings of the Twenty-Second Conference on Uncertainty in Artificial Intelligence*. AUAI Press, 2006, pp. 217–224. URL: <https://arxiv.org/abs/1206.6831>.
- [Ibá99] Ibáñez, C. F. ‘Researchers Face ‘Catch-22’ Grants Trap’. In: *Nature* vol. 398 (1999), p. 280.
- [Iem24] Iemhoff, R. ‘Intuitionism in the Philosophy of Mathematics’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Summer 2024. Metaphysics Research Lab, Stanford University, 2024. URL: <https://plato.stanford.edu/archives/sum2024/entries/intuitionism/>.
- [Ioa05] Ioannidis, J. P. A. ‘Why Most Published Research Findings Are False’. In: *PLOS Medicine* vol. 2, no. 8 (2005), e124.
- [Jay03] Jaynes, E. T. *Probability Theory: The Logic of Science*. Cambridge: Cambridge University Press, 2003.
- [Jef61] Jeffreys, H. *Theory of Probability*. 3rd ed. Oxford: Clarendon Press, 1961.
- [JLP12] John, L. K., Loewenstein, G. and Prelec, D. ‘Measuring the Prevalence of Questionable Research Practices with Incentives for Truth Telling’. In: *Psychological Science* vol. 23, no. 5 (2012), pp. 524–532.

B. Proofs and arguments

- [Kak26] Kakaes, K. *The AI Revolution in Math Has Arrived*. Quanta Magazine. 13th Apr. 2026. URL: <https://www.quantamagazine.org/the-ai-revolution-in-math-has-arrived-20260413/> (visited on 12/05/2026).
- [Kan98] Kant, I. *Critique of Pure Reason*. Ed. and trans. by Guyer, P. and Wood, A. W. The Cambridge Edition of the Works of Immanuel Kant. German original *Kritik der reinen Vernunft*, Riga: Hartknoch, 1st ed. 1781 (A) / 2nd ed. 1787 (B). Cambridge: Cambridge University Press, 1998.
- [Key21] Keynes, J. M. *A Treatise on Probability*. London: Macmillan, 1921. URL: <https://archive.org/details/treatiseonprobab007528mbp>.
- [Kie80] Kierkegaard, S. *The Concept of Anxiety: A Simple Psychologically Orienting Deliberation on the Dogmatic Issue of Hereditary Sin*. Ed. and trans. by Thomte, R. and Anderson, A. B. Kierkegaard's Writings VIII. Danish original *Begrebet Angest*, Reitzel, Copenhagen, 1844. Princeton, NJ: Princeton University Press, 1980.
- [Kie83] Kierkegaard, S. *Fear and Trembling / Repetition*. Ed. and trans. by Hong, H. V. and Hong, E. H. Kierkegaard's Writings VI. Danish original *Frygt og Bæven*, Reitzel, Copenhagen, 1843. Princeton, NJ: Princeton University Press, 1983.
- [Kie89] Kierkegaard, S. *The Concept of Irony, with Continual Reference to Socrates*. Ed. and trans. by Hong, H. V. and Hong, E. H. Kierkegaard's Writings II. Danish original *Om Begrebet Ironi med stadigt Hensyn til Socrates*, Reitzel, Copenhagen, 1841. Princeton, NJ: Princeton University Press, 1989.
- [Kle+14] Klein, R. A. et al. 'Investigating Variation in Replicability: A "Many Labs" Replication Project'. In: *Social Psychology* vol. 45, no. 3 (2014), pp. 142–152.
- [Kon15] Konnikova, M. *How a Gay-Marriage Study Went Wrong*. The New Yorker. 22nd May 2015. URL: <https://www.newyorker.com/science/maria-konnikova/how-a-gay-marriage-study-went-wrong> (visited on 12/05/2026).
- [Lan+22] Lane, J. N. et al. 'Conservatism Gets Funded? A Field Experiment on the Role of Negative Information in Novel Project Evaluation'. In: *Management Science* vol. 68, no. 6 (2022), pp. 4478–4495.
- [Lan16] Landhuis, E. 'Information Overload'. In: *Nature* vol. 535 (2016), pp. 457–458.
- [Los11] Los Angeles Times. *Dutch Scientist Accused of Falsifying Data*. Los Angeles Times. 6th Nov. 2011. URL: <https://www.latimes.com/science/la-sci-science-fraud-20111106-story.html> (visited on 12/05/2026).
- [Mar84] Martin-Löf, P. *Intuitionistic Type Theory*. Naples: Bibliopolis, 1984.
- [MAV05] Martinson, B. C., Anderson, M. S. and Vries, R. de. 'Scientists Behaving Badly'. In: *Nature* vol. 435 (2005), pp. 737–738.

- [McE20] McElreath, R. *Statistical Rethinking: A Bayesian Course with Examples in R and Stan*. 2nd ed. Boca Raton: CRC Press, 2020. URL: <https://xcelab.net/rm/>.
- [MO15] Marcus, A. and Oransky, I. *How the Biggest Fabricator in Science Got Caught*. Nautilus. 18th June 2015. URL: <https://nautil.us/how-the-biggest-fabricator-in-science-got-caught-235421/> (visited on 12/05/2026).
- [MU21] Moura, L. de and Ullrich, S. ‘The Lean 4 Theorem Prover and Programming Language’. In: *Automated Deduction — CADE 28*. Ed. by Platzer, A. and Sutcliffe, G. Vol. 12699. Lecture Notes in Computer Science. Cham: Springer, 2021, pp. 625–635. URL: https://doi.org/10.1007/978-3-030-79876-5_37.
- [Mun+17] Munafò, M. R. et al. ‘A Manifesto for Reproducible Science’. In: *Nature Human Behaviour* vol. 1 (10th Jan. 2017), p. 0021.
- [Nad26] Naddaf, M. *How Much for a Fake Authorship? Ad Database Reveals Secrets of Scientific Fraud*. Nature. 24th Apr. 2026. URL: <https://www.nature.com/articles/d41586-026-01340-y> (visited on 12/05/2026).
- [Nat26] Nature Editorial. ‘We Need to Talk about Failure in Science’. In: *Nature* vol. 652 (23rd Apr. 2026), p. 1091.
- [NDR18] NDR. *More than 5,000 German Scientists Have Published Papers in Pseudo-Scientific Journals*. NDR. 19th July 2018. URL: https://www.ndr.de/der_ndr/presse/More-than-5000-German-scientists-have-published-papers-in-pseudo-scientific-journals,fakescience178.html (visited on 12/05/2026).
- [NI12] Nicholson, J. M. and Ioannidis, J. P. A. ‘Conform and Be Funded’. In: *Nature* vol. 492 (2012), pp. 34–36.
- [Nos+15] Nosek, B. A. et al. ‘Promoting an Open Research Culture’. In: *Science* vol. 348, no. 6242 (2015), pp. 1422–1425.
- [NPS90] Nordström, B., Petersson, K. and Smith, J. M. *Programming in Martin-Löf’s Type Theory: An Introduction*. Oxford: Oxford University Press, 1990. URL: <https://intuitionistic.wordpress.com/wp-content/uploads/2010/07/nordstrom-et-al-type-theory.pdf>.
- [NSM12] Nosek, B. A., Spies, J. R. and Motyl, M. ‘Scientific Utopia: II. Restructuring Incentives and Practices to Promote Truth over Publishability’. In: *Perspectives on Psychological Science* vol. 7, no. 6 (2012), pp. 615–631.
- [OC10] Oreskes, N. and Conway, E. M. *Merchants of Doubt: How a Handful of Scientists Obscured the Truth on Issues from Tobacco Smoke to Global Warming*. New York: Bloomsbury Press, 2010.
- [OGr24] O’Grady, C. *Failure at Every Level: How Science Sleuths Exposed Massive Ethics Violations at a Famed French Institute*. Science. 5th Sept. 2024. URL: <https://www.science.org/content/article/failure-every-level-how-science-sleuths-exposed-massive-ethics-violations-famed-french> (visited on 12/05/2026).

B. Proofs and arguments

- [OGr25] O’Grady, C. *Scientific Fraud Has Become an ‘Industry,’ Alarming Analysis Finds*. Science. 4th Aug. 2025. URL: <https://www.science.org/content/article/scientific-fraud-has-become-industry-alarming-analysis-finds> (visited on 12/05/2026).
- [OM23] Oransky, I. and Marcus, A. *There’s Far More Scientific Fraud than Anyone Wants to Admit*. The Guardian. 9th Aug. 2023. URL: <https://www.theguardian.com/commentisfree/2023/aug/09/scientific-misconduct-retraction-watch> (visited on 12/05/2026).
- [Ope15] Open Science Collaboration. ‘Estimating the Reproducibility of Psychological Science’. In: *Science* vol. 349, no. 6251 (2015), aac4716.
- [Orn26] Ornes, S. *Math Long Resisted a Digital Disruption. AI Is Poised to Change That*. Science News. 15th Apr. 2026. URL: <https://www.sciencenews.org/article/math-disrupted-by-ai-verify-proofs> (visited on 12/05/2026).
- [PBR12] Pusey, M. F., Barrett, J. and Rudolph, T. ‘On the Reality of the Quantum State’. In: *Nature Physics* vol. 8 (2012), pp. 475–478.
- [Pea09] Pearl, J. *Causality: Models, Reasoning, and Inference*. 2nd ed. Cambridge: Cambridge University Press, 2009.
- [Pea12] Pearl, J. ‘The Do-Calculus Revisited’. In: *Proceedings of the Twenty-Eighth Conference on Uncertainty in Artificial Intelligence*. AUAI Press, 2012, pp. 4–11. URL: https://ftp.cs.ucla.edu/pub/stat_ser/r402.pdf.
- [Pfe20] Pfeijffer, I. L. *Grand Hotel Europa*. Trans. by Hutchison, M. Dutch original *Grand Hotel Europa*, De Arbeiderspers, Amsterdam, 2018. New York: Farrar, Straus and Giroux, 2020.
- [Pik14] Piketty, T. *Capital in the Twenty-First Century*. Trans. by Goldhammer, A. French original *Le Capital au XXIe siècle*, Éditions du Seuil, Paris, 2013. Cambridge, MA: The Belknap Press of Harvard University Press, 2014.
- [PLF23] Park, M., Leahey, E. and Funk, R. J. ‘Papers and Patents Are Becoming Less Disruptive over Time’. In: *Nature* vol. 613 (2023), pp. 138–144.
- [PM18] Pearl, J. and Mackenzie, D. *The Book of Why: The New Science of Cause and Effect*. New York: Basic Books, 2018. URL: <https://www.basicbooks.com/titles/judea-pearl/the-book-of-why/9780465097616/>.
- [Pra87] Pratchett, T. *Mort*. London: Gollancz, 1987.
- [Pub26] PubPeer Foundation. *PubPeer*. PubPeer Foundation. 2nd July 2026. URL: <https://pubpeer.com/> (visited on 02/07/2026).
- [Ram31] Ramsey, F. P. ‘Truth and Probability’. In: *The Foundations of Mathematics and Other Logical Essays*. Ed. by Braithwaite, R. B. Written in 1926. London: Kegan Paul, Trench, Trubner & Co., 1931, pp. 156–198.
- [Ret26] Retraction Watch. *Retraction Watch*. Retraction Watch. 2nd July 2026. URL: <https://retractionwatch.com/> (visited on 02/07/2026).

- [Ric+25] Richardson, R. A. K. et al. ‘The Entities Enabling Scientific Fraud at Scale Are Large, Resilient, and Growing Rapidly’. In: *Proceedings of the National Academy of Sciences of the United States of America* vol. 122, no. 32 (12th Aug. 2025), e2420092122.
- [Sch+21] Schweinsberg, M. et al. ‘Same Data, Different Conclusions: Radical Dispersion in Empirical Results When Independent Analysts Operationalize and Test the Same Hypothesis’. In: *Organizational Behavior and Human Decision Processes* vol. 165 (2021), pp. 228–249.
- [Sch13] Schekman, R. *How Journals Like Nature, Cell and Science Are Damaging Science*. The Guardian. 9th Dec. 2013. URL: <https://www.theguardian.com/commentisfree/2013/dec/09/how-journals-nature-science-cell-damage-science> (visited on 12/05/2026).
- [Sha04] Shafer, G. ‘Comments on “Constructing a Logic of Plausible Inference: A Guide to Cox’s Theorem”, by Kevin S. Van Horn’. In: *International Journal of Approximate Reasoning* vol. 35 (2004), pp. 97–105.
- [Sil+18] Silberzahn, R. et al. ‘Many Analysts, One Dataset: Making Transparent How Variations in Analytical Choices Affect Results’. In: *Advances in Methods and Practices in Psychological Science* vol. 1, no. 3 (2018), pp. 337–356.
- [Sku26] Skuse, B. *What It Means to Be a Mathematician When AI Does the Math*. IEEE Spectrum. 25th June 2026. URL: <https://spectrum.ieee.org/ai-in-mathematics> (visited on 09/07/2026).
- [Slo26] Sloman, L. *In Math, Rigor Is Vital. But Are Digitized Proofs Taking It Too Far?* Quanta Magazine. 25th Mar. 2026. URL: <https://www.quantamagazine.org/in-math-rigor-is-vital-but-are-digitized-proofs-taking-it-too-far-20260325/> (visited on 12/05/2026).
- [SM16] Smaldino, P. E. and McElreath, R. ‘The Natural Selection of Bad Science’. In: *Royal Society Open Science* vol. 3 (2016), p. 160384.
- [Smi06] Smith, R. ‘Peer Review: A Flawed Process at the Heart of Science and Journals’. In: *Journal of the Royal Society of Medicine* vol. 99, no. 4 (2006), pp. 178–182.
- [SN25] Steup, M. and Neta, R. ‘Epistemology’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Spring 2025. Metaphysics Research Lab, Stanford University, 2025. URL: <https://plato.stanford.edu/archives/spr2025/entries/epistemology/>.
- [Sno98] Snow, P. ‘On the Correctness and Reasonableness of Cox’s Theorem for Finite Domains’. In: *Computational Intelligence* vol. 14, no. 3 (1998), pp. 452–459.
- [SNS11] Simmons, J. P., Nelson, L. D. and Simonsohn, U. ‘False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant’. In: *Psychological Science* vol. 22, no. 11 (2011), pp. 1359–1366.

B. Proofs and arguments

- [SP06] Shpitser, I. and Pearl, J. ‘Identification of Joint Interventional Distributions in Recursive Semi-Markovian Causal Models’. In: *Proceedings of the Twenty-First National Conference on Artificial Intelligence*. AAAI Press, 2006, pp. 1219–1226.
- [SP08] Shpitser, I. and Pearl, J. ‘Complete Identification Methods for the Causal Hierarchy’. In: *Journal of Machine Learning Research* vol. 9 (2008), pp. 1941–1979. URL: <https://jmlr.org/papers/v9/shpitser08a.html>.
- [Spa18] Spaink, K. *Profileer- en Publicatiedwang: Wetenschap in de Greep van het Geld*. Follow the Money. 8th Aug. 2018. URL: <https://www.ftm.nl/artikelen/publicatiedwang> (visited on 12/05/2026).
- [Spe07] Spekkens, R. W. ‘Evidence for the Epistemic View of Quantum States: A Toy Theory’. In: *Physical Review A* vol. 75, no. 3 (2007), p. 032110. arXiv: [quant-ph/0401052](https://arxiv.org/abs/quant-ph/0401052).
- [SR14] Spicer, A. and Roulet, T. *Hate the Peer-Review Process? Einstein Did Too*. The Conversation. 2nd June 2014. URL: <https://theconversation.com/hate-the-peer-review-process-einstein-did-too-27405> (visited on 12/05/2026).
- [SS20] Schwaber, K. and Sutherland, J. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. 2020. URL: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf> (visited on 26/05/2026).
- [The16] The New Yorker. *The Stem-Cell Scandal*. The New Yorker. 29th Feb. 2016. URL: <https://www.newyorker.com/magazine/2016/02/29/the-stem-cell-scandal> (visited on 12/05/2026).
- [The24] ThePrint. *India’s Research Crime Is Getting Worse. Scientists Are Gaming Peer Review System*. ThePrint. 14th Sept. 2024. URL: <https://theprint.in/ground-reports/indias-research-crime-is-getting-worse-scientists-are-gaming-peer-review-system/2261884/> (visited on 12/05/2026).
- [The26] The Black Spatula Project. *The Black Spatula Project*. Project website. 2nd July 2026. URL: <https://the-black-spatula-project.github.io/> (visited on 02/07/2026).
- [Tho54] Thomson, J. F. ‘Tasks and Super-Tasks’. In: *Analysis* vol. 15, no. 1 (1st Oct. 1954), pp. 1–13. URL: <https://academic.oup.com/analysis/article-abstract/15/1/1/170170>.
- [TS09] Tian, J. and Shpitser, I. ‘On Identifying Causal Effects’. In: *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*. AUAI Press, 2009, pp. 722–729.
- [Van03] Van Horn, K. S. ‘Constructing a Logic of Plausible Inference: A Guide to Cox’s Theorem’. In: *International Journal of Approximate Reasoning* vol. 34, no. 1 (2003), pp. 3–24.
- [Vit05] Vitali, G. *Sul problema della misura dei gruppi di punti di una retta: Nota*. Bologna: Tip. Gamberini e Parmeggiani, 1905, p. 5. URL: <https://web.unica.it/unica/protected/417040/0/def/ref/MAT246835/>.

- [Wel26] Wellcome Open Research. *Wellcome Open Research*. Wellcome Open Research. 2nd July 2026. URL: <https://wellcomeopenresearch.org/> (visited on 02/07/2026).
- [Zha26] Zhao, C. ‘AI Has Helped Scientists—but May Have Hurt Science’. In: *Science* vol. 391, no. 6783 (22nd Jan. 2026), p. 337.
- [Zub19] Zuboff, S. *The Age of Surveillance Capitalism: The Fight for a Human Future at the New Frontier of Power*. London: Profile Books, 2019.

